The runner does not use error-message substring heuristics to decide whether a condition is fatal. Any condition exception terminates production and writes structured failure artifacts.
Durable condition status

Status is atomically rewritten after every committed condition. This is a major improvement over end-only status writing.
No unsafe resume

Correct:

Python

```
if resume:
raise UnsupportedPolicyError(...)
```

No overwrite

Every execution gets a fresh execution ID and create-only directory.
Logical condition transaction

The order is sensible:

simulate;
validate;
construct condition outputs;
write checkpoint;
commit to in-memory consolidation;
write success status.

A failed checkpoint does not enter the consolidated result.
Structured failure evidence

The runner preserves:

exception type;
message;
condition ID;
traceback;

elapsed time;
baseline certificate hash;
condition-status hash;
final non-freezeable manifest.

P0 certification issues
1. No independently pinned frozen authority is used here

The runner records:

Python

manifest_file_sha256
npz_file_sha256
scalar_csv_file_sha256

but _validate_authority_paths() only checks file existence. _run_baseline_gate() compares against the supplied authority set.

Unless oracle.py or the CLI performs an independent lock check before execute(), coordinated replacement of all three authority files remains possible.

Required production flow:

Python

authority_lock = load_authority_lock(...)
verify_authority_files(
authority_lock,
authority,
)

The lock should contain the already recorded hashes and must be independently stored.

This may be implemented in oracle.py; it remains pending until that file is reviewed.
2. Source and authority hashes are checked only at startup

The run lasts long enough that a source or authority file could be modified after initial hashing.

Before setting:

Python

"freezeable": True

recompute:

Python

final_source_hashes = compute_source_hashes(
spec,
registration,
engine,
)

if final_source_hashes != source_hashes:
raise ArtifactIntegrityError(
"Execution source files changed during the run"
)

Also rehash the frozen authorities:

Python

final_authority_hashes = {
"manifest_file_sha256":
file_sha256(authority["manifest"]),
"npz_file_sha256":
file_sha256(authority["npz"]),
"scalar_csv_file_sha256":
file_sha256(authority["scalar"]),
}

if final_authority_hashes != {
key: manifest["frozen_authority"][key]

for key in final_authority_hashes
}:
raise ArtifactIntegrityError(
"Frozen authority files changed during execution"
)

This is particularly important for a 70-minute production run.
3. Multiple collectors are recorded but only the first is executed

The manifest records all:

Python

"collectors": [
...
for item in spec.collectors
]

but the runner does:

Python

collector_spec = spec.collectors[0]
collector = build_collector(collector_spec)

Therefore, if a specification contains multiple collectors, the manifest claims they participated even though only one executed.

For modular v1, require exactly one collector:

Python

if len(spec.collectors) != 1:
raise UnsupportedPolicyError(
"Modular platform v1 supports exactly one collector"
)

Place this in validate_engine_compatibility() or validate_experiment_spec().

Future multi-collector support must execute, validate, namespace, and consolidate every collector.
4. “Freezeable” does not imply modular oracle certification

The runner marks a completed execution:

Python

"freezeable": True

after:

frozen baseline passes;
all conditions complete;
outputs are written.

That is sufficient for an internally complete execution, but it does not establish that the new modular orchestration reproduces runs/b57562a8ee77.

Keep two separate statuses:

JSON

{
"freezeable": true,
"platform_certification": {
"status": "not_evaluated",
"oracle_run": "runs/b57562a8ee77"
}
}

Once full logical oracle comparison passes:

JSON

{

"platform_certification": {
"status": "oracle_equivalent",
"oracle_id": "b57562a8ee77",
"comparison_manifest_sha256": "..."
}
}

Do not describe the modular runner as a certified replacement based only on its own complete run.
P1 integrity improvements
5. Validate all baseline seed checks, not only seed statuses

Current validation checks:

Python

item.get("status") == "passed"

Also require no false detailed checks and no reported differences:

Python

for seed, item in seed_results.items():
failed_checks = [
name
for name, passed in item.get(
"checks",
{},
).items()
if not passed
]

if failed_checks:
raise BaselineGateError(
f"Baseline seed {seed} contains false "
f"checks: {failed_checks[:10]}"
)

if item.get("max_diffs"):

```
raise BaselineGateError(
f"Baseline seed {seed} reports "
f"nonempty max_diffs"
)
```

Also verify:

Python

```
tuple(result["protocol"]["branches"])
== tuple(spec.branch_rng_namespace)
```

The engine gate should already enforce this, but the runner certificate should verify it independently.

6. Reopen and validate final outputs before freezeability

After:

Python

```
atomic_write_csv(...)
atomic_write_npz(...)
```

the runner hashes the files but does not read them back.

Before final manifest commitment:

Python

```
loaded_payload = load_npz(paths.branch_npz)
```

Require:

Python

```
set(loaded_payload) == set(consolidated_payload)
```

and exact NaN-aware logical equality for every value.

Reload the CSV and require:

text

expected field order
expected 800 rows
exact condition coverage
no duplicates

The existing collector/validation functions can assist.

This catches:

serialization errors;
truncation;
field-order drift;
unexpected type coercion;
missing keys.

oracle.py may provide logical comparison helpers; use them rather than duplicating logic.
7. Record source-hash closure in the final manifest

Add:

JSON

{
"source_integrity": {
"initial_hashes": {...},
"final_hashes": {...},
"unchanged": true
}
}

Likewise for frozen authorities.

This turns the final manifest into a closed execution record rather than an initial snapshot.
8. Failure count is mislabeled

This field:

Python

"committed_condition_rows": len(status_rows)

includes the failed status row. It is not the number of committed successful conditions.

Track explicitly near the top:

Python

successful_conditions = 0

Increment only after committed success. In failure output record:

Python

"successful_conditions_committed":
successful_conditions,
"status_rows_written":
len(status_rows),
"condition_checkpoints_written":
len(list(paths.conditions.glob("*.npz"))),

Do not call all status rows committed condition rows.
9. Set running/progress status in the manifest

The manifest remains:

Python

"status": "initialized"

until final completion/failure.

After acquiring the lock:

Python

```
manifest["status"] = "baseline_running"
```

After baseline:

Python

```
manifest["status"] = "conditions_running"
```

Optionally update progress periodically:

JSON

```
{
"progress": {
"completed": 83,
"expected": 200,
"last_condition_id": "...",
"updated_utc": "..."
}
}
```

Condition-status CSV already gives durable progress, so manifest updates need not occur every condition.

10. Lock before mutable authority writes

Current order:

Python

create_execution_directory(paths)
atomic_write_json(paths.manifest, manifest)

with ExecutionLock(...):
...

The unique execution directory and random execution ID make collision unlikely, but the stated lifecycle says locking precedes mutable writes.

Prefer:

Python

create_execution_directory(paths)

with ExecutionLock(paths.lock_file):
atomic_write_json(paths.manifest, manifest)
...

Directory creation necessarily precedes the internal lock, but the manifest and study artifacts should be written under the lock.
Generality concerns
11. _initial_manifest() is T-window-specific

It assumes:

Python

axis.name == "rho"
level.metadata["T_window"]
study_name = "AIEF_1_4_Twindow_sensitivity"

This means the runner is not yet a generic modular driver; it is a reusable execution skeleton with a hardcoded S01 manifest builder.

Move study-specific manifest content behind a study or collector hook:

Python

```
study_manifest_extension(
spec,
conditions,
) -> dict[str, Any]
```

The generic runner should describe factors generically:

JSON

```
{
"model_axes": [
{
"name": "rho",
"levels": [...]
}
]
}
```

The S01 extension can additionally emit legacy-compatible:

JSON

```
{
"study": {
"rho_values": [...],
"T_window_values": [...]
}
}
```

This change should preserve Analysis v3 compatibility while removing the generic runner’s direct dependency on rho.
12. Checkpoint counting is T-window-specific

Current:

Python

glob("condition_seed_*_rho_*.npz")

Use exact expected filenames from the collector:

Python

```
expected_checkpoints = {
collector.checkpoint_filename(condition)
for condition in conditions
}

observed_checkpoints = {
path.name
for path in paths.conditions.glob("*.npz")
}

if observed_checkpoints != expected_checkpoints:
...
```

This catches both missing and extra checkpoints and removes hardcoded rho naming from the runner.
13. Generic runner is tied to a concrete collector class

Current type and runtime checks:

Python

```
collector: TWindowLegacyCollectorV1
isinstance(
collector,
TWindowLegacyCollectorV1,
)
```

Use the Collector protocol:

Python

collector: Collector

For v1, build_collector() may still support only the T-window collector, but the runner should not import or type-check the concrete class.

The exact-one-collector restriction is sufficient.
Environment provenance

Record:

text

machine architecture
processor
bit generator
thread environment variables
BLAS/LAPACK configuration
float information

At minimum:

Python

"environment": {
...
"machine": platform.machine(),
"processor": platform.processor(),
"thread_environment": {
name: os.environ.get(name)
for name in (
"OMP_NUM_THREADS",
"MKL_NUM_THREADS",
"OPENBLAS_NUM_THREADS",
"NUMEXPR_NUM_THREADS",
)
},
"float_info": {
"eps": np.finfo(float).eps,
"tiny": np.finfo(float).tiny,

"max": np.finfo(float).max,
},
}

Capturing np.show_config() output may require redirecting stdout to a string.
Required runner tests
Full-regeneration spy test

Wrap or monkeypatch:

Python

engine.simulate_prefix

and assert exactly one call per condition:

text

200 conditions -> 200 prefix calls

Require each call to receive the correct seed and condition-specific T_window.
No split reuse

Return uniquely tagged split states and verify each is consumed only by branches from its own condition.
RNG namespace

For every condition:

text

make_branch_rngs called once
namespace exactly seven branches
future_shuffle child index 6

Baseline gate

Assert:

called before simulate_prefix for study conditions;
failure prevents any condition execution;
baseline-only executes zero study conditions.

Failure transaction

Force failure at:

simulation;
collector validation;
checkpoint write;
condition-status write;
consolidated CSV write;
consolidated NPZ write;
final manifest write.

Verify:

nonzero exception;
freezeable=false;
traceback exists;
previously committed checkpoints remain;
no false completed status.

Source mutation test

Modify a copied source file during a test run and require final integrity failure.
Oracle comparison

Compare a modular S01 run against runs/b57562a8ee77 by logical contents.
Updated provisional conclusion

The actual numerical execution path is suitable for S01 and correctly enforces full regeneration. The runner is substantially better than the monolithic orchestration.

Before calling it certification-grade, require:

pinned frozen-authority verification;
exact-one-collector enforcement;
final source/authority rehash;
final CSV/NPZ round-trip validation;
separate execution completeness from oracle certification;
removal of T-window assumptions from generic runner paths;
strict production JSON behavior from artifacts.py.

The remaining files—especially oracle.py, spec.py, and validation.py—may already address some of these.

spec.py

text

"""Immutable experiment specifications and canonical scientific identities.

The specification layer deliberately contains no execution code. It describes
only scientific design, engine compatibility, schemas, and artifact policy.
Runtime provenance (paths, timestamps, hosts, and execution mode) is kept out
of canonical scientific identities.
"""

from __future__ import annotations

import dataclasses
import hashlib
import json
import math
import re
from collections.abc import Iterator, Mapping
from dataclasses import dataclass, field
from typing import Any

JsonScalar = str | int | float | bool | None

```
class FrozenDict(Mapping[str, Any]):
"""Small immutable, deterministically ordered mapping.

Values are recursively frozen. This is used instead of mutable dictionaries
inside frozen specification dataclasses.
"""

__slots__ = ("_items", "_dict")

def __init__(self, values: Mapping[str, Any] | None = None):
source = {} if values is None else dict(values)
self._items = tuple(
(str(key), freeze_value(value))
for key, value in sorted(source.items(), key=lambda item: str(item[0]))
)
self._dict = dict(self._items)

def __getitem__(self, key: str) -> Any:
return self._dict[key]

def __iter__(self) -> Iterator[str]:
return iter(self._dict)

def __len__(self) -> int:
return len(self._dict)

def __hash__(self) -> int:
return hash(self._items)

def __repr__(self) -> str:
return f"FrozenDict({dict(self._items)!r})"



def freeze_value(value: Any) -> Any:
"""Recursively convert JSON-like containers to immutable containers."""
if isinstance(value, FrozenDict):
return value
```

```
if isinstance(value, Mapping):
return FrozenDict(value)
if isinstance(value, (list, tuple)):
return tuple(freeze_value(item) for item in value)
if isinstance(value, set):
return tuple(sorted((freeze_value(item) for item in value), key=repr))
return value



@dataclass(frozen=True)
class FactorLevel:
level_id: str
label: str
value: float
metadata: Mapping[str, Any] = field(default_factory=FrozenDict)

def __post_init__(self) -> None:
object.__setattr__(self, "metadata", FrozenDict(self.metadata))
if not self.level_id or not re.fullmatch(r"[A-Za-z0-9][A-Za-z0-9_.-]*", self.level_id):
raise ValueError(f"Invalid factor level ID: {self.level_id!r}")
if not self.label:
raise ValueError("Factor level label must be nonempty")
if isinstance(self.value, bool) or not math.isfinite(float(self.value)):
raise ValueError(f"Factor level {self.level_id!r} must have a finite numeric value")



@dataclass(frozen=True)
class ModelFactorSpec:
name: str
levels: tuple[FactorLevel, ...]
reference_level_id: str
regeneration_policy: str
resolver_id: str



@dataclass(frozen=True)
class ProtocolSpec:
T_sim: float
burn_end_time: float
```

split_time: float
future_duration: float
diagnostic_stride: int

@dataclass(frozen=True)
class SeedDesignSpec:
master_seed: int
count: int
paired_seeds: tuple[int, ...]
generation_rule: str

@dataclass(frozen=True)
class BranchSpec:
name: str
engine_branch: str
role: str
intervention_ids: tuple[str, ...] = ()
forcing_policy_id: str | None = None
requires_rng: bool = False

@dataclass(frozen=True)
class CollectorSpec:
collector_id: str
schema_version: str
core_schema_version: str
study_schema_version: str
requires_continue: bool = False
settings: Mapping[str, Any] = field(default_factory=FrozenDict)

def __post_init__(self) -> None:
object.__setattr__(self, "settings", FrozenDict(self.settings))

@dataclass(frozen=True)
class BaselineGateSpec:

required: bool
expected_seed_count: int
expected_branch_namespace: tuple[str, ...]
exact_arrays: bool = True
exact_scalars: bool = True
certificate_reuse: bool = False

@dataclass(frozen=True)
class ArtifactPolicySpec:
policy_id: str
scalar_filename_prefix: str
array_filename_prefix: str
checkpoint_role: str
expected_condition_count: int
expected_scalar_row_count: int

@dataclass(frozen=True)
class ValidatorSpec:
validator_id: str
schema_version: str
settings: Mapping[str, Any] = field(default_factory=FrozenDict)

def __post_init__(self) -> None:
object.__setattr__(self, "settings", FrozenDict(self.settings))

@dataclass(frozen=True)
class ExperimentSpec:
study_id: str
study_slug: str
study_version: str
title: str
purpose: str
engine_module: str
base_model: Any
model_axes: tuple[ModelFactorSpec, ...]
protocol: ProtocolSpec

```
seed_design: SeedDesignSpec
branch_rng_namespace: tuple[str, ...]
branches: tuple[BranchSpec, ...]
collectors: tuple[CollectorSpec, ...]
baseline_gate: BaselineGateSpec
artifact_policy: ArtifactPolicySpec
validators: tuple[ValidatorSpec, ...]

def __post_init__(self) -> None:
validate_experiment_spec(self)



class CanonicalizationError(ValueError):
"""Raised when an object cannot enter a canonical scientific identity."""



def _canonical_value(value: Any) -> Any:
"""Convert supported objects into canonical JSON-compatible values."""
if dataclasses.is_dataclass(value) and not isinstance(value, type):
return {
item.name: _canonical_value(getattr(value, item.name))
for item in dataclasses.fields(value)
}
if isinstance(value, Mapping):
converted: dict[str, Any] = {}
for key, item in value.items():
if not isinstance(key, str):
raise CanonicalizationError(f"Canonical mapping key must be str, got {type(key).__name__}")
converted[key] = _canonical_value(item)
return converted
if isinstance(value, (tuple, list)):
return [_canonical_value(item) for item in value]
if isinstance(value, bool) or value is None or isinstance(value, (str, int)):
return value
if isinstance(value, float):
if not math.isfinite(value):
raise CanonicalizationError("NaN and Infinity are forbidden in canonical JSON")
return value

# NumPy is optional at this layer. Handle scalar objects without making it
```

```
# a package import requirement.
item_method = getattr(value, "item", None)
if callable(item_method):
scalar = item_method()
if scalar is not value:
return _canonical_value(scalar)


raise CanonicalizationError(
f"Unsupported canonical value {type(value).__module__}.{type(value).__qualname__}"
)



def canonical_data(value: Any) -> Any:
"""Return the JSON-compatible canonical data tree for ``value``."""
return _canonical_value(value)



def canonical_json(value: Any) -> str:
"""Serialize with stable ordering and full round-trip float precision."""
return json.dumps(
canonical_data(value),
sort_keys=True,
separators=(",", ":"),
ensure_ascii=False,
allow_nan=False,
)



def sha256_canonical(value: Any) -> str:
return hashlib.sha256(canonical_json(value).encode("utf-8")).hexdigest()



def spec_sha256(spec: ExperimentSpec) -> str:
return sha256_canonical(spec)



def scientific_identity_payload(
spec: ExperimentSpec,
```

```
source_hashes: Mapping[str, str],
) -> dict[str, Any]:
"""Build the complete, path-free scientific identity payload.

Some fields are intentionally repeated outside the canonical specification.
This makes audit records explicit and guards against a future spec serializer
accidentally omitting a scientifically material field.
"""
axis_levels = {
axis.name: [
{
"level_id": level.level_id,
"value": level.value,
"metadata": level.metadata,
}
for level in axis.levels
]
for axis in spec.model_axes
}
return {
"identity_schema": "aief-scientific-run-identity-v1",
"study_id": spec.study_id,
"study_version": spec.study_version,
"canonical_spec": canonical_data(spec),
"spec_sha256": spec_sha256(spec),
"source_hashes": dict(sorted(source_hashes.items())),
"paired_seeds": list(spec.seed_design.paired_seeds),
"seed_generation_rule": spec.seed_design.generation_rule,
"model_factor_levels": axis_levels,
"protocol": canonical_data(spec.protocol),
"branch_rng_namespace": list(spec.branch_rng_namespace),
"executed_branches": [branch.engine_branch for branch in spec.branches],
"collectors": [
{
"collector_id": collector.collector_id,
"schema_version": collector.schema_version,
"core_schema_version": collector.core_schema_version,
"study_schema_version": collector.study_schema_version,
}
for collector in spec.collectors
],
}
```

```
def scientific_run_sha256(spec: ExperimentSpec, source_hashes: Mapping[str, str]) -> str:
return sha256_canonical(scientific_identity_payload(spec, source_hashes))



def _duplicates(values: list[str] | tuple[str, ...]) -> list[str]:
return sorted({value for value in values if values.count(value) > 1})



def validate_experiment_spec(spec: ExperimentSpec) -> None:
"""Validate static scientific and orchestration invariants."""
if not re.fullmatch(r"[A-Za-z0-9][A-Za-z0-9_.-]*", spec.study_id):
raise ValueError(f"Invalid study_id: {spec.study_id!r}")
if not re.fullmatch(r"[a-z0-9][a-z0-9_-]*", spec.study_slug):
raise ValueError(f"Invalid study_slug: {spec.study_slug!r}")
if not spec.study_version or not spec.title or not spec.purpose:
raise ValueError("study_version, title, and purpose must be nonempty")
if not spec.engine_module:
raise ValueError("engine_module must be nonempty")

factor_names = [axis.name for axis in spec.model_axes]
if len(factor_names) != len(set(factor_names)):
raise ValueError(f"Duplicate factor names: {_duplicates(factor_names)}")
for axis in spec.model_axes:
if not axis.levels:
raise ValueError(f"Factor {axis.name!r} has no levels")
ids = [level.level_id for level in axis.levels]
if len(ids) != len(set(ids)):
raise ValueError(f"Factor {axis.name!r} has duplicate level IDs: {_duplicates(ids)}")
if axis.reference_level_id not in ids:
raise ValueError(
f"Factor {axis.name!r} reference level {axis.reference_level_id!r} is absent"
)
if axis.regeneration_policy != "full_from_initialization":
raise ValueError(
f"Unsupported regeneration policy {axis.regeneration_policy!r}; "
"engine-v1 modular studies require full_from_initialization"
)
```

```
protocol = spec.protocol
times = (
protocol.T_sim,
protocol.burn_end_time,
protocol.split_time,
protocol.future_duration,
)
if any(isinstance(value, bool) or not math.isfinite(float(value)) for value in times):
raise ValueError("Protocol times must be finite numbers")
if not (0.0 < protocol.burn_end_time < protocol.split_time < protocol.T_sim):
raise ValueError("Protocol must satisfy 0 < burn < split < T_sim")
if protocol.future_duration <= 0.0:
raise ValueError("future_duration must be positive")
if protocol.split_time + protocol.future_duration != protocol.T_sim:
raise ValueError("split_time + future_duration must equal T_sim exactly")
if isinstance(protocol.diagnostic_stride, bool) or not isinstance(protocol.diagnostic_stride, int):
raise ValueError("diagnostic_stride must be a positive integer")
if protocol.diagnostic_stride <= 0:
raise ValueError("diagnostic_stride must be positive")


seeds = spec.seed_design
if isinstance(seeds.master_seed, bool) or not isinstance(seeds.master_seed, int):
raise ValueError("master_seed must be an integer")
if seeds.count <= 0 or len(seeds.paired_seeds) != seeds.count:
raise ValueError("Seed count does not match paired seed list")
if len(set(seeds.paired_seeds)) != seeds.count:
raise ValueError("Paired seed list contains duplicates")
if not seeds.generation_rule:
raise ValueError("seed generation rule must be recorded")


namespace = spec.branch_rng_namespace
if not namespace or len(namespace) != len(set(namespace)):
raise ValueError("branch_rng_namespace must be nonempty and unique")
branch_names = [branch.name for branch in spec.branches]
engine_names = [branch.engine_branch for branch in spec.branches]
if len(branch_names) != len(set(branch_names)):
raise ValueError(f"Duplicate branch names: {_duplicates(branch_names)}")
if len(engine_names) != len(set(engine_names)):
raise ValueError(f"Duplicate executed engine branches: {_duplicates(engine_names)}")
unknown = set(engine_names) - set(namespace)
```

```
if unknown:
raise ValueError(f"Executed branches outside RNG namespace: {sorted(unknown)}")


collector_names = [collector.collector_id for collector in spec.collectors]
if not collector_names or len(collector_names) != len(set(collector_names)):
raise ValueError("Collector IDs must be nonempty and unique")
if any(collector.requires_continue for collector in spec.collectors) and "continue" not in engine_names:
raise ValueError("At least one collector requires the continue branch")


validator_names = [validator.validator_id for validator in spec.validators]
if len(validator_names) != len(set(validator_names)):
raise ValueError(f"Duplicate validator IDs: {_duplicates(validator_names)}")


gate = spec.baseline_gate
if gate.required:
if gate.expected_seed_count != seeds.count:
raise ValueError("Baseline gate seed count differs from seed design")
if gate.expected_branch_namespace != namespace:
raise ValueError("Baseline gate branch namespace differs from RNG namespace")
if gate.certificate_reuse:
raise ValueError("Certificate reuse is unsupported in modular platform v1")


if spec.artifact_policy.expected_condition_count <= 0:
raise ValueError("Expected condition count must be positive")
if spec.artifact_policy.expected_scalar_row_count <= 0:
raise ValueError("Expected scalar row count must be positive")


# Verify that canonical serialization succeeds at construction time.
canonical_json(spec)
```

spec.py has the right conceptual design: immutable records, recursive metadata freezing, canonical JSON, explicit scientific identity, and separation of runtime provenance from scientific identity. However, several immutability and path-safety gaps should be corrected before relying on the resulting run ID as a certification identity.

Provisional verdict

Canonical identity concept: strong
S01 specification compatibility: likely correct
Actual deep immutability: incomplete

Path safety: incomplete
Specification/count consistency: incomplete
No obvious S01 numerical issue
Not yet certification-grade as a general specification layer

P0: FrozenDict can silently collapse keys

Current construction converts all keys to strings:

Python

```
self._items = tuple(
(str(key), freeze_value(value))
for key, value in ...
)
self._dict = dict(self._items)
```

A mapping such as:

Python

```
{1: "first", "1": "second"}
```

produces duplicate "1" entries in _items, while _dict retains only one. This creates disagreement among:

iteration;
length;
hash;
canonical serialization;
lookup.

Required fix

Require string keys before conversion:

Python

class FrozenDict(Mapping[str, Any]):
__slots__ = ("_items", "_dict")

def __init__(
self,
values: Mapping[str, Any] | None = None,
):
source = {} if values is None else dict(values)

for key in source:
if not isinstance(key, str):
raise CanonicalizationError(
f"FrozenDict key must be str, got "
f"{type(key).__name__}"
)

self._items = tuple(
(
key,
freeze_value(source[key]),
)
for key in sorted(source)
)
self._dict = MappingProxyType(
dict(self._items)
)

Import:

Python

from types import MappingProxyType

Do not silently stringify scientific metadata keys.
P0: internal _dict remains mutable

Although FrozenDict exposes no mutation method, this remains possible:

Python

frozen._dict["rho"] = 999

because _dict is an ordinary dictionary and is accessible.

That breaks the claim of deep immutability and can make:

text

hash
lookup
canonical serialization

disagree.

Use MappingProxyType, or implement __getitem__ directly from immutable _items.

A mapping proxy is preferable.
P0: frozen dataclasses do not canonicalize caller-provided lists

Python does not enforce annotations. A caller can construct:

Python

ExperimentSpec(
model_axes=[...],
branches=[...],
collectors=[...],
...
)

The outer dataclass is frozen, but those lists remain mutable.

The same applies to:

text

ModelFactorSpec.levels
SeedDesignSpec.paired_seeds
BranchSpec.intervention_ids
BaselineGateSpec.expected_branch_namespace
ExperimentSpec.branch_rng_namespace
ExperimentSpec.validators

Required fix

Canonicalize every sequence field in __post_init__():

Python

```
@dataclass(frozen=True)
class ExperimentSpec:
...

def __post_init__(self) -> None:
object.__setattr__(
self,
"model_axes",
tuple(self.model_axes),
)
object.__setattr__(
self,
"branch_rng_namespace",
tuple(self.branch_rng_namespace),
)
object.__setattr__(
self,
"branches",
tuple(self.branches),
)
object.__setattr__(
self,
"collectors",
```

tuple(self.collectors),
)
object.__setattr__(
self,
"validators",
tuple(self.validators),
)
validate_experiment_spec(self)

Do the same in each nested dataclass.

Test that mutating the caller’s original list after construction does not change the specification or its hash.
P0: unsafe path components are insufficiently validated

These values enter execution paths:

text

study_id
study_slug
study_version
artifact filename prefixes
policy ID

study_id and study_slug are constrained, but study_version is only checked for nonemptiness.

This is currently possible:

Python

study_version="../../other_directory"

Likewise, artifact prefixes could contain / or ...
Required fix

Define one path-component validator:

Python

```
SAFE_COMPONENT = re.compile(
r"[A-Za-z0-9][A-Za-z0-9._-]*"
)

def validate_path_component(
value: str,
field_name: str,
) -> None:
if not isinstance(value, str):
raise ValueError(
f"{field_name} must be a string"
)

if not SAFE_COMPONENT.fullmatch(value):
raise ValueError(
f"Invalid {field_name}: {value!r}"
)

if value in {".", ".."}:
raise ValueError(
f"Invalid {field_name}: {value!r}"
)
```

Apply it to:

text

```
study_id
study_slug
study_version
policy_id
scalar_filename_prefix
array_filename_prefix
collector IDs
validator IDs
factor names
resolver IDs where appropriate
```

branch names
engine branch names

Artifact-policy filenames should ideally be complete validated filenames, not loosely concatenated prefixes.
P0/P1: metadata freezing accepts unsupported objects too late

freeze_value() returns arbitrary scalar-like objects unchanged:

Python

return value

It also converts sets using:

Python

sorted(..., key=repr)

Problems:

arbitrary objects can enter a supposedly JSON-like specification;
repr ordering may not be a stable scientific canonicalization;
NumPy arrays can remain mutable;
Path, complex numbers, enums, and plugin objects can survive until later;
hashing may fail differently from serialization.

Preferred design

Allow only strict JSON-like values:

Python

def freeze_value(value: Any) -> Any:
if isinstance(value, FrozenDict):
return value

```
if isinstance(value, Mapping):
return FrozenDict(value)


if isinstance(value, (list, tuple)):
return tuple(
freeze_value(item)
for item in value
)


if isinstance(value, bool) or value is None:
return value


if isinstance(value, str):
return value


if isinstance(value, int):
return value


if isinstance(value, float):
if not math.isfinite(value):
raise CanonicalizationError(
"Metadata cannot contain NaN or Infinity"
)
return value


item_method = getattr(value, "item", None)
if callable(item_method):
try:
converted = item_method()
except Exception as exc:
raise CanonicalizationError(
f"Cannot convert metadata value "
f"{type(value).__name__}"
) from exc
if converted is not value:
return freeze_value(converted)


raise CanonicalizationError(
f"Unsupported immutable metadata value: "
```

f"{type(value).__module__}."
f"{type(value).__qualname__}"
)

Reject sets. If unordered-set semantics are needed, create a separately typed canonical set representation.
Numeric type canonicalization
FactorLevel.value

This permits a string:

Python

FactorLevel(
value="0.5",
)

because:

Python

float("0.5")

is finite. But the stored value remains a string, so:

execution resolves it as 0.5;
canonical specification stores "0.5";
another equivalent specification storing 0.5 gets a different identity.

That may be technically deterministic but is undesirable.

Require a numeric real type and canonicalize:

Python

from numbers import Real

if (
isinstance(self.value, bool)
or not isinstance(self.value, Real)
):
raise ValueError(...)

numeric_value = float(self.value)

if not math.isfinite(numeric_value):
raise ValueError(...)

object.__setattr__(
self,
"value",
numeric_value,
)

Apply similar canonicalization to protocol floats.
Seed validation needs strengthening

Current validation does not reject:

negative master seed;
negative paired seeds;
booleans in paired seeds;
floats in paired seeds;
paired seeds outside the intended unsigned 64-bit domain.

Use:

Python

UINT64_MAX = 2**64 - 1

if seeds.master_seed < 0:
raise ValueError(
"master_seed must be nonnegative"
)

for index, seed in enumerate(seeds.paired_seeds):
if (
isinstance(seed, bool)
or not isinstance(seed, int)
or seed < 0
or seed > UINT64_MAX
):
raise ValueError(
f"paired_seeds[{index}] must be "
f"a uint64-compatible integer"
)

Canonicalize NumPy integers to Python integers in SeedDesignSpec.__post_init__().

For S01, verify that the supplied paired seeds are exactly regenerated from the declared master seed and generation rule. A descriptive generation-rule string alone does not prove that relationship.
Baseline-gate semantics
required=False conflicts with the runner

BaselineGateSpec permits:

Python

required=False

but runner.execute() always runs the baseline.

For modular v1, make the specification contract explicit:

Python

if not gate.required:
raise ValueError(
"Baseline gate is mandatory in modular "
"platform v1"
)

Likewise require:

Python

```
gate.exact_arrays is True
gate.exact_scalars is True
gate.certificate_reuse is False
```

Otherwise the spec can claim different baseline semantics than the runner implements.
Exactly one collector in platform v1

The runner executes only:

Python

```
spec.collectors[0]
```

Therefore specification validation must require:

Python

```
if len(spec.collectors) != 1:
raise ValueError(
"Modular platform v1 supports exactly "
"one collector"
)
```

Without this, the canonical spec and manifest can list multiple collectors while only one is used.
S01 axis restrictions

The generic spec allows multiple axes, while factors.py only safely supports one T_window scale axis.

For platform v1/S01 require:

Python

if len(spec.model_axes) != 1:
raise ValueError(
"Modular platform v1 supports exactly one "
"model axis"
)

axis = spec.model_axes[0]

if axis.name != "rho":
raise ValueError(...)

if axis.resolver_id != "engine_v1.twindow_scale":
raise ValueError(...)

This can live in the S01 study builder or an S01 validator rather than the generic dataclass if later platform-v1 studies will support other axes.

Do not leave crossed-factor support apparently enabled when the resolver semantics are not implemented.
Expected-count derivation

Counts are currently trusted from:

Python

spec.artifact_policy.expected_condition_count
spec.artifact_policy.expected_scalar_row_count

Derive and compare:

Python

derived_conditions = (
spec.seed_design.count
* math.prod(
len(axis.levels)
for axis in spec.model_axes
)

)

derived_rows = (
derived_conditions
* len(spec.branches)
)

if (
spec.artifact_policy.expected_condition_count
!= derived_conditions
):
raise ValueError(...)

if (
spec.artifact_policy.expected_scalar_row_count
!= derived_rows
):
raise ValueError(...)

Longer term, move expected design counts out of ArtifactPolicySpec.
Branch validation additions

Require:

Python

if not spec.branches:
raise ValueError(
"At least one executed branch is required"
)

Validate every branch field:

text

name
engine_branch
role

intervention_ids
forcing_policy_id
requires_rng

Canonicalize intervention_ids to a tuple and require requires_rng to be a Boolean.

branches.py should perform the engine-v1-specific semantic validation.
Validator specifications are not yet executed

ExperimentSpec records:

Python

validators

but the shown runner does not iterate over or build them. If validation.py does not provide a registry used by the runner, this becomes the same problem as multiple collectors: the manifest claims validators that were never executed.

Either:

require exactly the fixed validator set and execute each validator, or
remove validators from v1 specs until the runner supports them.

The final manifest should distinguish:

text

validators_declared
validators_executed
validator_results

Canonical dataclass identity

_canonical_value() serializes dataclasses by fields only. It does not include the dataclass type.

Two different dataclass classes with identical fields can canonicalize identically.

This is not likely for the fixed S01 structure, but a modular platform should include a stable logical type tag:

Python

```
return {
"__type__": (
f"{type(value).__module__}."
f"{type(value).__qualname__}"
),
**{
field.name: _canonical_value(
getattr(value, field.name)
)
for field in dataclasses.fields(value)
},
}
```

Potential concern: module names become identity material. That is acceptable if they are stable logical plugin names. Alternatively, each spec class can define an explicit schema/type ID.

Explicit IDs are more stable:

text

```
aief-experiment-spec-v1
aief-model-factor-spec-v1
aief-engine-v1-model-config
```

Scientific identity assessment

The payload correctly excludes:

text

```
output paths
```

timestamps
host
execution mode

and includes:

text

canonical spec
source hashes
paired seeds
factor levels
protocol
RNG namespace
executed branches
collector schemas

That satisfies the stated design.

One terminology nuance:

Because source hashes and artifact policy are included, this is not only a scientific-design identity. It is a scientific-design plus implementation identity.

That is probably desirable here. Document it explicitly.
Required tests
Key-collision test

Python

with pytest.raises(CanonicalizationError):
FrozenDict({1: "a", "1": "b"})

Deep immutability

Python

branches = [branch_spec]

spec = ExperimentSpec(
branches=branches,
...
)

before = spec_sha256(spec)
branches.append(other_branch)

assert spec_sha256(spec) == before
assert len(spec.branches) == 1

Metadata mutation

Python

metadata = {"T_window": 17.5}
level = FactorLevel(..., metadata=metadata)

before = hash(level.metadata)
metadata["T_window"] = 99

assert level.metadata["T_window"] == 17.5
assert hash(level.metadata) == before

Also attempt direct mutation of level.metadata._dict and require failure.
Path traversal

Reject:

text

study_version = ../../x
scalar_filename_prefix = ../x
policy_id = /tmp/x

Canonical determinism

Specifications built from dictionaries with different insertion orders must produce identical canonical JSON and SHA-256.
Numeric canonicalization

Require equivalent numeric inputs to canonicalize identically, or explicitly reject noncanonical inputs:

text

0.5
np.float64(0.5)

Reject:

text

\"0.5\"
Decimal(\"0.5\") unless explicitly supported
NaN
Infinity

Count derivation

Test mismatched condition and scalar-row counts.
Baseline invariants

Reject:

text

required=False
exact_arrays=False
exact_scalars=False
certificate_reuse=True

Collector multiplicity

Reject zero or more than one collector in platform v1.
Provisional conclusion

spec.py has a strong conceptual core, but its current “frozen” guarantee is shallower than its name suggests because nested lists and the internal FrozenDict._dict can still mutate.

Before using spec_sha256 or scientific_run_sha256 as certification identities, fix:

string-key collision handling;
internal FrozenDict mutability;
tuple canonicalization of every sequence field;
strict metadata type acceptance;
safe path components;
seed domain validation;
exact-one-collector enforcement;
mandatory baseline semantics;
derived expected-count validation;
validator execution/claim consistency.

The scientific identity payload itself is appropriately path-free and execution-mode-free. Once deep immutability and strict validation are fixed, it will be suitable as the foundation for modular-run certification.

validation.py

text

"""Explicit validation gates and typed failure classes."""

from __future__ import annotations

import math
from typing import Any

import numpy as np

```
class AIEFModularError(Exception):
"""Base class for expected platform failures."""


class SpecValidationError(AIEFModularError):
pass


class BaselineGateError(AIEFModularError):
pass


class ConditionSchemaError(AIEFModularError):
pass


class ArtifactIntegrityError(AIEFModularError):
pass


class UnsupportedPolicyError(AIEFModularError):
pass


class ExecutionLockError(AIEFModularError):
pass


class CatalogError(AIEFModularError):
pass


def require_exact_branch_set(observed: dict[str, Any], expected: tuple[str, ...]) -> None:
if tuple(observed) != expected:
```

```
raise ConditionSchemaError(
f"Branch set/order mismatch: observed={tuple(observed)!r}, expected={expected!r}"
)



def validate_prefix_outputs(prefix: dict[str, Any], model: Any) -> None:
required = (
"B_at_split",
"R_physical_past",
"theta_at_split",
"coherence_acc_at_split",
"history_at_split",
"selected_at_split",
)
missing = set(required) - set(prefix)
if missing:
raise ConditionSchemaError(f"Prefix missing keys: {sorted(missing)}")


n = int(model.N)
delay_steps = int(model.delay_steps)
expected = {
"B_at_split": ((n, n), np.dtype("float64")),
"R_physical_past": ((n, n), np.dtype("float64")),
"theta_at_split": ((n,), np.dtype("float64")),
"coherence_acc_at_split": ((n, n), np.dtype("complex128")),
"history_at_split": ((delay_steps + 1, n), np.dtype("float64")),
"selected_at_split": ((n, n), np.dtype("uint8")),
}
for key, (shape, dtype) in expected.items():
value = np.asarray(prefix[key])
if value.shape != shape:
raise ConditionSchemaError(f"Prefix {key} shape {value.shape} != {shape}")
if value.dtype != dtype:
raise ConditionSchemaError(f"Prefix {key} dtype {value.dtype} != {dtype}")
if not np.all(np.isfinite(value)):
raise ConditionSchemaError(f"Prefix {key} contains non-finite values")


selected = np.asarray(prefix["selected_at_split"])
if not np.all((selected == 0) | (selected == 1)):
raise ConditionSchemaError("selected_at_split is not binary")
```

```
def _matrix_keys(engine: Any) -> tuple[str, ...]:
try:
keys = tuple(engine.REQUIRED_BRANCH_MATRIX_KEYS)
except Exception as exc:
raise ConditionSchemaError("Engine does not expose REQUIRED_BRANCH_MATRIX_KEYS")
from exc
return tuple(
key
for key in keys
if key not in {"sample_times", "density_trajectory", "similarity_trajectory"}
)



def validate_twindow_branch_output(
*,
engine: Any,
branch: str,
result: dict[str, Any],
model: Any,
protocol: Any,
rho: float,
required_scalar_keys: tuple[str, ...],
primary_outcomes: tuple[str, ...],
secondary_outcomes: tuple[str, ...],
) -> None:
"""Validate the complete legacy T_window condition branch contract."""
required_matrices = tuple(engine.REQUIRED_BRANCH_MATRIX_KEYS)
missing_scalar = set(required_scalar_keys) - set(result)
missing_matrix = set(required_matrices) - set(result)
if missing_scalar or missing_matrix:
raise ConditionSchemaError(
f"Branch {branch!r} missing scalars={sorted(missing_scalar)}, "
f"matrices={sorted(missing_matrix)}"
)

n = int(model.N)
for key in _matrix_keys(engine):
array = np.asarray(result[key])
if array.shape != (n, n):
raise ConditionSchemaError(f"{branch}.{key} shape {array.shape} != {(n, n)}")
```

```
if array.dtype != np.dtype("float64"):
raise ConditionSchemaError(f"{branch}.{key} dtype {array.dtype} != float64")
if not np.all(np.isfinite(array)):
raise ConditionSchemaError(f"{branch}.{key} contains non-finite values")


future_steps = int(protocol.future_steps(model))
stride = int(protocol.diagnostic_stride)
expected_times = (
np.arange(0, future_steps, stride, dtype=float) + 1.0
) * float(model.dt)
expected_length = len(expected_times)


times = np.asarray(result["sample_times"])
density = np.asarray(result["density_trajectory"])
similarity = np.asarray(result["similarity_trajectory"])
for key, array in (
("sample_times", times),
("density_trajectory", density),
("similarity_trajectory", similarity),
):
if array.ndim != 1 or len(array) != expected_length:
raise ConditionSchemaError(
f"{branch}.{key} shape {array.shape} incompatible with {expected_length} samples"
)
if array.dtype != np.dtype("float64"):
raise ConditionSchemaError(f"{branch}.{key} dtype {array.dtype} != float64")


if not np.array_equal(times, expected_times):
raise ConditionSchemaError(f"{branch}.sample_times has an unexpected schedule")
if not np.all(np.isfinite(density)) or np.any((density < 0.0) | (density > 1.0)):
raise ConditionSchemaError(f"{branch}.density_trajectory is outside [0,1] or non-finite")
if np.any(np.isinf(similarity)):
raise ConditionSchemaError(f"{branch}.similarity_trajectory contains Infinity")
finite_similarity = similarity[np.isfinite(similarity)]
if np.any((finite_similarity < -1.0 - 1e-12) | (finite_similarity > 1.0 + 1e-12)):
raise ConditionSchemaError(f"{branch}.similarity_trajectory is outside [-1,1]")


expected_twindow = float(model.T_window)
if "rho" not in result or float(result["rho"]) != float(rho):
raise ConditionSchemaError(f"{branch}.rho does not exactly match {rho!r}")
if "T_window" not in result or float(result["T_window"]) != expected_twindow:
```

```
raise ConditionSchemaError(
f"{branch}.T_window does not exactly match {expected_twindow!r}"
)


window_steps = int(round(expected_twindow / float(model.dt)))
recovery_end = min(5 * window_steps, future_steps)
fixed_immediate = min(int(round(17.5 / float(model.dt))), future_steps)
fixed_recovery = min(int(round(87.5 / float(model.dt))), future_steps)
metadata = {
"window_steps": window_steps,
"recovery_end_ema": recovery_end,
"fixed_immediate_end": fixed_immediate,
"fixed_recovery_end": fixed_recovery,
}
for key, expected in metadata.items():
observed = result.get(key)
if isinstance(observed, bool) or not isinstance(observed, (int, np.integer)):
raise ConditionSchemaError(f"{branch}.{key} is not an integer: {observed!r}")
if int(observed) != expected:
raise ConditionSchemaError(f"{branch}.{key}={observed} != {expected}")


for key in primary_outcomes + secondary_outcomes:
value = result.get(key)
if isinstance(value, bool) or not isinstance(value, (int, float, np.integer, np.floating)):
raise ConditionSchemaError(f"{branch}.{key} is not numeric: {value!r}")
if not np.isfinite(value):
raise ConditionSchemaError(f"{branch}.{key} is non-finite: {value!r}")



def exact_equal_with_nan(left: Any, right: Any) -> bool:
"""Exact scalar/array equality with matching NaN positions and dtypes."""
a = np.asarray(left)
b = np.asarray(right)
if a.shape != b.shape or a.dtype != b.dtype:
return False
if np.issubdtype(a.dtype, np.complexfloating):
# Compare real and imaginary components independently so a NaN in one
# component does not hide a mismatch in the other.
return exact_equal_with_nan(a.real, b.real) and exact_equal_with_nan(a.imag, b.imag)
if np.issubdtype(a.dtype, np.floating):
nan_a = np.isnan(a)
```

```
nan_b = np.isnan(b)
if not np.array_equal(nan_a, nan_b):
return False
return np.array_equal(a[~nan_a], b[~nan_b])
return np.array_equal(a, b)
```

```
def exact_scalar_equal_with_nan(left: Any, right: Any) -> bool:
try:
a = float(left)
b = float(right)
except (TypeError, ValueError):
return False
if math.isnan(a) and math.isnan(b):
return True
return a == b
```

validation.py is one of the stronger modules shown so far. It uses typed failures, validates exact array dtypes and schedules, permits the documented similarity NaNs, and fixes the partially-NaN complex comparison issue identified in the Engine v1 review.
Provisional verdict

Prefix validation: strong
Matrix/trajectory validation: strong
Condition metadata validation: correct
Primary/secondary outcome validation: correct
NaN-aware exact array equality: improved over Engine v1
All scalar-field schema validation: incomplete
Declared validator execution: unresolved
No immediate numerical blocker for valid S01 outputs

Positive assessment
Typed failures

The exception hierarchy is appropriate:

text

SpecValidationError

BaselineGateError
ConditionSchemaError
ArtifactIntegrityError
UnsupportedPolicyError
ExecutionLockError
CatalogError

This is a major improvement over message-string failure classification.
Exact prefix contract

The validator requires:

all six expected keys;
exact shapes;
exact dtypes;
finite values;
binary selected matrix.

The expected dtypes match Engine v1:

text

float64
complex128
uint8

Exact trajectory contract

The validator checks:

one-dimensional arrays;
exact expected length;
exact float64 dtype;
exact sample schedule;
density finite and in [0,1][0,1];
similarity infinity rejected;
similarity NaN permitted;
finite similarity in [−1,1][−1,1].

That is correctly aligned with Analysis v3's documented zero-vector cosine convention.
Exact condition metadata

Correctly validates:

text

rho
T_window
window_steps
recovery_end_ema
fixed_immediate_end
fixed_recovery_end

using independent expected values.
Improved complex equality

This is better than the original engine helper:

Python

return (
exact_equal_with_nan(a.real, b.real)
and exact_equal_with_nan(a.imag, b.imag)
)

A NaN in one complex component no longer hides a mismatch in the other.
P0/P1: validate every required scalar as a scalar

Current code validates presence of all required scalar keys:

Python

missing_scalar = (
set(required_scalar_keys) - set(result)
)

but validates type/finiteness only for primary and secondary outcomes.

A malformed result could contain:

Python

```
result["D_B_final"] = np.array([0.3])
```

and pass condition validation until CSV writing.
Required addition

Python

```
for key in required_scalar_keys:
value = result[key]
array = np.asarray(value)

if array.ndim != 0:
raise ConditionSchemaError(
f"{branch}.{key} must be scalar, "
f"got shape {array.shape}"
)

if isinstance(value, (bool, np.bool_)):
raise ConditionSchemaError(
f"{branch}.{key} must not be Boolean"
)

if not np.issubdtype(
array.dtype,
np.number,
):
raise ConditionSchemaError(
f"{branch}.{key} is not numeric: "
f"dtype={array.dtype}"
)
```

scalar = array.item()

Legacy condition-local fields may legitimately be NaN,
but Infinity is never valid.
if isinstance(scalar, complex):
if np.isinf(scalar.real) or np.isinf(
scalar.imag
):
raise ConditionSchemaError(
f"{branch}.{key} contains Infinity"
)
elif np.isinf(float(scalar)):
raise ConditionSchemaError(
f"{branch}.{key} contains Infinity"
)

Then retain the stronger finite requirement for primary and secondary outcomes.

Legacy fields that can validly be NaN include degenerate correlations and some projection diagnostics.
Do not impose blanket finiteness on every legacy scalar.
P1: reject extra prefix keys for the compatibility collector

The function checks missing keys but accepts extras.

For a strict legacy prefix contract:

Python

observed = set(prefix)
expected_keys = set(required)

missing = expected_keys - observed
extra = observed - expected_keys

if missing or extra:
raise ConditionSchemaError(
f"Prefix schema mismatch: "
f"missing={sorted(missing)}, "

```
f"extra={sorted(extra)}"
)
```

If future collectors need additional prefix fields, version their core schema rather than silently adding fields to the legacy contract.
P1: explicitly reject empty/truncated horizons for S01

The current metadata calculation matches Engine v1, including clamping:

Python

```
recovery_end = min(
5 * window_steps,
future_steps,
)
```

But it does not reject an empty normalized late horizon.

For the narrowly certified S01 validator, require:

Python

```
if not (
0 < window_steps
< recovery_end
< future_steps
):
raise ConditionSchemaError(
f"{branch}: normalized horizon layout is "
f"invalid: window={window_steps}, "
f"recovery_end={recovery_end}, "
f"future={future_steps}"
)

if not (
0 < fixed_immediate
< fixed_recovery
< future_steps
```

):
raise ConditionSchemaError(
f"{branch}: fixed horizon layout is invalid"
)

Also require the S01 future to be exactly 320 time units, preferably in spec validation.

This prevents Engine v1’s zero-matrix empty-horizon behavior from entering modular S01.
P1: scalar exact equality is overly coercive

Current:

Python

a = float(left)
b = float(right)

can accept:

numeric strings;
one-element arrays;
booleans;
large integers with precision loss.

Use a strict scalar normalizer:

Python

def _numeric_scalar(value: Any) -> float | None:
if isinstance(value, (bool, np.bool_)):
return None

array = np.asarray(value)

if array.ndim != 0:
return None

if not np.issubdtype(
array.dtype,
np.number,
):
return None

try:
return float(array.item())
except (TypeError, ValueError, OverflowError):
return None

Then:

Python

def exact_scalar_equal_with_nan(
left: Any,
right: Any,
) -> bool:
a = _numeric_scalar(left)
b = _numeric_scalar(right)

if a is None or b is None:
return False

if math.isnan(a) and math.isnan(b):
return True

return a == b

CSV parsing should convert authority values to numeric types before this comparison.
P1: validate exact engine matrix schema once

_matrix_keys() trusts:

Python

engine.REQUIRED_BRANCH_MATRIX_KEYS

Add checks for:

duplicate keys;
required trajectory names appearing exactly once;
no unexpected nonstring key;
compatibility with the collector’s pinned engine schema.

The engine source hash helps, but a clear schema error is preferable.
Optional matrix semantic validation

All 12×12 matrices are checked for shape, dtype, and finiteness. For stronger condition validation, add schema-specific checks:
Symmetry and diagonal

All returned BB, PP, and RR matrices should be symmetric with zero diagonal.
Bounds

BB: nonnegative;
PP: in [0,1][0,1];
RR: in [0,1][0,1].

These checks are inexpensive at N=12N=12.

Do not apply identical bounds blindly by name; define explicit matrix classes or key sets.
Cross-module P0: validator declarations are not executed

ExperimentSpec contains:

Python

validators: tuple[ValidatorSpec, ...]

but the shown runner does not build or iterate over validators. Instead, validation happens because the collector directly calls:

Python

validate_prefix_outputs(...)
validate_twindow_branch_output(...)

Therefore, any ValidatorSpec listed in the manifest may be declarative only and never executed.

This must be resolved before certification.
Option A — remove validator specs in v1

Require:

Python

spec.validators == ()

and document that collector v1 owns condition schema validation.
Option B — implement validator registry

Python

VALIDATORS = {
"aief.twindow.condition-v1":
validate_twindow_branch_output,
}

The runner should:

build every declared validator;
execute it at the proper gate;
record validator ID/version/result in the manifest.

Do not list validators in a scientific specification unless they are actually invoked.

For modular v1, Option A is simpler because the compatibility collector already owns complete condition validation.
Typed errors from spec.py

spec.py currently raises ordinary ValueError for many specification failures despite this module defining SpecValidationError.

Because validation.py does not import spec.py, spec.py can safely import:

Python

from .validation import SpecValidationError

and use it consistently.

If circular imports arise through future changes, move the exception classes into a dependency-neutral errors.py.
Required tests
Prefix tests

missing key;
extra key;
wrong shape;
wrong dtype;
nonfinite value;
nonbinary selection matrix.

Matrix tests

missing matrix;
wrong shape;
wrong dtype;
Infinity;
NaN;
optional semantic symmetry/bounds tests.

Trajectory tests

wrong length;
wrong dtype;
wrong schedule;
density NaN;

density outside [0,1][0,1];
similarity Infinity;
similarity finite value outside [−1,1][−1,1];
permitted similarity NaN.

Metadata tests

wrong rho;
wrong TwindowTwindow;
wrong window steps;
wrong recovery end;
empty normalized late horizon;
truncated fixed horizon.

Exact equality tests

text

matching finite arrays
different dtype
different shape
matching NaN locations
different NaN locations
complex real-part NaN only
complex imaginary-part NaN only
finite mismatch hidden beside complex NaN
scalar NaN equality
one-element array rejection
string rejection
Boolean rejection

Declared validator test

Construct a spec with a fake validator ID and require either:

spec rejection; or
runner UnsupportedPolicyError.

It must not appear in a manifest as though it ran.
Provisional conclusion

validation.py is suitable for S01’s current numerical outputs after adding strict scalar-shape validation and resolving the declared-validator issue.

The most important remaining cross-file question is now oracle.py: whether frozen authority hashes and full logical oracle equivalence are independently authenticated and recorded.

registry.py

text

"""Explicit, deterministic registry for trusted built-in AIEF studies."""

from __future__ import annotations

from dataclasses import dataclass
from typing import Callable

from .spec import ExperimentSpec

StudyBuilder = Callable[[], ExperimentSpec]

@dataclass(frozen=True)
class StudyRegistration:
study_id: str
slug: str
version: str
title: str
source_module: str
builder: StudyBuilder

def build(self) -> ExperimentSpec:
spec = self.builder()

```
expected = (self.study_id, self.slug, self.version, self.title)
observed = (spec.study_id, spec.study_slug, spec.study_version, spec.title)
if observed != expected:
raise RuntimeError(
"Registered study metadata differs from built specification: "
f"registered={expected!r}, built={observed!r}"
)
return spec



class StudyRegistry:
"""Registry that rejects duplicate IDs and slugs."""


def __init__(self) -> None:
self._by_id: dict[str, StudyRegistration] = {}
self._slug_to_id: dict[str, str] = {}


def register(
self,
study_id: str,
builder: StudyBuilder,
*,
slug: str,
version: str,
title: str,
source_module: str | None = None,
) -> StudyRegistration:
if study_id in self._by_id:
raise RuntimeError(f"Duplicate study ID: {study_id}")
if slug in self._slug_to_id:
raise RuntimeError(
f"Duplicate study slug {slug!r}: already registered by {self._slug_to_id[slug]}"
)
registration = StudyRegistration(
study_id=study_id,
slug=slug,
version=version,
title=title,
source_module=source_module or builder.__module__,
builder=builder,
)
```

```
self._by_id[study_id] = registration
self._slug_to_id[slug] = study_id
return registration


def get(self, study_id: str) -> StudyRegistration:
try:
return self._by_id[study_id]
except KeyError as exc:
available = ", ".join(self.ids()) or "<none>"
raise KeyError(f"Unknown study {study_id!r}; available: {available}") from exc


def ids(self) -> tuple[str, ...]:
return tuple(sorted(self._by_id))


def entries(self) -> tuple[StudyRegistration, ...]:
return tuple(self._by_id[key] for key in self.ids())


def clear_for_testing(self) -> None:
self._by_id.clear()
self._slug_to_id.clear()



REGISTRY = StudyRegistry()



def register_study(
study_id: str,
builder: StudyBuilder,
*,
slug: str,
version: str,
title: str,
source_module: str | None = None,
) -> StudyRegistration:
return REGISTRY.register(
study_id,
builder,
slug=slug,
version=version,
```

title=title,
source_module=source_module,
)

def load_builtin_studies() -> None:
"""Import only trusted package-owned study modules.

No path-based dynamic loading is supported.
"""
from .studies import aief_1_4_s01_twindow as _twindow # noqa: F401

registry.py is simple and appropriately conservative. It uses explicit package-owned imports, deterministic ordering, and rejects duplicate IDs/slugs. That is preferable to a dynamic plugin loader for certification-oriented work.
Provisional verdict

Explicit trusted registration: correct
Deterministic listing: correct
Duplicate protection: correct
No untrusted path loading: correct
Source provenance binding: needs hardening
Test reset/idempotence: currently defective
No S01 numerical blocker

Positive assessment
Trusted built-ins only

Good:

Python

from .studies import aief_1_4_s01_twindow

There is no:

arbitrary filesystem import;
entry-point discovery;

configuration-supplied module path;
eval;
untrusted dynamic plugin loading.

Duplicate rejection

Correctly rejects:

text

duplicate study ID
duplicate study slug

before mutating both indexes.
Deterministic listing

Good:

Python

tuple(sorted(self._by_id))

and entries follow sorted IDs.
Built-spec metadata check

StudyRegistration.build() ensures the registration label agrees with the actual specification:

text

study ID
slug
version
title

This prevents a registry entry from describing one study while building another.
P0 provenance issue: source_module can differ from the builder’s module

Registration permits:

Python

source_module="some.other.module"

even if:

Python

builder.__module__

is the actual study implementation module.

The runner later hashes:

Python

importlib.import_module(
registration.source_module
)

Therefore, a registration can cause the manifest to hash the wrong study source while executing the real builder from another module.
Required correction

For modular v1, require equality:

Python

builder_module = builder.__module__

if source_module is None:
resolved_source_module = builder_module
elif source_module != builder_module:
raise RuntimeError(

f"Registered source_module "
f"{source_module!r} differs from "
f"builder module {builder_module!r}"
)
else:
resolved_source_module = source_module

Then store:

Python

source_module=resolved_source_module

Alternatively, the runner must hash both modules, but requiring equality is simpler and safer.

Also restrict trusted study modules:

Python

if not resolved_source_module.startswith(
"aief_modular.studies."
):
raise RuntimeError(
"Built-in study builder must come from "
"aief_modular.studies"
)

This is important because the scientific run ID includes the recorded study source hash.
P1 bug: clear_for_testing() breaks load_builtin_studies()

Initial behavior:

Python

load_builtin_studies()

imports the study module and triggers registration.

But after:

Python

REGISTRY.clear_for_testing()

calling:

Python

load_builtin_studies()

does not restore the study because Python returns the already cached module without rerunning its import-time registration.

Thus this test sequence fails:

Python

load_builtin_studies()
assert REGISTRY.ids()

REGISTRY.clear_for_testing()
load_builtin_studies()
assert REGISTRY.ids() # likely empty

Preferred correction: explicit registration function

In the study module:

Python

def register_builtin_study() -> None:
if "AIEF_1_4_S01" in REGISTRY:
return

register_study(
"AIEF_1_4_S01",
build_spec,
slug="twindow_sensitivity",
version="v1",
title="...",
)

Then:

Python

def load_builtin_studies() -> None:
from .studies.aief_1_4_s01_twindow import (
register_builtin_study,
)

register_builtin_study()

Add:

Python

def __contains__(
self,
study_id: str,
) -> bool:
return study_id in self._by_id

This makes loading explicitly idempotent and allows tests to clear and reload safely.

Do not use importlib.reload() as the normal registry mechanism.
P1: validate registration metadata before insertion

Currently malformed registrations can enter the registry and fail only when built.

Validate:

text

study_id
slug
version
title
source_module
builder callable

at registration time.

Use the same safe component rules as spec.py.

Python

if not callable(builder):
raise TypeError("Study builder must be callable")

validate_path_component(
study_id,
"study_id",
)
validate_path_component(
slug,
"slug",
)
validate_path_component(
version,
"version",
)

if not isinstance(title, str) or not title.strip():
raise ValueError(
"Study title must be a nonempty string"
)

For source_module, require a dotted Python module path rather than a filesystem path.
P1: validate the builder at registration or built-in load

The registry currently stores the builder without calling it. A malformed builder remains visible through:

Python

list-studies

until build() is called later.

A safe built-in loading process should validate each registered study:

Python

registration = register_study(...)
registration.build()

or after all built-ins load:

Python

for registration in REGISTRY.entries():
registration.build()

This causes import/dry-run to fail early if the built-in specification is invalid.

Do not cache the returned mutable object unless deep immutability is guaranteed by the corrected spec.py.
P1: deterministic-builder requirement

A study builder must not depend on:

current time;
random numbers;

output path;
host;
environment variables not represented in the spec;
mutable global state.

Add a test:

Python

```
first = registration.build()
second = registration.build()

assert canonical_json(first) == canonical_json(second)
assert spec_sha256(first) == spec_sha256(second)
```

This need not necessarily run twice in every production invocation, but it must be part of study-registration tests.

A stronger build() implementation could optionally perform the comparison in debug/certification mode.
Registry version semantics

The current registry rejects duplicate study_id even when versions differ:

text

```
AIEF_1_4_S01 v1
AIEF_1_4_S01 v2
```

cannot coexist.

This follows the original “unique study ID” requirement, but decide the intended version model explicitly.
Option A — IDs include release identity

text

AIEF_1_4_S01
AIEF_1_4_S01_V2

Current registry is sufficient.
Option B — one study ID has multiple versions

Use:

Python

(study_id, version)

as the registry key, and allow:

Bash

describe AIEF_1_4_S01 --version v2

For modular platform v1, Option A is simpler. Document it so future developers do not assume versions can coexist under one ID.
Typed errors

The registry currently raises:

Python

RuntimeError
KeyError

Consider dedicated types:

Python

RegistryError
UnknownStudyError
DuplicateStudyError

These could derive from AIEFModularError.

This is operational clarity rather than a correctness blocker.
Encapsulation

Rename:

Python

clear_for_testing()

to:

Python

_clear_for_testing()

or move it to test support. It should not appear as an ordinary production operation.

The global registry is intentionally mutable during built-in loading, but should effectively become read-only after loading. A seal() operation could reject later registration:

Python

REGISTRY.seal()

The CLI can:

load built-ins;
validate entries;
seal registry;
execute commands.

This prevents accidental late registration from changing available study definitions during a process.
Required tests

Duplicate protection

Python

register study A
register same ID -> failure
register same slug -> failure

Idempotent loading

Python

load_builtin_studies()
first = REGISTRY.ids()

load_builtin_studies()
second = REGISTRY.ids()

assert first == second

Clear and reload

Python

REGISTRY._clear_for_testing()
load_builtin_studies()
assert REGISTRY.ids() == (
"AIEF_1_4_S01",
)

Metadata mismatch

Builder returns a spec with wrong:

text

ID
slug
version
title

Each must fail.
Source-module mismatch

Register builder from module A while claiming module B. Must fail.
Untrusted source module

Reject a builder outside:

text

aief_modular.studies

for built-in registration.
Deterministic build

Two builds produce identical canonical data and hash.
Sorted listing

Register IDs out of order and require sorted output.
Seal behavior

After registry seal, further registration fails.
Provisional conclusion

registry.py is appropriate for the narrowly trusted S01 platform after:

binding source_module to builder.__module__;
making built-in loading explicitly idempotent;
fixing clear/reload behavior;
validating registrations before insertion.

It introduces no numerical or experimental-design drift. The remaining decisive module is oracle.py, which must authenticate the frozen authority and compare modular outputs logically against the successful S01 oracle.

oracle.py

text

"""Logical-content comparison against the successful T_window production oracle."""

from __future__ import annotations

import csv
import math
from dataclasses import dataclass
from pathlib import Path
from typing import Any

import numpy as np

from .validation import ArtifactIntegrityError, exact_equal_with_nan, exact_scalar_equal_with_nan

@dataclass(frozen=True)
class OracleComparison:
status: str
scalar_rows: int
npz_entries: int
condition_rows: int
errors: tuple[str, ...]

def _read_csv(path: Path) -> tuple[tuple[str, ...], list[dict[str, str]]]:
with open(path, "r", encoding="utf-8", newline="") as handle:
reader = csv.DictReader(handle)
if reader.fieldnames is None:
raise ArtifactIntegrityError(f"CSV has no header: {path}")
return tuple(reader.fieldnames), list(reader)

```
def _is_numeric_text(value: str) -> bool:
try:
float(value)
return True
except (TypeError, ValueError):
return False



def _compare_scalar_csv(candidate: Path, oracle: Path, errors: list[str]) -> int:
candidate_fields, candidate_rows = _read_csv(candidate)
oracle_fields, oracle_rows = _read_csv(oracle)
if candidate_fields != oracle_fields:
errors.append(
f"Scalar CSV columns/order differ: candidate={candidate_fields!r}, oracle={oracle_fields!r}"
)
return len(candidate_rows)
if len(candidate_rows) != len(oracle_rows):
errors.append(
f"Scalar CSV row count differs: candidate={len(candidate_rows)}, oracle={len(oracle_rows)}"
)
return len(candidate_rows)

text_fields = {"seed_index", "seed", "branch"}
for index, (left, right) in enumerate(zip(candidate_rows, oracle_rows, strict=True)):
for field in candidate_fields:
a = left[field]
b = right[field]
if field in text_fields:
if a != b:
errors.append(f"Scalar row {index} field {field}: {a!r} != {b!r}")
elif not exact_scalar_equal_with_nan(a, b):
errors.append(f"Scalar row {index} field {field}: {a!r} != {b!r}")
if len(errors) >= 100:
return len(candidate_rows)
return len(candidate_rows)
```

```
def _load_npz(path: Path) -> dict[str, np.ndarray]:
try:
with np.load(path, allow_pickle=False) as archive:
return {key: archive[key].copy() for key in archive.files}
except Exception as exc:
raise ArtifactIntegrityError(f"Cannot load NPZ {path}: {exc}") from exc



def _compare_npz(candidate: Path, oracle: Path, errors: list[str]) -> int:
left = _load_npz(candidate)
right = _load_npz(oracle)
if set(left) != set(right):
missing = sorted(set(right) - set(left))
extra = sorted(set(left) - set(right))
errors.append(f"NPZ key mismatch: missing={missing[:20]}, extra={extra[:20]}")
for key in sorted(set(left) & set(right)):
if not exact_equal_with_nan(left[key], right[key]):
errors.append(
f"NPZ value mismatch for {key}: "
f"candidate(shape={left[key].shape},dtype={left[key].dtype}), "
f"oracle(shape={right[key].shape},dtype={right[key].dtype})"
)
if len(errors) >= 100:
break
return len(left)



def _compare_condition_coverage(candidate: Path, oracle: Path, errors: list[str]) -> int:
left_fields, left_rows = _read_csv(candidate)
right_fields, right_rows = _read_csv(oracle)
required = {"seed", "rho", "status"}
if not required.issubset(left_fields) or not required.issubset(right_fields):
errors.append("Condition status files lack seed/rho/status columns")
return len(left_rows)
left_keys = {(row["seed"], float(row["rho"]), row["status"]) for row in left_rows}
right_keys = {(row["seed"], float(row["rho"]), row["status"]) for row in right_rows}
if left_keys != right_keys:
errors.append(
f"Condition status coverage differs: missing={sorted(right_keys-left_keys)[:10]}, "
f"extra={sorted(left_keys-right_keys)[:10]}"
)
```

```
return len(left_rows)



def _single_file(directory: Path, pattern: str) -> Path:
files = sorted(directory.glob(pattern))
if len(files) != 1:
raise ArtifactIntegrityError(
f"Expected exactly one {pattern!r} in {directory}, found {[item.name for item in files]}"
)
return files[0]



def compare_twindow_run(candidate_dir: str | Path, oracle_dir: str | Path) -> OracleComparison:
"""Compare decompressed logical contents, never compressed NPZ bytes."""
candidate = Path(candidate_dir)
oracle = Path(oracle_dir)
errors: list[str] = []
candidate_csv = _single_file(candidate, "AIEF_1_4_Twindow_sensitivity_*_scalar_summary.csv")
oracle_csv = _single_file(oracle, "AIEF_1_4_Twindow_sensitivity_*_scalar_summary.csv")
candidate_npz = _single_file(candidate, "AIEF_1_4_Twindow_sensitivity_*_branch_diagnostics.npz")
oracle_npz = _single_file(oracle, "AIEF_1_4_Twindow_sensitivity_*_branch_diagnostics.npz")
scalar_rows = _compare_scalar_csv(candidate_csv, oracle_csv, errors)
npz_entries = _compare_npz(candidate_npz, oracle_npz, errors)
condition_rows = _compare_condition_coverage(
candidate / "twindow_condition_status.csv",
oracle / "twindow_condition_status.csv",
errors,
)
return OracleComparison(
status="passed" if not errors else "failed",
scalar_rows=scalar_rows,
npz_entries=npz_entries,
condition_rows=condition_rows,
errors=tuple(errors),
)
```

oracle.py correctly focuses on logical content equivalence rather than compressed NPZ byte equality. That is the right comparison for determining whether the modular runner reproduces the successful monolithic S01 workflow.

Provisional verdict

Scalar logical comparison: strong
NPZ logical comparison: strong
Condition coverage comparison: incomplete
Oracle authentication: absent
Durable certification artifact: absent in this module
Runner integration: not present in the shown runner.py
Suitable as a comparison helper, not yet as a certification gate

Positive assessment
Logical rather than compressed-byte comparison

Correct:

Python

with np.load(...):
...
exact_equal_with_nan(...)

This allows equivalent NPZ archives with different compression/container bytes to pass.
Strict scalar-table structure

The comparator checks:

exact column names;
exact column order;
exact row count;
exact row order;
exact seed index;
exact seed text;
exact branch;
NaN-aware numeric equality.

That is appropriate for the legacy collector’s compatibility contract.
Exact NPZ schema

It compares:

text

missing keys
extra keys
shape
dtype
NaN-aware values

using the improved complex-aware equality helper.
Error accumulation

Collecting up to 100 mismatches is more useful than failing on the first mismatch.
Oracle path discovery

Requiring exactly one matching scalar CSV and NPZ prevents accidental selection among multiple candidate outputs.
P0: this does not authenticate the oracle

compare_twindow_run() accepts any directory as oracle_dir.

It does not verify that the supplied directory is the independently trusted:

text

runs/b57562a8ee77

A modified oracle directory could be compared successfully against an equally modified candidate.
Required oracle lock

Create an independently stored lock, e.g.:

JSON

{
"oracle_id": "AIEF_1_4_S01_MONOLITHIC_b57562a8ee77",
"manifest_sha256": "...",

"scalar_csv_sha256": "...",
"branch_npz_sha256": "...",
"condition_status_sha256": "...",
"baseline_certificate_sha256": "...",
"engine_sha256": "...",
"driver_sha256": "...",
"seed_count": 40,
"condition_count": 200,
"scalar_row_count": 800
}

Before logical comparison:

verify the lock itself comes from the evidence lineage;
verify every oracle file against the lock;
load the oracle manifest;
verify its own recorded output hashes;
only then compare candidate logical contents.

Use separate concepts:

text

FrozenAuthorityLock
MonolithicOracleLock

The frozen authority certifies Engine v1 baseline behavior. The monolithic oracle certifies modular orchestration equivalence.
P0: the runner does not use the oracle comparator

The shown runner.py never imports or calls:

Python

compare_twindow_run(...)

Therefore, a modular production execution is currently marked:

Python

"freezeable": True

without demonstrating equivalence to runs/b57562a8ee77.

That can be acceptable if freezeable means only “internally complete execution,” but it must not be interpreted as “modular platform certified.”
Required certification command

Implement a separate command:

Bash

```
python AIEF_modular_driver_v1.py certify-oracle \
AIEF_1_4_S01 \
--candidate <modular-production-leaf> \
--oracle runs/b57562a8ee77 \
--oracle-lock authority/S01_b57562a8ee77.lock.json
```

It should:

authenticate the oracle;
validate the candidate manifest and hashes;
perform logical comparison;
write an atomic comparison report;
update or supplement platform certification state.

Do not mutate a completed execution manifest after freezing it. Prefer a separate certification artifact:

text

```
certifications/
oracle_equivalence_<execution_id>.json
```

A catalog can join that certification to the run.
P0/P1: condition-status comparison ignores duplicates

Current code converts rows to sets:

Python

```
left_keys = {
(seed, rho, status)
for row in left_rows
}
```

A candidate with duplicate rows can pass if its set matches the oracle.

Example:

text

```
candidate: 201 rows, including one duplicate
oracle: 200 rows
```

The sets can still be equal.
Required correction

Validate headers, row counts, duplicates, and ordered logical rows:

Python

```
required = ("seed", "rho", "status")

if len(left_rows) != len(right_rows):
errors.append(
f"Condition status row count differs: "
f"candidate={len(left_rows)}, "
f"oracle={len(right_rows)}"
)

def condition_keys(rows):
return [
```

```
(
row["seed"],
float(row["rho"]),
row["status"],
)
for row in rows
]

left_keys = condition_keys(left_rows)
right_keys = condition_keys(right_rows)

if len(left_keys) != len(set(left_keys)):
errors.append(
"Candidate condition status contains duplicates"
)

if len(right_keys) != len(set(right_keys)):
errors.append(
"Oracle condition status contains duplicates"
)

if left_keys != right_keys:
errors.append(
"Condition status row order/content differs"
)
```

If elapsed time is intentionally ignored, compare only the required logical columns.

Also require all statuses to equal "success".
P1: CSV numeric comparison must not depend on a permissive generic scalar helper

_read_csv() returns strings. Current comparison delegates to:

Python

exact_scalar_equal_with_nan(a, b)

The current helper accepts strings through float(), but the recommended hardened helper in validation.py should reject strings and one-element arrays.

Do not make those two requirements conflict.

Implement CSV parsing locally:

Python

```
def parse_numeric_csv(
value: str,
*,
field: str,
row: int,
) -> float:
text = value.strip()

if text.lower() in {
"nan",
"+nan",
"-nan",
}:
return float("nan")

if text == "":
raise ArtifactIntegrityError(
f"Blank numeric CSV value at "
f"row={row}, field={field}"
)

try:
result = float(text)
except ValueError as exc:
raise ArtifactIntegrityError(
f"Invalid numeric CSV value "
f"{value!r} at row={row}, field={field}"
) from exc

if math.isinf(result):
```

raise ArtifactIntegrityError(
f"Infinite numeric CSV value at "
f"row={row}, field={field}"
)

return result

Then compare parsed floats with an explicit NaN-aware equality function.

Remove unused:

Python

_is_numeric_text()

P1: duplicate NPZ archive entries

_load_npz() converts the archive directly to a dictionary. If the underlying ZIP contains duplicate names, dictionary construction can hide them.

Use the package artifact loader after it is hardened, or inspect the ZIP member list:

Python

import zipfile

with zipfile.ZipFile(path, "r") as archive:
names = archive.namelist()
array_names = [
name[:-4]
for name in names
if name.endswith(".npy")
]

if len(array_names) != len(set(array_names)):
raise ArtifactIntegrityError(
f"NPZ contains duplicate entries: {path}"

)

Then load with NumPy.

The candidate writer uses a mapping and should not produce duplicates, but the oracle is external input and should be authenticated.
P1: compare manifests and scientific design

Exact scalar/array output comparison is strong evidence, but the certification report should also compare key design metadata:

text

40 exact paired seeds
rho values and order
four executed branches and order
seven-branch RNG namespace
protocol times
diagnostic stride
engine hash
collector schema
condition count
scalar row count

Some of this is implied by exact output keys, but explicit comparison gives a clearer certificate.

Do not require runner source equality—the modular runner is intentionally different. Do require the same Engine v1 source hash.
P1: improve OracleComparison

Current result:

Python

@dataclass(frozen=True)
class OracleComparison:
status: str
scalar_rows: int

npz_entries: int
condition_rows: int
errors: tuple[str, ...]

Add:

text

candidate path
oracle ID
candidate manifest hash
oracle manifest hash
candidate scalar hash
oracle scalar hash
candidate NPZ hash
oracle NPZ hash
candidate status hash
oracle status hash
comparison source hash
comparison timestamp
matched scalar fields
matched NPZ entries
expected counts

Do not put timestamps into scientific run identity, but include them in certification execution provenance.

Use explicit counts:

text

scalar_rows_expected = 800
scalar_rows_matched
npz_entries_expected
npz_entries_matched
condition_rows_expected = 200
condition_rows_matched

A candidate entry count alone is ambiguous after mismatch.

P1: write a durable atomic certification report

compare_twindow_run() returns an in-memory result. The CLI should write:

text

oracle_equivalence_certificate.json

atomically, containing:

authenticated oracle lock;
candidate hashes;
oracle hashes;
logical comparison result;
error details;
comparator source hash;
exact expected and observed counts;
oracle_equivalent: true|false.

On failure, write the report and exit nonzero.
P1: validate candidate completeness before comparison

Before comparison, require candidate:

text

manifest status = completed
freezeable = true
baseline status = passed
200/200 conditions
800 scalar rows
output file hashes match manifest
source hashes unchanged

The oracle comparator should not treat raw file equality alone as sufficient if the candidate execution manifest says failed or incomplete.

Likewise authenticate and validate the monolithic oracle manifest.

Error cap behavior

Currently, after 100 scalar errors, NPZ and condition comparison still run. That can be useful but expensive.

Make policy explicit:

Python

MAX_ORACLE_ERRORS = 100

All comparison functions should respect a shared remaining-error budget. Alternatively, continue all categories but cap errors per category.

Record whether the report was truncated:

JSON

{
"errors_truncated": true,
"max_errors": 100
}

Required oracle tests
Exact copy

Copy/recompress the oracle NPZ without changing logical arrays. Comparison must pass despite different file SHA-256.
Scalar mismatch

Change one scalar by one ULP. Must fail.
Scalar NaN

matching nan passes;
nan versus finite fails;
blank numeric value fails clearly.

CSV structure

reordered column fails;
reordered row fails;
duplicate row fails;
missing row fails;
extra row fails.

NPZ structure

missing key fails;
extra key fails;
dtype change fails;
shape change fails;
finite value change fails;
NaN-position change fails;
partial complex NaN mismatch fails;
duplicate archive entry fails.

Condition status

duplicate condition fails;
failed status fails;
missing condition fails;
extra condition fails;
reordered condition behavior should follow the explicitly chosen policy.

Authentication

wrong oracle lock fails before logical comparison;
modified oracle manifest fails;
modified oracle scalar/NPZ/status file fails;
candidate output hash mismatch fails.

Certification output

passing report written atomically;
failing report written atomically;

failure exits nonzero;
report records comparator hash and input hashes.

Provisional conclusion

oracle.py correctly implements the core numerical comparison needed for modular S01 equivalence. It should remain separate from ordinary production execution.

Before it can certify the modular platform, add:

independently pinned oracle authentication;
candidate/oracle manifest validation;
duplicate-safe condition comparison;
local strict numeric CSV parsing;
duplicate-safe NPZ loading;
durable atomic certification output;
explicit linkage between the certification report and the modular execution.

Once those are implemented and the full 200-condition logical comparison passes, the modular runner can be described as oracle-equivalent to the successful monolithic S01 workflow.

Evaluate this:

text

#!/usr/bin/env python3
"""AIEF-1.4 T_window sensitivity reanalysis v4.

Analysis v4 is a separately versioned reanalysis of the existing S01 raw run.
It does not regenerate trajectories and does not modify Engine v1, Modular
Runner v1, the legacy collector, or Analysis v3.

Main changes from Analysis v3
----
* Recovery target definitions are separated from observation support.
* Recovery events are explicitly saved-grid events on [0.01, 319.51].
* Held recovery uses timestamps: times[j] - times[i] >= duration.
* Both a fixed 17.5-unit hold and one-current-T_window hold are reported.

* Density events and threshold-independent AUCs are not duplicated under two horizon labels.
* Undefined zero-vector cosine is retained as NaN in raw input and converted to zero in one explicit derived recovery representation used consistently by recovery events and AUC.
* Restricted event times and event indicators are full-cohort outcomes.
* Raw event-time interactions are explicitly four-cell complete-case outcomes.
* The same bootstrap seed resamples the same seed indices across all groups. Primary regime/DID/topology families also receive simultaneous max-t bands.
* Outputs are written to a staging directory and promoted atomically as one immutable analysis bundle.

The analysis cannot reconstruct unsaved t=0/t=320 observations or exact step-level crossing times. Those require a new engine/collector version.
"""

```
from __future__ import annotations

import argparse
import datetime as dt
import hashlib
import json
import math
import os
import platform
import secrets
import sys
import time
import traceback
from dataclasses import dataclass
from pathlib import Path
from typing import Any, Iterable, Mapping, Sequence

import numpy as np
import pandas as pd

import aief_modular.artifacts as modular_artifacts
import aief_modular.validation as modular_validation
from aief_modular.artifacts import (
atomic_write_json,
```

```
atomic_write_text,
file_sha256,
)
from aief_modular.validation import ArtifactIntegrityError



ANALYSIS_ID = "AIEF_1_4_TWINDOW_ANALYSIS_V4"
ANALYSIS_VERSION = "v4.0"
STUDY_NAME = "AIEF_1_4_Twindow_sensitivity"
EXPECTED_N_SEEDS = 40
EXPECTED_RHO_VALUES = (0.5, 0.75, 1.0, 1.5, 2.0)
EXPECTED_BRANCHES = ("continue", "reset_ema", "reset_B", "future_shuffle")
NON_CONTINUE_BRANCHES = ("reset_ema", "reset_B", "future_shuffle")
EXPECTED_SAMPLE_TIMES = (np.arange(0, 32000, 50, dtype=float) + 1.0) * 0.01
SUPPORT_START = float(EXPECTED_SAMPLE_TIMES[0])
SUPPORT_END = float(EXPECTED_SAMPLE_TIMES[-1])
SAMPLE_SPACING = 0.5
FIXED_HOLD_DURATION = 17.5

PRIMARY_OUTCOMES = (
"Weighted_similarity_late_fixed_physical",
"Retention_contrast_late_fixed_physical",
"P_mean_late_fixed_physical",
)
SECONDARY_OUTCOMES = (
"Weighted_similarity_late_normalized_EMA",
"Retention_contrast_late_normalized_EMA",
"P_mean_late_normalized_EMA",
)



@dataclass(frozen=True)
class RunInputs:
run_dir: Path
scalar_csv: Path
branch_npz: Path
manifest: Path
condition_status: Path
baseline_certificate: Path
```

```
@dataclass(frozen=True)
class ValidatedRun:
inputs: RunInputs
scalar: pd.DataFrame
manifest: dict[str, Any]
seeds: tuple[str, ...]
rho_values: tuple[float, ...]
branches: tuple[str, ...]
b_c: float
t_star: float
input_hashes: dict[str, str]



# ---------------------------------------------------------------------------
# Atomic analysis-bundle utilities
# ---------------------------------------------------------------------------



def _fsync_directory(path: Path) -> None:
descriptor = os.open(str(path), os.O_RDONLY | getattr(os, "O_DIRECTORY", 0))
try:
os.fsync(descriptor)
finally:
os.close(descriptor)



def save_dataframe_atomic(path: Path, frame: pd.DataFrame) -> None:
path.parent.mkdir(parents=True, exist_ok=True)
temporary = path.with_name(f".{path.name}.{os.getpid()}.{time.time_ns()}.tmp")
try:
with open(temporary, "w", encoding="utf-8", newline="") as handle:
frame.to_csv(handle, index=False)
handle.flush()
os.fsync(handle.fileno())
os.replace(temporary, path)
_fsync_directory(path.parent)
finally:
if temporary.exists():
```

```
temporary.unlink()



def utc_now() -> str:
return dt.datetime.now(dt.timezone.utc).replace(microsecond=0).isoformat().replace(
"+00:00", "Z"
)



def analysis_execution_id() -> str:
stamp = dt.datetime.now(dt.timezone.utc).strftime("%Y%m%dT%H%M%SZ")
return f"analysis_v4_{stamp}_{secrets.token_hex(4)}"



def source_sha256() -> str:
return file_sha256(Path(__file__))



def seed_set_sha256(seeds: Sequence[str]) -> str:
text = json.dumps(list(seeds), separators=(",", ":"), ensure_ascii=False)
return hashlib.sha256(text.encode("utf-8")).hexdigest()



def analysis_contract() -> dict[str, Any]:
return {
"raw_trajectory_unchanged": True,
"observation_support": [SUPPORT_START, SUPPORT_END],
"sample_spacing": SAMPLE_SPACING,
"event_time_semantics": "saved_grid_interval_censored",
"undefined_similarity": "raw_NaN_derived_recovery_zero",
"hold_duration_systems": [
"fixed_physical_17p5",
"one_current_Twindow_nominal",
],
"AUC_support": "saved_grid_0p01_to_319p51",
"full_cohort_recovery_estimands": [
"restricted_time",
"restricted_time_over_Twindow",
```

```
"event_observed",
"auc_deficit",
],
"conditional_secondary_estimands": [
"event_time",
"event_time_over_Twindow",
"four_cell_complete_case_event_DID",
],
"primary_interval_family": "fixed_physical_regime_and_branch_DID",
"secondary_interval_family": "normalized_EMA_regime_and_branch_DID",
}



def canonical_sha256(value: Mapping[str, Any]) -> str:
text = json.dumps(
value,
sort_keys=True,
separators=(",", ":"),
ensure_ascii=False,
allow_nan=False,
)
return hashlib.sha256(text.encode("utf-8")).hexdigest()



# ---------------------------------------------------------------------------
# Input discovery and validation
# ---------------------------------------------------------------------------



def _one(directory: Path, pattern: str) -> Path:
matches = sorted(directory.glob(pattern))
if len(matches) != 1:
raise ArtifactIntegrityError(
f"Expected exactly one {pattern!r} in {directory}, found {[p.name for p in matches]}"
)
return matches[0]



def discover_inputs(run_dir: str | Path) -> RunInputs:
```

```
root = Path(run_dir)
if not root.is_dir():
raise ArtifactIntegrityError(f"Run directory not found: {root}")
return RunInputs(
run_dir=root,
scalar_csv=_one(root, f"{STUDY_NAME}_*_scalar_summary.csv"),
branch_npz=_one(root, f"{STUDY_NAME}_*_branch_diagnostics.npz"),
manifest=root / "manifest.json",
condition_status=root / "twindow_condition_status.csv",
baseline_certificate=root / "baseline_equivalence_certificate.json",
)



def _load_scalar(path: Path) -> pd.DataFrame:
frame = pd.read_csv(path, dtype={"seed": str, "branch": str})
if "rho" in frame:
frame["rho"] = pd.to_numeric(frame["rho"], errors="raise")
for column in frame.columns:
if column not in {"seed", "branch"}:
frame[column] = pd.to_numeric(frame[column], errors="coerce")
return frame



def _require_hash(manifest: Mapping[str, Any], label: str, path: Path) -> None:
expected = manifest.get("output_hashes", {}).get(label)
if not isinstance(expected, str):
raise ArtifactIntegrityError(f"Manifest lacks output hash {label!r}")
observed = file_sha256(path)
if observed != expected:
raise ArtifactIntegrityError(
f"Input hash mismatch for {label}: expected={expected}, observed={observed}"
)



def validate_run(inputs: RunInputs) -> ValidatedRun:
for path in (
inputs.manifest,
inputs.condition_status,
inputs.baseline_certificate,
inputs.scalar_csv,
```

```
inputs.branch_npz,
):
if not path.is_file():
raise ArtifactIntegrityError(f"Required input missing: {path}")


with open(inputs.manifest, "r", encoding="utf-8") as handle:
manifest = json.load(handle)
if not isinstance(manifest, dict):
raise ArtifactIntegrityError("Simulation manifest root must be an object")
if manifest.get("freezeable") is not True:
raise ArtifactIntegrityError("Simulation run is not explicitly freezeable")
completion = manifest.get("completion", {})
if completion.get("status") != "completed":
raise ArtifactIntegrityError("Simulation run is not completed")
if manifest.get("baseline_equivalence", {}).get("status") != "passed":
raise ArtifactIntegrityError("Simulation baseline equivalence is not passed")


for label, path in (
("scalar_csv", inputs.scalar_csv),
("branch_npz", inputs.branch_npz),
("condition_status_csv", inputs.condition_status),
("baseline_certificate", inputs.baseline_certificate),
):
_require_hash(manifest, label, path)


with open(inputs.baseline_certificate, "r", encoding="utf-8") as handle:
certificate = json.load(handle)
if certificate.get("status") != "passed" or certificate.get("n_seeds") != 40:
raise ArtifactIntegrityError("Baseline certificate is not a passing 40-seed certificate")


frame = _load_scalar(inputs.scalar_csv)
required_columns = {
"seed",
"seed_index",
"rho",
"T_window",
"branch",
"D_B_final",
*PRIMARY_OUTCOMES,
*SECONDARY_OUTCOMES,
}
```

```
missing = required_columns - set(frame)
if missing:
raise ArtifactIntegrityError(f"Scalar CSV missing fields: {sorted(missing)}")
if len(frame) != 800:
raise ArtifactIntegrityError(f"Scalar CSV has {len(frame)} rows, expected 800")
if frame.duplicated(["seed", "rho", "branch"]).any():
raise ArtifactIntegrityError("Scalar CSV contains duplicate seed/rho/branch rows")


randomization = manifest.get("randomization", {})
study = manifest.get("study", {})
protocol = manifest.get("protocol", {})
model = manifest.get("model", {})
seeds = tuple(str(seed) for seed in randomization.get("paired_seeds", ()))
rho_values = tuple(float(value) for value in study.get("rho_values", ()))
branches = tuple(protocol.get("branches", ()))
if len(seeds) != EXPECTED_N_SEEDS or len(set(seeds)) != EXPECTED_N_SEEDS:
raise ArtifactIntegrityError("Manifest does not contain 40 unique paired seeds")
if rho_values != EXPECTED_RHO_VALUES:
raise ArtifactIntegrityError(f"Unexpected rho levels/order: {rho_values}")
if branches != EXPECTED_BRANCHES:
raise ArtifactIntegrityError(f"Unexpected branch set/order: {branches}")


expected_conditions = {
(seed, rho, branch)
for seed in seeds
for rho in rho_values
for branch in branches
}
observed_conditions = set(zip(frame["seed"], frame["rho"], frame["branch"]))
if observed_conditions != expected_conditions:
raise ArtifactIntegrityError("Scalar condition coverage differs from manifest design")


for outcome in (*PRIMARY_OUTCOMES, *SECONDARY_OUTCOMES, "D_B_final"):
values = pd.to_numeric(frame[outcome], errors="coerce").to_numpy(float)
if not np.all(np.isfinite(values)):
raise ArtifactIntegrityError(f"Scalar outcome {outcome} contains nonfinite values")
density = frame["D_B_final"].to_numpy(float)
if np.any((density < 0.0) | (density > 1.0)):
raise ArtifactIntegrityError("D_B_final is outside [0,1]")


t_star = float(model.get("T_star", float("nan")))
```

```
if not np.isfinite(t_star) or t_star <= 0.0:
raise ArtifactIntegrityError("Manifest model.T_star is invalid")
for rho in rho_values:
observed = frame.loc[frame["rho"] == rho, "T_window"].unique()
if len(observed) != 1 or float(observed[0]) != rho * t_star:
raise ArtifactIntegrityError(f"rho={rho} has inconsistent nominal T_window")


status = pd.read_csv(inputs.condition_status, dtype={"seed": str})
if len(status) != 200 or status.duplicated(["seed", "rho"]).any():
raise ArtifactIntegrityError("Condition-status table does not have 200 unique conditions")
if set(status["status"]) != {"success"}:
raise ArtifactIntegrityError("Condition-status table contains non-success conditions")
observed_status = set(zip(status["seed"], status["rho"].astype(float)))
expected_status = {(seed, rho) for seed in seeds for rho in rho_values}
if observed_status != expected_status:
raise ArtifactIntegrityError("Condition-status coverage differs from design")


b_c = float(model.get("B_c", float("nan")))
if not np.isfinite(b_c) or b_c < 0.0:
raise ArtifactIntegrityError("Manifest model.B_c is invalid")


input_hashes = {
"scalar_csv": file_sha256(inputs.scalar_csv),
"branch_npz": file_sha256(inputs.branch_npz),
"simulation_manifest": file_sha256(inputs.manifest),
"condition_status": file_sha256(inputs.condition_status),
"baseline_certificate": file_sha256(inputs.baseline_certificate),
}
return ValidatedRun(
inputs=inputs,
scalar=frame,
manifest=manifest,
seeds=seeds,
rho_values=rho_values,
branches=branches,
b_c=b_c,
t_star=t_star,
input_hashes=input_hashes,
)
```

```
# ----------------------------------------------------------------------
# Common scientific metrics
# ----------------------------------------------------------------------



def cosine_edges(left: np.ndarray, right: np.ndarray) -> float:
if left.shape != (12, 12) or right.shape != (12, 12):
raise ArtifactIntegrityError("Topology matrix shape must be (12,12)")
if not np.all(np.isfinite(left)) or not np.all(np.isfinite(right)):
raise ArtifactIntegrityError("Topology matrix contains nonfinite values")
edges = np.triu_indices(12, 1)
x = left[edges]
y = right[edges]
denominator = float(np.linalg.norm(x) * np.linalg.norm(y))
if denominator <= 1e-12:
return float("nan")
return float(np.dot(x, y) / denominator)



def jaccard_edges(left: np.ndarray, right: np.ndarray, threshold: float) -> float:
edges = np.triu_indices(12, 1)
x = left[edges] > threshold
y = right[edges] > threshold
union = np.count_nonzero(x | y)
return float(np.count_nonzero(x & y) / union) if union else float("nan")



def normalized_distance(left: np.ndarray, baseline: np.ndarray) -> float:
edges = np.triu_indices(12, 1)
denominator = float(np.linalg.norm(baseline[edges]))
if denominator <= 1e-12:
return float("nan")
return float(np.linalg.norm(left[edges] - baseline[edges]) / denominator)



def regime_contrasts(frame: pd.DataFrame, outcomes: Sequence[str], system: str) -> pd.DataFrame:
control = frame.loc[frame["branch"] == "continue", ["seed", "rho", *outcomes]].copy()
baseline = control.loc[control["rho"] == 1.0, ["seed", *outcomes]].rename(
columns={outcome: f"{outcome}_baseline" for outcome in outcomes}
```

```
)
merged = control.merge(baseline, on="seed", validate="many_to_one")
rows: list[dict[str, Any]] = []
for row in merged.loc[merged["rho"] != 1.0].itertuples(index=False):
values = row._asdict()
for outcome in outcomes:
current = float(values[outcome])
reference = float(values[f"{outcome}_baseline"])
rows.append(
{
"seed": values["seed"],
"rho": float(values["rho"]),
"target_system": system,
"outcome": outcome,
"Y_rho_continue": current,
"Y_baseline_continue": reference,
"paired_regime_contrast": current - reference,
}
)
return pd.DataFrame(rows)



def branch_interactions(frame: pd.DataFrame, outcomes: Sequence[str], system: str) -> pd.DataFrame:
indexed = frame.set_index(["seed", "rho", "branch"], verify_integrity=True)
rows: list[dict[str, Any]] = []
for seed in frame["seed"].unique():
for rho in EXPECTED_RHO_VALUES:
if rho == 1.0:
continue
for branch in NON_CONTINUE_BRANCHES:
for outcome in outcomes:
y_rho_b = float(indexed.loc[(seed, rho, branch), outcome])
y_rho_c = float(indexed.loc[(seed, rho, "continue"), outcome])
y_base_b = float(indexed.loc[(seed, 1.0, branch), outcome])
y_base_c = float(indexed.loc[(seed, 1.0, "continue"), outcome])
effect_rho = y_rho_b - y_rho_c
effect_base = y_base_b - y_base_c
rows.append(
{
"seed": seed,
"rho": rho,
"branch": branch,
```

```
"target_system": system,
"outcome": outcome,
"branch_effect_rho": effect_rho,
"branch_effect_baseline": effect_base,
"difference_in_differences": effect_rho - effect_base,
}
)
return pd.DataFrame(rows)



def _npz_key(branch: str, seed: str, rho: float, field: str) -> str:
return f"{branch}_seed_{seed}_rho_{rho}_{field}"



def load_npz_keys(path: Path, keys: Iterable[str]) -> dict[str, np.ndarray]:
requested = tuple(dict.fromkeys(keys))
result: dict[str, np.ndarray] = {}
try:
with np.load(path, allow_pickle=False) as archive:
missing = [key for key in requested if key not in archive]
if missing:
raise ArtifactIntegrityError(
f"NPZ missing {len(missing)} required entries: {missing[:10]}"
)
for key in requested:
result[key] = archive[key].copy()
except ArtifactIntegrityError:
raise
except Exception as exc:
raise ArtifactIntegrityError(f"Cannot read NPZ: {exc}") from exc
return result



def cross_t_topology(run: ValidatedRun) -> pd.DataFrame:
keys = [
_npz_key("continue", seed, rho, "B_final")
for seed in run.seeds
for rho in run.rho_values
]
arrays = load_npz_keys(run.inputs.branch_npz, keys)
```

```
rows = []
for seed in run.seeds:
baseline = np.asarray(arrays[_npz_key("continue", seed, 1.0, "B_final")], float)
for rho in run.rho_values:
if rho == 1.0:
continue
current = np.asarray(arrays[_npz_key("continue", seed, rho, "B_final")], float)
metrics = {
"cross_T_cosine": cosine_edges(current, baseline),
"cross_T_Jaccard": jaccard_edges(current, baseline, run.b_c),
"cross_T_normalized_distance": normalized_distance(current, baseline),
}
if not all(np.isfinite(value) for value in metrics.values()):
raise ArtifactIntegrityError(f"Nonfinite topology metric for seed={seed}, rho={rho}")
rows.append({"seed": seed, "rho": rho, **metrics})
return pd.DataFrame(rows)



# ---------------------------------------------------------------------------
# Saved-grid recovery semantics
# ---------------------------------------------------------------------------



def _validate_trajectory(times: np.ndarray, density: np.ndarray, similarity: np.ndarray, label: str) ->
None:
if times.shape != EXPECTED_SAMPLE_TIMES.shape or not np.array_equal(times,
EXPECTED_SAMPLE_TIMES):
raise ArtifactIntegrityError(f"{label} has an unexpected saved-time schedule")
if density.shape != times.shape or similarity.shape != times.shape:
raise ArtifactIntegrityError(f"{label} trajectory shapes do not agree")
if not np.all(np.isfinite(density)) or np.any((density < 0.0) | (density > 1.0)):
raise ArtifactIntegrityError(f"{label} density is nonfinite or outside [0,1]")
if np.any(np.isinf(similarity)):
raise ArtifactIntegrityError(f"{label} similarity contains Infinity")
finite = similarity[np.isfinite(similarity)]
if np.any((finite < -1.0 - 1e-12) | (finite > 1.0 + 1e-12)):
raise ArtifactIntegrityError(f"{label} similarity is outside [-1,1]")



def _first_index(condition: np.ndarray) -> int | None:
```

```
indices = np.flatnonzero(condition)
return int(indices[0]) if len(indices) else None



def _sustained_index(condition: np.ndarray) -> int | None:
suffix = np.logical_and.accumulate(condition[::-1])[::-1]
return _first_index(suffix)



def _held_index(times: np.ndarray, condition: np.ndarray, duration: float) -> int | None:
"""Earliest saved index with a true run spanning at least ``duration``."""
if not np.isfinite(duration) or duration <= 0.0:
raise ValueError("Hold duration must be finite and positive")
run_end = np.empty(len(condition), dtype=np.int64)
end = -1
for index in range(len(condition) - 1, -1, -1):
if condition[index]:
if index == len(condition) - 1 or not condition[index + 1]:
end = index
run_end[index] = end
else:
run_end[index] = index - 1
for index in np.flatnonzero(condition):
required_end = int(np.searchsorted(times, times[index] + duration, side="left"))
if required_end < len(times) and required_end <= run_end[index]:
return int(index)
return None



def _event_row(
*,
seed: str,
rho: float,
t_window: float,
branch: str,
metric: str,
target_definition: str,
target_value: float,
event_type: str,
hold_system: str,
```

```
hold_duration: float | None,
event_index: int | None,
imputed_count: int,
) -> dict[str, Any]:
observed = event_index is not None
event_time = float(EXPECTED_SAMPLE_TIMES[event_index]) if observed else float("nan")
restricted = event_time if observed else SUPPORT_END
lower = (
0.0
if observed and event_index == 0
else float(EXPECTED_SAMPLE_TIMES[event_index - 1])
if observed
else float("nan")
)
upper = event_time if observed else float("nan")
window_steps = int(round(t_window / 0.01))
effective = window_steps * 0.01
return {
"seed": seed,
"rho": rho,
"T_window_nominal": t_window,
"window_steps": window_steps,
"T_window_effective": effective,
"T_window_rounding_error": effective - t_window,
"branch": branch,
"metric": metric,
"target_definition": target_definition,
"target_value": target_value,
"observation_support": "saved_grid_0p01_to_319p51",
"support_start": SUPPORT_START,
"support_end": SUPPORT_END,
"sample_spacing": SAMPLE_SPACING,
"event_type": event_type,
"hold_duration_system": hold_system,
"hold_duration": hold_duration,
"event_time_semantics": "first_qualifying_saved_sample_interval_censored",
"event_time": event_time,
"event_observed": bool(observed),
"restricted_time": restricted,
"event_time_over_Twindow": event_time / t_window if observed else float("nan"),
"restricted_time_over_Twindow": restricted / t_window,
"event_interval_lower": lower,
```

```
"event_interval_upper": upper,
"undefined_samples_imputed_zero": imputed_count,
"undefined_similarity_convention": (
"raw_NaN_to_zero_for_all_derived_recovery_metrics"
if metric == "similarity"
else "not_applicable"
),
}



def _auc(times: np.ndarray, reference: np.ndarray, target: np.ndarray) -> float:
deficit = np.maximum(0.0, reference - target)
if hasattr(np, "trapezoid"):
return float(np.trapezoid(deficit, x=times))
return float(np.trapz(deficit, x=times))



def compute_recovery(run: ValidatedRun) -> tuple[pd.DataFrame, pd.DataFrame]:
fields = ("sample_times", "density_trajectory", "similarity_trajectory")
keys = [
_npz_key(branch, seed, rho, field)
for seed in run.seeds
for rho in run.rho_values
for branch in run.branches
for field in fields
]
arrays = load_npz_keys(run.inputs.branch_npz, keys)
scalar = run.scalar.set_index(["seed", "rho", "branch"], verify_integrity=True)
event_rows: list[dict[str, Any]] = []
auc_rows: list[dict[str, Any]] = []


for seed in run.seeds:
for rho in run.rho_values:
t_window = float(scalar.loc[(seed, rho, "continue"), "T_window"])
density_target = 0.9 * float(scalar.loc[(seed, rho, "continue"), "D_B_final"])
similarity_targets = (
(
"similarity_90pct_continue_late_fixed_physical",
0.9
* float(
```

```
scalar.loc[
(seed, rho, "continue"),
"Weighted_similarity_late_fixed_physical",
]
),
),
(
"similarity_90pct_continue_late_normalized_EMA",
0.9
* float(
scalar.loc[
(seed, rho, "continue"),
"Weighted_similarity_late_normalized_EMA",
]
),
),
)
cont_times = np.asarray(
arrays[_npz_key("continue", seed, rho, "sample_times")], float
)
cont_density = np.asarray(
arrays[_npz_key("continue", seed, rho, "density_trajectory")], float
)
cont_similarity_raw = np.asarray(
arrays[_npz_key("continue", seed, rho, "similarity_trajectory")], float
)
_validate_trajectory(
cont_times, cont_density, cont_similarity_raw, f"continue/{seed}/{rho}"
)
cont_similarity = np.nan_to_num(cont_similarity_raw, nan=0.0)


for branch in run.branches:
times = np.asarray(arrays[_npz_key(branch, seed, rho, "sample_times")], float)
density = np.asarray(
arrays[_npz_key(branch, seed, rho, "density_trajectory")], float
)
similarity_raw = np.asarray(
arrays[_npz_key(branch, seed, rho, "similarity_trajectory")], float
)
_validate_trajectory(times, density, similarity_raw, f"{branch}/{seed}/{rho}")
similarity = np.nan_to_num(similarity_raw, nan=0.0)
undefined_count = int(np.isnan(similarity_raw).sum())
```

```
metric_targets = (
(
"density",
density,
(("density_90pct_continue_final", density_target),),
0,
),
("similarity", similarity, similarity_targets, undefined_count),
)
for metric, values, targets, imputed_count in metric_targets:
for target_definition, target_value in targets:
if not np.isfinite(target_value):
raise ArtifactIntegrityError(
f"Nonfinite recovery target {target_definition} for {seed}/{rho}"
)
condition = values >= target_value
first = _first_index(condition)
sustained = _sustained_index(condition)
event_rows.append(
_event_row(
seed=seed,
rho=rho,
t_window=t_window,
branch=branch,
metric=metric,
target_definition=target_definition,
target_value=target_value,
event_type="first_crossing",
hold_system="not_applicable",
hold_duration=None,
event_index=first,
imputed_count=imputed_count,
)
)
event_rows.append(
_event_row(
seed=seed,
rho=rho,
t_window=t_window,
branch=branch,
metric=metric,
```

```
target_definition=target_definition,
target_value=target_value,
event_type="sustained_through_last_saved_observation",
hold_system="not_applicable",
hold_duration=None,
event_index=sustained,
imputed_count=imputed_count,
)
)
for hold_system, hold_duration in (
("fixed_physical_17p5", FIXED_HOLD_DURATION),
("one_current_Twindow_nominal", t_window),
):
held = _held_index(times, condition, hold_duration)
event_rows.append(
_event_row(
seed=seed,
rho=rho,
t_window=t_window,
branch=branch,
metric=metric,
target_definition=target_definition,
target_value=target_value,
event_type="held",
hold_system=hold_system,
hold_duration=hold_duration,
event_index=held,
imputed_count=imputed_count,
)
)


for metric, reference, target, target_nan, reference_nan in (
("density", cont_density, density, 0, 0),
(
"similarity",
cont_similarity,
similarity,
undefined_count,
int(np.isnan(cont_similarity_raw).sum()),
),
):
auc_value = _auc(times, reference, target)
```

```
auc_rows.append(
{
"seed": seed,
"rho": rho,
"T_window_nominal": t_window,
"branch": branch,
"metric": metric,
"auc_definition": "one_sided_deficit_vs_same_condition_continue",
"observation_support": "saved_grid_0p01_to_319p51",
"support_start": SUPPORT_START,
"support_end": SUPPORT_END,
"support_width": SUPPORT_END - SUPPORT_START,
"auc_deficit": auc_value,
"auc_deficit_per_support_unit": auc_value
/ (SUPPORT_END - SUPPORT_START),
"target_undefined_samples_imputed_zero": target_nan,
"reference_undefined_samples_imputed_zero": reference_nan,
}
)


events = pd.DataFrame(event_rows)
auc = pd.DataFrame(auc_rows)
if len(events) != 9600:
raise ArtifactIntegrityError(f"Recovery event row count {len(events)} != 9600")
if len(auc) != 1600:
raise ArtifactIntegrityError(f"Recovery AUC row count {len(auc)} != 1600")
return events, auc



# ---------------------------------------------------------------------------
# Recovery interactions
# ---------------------------------------------------------------------------



def recovery_event_interactions(events: pd.DataFrame) -> pd.DataFrame:
keys = [
"seed",
"rho",
"branch",
"metric",
```

```
"target_definition",
"event_type",
"hold_duration_system",
]
indexed = events.set_index(keys, verify_integrity=True)
rows: list[dict[str, Any]] = []
definitions = events[
["metric", "target_definition", "event_type", "hold_duration_system"]
].drop_duplicates()
for seed in sorted(events["seed"].unique()):
for rho in EXPECTED_RHO_VALUES:
if rho == 1.0:
continue
for branch in NON_CONTINUE_BRANCHES:
for definition in definitions.itertuples(index=False):
suffix = tuple(definition)
cells = {
"rho_branch": indexed.loc[(seed, rho, branch, *suffix)],
"rho_continue": indexed.loc[(seed, rho, "continue", *suffix)],
"base_branch": indexed.loc[(seed, 1.0, branch, *suffix)],
"base_continue": indexed.loc[(seed, 1.0, "continue", *suffix)],
}
restricted = {
name: float(value["restricted_time"]) for name, value in cells.items()
}
restricted_norm = {
name: float(value["restricted_time_over_Twindow"])
for name, value in cells.items()
}
observed = {
name: float(bool(value["event_observed"])) for name, value in cells.items()
}
complete = all(bool(value["event_observed"]) for value in cells.values())
raw = {
name: float(value["event_time"]) for name, value in cells.items()
}
raw_norm = {
name: float(value["event_time_over_Twindow"])
for name, value in cells.items()
}


def did(values: Mapping[str, float]) -> float:
```

```
return (values["rho_branch"] - values["rho_continue"]) - (
values["base_branch"] - values["base_continue"]
)


rows.append(
{
"seed": seed,
"rho": rho,
"branch": branch,
"metric": definition.metric,
"target_definition": definition.target_definition,
"event_type": definition.event_type,
"hold_duration_system": definition.hold_duration_system,
"rho_branch_restricted_time": restricted["rho_branch"],
"rho_continue_restricted_time": restricted["rho_continue"],
"baseline_branch_restricted_time": restricted["base_branch"],
"baseline_continue_restricted_time": restricted["base_continue"],
"restricted_branch_effect_rho": (
restricted["rho_branch"] - restricted["rho_continue"]
),
"restricted_branch_effect_baseline": (
restricted["base_branch"] - restricted["base_continue"]
),
"restricted_time_DID": did(restricted),
"restricted_time_over_Twindow_DID": did(restricted_norm),
"rho_branch_event_observed": bool(observed["rho_branch"]),
"rho_continue_event_observed": bool(observed["rho_continue"]),
"baseline_branch_event_observed": bool(observed["base_branch"]),
"baseline_continue_event_observed": bool(observed["base_continue"]),
"event_observed_DID": did(observed),
"four_cell_complete_case": complete,
"rho_branch_event_time": raw["rho_branch"],
"rho_continue_event_time": raw["rho_continue"],
"baseline_branch_event_time": raw["base_branch"],
"baseline_continue_event_time": raw["base_continue"],
"conditional_event_time_DID": did(raw) if complete else float("nan"),
"conditional_event_time_over_Twindow_DID": (
did(raw_norm) if complete else float("nan")
),
}
)
result = pd.DataFrame(rows)
```

```
if len(result) != 5760:
raise ArtifactIntegrityError(f"Recovery event DID row count {len(result)} != 5760")
return result



def recovery_auc_interactions(auc: pd.DataFrame) -> pd.DataFrame:
indexed = auc.set_index(["seed", "rho", "branch", "metric"], verify_integrity=True)
rows = []
for seed in sorted(auc["seed"].unique()):
for rho in EXPECTED_RHO_VALUES:
if rho == 1.0:
continue
for branch in NON_CONTINUE_BRANCHES:
for metric in ("density", "similarity"):
values = {
"rho_branch": float(indexed.loc[(seed, rho, branch, metric), "auc_deficit"]),
"rho_continue": float(
indexed.loc[(seed, rho, "continue", metric), "auc_deficit"]
),
"base_branch": float(
indexed.loc[(seed, 1.0, branch, metric), "auc_deficit"]
),
"base_continue": float(
indexed.loc[(seed, 1.0, "continue", metric), "auc_deficit"]
),
}
did = (values["rho_branch"] - values["rho_continue"]) - (
values["base_branch"] - values["base_continue"]
)
rows.append(
{
"seed": seed,
"rho": rho,
"branch": branch,
"metric": metric,
"rho_branch_auc_deficit": values["rho_branch"],
"rho_continue_auc_deficit": values["rho_continue"],
"baseline_branch_auc_deficit": values["base_branch"],
"baseline_continue_auc_deficit": values["base_continue"],
"auc_branch_effect_rho": (
values["rho_branch"] - values["rho_continue"]
),
```

```
"auc_branch_effect_baseline": (
values["base_branch"] - values["base_continue"]
),
"auc_deficit_DID": did,
}
)
result = pd.DataFrame(rows)
if len(result) != 960:
raise ArtifactIntegrityError(f"Recovery AUC DID row count {len(result)} != 960")
return result



# ---------------------------------------------------------------------------
# Joint seed bootstrap summaries
# ---------------------------------------------------------------------------



def bootstrap_draws(seeds: Sequence[str], iterations: int, seed: int) -> np.ndarray:
if iterations <= 0:
raise ValueError("Bootstrap iterations must be positive")
rng = np.random.default_rng(seed)
return rng.integers(0, len(seeds), size=(iterations, len(seeds)))



def _aligned_values(group: pd.DataFrame, value: str, seeds: Sequence[str]) -> np.ndarray:
if group["seed"].duplicated().any():
raise ArtifactIntegrityError("Summary group contains duplicate seeds")
mapping = dict(zip(group["seed"], group[value], strict=True))
return np.asarray([mapping.get(seed, float("nan")) for seed in seeds], dtype=float)



def pointwise_summary(
frame: pd.DataFrame,
*,
group_columns: Sequence[str],
value_column: str,
seeds: Sequence[str],
draws: np.ndarray,
) -> pd.DataFrame:
```

```
rows = []
for group_key, group in frame.groupby(list(group_columns), sort=True, dropna=False):
if not isinstance(group_key, tuple):
group_key = (group_key,)
values = _aligned_values(group, value_column, seeds)
finite = np.isfinite(values)
n = int(finite.sum())
mean = float(np.mean(values[finite])) if n else float("nan")
std = float(np.std(values[finite], ddof=1)) if n > 1 else float("nan")
sampled = values[draws]
counts = np.isfinite(sampled).sum(axis=1)
means = np.divide(
np.nansum(sampled, axis=1),
counts,
out=np.full(len(draws), np.nan),
where=counts > 0,
)
finite_boot = means[np.isfinite(means)]
lower = float(np.percentile(finite_boot, 2.5)) if len(finite_boot) else float("nan")
upper = float(np.percentile(finite_boot, 97.5)) if len(finite_boot) else float("nan")
included_seeds = [seed for seed, keep in zip(seeds, finite, strict=True) if keep]
rows.append(
{
**dict(zip(group_columns, group_key, strict=True)),
"estimand": value_column,
"mean": mean,
"std": std,
"n": n,
"pointwise_ci_lower": lower,
"pointwise_ci_upper": upper,
"interval_type": "joint-seed-resampling_pointwise_percentile_95pct",
"included_seed_set_sha256": seed_set_sha256(included_seeds),
"included_seed_fraction": n / len(seeds),
}
)
return pd.DataFrame(rows)



def simultaneous_complete_summary(
frame: pd.DataFrame,
*,
group_columns: Sequence[str],
```

```
value_column: str,
seeds: Sequence[str],
draws: np.ndarray,
family_id: str,
) -> pd.DataFrame:
groups = []
keys = []
for key, group in frame.groupby(list(group_columns), sort=True, dropna=False):
if not isinstance(key, tuple):
key = (key,)
values = _aligned_values(group, value_column, seeds)
if not np.all(np.isfinite(values)):
raise ArtifactIntegrityError(f"Simultaneous family {family_id} is not full cohort")
keys.append(key)
groups.append(values)
matrix = np.asarray(groups, dtype=float)
means = matrix.mean(axis=1)
std = matrix.std(axis=1, ddof=1)
standard_error = std / math.sqrt(len(seeds))
bootstrap_means = matrix[:, draws].mean(axis=2)
point_lower = np.percentile(bootstrap_means, 2.5, axis=1)
point_upper = np.percentile(bootstrap_means, 97.5, axis=1)
denominators = np.where(standard_error > 0.0, standard_error, np.inf)
max_t = np.max(np.abs((bootstrap_means - means[:, None]) / denominators[:, None]), axis=0)
critical = float(np.percentile(max_t, 95.0))
rows = []
for index, key in enumerate(keys):
rows.append(
{
**dict(zip(group_columns, key, strict=True)),
"estimand": value_column,
"mean": float(means[index]),
"std": float(std[index]),
"n": len(seeds),
"pointwise_ci_lower": float(point_lower[index]),
"pointwise_ci_upper": float(point_upper[index]),
"simultaneous_ci_lower": float(means[index] - critical * standard_error[index]),
"simultaneous_ci_upper": float(means[index] + critical * standard_error[index]),
"simultaneous_family": family_id,
"simultaneous_method": "joint-seed-bootstrap_max_t_95pct",
"included_seed_set_sha256": seed_set_sha256(seeds),
}
```

```
)
return pd.DataFrame(rows)



def recovery_summaries(
events: pd.DataFrame,
auc: pd.DataFrame,
event_did: pd.DataFrame,
auc_did: pd.DataFrame,
seeds: Sequence[str],
draws: np.ndarray,
) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame, pd.DataFrame]:
event_groups = (
"rho",
"branch",
"metric",
"target_definition",
"event_type",
"hold_duration_system",
)
event_parts = [
pointwise_summary(
events,
group_columns=event_groups,
value_column=field,
seeds=seeds,
draws=draws,
)
for field in (
"restricted_time",
"restricted_time_over_Twindow",
"event_observed",
"event_time",
"event_time_over_Twindow",
)
]
event_summary = pd.concat(event_parts, ignore_index=True)

auc_parts = [
pointwise_summary(
auc,
```

```
group_columns=("rho", "branch", "metric"),
value_column=field,
seeds=seeds,
draws=draws,
)
for field in ("auc_deficit", "auc_deficit_per_support_unit")
]
auc_summary = pd.concat(auc_parts, ignore_index=True)


interaction_groups = (
"rho",
"branch",
"metric",
"target_definition",
"event_type",
"hold_duration_system",
)
interaction_parts = [
pointwise_summary(
event_did,
group_columns=interaction_groups,
value_column=field,
seeds=seeds,
draws=draws,
)
for field in (
"restricted_time_DID",
"restricted_time_over_Twindow_DID",
"event_observed_DID",
"conditional_event_time_DID",
"conditional_event_time_over_Twindow_DID",
)
]
interaction_summary = pd.concat(interaction_parts, ignore_index=True)
auc_interaction_summary = pointwise_summary(
auc_did,
group_columns=("rho", "branch", "metric"),
value_column="auc_deficit_DID",
seeds=seeds,
draws=draws,
)
return event_summary, auc_summary, interaction_summary, auc_interaction_summary
```

```
# ---------------------------------------------------------------------------
# Report and bundle execution
# ---------------------------------------------------------------------------



def build_report(
run: ValidatedRun,
regime_summary: pd.DataFrame,
did_summary: pd.DataFrame,
topology_summary: pd.DataFrame,
events: pd.DataFrame,
event_did_summary: pd.DataFrame,
) -> str:
complete_case = event_did_summary[
event_did_summary["estimand"] == "conditional_event_time_DID"
]
lines = [
"=" * 80,
"AIEF-1.4 T_window Sensitivity — Analysis v4",
"=" * 80,
"",
"STATUS",
" Separately versioned reanalysis of the existing S01 raw trajectories.",
" No trajectories were regenerated.",
"",
"RECOVERY SEMANTICS",
f" Saved observation support: [{SUPPORT_START}, {SUPPORT_END}]",
f" Saved-grid spacing: {SAMPLE_SPACING}",
" Event times: first qualifying saved observation; interval-censored.",
" Undefined similarity: raw NaN, derived recovery representation uses zero.",
" Hold systems: fixed 17.5 units and one nominal current T_window.",
" AUC: one-sided deficit over saved support; not labelled full [0,320].",
"",
"INFERENCE",
" Seed/graph is the inferential unit.",
" One joint seed-resampling plan is reused across all estimands.",
" Primary regime, branch-DID, and topology families have max-t simultaneous bands.",
" Recovery intervals are pointwise; complete-case raw-event estimates are secondary.",
"",
```

```
"COUNTS",
f" Seeds: {len(run.seeds)}",
f" Regime summary groups: {len(regime_summary)}",
f" Branch-DID summary groups: {len(did_summary)}",
f" Topology summary groups: {len(topology_summary)}",
f" Recovery event rows: {len(events)}",
f" Conditional recovery-DID groups: {len(complete_case)}",
"",
"INTERPRETATION LIMITS",
" Exact step-level crossings and t=0/t=320 endpoint values are unavailable.",
" Analysis v3 remains the prespecified legacy-compatible analysis.",
" Analysis v4 is not an independent experiment.",
"",
"=" * 80,
]
return "\n".join(lines) + "\n"



def _write_bundle(
*,
run: ValidatedRun,
frames: Mapping[str, pd.DataFrame],
report: str,
output_root: Path,
bootstrap_seed: int,
n_bootstrap: int,
draws: np.ndarray,
command_line: str,
) -> Path:
execution_id = analysis_execution_id()
version_root = output_root / "v4"
version_root.mkdir(parents=True, exist_ok=True)
final = version_root / execution_id
staging = version_root / f".{execution_id}.tmp"
failed = version_root / f"failed_{execution_id}"
if final.exists() or staging.exists() or failed.exists():
raise ArtifactIntegrityError("Analysis execution path collision")
staging.mkdir()
started = time.time()
try:
output_paths: dict[str, Path] = {}
for filename, frame in frames.items():
```

```
path = staging / filename
save_dataframe_atomic(path, frame)
output_paths[filename] = path
report_path = staging / "twindow_sensitivity_report_v4.txt"
atomic_write_text(report_path, report)
output_paths[report_path.name] = report_path


# Reopen every authoritative table before bundle promotion. This checks
# logical readability, exact header order, and row cardinality rather
# than relying only on successful writes and byte hashes.
for filename, expected_frame in frames.items():
observed_frame = pd.read_csv(staging / filename)
if list(observed_frame.columns) != list(expected_frame.columns):
raise ArtifactIntegrityError(f"Staged CSV header mismatch: {filename}")
if len(observed_frame) != len(expected_frame):
raise ArtifactIntegrityError(
f"Staged CSV row count mismatch for {filename}: "
f"{len(observed_frame)} != {len(expected_frame)}"
)
if report_path.read_text(encoding="utf-8") != report:
raise ArtifactIntegrityError("Staged analysis report failed read-back validation")


output_hashes = {
filename: file_sha256(path) for filename, path in sorted(output_paths.items())
}
dependency_hashes = {
"analysis_source": source_sha256(),
"aief_modular.artifacts": file_sha256(Path(modular_artifacts.__file__)),
"aief_modular.validation": file_sha256(Path(modular_validation.__file__)),
}
identity_payload = {
"identity_schema": "aief-analysis-scientific-identity-v1",
"analysis_id": ANALYSIS_ID,
"analysis_version": ANALYSIS_VERSION,
"dependency_hashes": dependency_hashes,
"input_hashes": run.input_hashes,
"ordered_seed_set_sha256": seed_set_sha256(run.seeds),
"bootstrap": {
"seed": bootstrap_seed,
"iterations": n_bootstrap,
"resampling": "same_seed_indices_reused_across_all_groups",
},
```

```
"analysis_contract": analysis_contract(),
}
manifest = {
"analysis_manifest_schema": "aief-analysis-execution-manifest-v1",
"analysis_id": ANALYSIS_ID,
"analysis_version": ANALYSIS_VERSION,
"analysis_execution_id": execution_id,
"created_utc": utc_now(),
"status": "completed",
"analysis_identity_sha256": canonical_sha256(identity_payload),
"analysis_identity_payload": identity_payload,
"dependency_hashes": dependency_hashes,
"source_sha256": dependency_hashes["analysis_source"],
"command_line": command_line,
"input_run": {
"path_recorded_for_provenance": str(run.inputs.run_dir),
"input_hashes": run.input_hashes,
"simulation_run_id": run.manifest.get("run_id"),
"simulation_spec_sha256": run.manifest.get("spec_sha256"),
},
"cohort": {
"seed_count": len(run.seeds),
"ordered_seed_set_sha256": seed_set_sha256(run.seeds),
"rho_values": list(run.rho_values),
"branches": list(run.branches),
"inferential_unit": "paired_seed_graph",
},
"bootstrap": {
"seed": bootstrap_seed,
"iterations": n_bootstrap,
"draw_matrix_sha256": hashlib.sha256(
np.ascontiguousarray(draws, dtype=np.int64).tobytes()
).hexdigest(),
"resampling": "same_seed_indices_reused_across_all_groups",
"pointwise_method": "percentile_95pct",
"primary_simultaneous_method": "max_t_95pct",
},
"analysis_contract": analysis_contract(),
"output_hashes": output_hashes,
"elapsed_seconds": time.time() - started,
"environment": {
"python": platform.python_version(),
```

```
"numpy": np.__version__,
"pandas": pd.__version__,
"platform": platform.platform(),
},
}
manifest_path = staging / "analysis_manifest.json"
atomic_write_json(manifest_path, manifest)
with open(manifest_path, "r", encoding="utf-8") as handle:
observed_manifest = json.load(handle)
if observed_manifest.get("analysis_identity_sha256") != canonical_sha256(identity_payload):
raise ArtifactIntegrityError("Staged analysis manifest identity failed read-back")
for filename, expected_hash in output_hashes.items():
if file_sha256(staging / filename) != expected_hash:
raise ArtifactIntegrityError(f"Staged output hash changed before promotion: {filename}")
_fsync_directory(staging)
os.replace(staging, final)
_fsync_directory(version_root)
return final
except Exception as exc:
failure = {
"analysis_id": ANALYSIS_ID,
"analysis_execution_id": execution_id,
"status": "failed",
"exception_type": type(exc).__name__,
"message": str(exc),
"traceback": traceback.format_exc(),
"created_utc": utc_now(),
}
try:
atomic_write_json(staging / "FAILED_analysis_manifest.json", failure)
_fsync_directory(staging)
os.replace(staging, failed)
_fsync_directory(version_root)
except Exception:
pass
raise



def run_analysis(
run_dir: str | Path,
*,
output_root: str | Path | None,
```

```
bootstrap_seed: int,
n_bootstrap: int,
command_line: str,
) -> Path:
run = validate_run(discover_inputs(run_dir))
draws = bootstrap_draws(run.seeds, n_bootstrap, bootstrap_seed)


regime = pd.concat(
[
regime_contrasts(run.scalar, PRIMARY_OUTCOMES, "fixed_physical"),
regime_contrasts(run.scalar, SECONDARY_OUTCOMES, "normalized_EMA"),
],
ignore_index=True,
)
branch_did = pd.concat(
[
branch_interactions(run.scalar, PRIMARY_OUTCOMES, "fixed_physical"),
branch_interactions(run.scalar, SECONDARY_OUTCOMES, "normalized_EMA"),
],
ignore_index=True,
)
topology = cross_t_topology(run)
if len(regime) != 960:
raise ArtifactIntegrityError(f"Regime contrast row count {len(regime)} != 960")
if len(branch_did) != 2880:
raise ArtifactIntegrityError(f"Branch DID row count {len(branch_did)} != 2880")
if len(topology) != 160:
raise ArtifactIntegrityError(f"Topology row count {len(topology)} != 160")


events, auc = compute_recovery(run)
event_did = recovery_event_interactions(events)
auc_did = recovery_auc_interactions(auc)


regime_summary = pd.concat(
[
simultaneous_complete_summary(
regime.loc[regime["target_system"] == system],
group_columns=("rho", "target_system", "outcome"),
value_column="paired_regime_contrast",
seeds=run.seeds,
draws=draws,
```

```
family_id=(
"primary_regime_fixed_physical"
if system == "fixed_physical"
else "secondary_regime_normalized_EMA"
),
)
for system in ("fixed_physical", "normalized_EMA")
],
ignore_index=True,
)
did_summary = pd.concat(
[
simultaneous_complete_summary(
branch_did.loc[branch_did["target_system"] == system],
group_columns=("rho", "branch", "target_system", "outcome"),
value_column="difference_in_differences",
seeds=run.seeds,
draws=draws,
family_id=(
"primary_branch_DID_fixed_physical"
if system == "fixed_physical"
else "secondary_branch_DID_normalized_EMA"
),
)
for system in ("fixed_physical", "normalized_EMA")
],
ignore_index=True,
)
topology_long = topology.melt(
id_vars=("seed", "rho"), var_name="topology_metric", value_name="topology_value"
)
topology_summary = simultaneous_complete_summary(
topology_long,
group_columns=("rho", "topology_metric"),
value_column="topology_value",
seeds=run.seeds,
draws=draws,
family_id="cross_T_topology",
)
(
event_summary,
auc_summary,
```

```
event_did_summary,
auc_did_summary,
) = recovery_summaries(events, auc, event_did, auc_did, run.seeds, draws)
expected_summary_counts = {
"regime": (len(regime_summary), 24),
"branch_DID": (len(did_summary), 72),
"topology": (len(topology_summary), 12),
"recovery_event": (len(event_summary), 1200),
"recovery_AUC": (len(auc_summary), 80),
"recovery_event_DID": (len(event_did_summary), 720),
"recovery_AUC_DID": (len(auc_did_summary), 24),
}
for label, (observed, expected) in expected_summary_counts.items():
if observed != expected:
raise ArtifactIntegrityError(
f"{label} summary row count {observed} != {expected}"
)


report = build_report(
run, regime_summary, did_summary, topology_summary, events, event_did_summary
)
frames = {
"twindow_paired_regime_contrasts_v4.csv": regime,
"twindow_branch_interactions_v4.csv": branch_did,
"twindow_cross_condition_topology_v4.csv": topology,
"twindow_recovery_events_by_seed_v4.csv": events,
"twindow_recovery_auc_by_seed_v4.csv": auc,
"twindow_recovery_event_interactions_v4.csv": event_did,
"twindow_recovery_auc_interactions_v4.csv": auc_did,
"twindow_regime_summary_v4.csv": regime_summary,
"twindow_branch_DID_summary_v4.csv": did_summary,
"twindow_topology_summary_v4.csv": topology_summary,
"twindow_recovery_event_summary_v4.csv": event_summary,
"twindow_recovery_auc_summary_v4.csv": auc_summary,
"twindow_recovery_event_interaction_summary_v4.csv": event_did_summary,
"twindow_recovery_auc_interaction_summary_v4.csv": auc_did_summary,
}
root = Path(output_root) if output_root is not None else Path(run_dir) / "analysis"
return _write_bundle(
run=run,
frames=frames,
report=report,
```

```
output_root=root,
bootstrap_seed=bootstrap_seed,
n_bootstrap=n_bootstrap,
draws=draws,
command_line=command_line,
)



def parse_arguments(argv: Sequence[str] | None = None) -> argparse.Namespace:
parser = argparse.ArgumentParser(description=__doc__)
parser.add_argument("run_dir", help="completed freezeable S01 run directory")
parser.add_argument(
"--output-root",
default=None,
help="analysis root (default: <run_dir>/analysis)",
)
parser.add_argument("--bootstrap-seed", required=True, type=int)
parser.add_argument("--n-bootstrap", type=int, default=2000)
return parser.parse_args(argv)



def main(argv: Sequence[str] | None = None) -> int:
args = parse_arguments(argv)
if args.n_bootstrap <= 0:
raise ValueError("--n-bootstrap must be positive")
final = run_analysis(
args.run_dir,
output_root=args.output_root,
bootstrap_seed=args.bootstrap_seed,
n_bootstrap=args.n_bootstrap,
command_line=" ".join(sys.argv),
)
print(f"Analysis v4 bundle: {final}")
print(f"Manifest: {final / 'analysis_manifest.json'}")
return 0



if __name__ == "__main__":
raise SystemExit(main())
```

This is a substantial improvement over Analysis v3. It cleanly separates target definitions, saved observation support, hold-duration systems, AUC, event tables, and recovery interactions. The use of one joint seed-resampling matrix and transactionally promoted analysis bundles is especially good.
Verdict

Analysis architecture: strong
Recovery semantics: materially improved
Primary/secondary contrast preservation: correct in design
Censoring representation: improved
Joint bootstrap: strong
Transactional outputs: strong
No obvious deterministic runtime blocker
Ready for staged testing
Not yet freeze-ready until the validation points below and oracle comparisons pass

Major improvements over v3
Saved-grid semantics are explicit

Correctly records:

text

support = [0.01, 319.51]
spacing = 0.5
events = first qualifying saved observation
exact step-level crossing unavailable

Correct held-duration implementation

_held_index() now uses:

Python

times[j] - times[i] >= duration

through searchsorted, correcting the one-sample-interval problem.

It also provides both:

text

fixed physical 17.5 hold
one nominal current T_window hold

Unified undefined-similarity representation

Correct:

Python

similarity = np.nan_to_num(
similarity_raw,
nan=0.0,
)

The same derived representation is used for:

first crossing;
sustained event;
held event;
AUC.

Raw inputs remain unchanged.
Deduplicated recovery schema

Correctly avoids:

duplicate density rows under target-system labels;
duplicate similarity AUC rows.

Full-cohort versus conditional separation

Correctly provides:

text

restricted time DID
restricted normalized time DID
event-observed DID
conditional raw-event DID
conditional raw normalized DID
four-cell complete-case indicator

Joint seed bootstrap

One draw matrix is generated and reused across all estimands. This preserves paired cross-condition covariance.
Transactional analysis bundle

The staging-directory workflow is appropriate:

write all outputs;
read them back;
hash them;
write manifest;
validate hashes;
atomically promote the directory.

This resolves Analysis v3's bundle-level replacement problem.
P0/P1: require exact physical seed coverage in every summary group

_aligned_values() currently does:

Python

mapping.get(seed, float("nan"))

A genuinely missing seed row is therefore indistinguishable from a present row whose scientific value is NaN due to censoring.

For conditional estimands, NaN values are legitimate. Missing physical rows are not.
Required fix

Python

```
def _aligned_values(
group: pd.DataFrame,
value: str,
seeds: Sequence[str],
) -> np.ndarray:
if group["seed"].duplicated().any():
raise ArtifactIntegrityError(
"Summary group contains duplicate seeds"
)

expected = set(seeds)
observed = set(group["seed"])

if observed != expected:
missing = sorted(expected - observed)
extra = sorted(observed - expected)
raise ArtifactIntegrityError(
f"Summary group seed coverage mismatch: "
f"missing={missing[:10]}, "
f"extra={extra[:10]}"
)

mapping = dict(
zip(
group["seed"],
group[value],
strict=True,
)
)

return np.asarray(
[mapping[seed] for seed in seeds],
dtype=float,
)
```

Then NaN means “row exists but value is unavailable/censored,” never “row missing.”

Input-certificate validation should be as strict as v3

Current validation checks only:

Python

```
certificate.status
certificate.n_seeds
```

It should also require:

exact 40-seed cohort;
all seed statuses passed;
no false detailed checks;
no nonempty max_diffs;
exact seven frozen branches;
certificate engine hash consistent with the simulation manifest.

Add:

Python

```
expected_certificate_seeds = set(run_manifest_seeds)
observed_certificate_seeds = set(
certificate.get("seed_results", {})
)

if observed_certificate_seeds != expected_certificate_seeds:
raise ArtifactIntegrityError(
"Baseline certificate seed cohort mismatch"
)

expected_frozen_branches = (
"continue",
"reset_ema",
"reset_B",
"reset_B_ema",
"B_carrier_only",
```

```
"full_reset",
"future_shuffle",
)

if tuple(
certificate.get("protocol", {}).get(
"branches",
(),
)
) != expected_frozen_branches:
raise ArtifactIntegrityError(
"Baseline certificate branch namespace mismatch"
)

for seed, result in certificate["seed_results"].items():
if result.get("status") != "passed":
raise ArtifactIntegrityError(
f"Baseline seed {seed} did not pass"
)

failed_checks = [
name
for name, passed in result.get(
"checks",
{},
).items()
if not passed
]
if failed_checks:
raise ArtifactIntegrityError(
f"Baseline seed {seed} contains false "
f"checks: {failed_checks[:10]}"
)

if result.get("max_diffs"):
raise ArtifactIntegrityError(
f"Baseline seed {seed} reports differences"
)
```

Also validate completion counts explicitly:

text

200 requested
200 successful
0 failed
800 scalar rows

Topology validation should retain structural checks

cosine_edges() checks shape and finiteness, but the input matrices are not checked for:

symmetry;
zero diagonal;
nonnegative weights.

Add a dedicated validator before topology calculations:

Python

```
def validate_topology_matrix(
matrix: np.ndarray,
label: str,
) -> np.ndarray:
matrix = np.asarray(matrix, dtype=float)

if matrix.shape != (12, 12):
raise ArtifactIntegrityError(...)

if not np.all(np.isfinite(matrix)):
raise ArtifactIntegrityError(...)

if not np.allclose(
matrix,
matrix.T,
rtol=0.0,
atol=1e-12,
):
```

raise ArtifactIntegrityError(
f"{label} is not symmetric"
)

if not np.allclose(
np.diag(matrix),
0.0,
rtol=0.0,
atol=1e-12,
):
raise ArtifactIntegrityError(
f"{label} diagonal is not zero"
)

if np.any(matrix < -1e-12):
raise ArtifactIntegrityError(
f"{label} contains negative weights"
)

return matrix

Use it for every baseline and current BfinalBfinal.
Scientific interpretation of the bootstrap
Complete-case bootstrap method

For conditional outcomes, pointwise_summary() uses the joint 40-seed draw matrix and drops NaNs inside every resample. This is an available-case ratio estimator bootstrap, not a bootstrap restricted to a fixed complete-case subset.

That can be defensible because it:

retains the same seed draws across all outcomes;
treats event observability as part of the empirical seed-level record;
preserves cross-estimand resampling alignment.

But document it accurately:

text

joint full-cohort seed resampling with available-case mean recomputation

Do not describe it as:

text

bootstrap only over the fixed complete-case subset

Add this to analysis_contract() and the output table’s interval metadata.

If the intended estimand is explicitly conditional on the observed complete-case set, then bootstrap only the included seeds. That would sacrifice one common 40-seed draw matrix across differently selected groups.

The current choice is reasonable if explicitly named.
Simultaneous-band terminology

simultaneous_complete_summary() computes a maximum standardized bootstrap deviation using the observed standard errors:

Python

(bootstrap_mean - observed_mean) / observed_SE

This is a max-standardized-deviation bootstrap band. Calling it “max-t” is common, but it is not a fully bootstrap-studentized tt statistic with a separate bootstrap SE in each resample.

More precise wording:

text

joint-seed bootstrap maximum standardized-deviation 95% band

If you retain max_t, document that it uses observed plug-in standard errors.
Primary versus secondary simultaneous intervals

The report says:

Primary regime, branch-DID, and topology families have max-t simultaneous bands.

But the code also computes simultaneous bands for normalized-EMA secondary regime and DID families.

Either change the report to:

Fixed-primary and normalized-secondary regime/DID families receive separate simultaneous bands; topology receives its own simultaneous family.

or make the secondary summaries pointwise only.

The current separate-family implementation is reasonable.
Event interval metadata

For an event at the first saved sample, _event_row() sets:

Python

event_interval_lower = 0.0
event_interval_upper = 0.01

That is defensible as a post-intervention interval bound, but the metric itself is not observed at t=0t=0.

Add:

text

lower-bound metric unobserved when event_index == 0

or a Boolean field:

Python

"interval_lower_observed": event_index not in (None, 0)

For censored events, consider recording:

Python

"censoring_time": SUPPORT_END

rather than leaving both event interval fields NaN without an explicit censor field.
AUC validation

_auc() assumes arrays are finite because similarity has already been imputed and density validated. Add defensive checks:

Python

```
if not (
len(times)
== len(reference)
== len(target)
):
raise ArtifactIntegrityError(
"AUC input length mismatch"
)

if not np.all(np.isfinite(reference)):
raise ArtifactIntegrityError(...)

if not np.all(np.isfinite(target)):
raise ArtifactIntegrityError(...)
```

This makes _auc() safe as a standalone function.
Bundle read-back should check logical values, not only rows and headers

The staging validator reopens tables and checks:

text

header order
row count

A serialization/type conversion problem could preserve both while changing values.

For important numerical tables, compare loaded values to the original frame:

exact text fields;
NaN-aware numeric equality;
column order;
row count.

At minimum use:

Python

```
pd.testing.assert_frame_equal(
observed_frame,
expected_frame,
check_dtype=False,
check_exact=False,
rtol=0.0,
atol=0.0,
)
```

CSV round-tripping may alter inferred dtypes, so normalize expected/observed columns before comparison. Exact numeric values should round-trip under Python/pandas's current float formatting.

Alternatively, write a per-table schema defining text versus numeric columns.
Analysis identity and dependency closure

The identity includes:

text

analysis source
modular artifacts source
modular validation source
input hashes
seed order
bootstrap parameters
analysis contract

That is good.

Before final promotion, rehash these dependency files and inputs to ensure none changed during analysis:

Python

final_dependency_hashes = {...}
final_input_hashes = {...}

if final_dependency_hashes != dependency_hashes:
raise ArtifactIntegrityError(
"Analysis dependencies changed during execution"
)

if final_input_hashes != run.input_hashes:
raise ArtifactIntegrityError(
"Analysis inputs changed during execution"
)

Analysis is shorter than simulation, but closure is still appropriate.
Bundle-promotion edge case

If:

Python

os.replace(staging, final)

succeeds but _fsync_directory(version_root) fails, the exception handler tries to write into the now-nonexistent staging path and may report failure even though the final directory exists.

Handle promotion state explicitly:

Python

```
promoted = False

try:
...
os.replace(staging, final)
promoted = True
_fsync_directory(version_root)
return final
except Exception:
if promoted:
# Do not pretend staging still exists.
# Write a sibling promotion-warning/failure record.
...
```

Rare, but relevant for transaction semantics.
Input-manifest format

This v4 script appears intended for both the monolithic and modular S01 run manifests. The shown validation relies on:

text

```
manifest.study.rho_values
manifest.randomization.paired_seeds
manifest.protocol.branches
manifest.model.T_star
```

Both current formats provide these, which is good.

Do not add permissive fallbacks that silently skip validation. Missing fields should remain fatal.
Required v4 acceptance tests

Exact preservation of non-recovery raw estimands

Compare v4 and v3:

960 regime-contrast rows;
2,880 branch-DID rows;
160 topology rows.

After accounting for renamed target_system/column names, point estimates must be exactly equal.
Recovery counts

Require:

text

events: 9600
AUC rows: 1600
event interactions: 5760
AUC interactions: 960
event summaries: 1200
AUC summaries: 80
event-DID summaries: 720
AUC-DID summaries: 24

Held-event tests

Construct trajectories where:

35 samples at 0.5 spacing must not satisfy 17.5;
36 samples must satisfy 17.5;
a run interrupted before the required timestamp fails;
a run ending exactly at the required timestamp passes;
nominal TwindowTwindow and fixed 17.5 holds differ where expected.

NaN tests

raw NaN remains unchanged;
derived similarity replaces NaN with zero;

event and AUC use the same derived representation;
Infinity fails;
NaN counts are correct.

Bootstrap tests

draw matrix deterministic from seed;
same draws reused across all summaries;
missing physical seed row fails;
present NaN conditional value is permitted;
complete-case seed hashes are correct;
simultaneous family has expected group coverage;
zero-variance group behaves deterministically.

Transaction tests

Force failure:

during one CSV write;
during report write;
during manifest write;
before promotion;
after promotion.

Require no partially promoted “completed” bundle.
Provisional conclusion

Analysis v4 correctly addresses the principal scientific-semantic limitations identified in Analysis v3:

timestamp-corrected hold duration;
fixed and normalized hold systems;
target/support separation;
deduplicated density and AUC outcomes;
consistent undefined-similarity representation;
full-cohort versus conditional interactions;
joint seed bootstrap;
simultaneous primary-family bands;
transactional immutable bundles.

Before freezing it, add:

exact physical seed coverage in every summary group;
full baseline-certificate validation;
topology structural validation;
dependency/input rehash at completion;
stronger staged CSV logical read-back;
precise bootstrap and simultaneous-band terminology.

After those tests pass, this is a credible corrected reanalysis layer that can operate on the existing S01 trajectories without regenerating them.

try again

On a second, stricter pass: the core v4 algorithms are internally coherent. My prior response overclassified several defensive improvements as blockers.
Revised verdict

No obvious deterministic or scientific-semantic blocker.
The script is ready for staged execution against the completed S01 run.

It successfully implements the intended v4 corrections:

timestamp-based held events;
fixed and current-TwindowTwindow hold definitions;
saved-support terminology;
consistent NaN-to-zero recovery representation;
no duplicated density target systems;
no duplicated threshold-independent AUC systems;
full-cohort restricted/indicator interactions;
explicitly conditional raw-event interactions;
shared seed-resampling draws;
separate simultaneous families;
immutable execution-specific output bundles.

The remaining issues are certification hardening and terminology, not fundamental analysis errors.
What I would still fix before freezing v4
1. Require every summary group to contain all 40 physical seed rows

This is the most important remaining integrity check.

Current _aligned_values() silently inserts NaN for a missing row:

Python

```
mapping.get(seed, float("nan"))
```

That conflates:

a present but censored/undefined value;
a physically missing seed row.

Use:

Python

```
def _aligned_values(
group: pd.DataFrame,
value: str,
seeds: Sequence[str],
) -> np.ndarray:
if group["seed"].duplicated().any():
raise ArtifactIntegrityError(
"Summary group contains duplicate seeds"
)

expected = set(seeds)
observed = set(group["seed"])

if observed != expected:
raise ArtifactIntegrityError(
"Summary group physical seed coverage "
f"differs: missing={sorted(expected-observed)}, "
f"extra={sorted(observed-expected)}"
)

mapping = dict(
```

zip(group["seed"], group[value], strict=True)
)

return np.asarray(
[mapping[seed] for seed in seeds],
dtype=float,
)

After that, NaN unambiguously means the row exists but the conditional event time is unavailable.
2. Close source/input hashes before bundle promotion

The initial input and dependency hashes are strong, but recompute them immediately before final promotion:

Python

final_input_hashes = {
"scalar_csv": file_sha256(run.inputs.scalar_csv),
"branch_npz": file_sha256(run.inputs.branch_npz),
"simulation_manifest": file_sha256(run.inputs.manifest),
"condition_status": file_sha256(
run.inputs.condition_status
),
"baseline_certificate": file_sha256(
run.inputs.baseline_certificate
),
}

if final_input_hashes != run.input_hashes:
raise ArtifactIntegrityError(
"Analysis inputs changed during execution"
)

Do the same for:

text

analysis source

aief_modular.artifacts
aief_modular.validation

This closes the execution provenance before declaring the bundle complete.
3. Strengthen certificate validation

For the known successful S01 run this is unlikely to change anything, because:

the certificate hash is authenticated through the simulation manifest;
the simulation manifest is marked freezeable;
the certificate reports passed/40 seeds.

Still, before freezing v4, require:

text

exact seed cohort
all seed statuses passed
no false detailed checks
no max_diffs
exact seven frozen branches

This is certification defense, not an analysis-algorithm correction.
Statistical terminology refinements
Shared bootstrap with missing conditional outcomes

The current conditional summary bootstrap:

resamples all 40 seeds using the shared draw matrix;
drops NaN conditional event values inside each resample;
recomputes the available-case mean.

That is coherent, but describe it as:

joint full-cohort seed resampling with available-case recomputation.

It is not exactly the same as fixing the observed complete-case subset and bootstrapping only those seeds.

The current approach has a real advantage: all outcomes share the same underlying seed draws, preserving cross-outcome dependence and observability patterns.
“Max-t” label

The simultaneous bands use observed plug-in standard errors rather than bootstrap-specific standard errors in each resample. A more exact label is:

joint-seed bootstrap maximum standardized-deviation band.

Calling it max-tt is common shorthand, but the manifest should state the precise denominator.
Secondary simultaneous families

The code gives separate simultaneous bands to both:

fixed-primary regime/DID families;
normalized-secondary regime/DID families.

The report currently emphasizes only primary families. Update it to say both receive separate familywise bands.
One small report issue

This line:

text

Conditional recovery-DID groups: len(complete_case)

counts the number of conditional summary groups, not the number of groups with at least one complete case.

Use wording such as:

text

Conditional recovery-DID summary groups

and separately report:

text

groups with n > 0
minimum n
median n
maximum n

That will expose complete-case support directly.
Recommended empirical acceptance tests
1. Compile/import

Bash

python3 -m py_compile AIEF_1_4_Twindow_sensitivity_analysis_v4.py
python3 AIEF_1_4_Twindow_sensitivity_analysis_v4.py --help

2. Synthetic held-event tests

Require:

text

35 samples at 0.5 spacing do not satisfy 17.5
36 samples at 0.5 spacing satisfy 17.5
interrupted runs fail
runs ending exactly at the required timestamp pass

3. Run against the completed S01 raw run

Bash

python3 AIEF_1_4_Twindow_sensitivity_analysis_v4.py \
runs/b57562a8ee77 \

--bootstrap-seed 20260808 \
--n-bootstrap 5000

4. Required output counts

text

regime rows 960
branch-DID rows 2,880
topology rows 160
recovery event rows 9,600
recovery AUC rows 1,600
recovery event interactions 5,760
recovery AUC interactions 960
regime summary rows 24
branch-DID summary rows 72
topology summary rows 12
recovery event summary rows 1,200
recovery AUC summary rows 80
recovery event-DID summary rows 720
recovery AUC-DID summary rows 24

5. Legacy-preservation check

After accounting for renamed columns:

v4 regime contrast point estimates must exactly match v3;
v4 ordinary branch DID point estimates must exactly match v3;
v4 topology point estimates must exactly match v3.

Recovery results are expected to differ because their semantics were intentionally revised.
Final assessment

Analysis v4 is suitable for execution as a corrected, separately versioned reanalysis of the existing S01 trajectories.

I would make the physical-seed-coverage check before freezing it. The source/input closure and certificate checks are highly recommended for certification, but the central recovery and inferential algorithms no longer show the defects identified in v3.

Think about this thought:

text

Yes. I think there are some **real conceptual resonances**, but they are not where the attached document initially seems to point. The useful nuggets are mostly about **memory, phase coherence, delayed feedback, multiscale separation, and invariant subspaces**. The more specific SSRH physical claims should not be imported into AIEF as evidence.

The strongest connection is surprisingly close to the direction we've just arrived at.

1. The most important resonance: coherence is a relationship, not a phase value

The attached conversation arrives at a distinction that maps almost perfectly onto our AIEF findings:

> coherence is not "being in phase"; it is having a **stable/predictable phase relationship**.

That is almost exactly the correction we needed when interpreting the K3.

Our earlier language occasionally drifted toward:

[
\text{phase alignment}\Rightarrow\text{coherence}\Rightarrow\text{amplification}.
]

But the actual AIEF mechanism appears more like:

[
\boxed{
\text{stable phase relationships}
\rightarrow
\text{stable coupling projection}
\rightarrow

\text{frequency locking}
\rightarrow
\text{persistent coherent state}.
}
]

And importantly, the attached discussion points out that **coherent cancellation is still coherence**. Two perfectly phase-locked waves can cancel exactly.

That resonates strongly with our:

[
T_1+T_2\rightarrow0
]

observations.

The fact that (T_1/T_2\to-1) should **not** be interpreted as the system losing coherence. Quite the opposite: extremely precise cancellation is evidence of a highly organized relationship between the components.

That is a useful conceptual correction.

---

2. The laser analogy contains a surprisingly good dynamical analogy

The attached discussion describes the laser as:

[
\text{spontaneous emission}
\rightarrow
\text{one organized emission}
\rightarrow
\text{stimulated copies}
\rightarrow
\text{feedback through cavity}
\rightarrow

\text{coherent field}.
]

The useful part is not the physics analogy itself. It's the **architecture**:

```text
weak/random local events
↓
selection/amplification
↓
feedback
↓
phase-locked collective state
↓
persistent coherent output
```

That resembles the architecture we're seeing in the K3 much more than I initially would have expected.

Our empirical sequence is roughly:

text

local intrinsic frequencies
↓
adaptive coupling
↓
phase relationships
↓
frequency locking
↓
EMA/coherence memory
↓
persistent slow collective state
↓
observable/proposal inheritance

The important common idea is not "AIEF is a laser." It obviously isn't.

The common mathematical idea is:

feedback can convert repeated local interactions into a coherent collective mode whose persistence is determined by memory.

That's a useful abstraction.
3. The delay/memory discussion is even more relevant

The other attached material talks explicitly about delay-coupled systems in which fast local transduction can produce slow global coherence through a ratio like
[
\Gamma_{\mathrm{DDE}}

\frac{T_{\mathrm{global}}}{\tau_{\mathrm{local}}}.
]

It describes the conceptual mechanism as:

finite propagation delays convert fast local transduction into slow global coherence.

That is very close to a question we've been circling:

How can phase synchronization be fast ((\mu\approx0.69)) while the observable relaxation is slow ((r_{\mathrm{EMA}}\approx0.057))?

Our AIEF answer is not spatial propagation delay, but the architecture is analogous:

[
\boxed{
\text{fast local phase/coupling relaxation}
+
\text{slow memory}
\rightarrow
\text{slow collective observable}.
}
]

The relevant time-scale ratio in our system is roughly

$$
\frac{\mu_{\rm phase}}{r_{\rm EMA}}
\approx
\frac{0.69}{0.0571}
\approx12.1.
$$

So we actually have direct evidence for the same type of timescale separation.

The attached document's language gives us a useful conceptual vocabulary:

$$
\text{fast transduction}
\quad\longrightarrow\quad
\text{slow coherent organization}.
$$

But we should keep it as analogy, not claim that the mechanisms are mathematically identical.
4. The DDE stability-margin idea may be genuinely useful

This is probably the most technically interesting nugget.

The attached material describes a delayed system schematically as

$$
\dot x(t)=a(t)x(t)+\alpha(t)x(t-\tau(t))
$$

and proposes a stability condition based on the relative strength of delayed versus instantaneous feedback.

That idea maps naturally onto the thing we have been trying to prove:

$$
\text{fast phase/coupling contraction}

\text{slow memory forcing}.
$$

Rather than attempting to identify a beautiful exact eigenbasis, one could seek a comparison inequality of the form

$$
\text{instantaneous contraction}
$$

$$
\text{delayed-memory gain}
$$

That would be much closer to a genuine stability theorem.

In our language, something like

$$
\mu_{\rm fast}>\text{memory feedback gain}
$$

could yield a stable fiber over a slower coherence process.

This potentially gives us a bridge to the architecture we just established:

$$
\boxed{
\text{fast stable fiber}
;\longrightarrow;
\text{slow memory base}
}
$$

rather than trying to prove everything by diagonalizing a questionable compressed Jacobian.

That is a real mathematical direction worth investigating.

5. The Razumikhin idea is surprisingly compatible with what we need

The attached discussion describes the Razumikhin approach as a way to prove stability of delay systems without constructing an entire infinite-dimensional Lyapunov-Krasovskii functional. It works with a finite-dimensional (V(x(t))), under a condition controlling the recent history.

That resonates with our situation because we do not want to prove a gigantic spectral theorem over the entire delay state just to establish that the observed branch is stable.

We potentially want something more like:

$$
V(\text{relative phase error},\text{coherence error})
$$

with a condition on the recent delay history.

Then establish something like

$$
V_{n+1}-V_n
\le
-\gamma V_n
+
\text{controlled EMA forcing}.
$$

If the forcing itself obeys the exact EMA recurrence, we could potentially derive the asymptotic envelope without ever claiming that the seven near-(\alpha) numerical eigenvectors represent seven physical modes.

That is a much more attractive proof strategy.
6. This reinforces why "4-dimensional slow bundle" is not yet the thing to worship

There's another striking match.

The older attached AIEF material explicitly says:

"Do not expect universal rank one. The correct generalization is an invariant biorthogonal subspace..."

And later warns:

"Do not infer a torus, slow manifold, or attractor dimension from weight-tangent POD."

That is exactly where our conversation landed after critically examining the augmented-spectrum work.

So the attached document actually reinforces our retreat from the claim:

[
\text{7 numerical eigenvectors}
\Rightarrow
\text{4 physical slow dimensions}.
]

The safer statement is:

[
\boxed{
\text{there is evidence for a C-dominated slow spectral cluster}
}
]

and then separately ask whether it corresponds to an invariant geometric bundle.

The attachment's recommendation to test biorthogonal invariant subspaces directly is much better than interpreting the raw SVD rank of nearly degenerate eigenvectors.
7. The matched-phase memory ablation is almost exactly what our latest pivot work wants

This was one of the most interesting things I noticed.

The attachment recommends completing:

text

B_carrier_only
full_reset

using identical randomized phase/history resets.

That is almost exactly the experimental-control principle we're now moving toward.

Our pivot study found:

$$
\text{reset_B}\approx0
$$

while EMA/reset and future-shuffle interventions matter.

But there is a possible confound:

Did the interventions alter the phase/history distribution differently?

A matched-phase/history ablation is the clean way to isolate the variable.

That connects directly to the modular-driver architecture we've been building.

You could implement a family of intervention drivers:

$$
\begin{array}{c}
\text{reset B only} \\
\text{reset EMA only} \\
\text{reset phase/history only} \\
\text{reset B+EMA} \\
\text{reset phase/history+B}
\end{array}
$$

using the exact same randomized perturbations wherever appropriate.

Then the interaction effects become much more interpretable.

That is probably more scientifically valuable than another eigenvector decomposition.

8. The attachment even gives us a warning about the next T_window study

It says, very explicitly:

don't reuse the original split state under a changed EMA parameter; regenerate the complete trajectory state, including phase prefix, delay history, complex EMA, weights, and split state.

This is important.

Our current T_window sensitivity pipeline deliberately studies a defined intervention protocol, and the baseline equivalence is excellent.

But this attached recommendation highlights a distinction we need to keep explicit:
Question A — intervention sensitivity

"Given the S01 split state, what happens if we alter the intervention/EMA condition?"

versus
Question B — parameterized dynamical system sensitivity

"If we had actually run the entire system with a different ($T_{\rm window}$), what trajectory would it naturally generate?"

Those are different estimands.

The current modular T_window study can legitimately answer A.

It cannot automatically be interpreted as B unless the engine regenerates the upstream state consistently with the changed parameter.

That's an important conceptual boundary.
9. The best unexpected connection: the "map-territory gap"

This part of the attachment was philosophical, but I think it has direct relevance to the project.

The conversation repeatedly emphasizes that models such as:

photon as a ball,
electron as a pendulum,
band as a highway,
phase as a clock,

are maps, not literal descriptions of the underlying system.

We've had exactly the same problem in AIEF.

We've accumulated maps:

"EMA pole"
"slow bundle"
"phase synchronization manifold"
"B as fast slave"
"C as slow state"
"laser-like coherence"
"triangular cascade"

Some may be excellent approximations.

But the raw engine is the territory.

The new analysis architecture you've built is valuable precisely because it makes us less dependent on any particular map.
What I think is actually worth carrying forward

I would extract five nuggets from this otherwise unrelated conversation:
1. Stable phase relationships, not "being in phase"

This gives a cleaner language for the K3:

[
\boxed{\text{coherence}=\text{persistent relational phase structure}}
]

rather than "alignment means phases become equal."
2. Fast local dynamics + slow memory can produce slow collective behavior

This fits the empirical

[
0.69 \gg 0.057
]

timescale separation remarkably well.
3. Delayed-feedback stability may be better attacked through inequalities than eigenvectors

A Razumikhin/Lyapunov-style comparison argument could be a promising route toward the theorem you actually want.
4. Matched interventions are more valuable than more observational geometry

The B_carrier_only / full_reset recommendation fits beautifully with the modular-driver architecture and could separate memory channels experimentally.
5. Don't confuse a useful representation with the underlying invariant object

This is probably the most important methodological lesson.

The attached material itself says the full geometry work should wait until global phase symmetry, circular variables, delay consistency, scaling, and stable-fiber versus tangent directions are handled properly.

That is almost exactly where we ended up.
So my strongest synthesis is

I wouldn't graft the attached document's physical model onto AIEF.

But I would steal its mathematical questions:

[
\boxed{
\begin{aligned}
&\text{What is the fast contracting fiber?}
&\text{What is the slowly evolving coherent base?}

&\text{How much delayed feedback can the fast fiber tolerate?}
&\text{Can stability be proved by a comparison functional rather than fragile spectrum?}
&\text{Which intervention uniquely destroys the slow memory?}
\end{aligned}}
]

Those questions line up remarkably well with what we've already uncovered.

And the last one may be the most immediately actionable: the next generation of modular drivers could experimentally separate phase/history memory, EMA memory, and B memory with matched perturbations. That would test the architecture directly rather than asking another compressed Jacobian to tell us what the architecture is.

There is a valuable mathematical core here, but the thought contains two major factual mistakes about completed work and several places where analogy becomes stronger than the evidence.
Two important corrections
1. The current TwindowTwindow study answers Question B, not Question A

This section is incorrect:

The current modular T_window study can legitimately answer A. It cannot automatically be interpreted as B unless the engine regenerates the upstream state consistently.

The completed S01 study does regenerate the complete trajectory for every (seed,ρ)(seed,ρ):

text

initial theta
initial B
zero EMA
delay history
entire prefix
split state
future branches

No split state is shared across rho levels.

Thus S01 primarily answers:

If the entire system had evolved from initialization with a different TwindowTwindow, what prefix, mature split state, and future branch response would it generate?

That is Question B.

Within each regenerated condition, the four future branches then answer a nested intervention question: parameterized model regime+split-state branch intervention.
parameterized model regime+split-state branch intervention.

There is no current experiment that preserves the ρ=1ρ=1 split state and changes TwindowTwindow only at the split. Such a study would answer Question A, but it would be a different estimand and would require very careful state-compatibility semantics.
2. The matched-phase ablation is already complete

The thought treats matched B_carrier_only versus full_reset phase/history intervention as a proposed next experiment. It has already been completed across all 40 seeds with two matched reset realizations per seed.

The saved conclusion was:

mature BB improves recovery of prior continuous-weight organization when phase/history and EMA are reset;
it increases sustained similarity-recovery probability;
it suppresses emergence and produces somewhat sparser/lower-weight networks;
it does not clearly increase selected-edge retention;
reset-realization interactions were small;
BB is a conditional scaffold, not a unique memory carrier.

Therefore, another broad matched-phase rerun is not the immediate next step.
Coherence and cancellation

The conceptual correction is good:
coherence≠equal phase values.
coherence =equal phase values.

A better description is:
\boxed{ \text{coherence} =

\text{persistent, predictable relational phase structure}
}
]

That fits the rotating-frame lock:
ωi+Gi≈Ω,
ωi+Gi≈Ω,

rather than an incorrect claim that phases or corrected frequencies approach zero.

The cancellation interpretation also has value:
T1/T2→−1
T1/T2→−1

does not indicate disorganization. In the saved exceptional-K3 decomposition, it indicates increasingly precise cancellation between two resolved terms.

But one qualification is essential:

T1T1 and T2T2 are terms in a chosen mathematical decomposition, not physical wave channels.

The ratio T1/T2→−1T1/T2→−1 alone does not prove coherence. Its significance comes from the larger finite-replay chain:

rotating frequency lock;
stable resonance ordering;
fixed winner map;
exact identity T1+T2=eT1+T2=e;
shrinking cancellation residual;
consistent forcing sign;
inheritance by ΔMΔM, ΔRΔR, and proposal differences.

So the safe statement is:

The near-−1−1 ratio is a signature of highly organized cancellation within the validated decomposition, not evidence of lost coherence.

Laser analogy

The laser analogy is acceptable as an informal systems-level metaphor:

text

local events
→ selection
→ feedback
→ organized collective behavior
→ persistence

But it should remain outside theorem statements and primary scientific claims.

AIEF does not contain:

stimulated emission;
cavity modes;
photons;
population inversion;
optical gain.

The reusable abstraction is simply:

Feedback and memory can stabilize a collective relational mode generated by repeated local interactions.

Even that should be presented as a general dynamical motif, not evidence that the two mechanisms are mathematically equivalent.
Timescale separation: promising but currently overstated

The ratio
$0.690.0571 \approx 12.1$
$0.05710.69 \approx 12.1$

is suggestive, but the numerator and denominator do not have equal evidential status.
$rEMA \approx 0.0571rEMA \approx 0.0571$

This is a finite-replay late decay rate associated with the EMA envelope.
μphase≈0.69μphase≈0.69

This came from a frozen/quasi-static weighted-Laplacian surrogate. It is not the certified contraction rate of the complete delayed tangent system.

Therefore, do not state:

We have direct evidence for a 12.1-fold full-system fast/slow spectral gap.

Use:

A frozen phase-locking surrogate is substantially faster than the observed late EMA envelope, providing finite-horizon evidence consistent with fast/slow organization. A uniform contraction rate for the complete delayed system has not been established.

There are also other relevant rates:

fixed-left proposal mode intrinsic rate ∼0.126∼0.126;
alignment residual rates ∼0.105∼0.105–0.1080.108;
EMA rate ∼0.0571∼0.0571;
right proposal mode much faster.

So the system does not reduce to one universal fast rate and one universal slow rate.
Slow base / fast fiber language

This is a useful proposed architecture, but it is not yet an exact skew product.

The EMA state is not an autonomous slow base because:
Cn+1=
Cn+1=

\alpha C_n
+
(1-\alpha)z(\theta_n,\theta_{n-\tau}),
]

and phase dynamics depend on BB, while BB depends on R=|C|R=|C|.

Thus:
θ/history↔C↔B
θ/history↔C↔B

remain coupled.

The safe language is:

There is evidence for a candidate slow memory coordinate and faster conditionally tracking directions.

Not yet:

The system has a proved slow base and invariant fast fiber.

A graph-transform or input-to-state-stability argument would be needed to promote the candidate architecture to an invariant bundle.
Razumikhin/Halanay direction

This is the strongest new mathematical suggestion.

But because the implementation is a discrete delayed map, the first theorem should target that map rather than an idealized continuous DDE.

A plausible target inequality is:
Vn+1≤(1−a dt)Vn+b dtmaxn−m≤k≤nVk+LC‖ΔCn‖,
Vn+1≤(1−adt)Vn+bdtn−m≤k≤nmaxVk+LC‖ΔCn‖,

where:

VnVn measures quotient phase/history and fast adaptive error;
m=τ/dtm=τ/dt;
aa is instantaneous/local contraction;
bb bounds delayed feedback gain;
LC‖ΔCn‖LC‖ΔCn‖ is slow-memory forcing.

If:
$a>b>0$,
$a>b>0$,

a discrete Halanay/Razumikhin argument may establish exponential tracking or input-to-state stability.
If the forcing then obeys:
$\|\Delta Cn\| \sim A\alpha n$,
$\|\Delta Cn\| \sim A\alpha n$,

one may derive an inherited αα-envelope for the tracked observables.

This is much more defensible than identifying individual eigenvectors in a near-degenerate compressed spectrum.
Obstacles that must remain explicit

A proof must address:

rotational common-phase symmetry;
the full delay buffer;
CC-to-BB-to-phase feedback;
positivity/support boundaries;
global scaling;
fixed winner-map assumptions;
one-sided branch differentiation;
the approaching triple controller tie;
possible loss of a uniform winner margin;
the distinction between local/conditional and global stability.

Because the exceptional K3 approaches a tie boundary, a uniform infinite-time branch margin may not exist. The first result may need to be:

local;
one-sided;
conditional on branch invariance;
finite-horizon or nonuniform.

Relation to the spectral retreat

The thought is correct here.

The exact controller-fiber and two-channel results are more reliable than interpreting nearly degenerate numerical eigenvectors.

Safe statement:

A selected-coordinate response map exhibits a provisional CC-dominated near-αα cluster. Its relationship to a full invariant slow bundle is unvalidated.

Do not infer:
7 compressed eigenvectors⇒7 physical slow modes
7 compressed eigenvectors⇒7 physical slow modes

or:
eigenvector-stack SVD rank⇒attractor/manifold dimension.
eigenvector-stack SVD rank⇒attractor/manifold dimension.

A direct invariant-subspace, tracking, or comparison-functional proof is the better direction.
Memory-channel experiments

The proposed intervention family substantially overlaps experiments already completed:

text

reset EMA
reset B
reset B + EMA
preserve B, reset EMA and phase/history
zero B, reset EMA and phase/history
future shuffle
matched phase/history with preserved versus zero B

Existing evidence already establishes:

reset_B reconstructs mature topology when EMA and phase/history are retained;
future edge-specific identity is required for maintenance;
mature BB helps as a scaffold when other mature channels are reset;

no single variable is the unique register.

A future modular intervention study should not simply repeat these branches. A worthwhile extension would need a new estimand, such as:

graded reset amplitudes;
matched perturbation directions;
controlled finite-duration channel forcing;
recovery response surfaces;
fixed perturbation energy across channels.

Use the existing 40 paired seeds. Do not introduce another broad cohort before exhausting the regenerated parameter and existing trajectory analyses.
Map–territory correction

The methodological warning is good, but this sentence needs refinement:

The raw engine is the territory.

The engine is still a model.

A better hierarchy is:

text

implemented engine
= territory for implementation audits

analysis reductions
= maps of the implemented engine

physical interpretation
= a further map from model to proposed real mechanism

For exact implementation identities, the engine is the object being studied. For physical claims, neither the engine nor its reductions are literal territory.

This distinction helps keep separate:

exact implementation identities;
saved finite replay;
controlled counterfactual evidence;
conditional mathematical lemmas;
open physical/general claims.

What should actually be carried forward
Strong and usable now

Coherence as persistent relational structure.
Structured cancellation can coexist with coherence.
Fast/slow tracking is a better candidate architecture than isolated eigenmodes.
A discrete Razumikhin/Halanay or ISS inequality is a promising proof route.
Invariant/fiber claims should be tested directly.
Representations must not be mistaken for invariant objects.

Keep only as analogy

Laser feedback.
Fast transduction versus global coherence.
Scalar delayed-feedback margin intuition.

Reject as outdated or incorrect

Matched-phase ablation as unfinished.
Current TwindowTwindow study as split-state parameter intervention.
A direct 12.1-fold certified full-system spectral gap.
reset_B as evidence that BB has no role.
The engine as literal physical territory.

Best next mathematical task

A productive proof program would be:

Define quotient phase/history error on the fixed exceptional-K3 branch.
Derive the exact discrete delayed error recurrence.

Bound instantaneous and delayed terms in a weighted norm.
Treat EMA/proposal differences as an input.
Establish a conditional Halanay/Razumikhin inequality.
Prove input-to-state tracking of fast variables.
Insert the exact radial-filter recurrence.
Derive the inherited αα-rate conditionally.
State explicitly where branch-margin/tie-boundary control is still missing.

That uses the validated implementation identities and finite-replay observations without relying on the quarantined compressed-spectrum claims.
Bottom line

The thought contains a genuinely useful synthesis:
organized relational phase structure+feedback+slow memory⇒candidate fast-tracking/slow-envelope architecture
organized relational phase structure+feedback+slow memory⇒candidate fast-tracking/slow-envelope architecture

But it must be corrected for the already completed matched-phase experiment and for the fact that S01 fully regenerates every TwindowTwindow trajectory. The Razumikhin/Halanay direction is the most promising genuinely new mathematical idea in it.

What is the new modular systems role in this:

A productive proof program would be:

Define quotient phase/history error on the fixed exceptional-K3 branch.
Derive the exact discrete delayed error recurrence.
Bound instantaneous and delayed terms in a weighted norm.
Treat EMA/proposal differences as an input.
Establish a conditional Halanay/Razumikhin inequality.
Prove input-to-state tracking of fast variables.
Insert the exact radial-filter recurrence.
Derive the inherited αα-rate conditionally.
State explicitly where branch-margin/tie-boundary control is still missing.

The modular system’s role is best understood as:

a proof-support laboratory and evidence ledger, not the proof itself.

It can make assumptions explicit, generate controlled counterexamples, verify exact implementation identities, and test proposed inequalities over regenerated trajectories. It cannot turn finite numerical success into an infinite-time theorem.
Role by proof step
Proof task Modular system contribution What remains mathematical
1. Define quotient error Reproducibly extract phase/history states and compute gauge-fixed errors Prove the quotient is well-defined and appropriate
2. Derive exact recurrence Replay one engine step and verify the derived recurrence to machine precision Derive the recurrence symbolically
3. Bound delayed terms Collect every term needed for candidate norm inequalities; search for violations Establish uniform analytic bounds
4. Treat EMA as input Save homogeneous/forcing decompositions and controlled input-response experiments Justify the input/state decomposition for the original coupled system
5. Halanay inequality Check proposed inequalities at every saved/tested state and produce counterexamples Prove inequality over the required state domain
6. ISS tracking Run deterministic matched perturbations and estimate finite-horizon gains Prove the ISS estimate and invariant tracking domain
7. Radial recurrence Verify exact recurrence identities across all generated conditions Connect the recurrence to the larger delayed system
8. Inherited αα-rate Test transfer-gain predictions and normalized envelopes Prove asymptotic hypotheses and nonzero limiting amplitude
9. Branch margins Monitor support, gate, controller map, and margins at every step Prove branch invariance or state where it fails
1. Quotient phase/history error

The modular platform can provide a dedicated proof-support collector that computes a rotationally invariant error.

For example, on the exceptional K3 nodes $K=\{2,7,11\}K=\{2,7,11\}$, define relative phase coordinates rather than raw phases:
$\psi n=(\theta 2n-\theta 11n\theta 7n-\theta 11n).$
$\psi n=(\theta 2n-\theta 11n\theta 7n-\theta 11n).$

Because the system has delay, the state needed by a proof is not only $\psi n\psi n$, but the relative-history buffer:
$\Psi n=(\psi n,\psi n-1,\ldots,\psi n-m),m=\tau/dt=50.$
$\Psi n=(\psi n,\psi n-1,\ldots,\psi n-m),m=\tau/dt=50.$

A modular collector can save:

text

quotient phase error
quotient delay-history error
phase-mean/common-phase coordinate
K3 external forcing terms
B subblock error
C subblock error
proposal differences
controller margins

It should explicitly verify invariance under common phase rotation numerically:
$\theta_k \mapsto \theta_k + \chi$
$\theta_k \mapsto \theta_k + \chi$

must leave quotient observables unchanged.

But the mathematical work still has to prove that this quotient coordinate and chosen history norm are sufficient for the theorem.
2. Exact delayed recurrence

After deriving a candidate recurrence on paper, add a collector that evaluates both sides from the same replay state.

Conceptually:
e_{n+1}observedversus$\Phi(e_n, e_{n-1}, \ldots, e_{n-m}, \Delta C_n, \Delta B_n)$.
e_{n+1}observedversus$\Phi(e_n, e_{n-1}, \ldots, e_{n-m}, \Delta C_n, \Delta B_n)$.

Output:

text

observed next error
predicted next error
identity residual
individual instantaneous terms
individual delayed terms

memory forcing
external-node forcing

This is the same successful strategy used for:

radial recurrence;
proposal equalization;
phase-balance identities;
T1+T2=eT1+T2=e;
symmetrized resonance identities.

A residual near machine precision establishes an implementation identity for the tested step map. It does not prove stability.
3. Candidate weighted norm and delayed gain

The modular system can test a candidate Lyapunov/Razumikhin quantity such as:
Vn=
Vn=

|\Psi_n|_P
+
c_B|\Delta B_n|
+
c_R|\Delta C_n|.
]

Or, for a fast subsystem driven by slow memory:
Vnfast=
Vnfast=

|\Psi_n|_P
+
c_B|\Delta B_n|.
]

A collector can calculate candidate inequality terms:
Vn+1,Vn,maxn−m≤k≤nVk,‖ΔCn‖.
Vn+1,Vn,n−m≤k≤nmaxVk,‖ΔCn‖.

Then test:
$V_{n+1} \le (1 - a\,dt) V_n + b\,dt \max_{n-m \le k \le n} V_k + LC\|\Delta C_n\|$.
$V_{n+1} \le (1 - adt) V_n + bdt \max_{n-m \le k \le n} V_k + LC\|\Delta C_n\|$.

The modular platform should treat any violation as a falsification of the proposed constants.

A finite trajectory with no violations is only a finite audit. A theorem still requires analytic or interval-certified bounds over the intended domain.
4. Input decomposition

The proposed “slow memory input” interpretation must not silently turn the fully coupled system into an exact triangular skew product.

The platform can help distinguish:
Original-system decomposition

Use exact saved terms from the coupled dynamics:

text

fast state
EMA input term
B feedback term
delay-history term
external-node term

This preserves the original system.
Auxiliary forced dynamics

Experiments such as:

text

hold C fixed
inject prescribed C forcing
remove B-to-phase feedback
freeze external nodes

change the dynamics. They must be labelled:

text

auxiliary comparison model

not an intervention within the original model.

The modular specification should encode:

Python

evidence_class="auxiliary_forced_dynamics"

and require a new model/operator version if engine hooks change.
5. Conditional Halanay/Razumikhin audit

A proof-support validator could accept candidate constants:

Python

HalanayCandidate(
instantaneous_rate=a,
delayed_gain=b,
memory_gain=L_C,
norm_weights=P,
history_length=50,
)

For every replay step, it would save:

text

left side
right side
slack = right - left
minimum slack

first violating step
branch state
controller margins
support pattern

The key mathematical condition might be:
$a>b>0$.
$a>b>0$.

But the modular output must not label this “proved” based on floating-point samples. Appropriate statuses are:

text

falsified_on_tested_state
passed_finite_replay_audit
interval_certified_on_box
analytically_proved

Only the last two can support a theorem, depending on the certification method.
6. Input-to-state tracking perturbations

The modular architecture is particularly useful for deterministic matched perturbations.

A proof-support study could use:

the exceptional seed 461912217115876852446191221711587685 24;
selected saved K3 states;
sum-zero phase perturbations;
delay-history-consistent perturbations;
matched $\pm v \pm v$ directions;
a geometric amplitude grid;
the same perturbation realization across comparison branches.

Possible conditions:

text

perturb current relative phase only
perturb the entire history buffer coherently
perturb B along controller-fiber direction
perturb EMA radial difference
combined matched perturbations

Outputs:
‖en‖‖e0‖,supn‖en‖‖e0‖,‖en‖supk≤n‖ΔCk‖,
‖e0‖‖en‖,‖e0‖supn‖en‖,supk≤n‖ΔCk‖‖en‖,

plus branch and margin diagnostics.

These experiments can:

falsify a proposed tracking domain;
estimate useful candidate gains;
reveal transient amplification;
detect branch switching.

They do not prove ISS by themselves.
7. Radial-filter integration

The exact radial recurrence already exists:
ΔMn+1=
ΔMn+1=

\alpha\Delta M_n
+
\alpha(1-\alpha)e_n.
]

The modular proof collector should link the fast tracking audit to the radial recurrence without merging their evidential status.

For each step save:

text

fast tracking error
alignment forcing e_n
Delta M_n
Delta R_n
proposal difference d_n
predicted transfer gain
observed transfer gain

Then audit the finite chain:
fast tracking $\rightarrow$ en $\rightarrow$ ΔMn $\rightarrow$ ΔRn $\rightarrow$ dn.
fast tracking $\rightarrow$ en $\rightarrow$ ΔMn $\rightarrow$ ΔRn $\rightarrow$ dn.

The recurrence identity may be exact while the decay assumptions on enen remain conditional.
8. Conditional inheritance of the EMA rate

The platform can test the hypotheses needed by the radial-filter lemma:
$\sigma\Delta Mn0>0,\sigma en\geq 0,$
$\sigma\Delta Mn0>0,\sigma en\geq 0,$

and finite-horizon analogues of:
$\sum n|en|\alpha n<\infty.$
$n\sum\alpha n|en|<\infty.$

Useful diagnostics include:

text

sign changes
minimum signed forcing
normalized forcing e_n / alpha^n
partial weighted sums
Delta M_n / alpha^n
Delta R_n / alpha^n
d_n / alpha^n
estimated transfer gain

But finite partial sums do not prove infinite summability.

The resulting statement must remain:

Conditional on continued branch invariance, sign persistence, and weighted forcing summability, the radial and proposal differences inherit the EMA envelope.

9. Branch-margin and tie-boundary control

This is where the modular system is most valuable as a falsification apparatus.

Every proof-support condition should record:

text

positive support
active gate
controller winner map
all controller margins
minimum winner margin
distance to positivity boundary
distance to global-capacity boundary
K3 isolation error

A perturbation condition should be marked invalid for the fixed-branch theorem immediately after:

text

winner-map change
support change
gate change
controller tie
loss of K3 isolation

Do not continue using the fixed branch recurrence after that event.

Because the exceptional trajectory approaches a triple tie, use margin-adaptive perturbation amplitudes:
$\varepsilon_n < c\,m_n$,
$\varepsilon_n < c m_n$,

where mnmn is the current minimum branch margin.

A uniform perturbation radius may not exist asymptotically.
Recommended modular extension

Do not put this into the ordinary S01 experiment registry as another inferential seed study. Create a separate proof-support category.

For example:

text

AIEF_1_4_P01_exceptional_K3_tracking

with manifest fields:

JSON

```
{
"evidence_class": "proof_support_falsification",
"inferential_scope": "single exceptional saved component",
"seed": 4619122171158768524,
"replicate_claim": false,
"theorem_status": "conditional",
"branch_assumptions": [
"fixed positive support",
"fixed gate",
"fixed controller map",
"K3 isolation"
]
}
```

Its outputs should not be pooled with the 40-seed S01 statistical study.
Engine requirements

The current modular runner supports only the existing four S01 branches. Arbitrary phase/history/EMA perturbations are not yet legitimate plugins under Engine v1.

Therefore:
First phase: offline replay analysis

Use existing V4 trajectories and saved states to:

define quotient errors;
derive recurrences;
test candidate inequalities;
identify required intervention hooks.

No engine change is required.
Second phase: versioned proof-support kernel

If controlled perturbations require new hooks, create:

text

aief_local_replay_kernel_v1

or Engine v2 proof-support APIs.

Do not modify certified Engine v1 in place.

The new kernel must preserve the original step map when no perturbation is applied and pass a no-perturbation oracle test against Engine v1.
Suggested work order
Mathematical phase

Define the exact quotient state and norm.
Derive the discrete delayed recurrence.
Identify every delayed, instantaneous, memory, and external forcing term.
State the branch assumptions.
Propose candidate inequality constants.

Existing-data audit

Implement a read-only collector over existing V4 artifacts.

Verify the recurrence identity.
Search for finite-replay inequality violations.
Audit branch margins and determine admissible perturbation scales.

Controlled falsification

Add a versioned local perturbation kernel only if necessary.
Run deterministic matched ±± perturbations.
Test gain, decay, and branch invariance.
Refine or falsify the proposed constants.

Proof

Replace empirical extrema with analytic or interval bounds.
Prove the conditional delayed tracking inequality.
Compose it with the exact radial-filter lemma.
State explicitly what remains open at the tie boundary.

Bottom line

The modular system contributes:
reproducible state extraction+controlled perturbations+exact identity audits+systematic counterexample search+branch-assumption tracking
reproducible state extraction+controlled perturbations+exact identity audits+systematic counterexample search+branch-assumption tracking

The proof still contributes:
uniform bounds+domain control+infinite-time argument+tie-boundary treatment
uniform bounds+domain control+infinite-time argument+tie-boundary treatment

The modular platform should be used to make the theorem precise and difficult to fool—not to replace the theorem with more numerical evidence.

Would it be appropriate to integrate Lean Proofs into our workflow?

Yes—targeted Lean integration would be appropriate and valuable, especially for the exact controller-fiber algebra and conditional recurrence lemmas.

It would not be appropriate to ask Lean to certify the entire NumPy simulation or infer scientific conclusions from finite runs.

The clean workflow would have three independent layers:
Lean theorem proofs+Python implementation audits+modular experiments
Lean theorem proofs+Python implementation audits+modular experiments

Each answers a different question.
What Lean should prove
1. Controller-fiber algebra

This is the best starting point because it is finite-dimensional and mostly exact algebra.

Candidate theorems:
ker(110101)=
ker(111001)=

\operatorname{span}{(1,-1,-1)}.
]

For the selected K3 block:
$Kv=\lambda v \iff s2=s7.$
$Kv=\lambda v \iff s2=s7.$

For the fixed left covector:
$u*TK=\lambda u*T \iff \beta25=\beta59.$
$u*TK=\lambda u*T \iff \beta25=\beta59.$

Also:
m59=m25+m29,
m59=m25+m29,

and the proposal-difference identities:
s2−s7=
s2−s7=

c\frac{x_{59}-x_{29}}
{(q_2+\varepsilon)(q_7+\varepsilon)},

]
β25−β59=
β25−β59=

\frac{s_7}{q_7+\varepsilon}
(x_{25}-x_{59}).
]

These claims are exact and largely independent of numerical trajectories.
2. Controller-fiber theorem package

Lean could formalize:

lineality of the relevant polyhedral branch;
affine dependence on multiplier-controller fibers;
kernel invariance;
the intertwining factorization;
fixed versus moving dual closure;
the tied-controller kernel becoming trivial;
one-sided branch interpretation.

The one-sided boundary statement must remain conditional on the selected branch. Lean should not silently assert differentiability of the complete nonsmooth global map.
3. Proposal-equalization recurrence

Formalize:
dn+1=
dn+1=

\lambda_n d_n
+
dt(\eta+\mu)r_{n+1},
\qquad
0<\lambda_n<1-\gamma dt<1.
]

Useful initial results:

positivity/sign persistence under nonnegative forcing;
variation-of-constants formula;
geometric convolution bounds;
constant-coefficient transfer gain;
convergence under $\lambda_n \to \lambda_*$ $\lambda_n \to \lambda_*$ and suitable forcing assumptions.

4. Radial-filter lemma

The existing conditional lemma is an excellent Lean target:
$\Delta M_{n+1} =$
$\Delta M_{n+1} =$

$$
\alpha\Delta M_n
+
\alpha(1-\alpha)e_n.
$$

Under:
$\sigma_{\Delta M_{n_0}} > 0, \sigma_{e_n} \geq 0, \sum_{n=n_0}^{\infty} |e_n| \alpha_n < \infty,$
$\sigma_{\Delta M_{n_0}} > 0, \sigma_{e_n} \geq 0, n=n_0 \sum_{\infty} \alpha_n |e_n| < \infty,$

prove:
$\sigma_{\Delta M_n} > 0,$
$\sigma_{\Delta M_n} > 0,$
$\Delta M_n \alpha_n \to A, \sigma_A > 0,$
$\alpha_n \Delta M_n \to A, \sigma_A > 0,$

and:
$\Delta M_n = A\alpha_n + o(\alpha_n).$
$\Delta M_n = A\alpha_n + o(\alpha_n).$

This would sharply separate:

the exact conditional theorem;
the finite V4 evidence for its assumptions;
the still-open infinite-time continuation.

5. Discrete Halanay/Razumikhin theorem

After the exact delayed error recurrence is derived, Lean could prove a general theorem of the form:
$V_{n+1} \le (1 - a\,dt) V_n + b\,dt \max_{n-m \le k \le n} V_k + u_n.$
$V_{n+1} \le (1 - adt) V_n + bdt \max_{n-m \le k \le n} V_k + u_n.$

Under an appropriate contraction condition such as:
$a > b > 0,$
$a > b > 0,$

derive a tracking bound driven by u_n.

This should first be a generic sequence theorem. Only afterward should it be instantiated with AIEF quantities.
What Lean should not initially attempt

Do not begin with:

the complete 12-node delayed nonlinear system;
the full floating-point NumPy implementation;
the global nonsmooth admissibility map;
global branch-cone invariance;
the complete delayed-state spectrum;
invariant-manifold dimension;
KAM, torus, or attractor claims;
bootstrap/statistical inference;
physical interpretations of the model.

Those targets are either too large, not yet precisely stated, or not presently justified.
Real-valued theorem versus floating implementation

Lean should initially formalize an exact mathematical model over:

lean

$\mathbb{R}$

or rational/algebraic finite matrices where possible.

For example:

lean

dt = 1 / 100

can be represented exactly as a real rational value.

But the Python engine uses IEEE floating-point operations and NumPy transcendental functions. Therefore:

A Lean proof of the real-valued recurrence is not automatically a proof that NumPy implements it bit-for-bit.

The bridge remains:

Lean proves the mathematical implication.
Python verifies that the implemented finite-step identity matches the formal recurrence on tested states.
The modular runner records the implementation and proof dependencies.
Interval or floating-point formalization may later strengthen the bridge.

Recommended project layout

text

lean/
lean-toolchain
lakefile.toml
AIEF/
Basic.lean
ControllerFiber.lean
ExceptionalK3.lean
ProposalEqualization.lean
RadialFilter.lean
DiscreteHalanay.lean
Assumptions.lean
ClaimRegistry.lean

Suggested module roles:
Basic.lean

finite vectors and matrices;
scalar parameter assumptions;
helper lemmas;
normalization conventions.

ControllerFiber.lean

controller matrices;
kernels;
affine fibers;
intertwining identities.

ExceptionalK3.lean

selected K3 block;
right and left closure;
pointwise covectors;
tied-controller limit.

ProposalEqualization.lean

exact recurrence;
positivity;
convolution solution;
transfer gain.

RadialFilter.lean

sign persistence;
weighted summability;
normalized convergence;
asymptotic envelope.

DiscreteHalanay.lean

generic delayed sequence inequalities;
forced tracking estimates;
eventual geometric envelopes.

ClaimRegistry.lean

Map manuscript claim IDs to theorem names.
Proof-quality requirements

Every Lean contribution should satisfy:

text

no sorry
no admit
no unexplained axioms
pinned Lean toolchain
pinned Mathlib revision
lake build passes
theorem assumptions documented

Use:

lean

set_option autoImplicit false

and audit important theorems with:

lean

#print axioms theorem_name

The report should distinguish ordinary foundational axioms used by Mathlib from project-specific assumptions.

Do not allow the coding agent to “complete” proofs by introducing an axiom equivalent to the desired conclusion.

Integration with the modular runner

The experiment manifest can record formal dependencies:

JSON

```
{
"formal_proof_dependencies": [
{
"proof_id": "AIEF_CONTROLLER_FIBER_V1",
"theorem": "AIEF.ControllerFiber.kernel_eq_span",
"source_sha256": "...",
"lean_version": "...",
"mathlib_revision": "...",
"build_status": "passed",
"assumption_registry": [
"fixed_controller_branch",
"positive_denominators"
]
}
]
}
```

But keep proof status separate from experiment status:

text

```
experiment freezeable
formal theorem checked
theorem assumptions verified in finite replay
infinite-time assumptions open
```

A passing Lean build must not make an experimental assumption automatically true.

Best first pilot

Start with two small packages:

Pilot A — Exceptional K3 algebra

Formalize:

controller-kernel carrier;
$Kv=\lambda v \iff s2=s7Kv=\lambda v \Longleftrightarrow s2=s7$;
fixed-left closure;
tied-controller trivial kernel.

Pilot B — Radial-filter recurrence

Formalize:

exact normalized recurrence;
sign persistence;
summability-based limit;
asymptotic αα-envelope.

These are sufficiently precise, useful, and bounded in scope.

Only after those pass should you attempt the delayed Halanay/Razumikhin theorem.
Relationship to the modular proof-support workflow

The roles would be:
Layer Responsibility
Lean Prove conditional exact mathematical implications
Python identity audits Verify that implemented recurrences match formal equations on replayed states
Modular perturbation studies Search for violations of assumptions and candidate bounds
Evidence lineage Link proof source, engine source, artifacts, and assumptions
Manuscript State only the strongest claim supported by all relevant layers
Final recommendation

Integrate Lean, but as a small formal theorem track, not as a rewrite of the simulation project.

The highest-value sequence is:

freeze the current empirical evidence;
establish a Lean project with pinned dependencies;
formalize the exceptional-K3 algebra;

formalize the radial-filter lemma;
derive the exact delayed quotient recurrence on paper;
formalize a generic discrete Halanay theorem;
use the modular system to falsify or support the AIEF-specific assumptions;
attempt an AIEF instantiation only after the assumptions are precise.

Lean is particularly well suited to preventing algebraic drift and assumption slippage in this project. Its main value is not proving the numerical experiment “correct”; it is making the exact conditional theorem impossible to state ambiguously.

What and how should I download Lean on to my steam Deck?

Install Lean 4 using elan, its official user-level toolchain manager. Do not install Lean through pacman, and you do not need to disable SteamOS read-only mode. The official Lean quick start and manual recommend elan on Linux; it installs under your home directory and manages project-specific Lean versions 2 4.

Your Python sci-env is unrelated. Lean can coexist with it.
1. Check architecture, tools, and disk space

In Desktop Mode, open Konsole:

Bash

```
uname -m
command -v curl
command -v git
df -h "$HOME"
```

Expected architecture:

text

x86_64

Leave several gigabytes free. Mathlib plus its compiled cache can require substantial storage.

If either curl or git is missing, stop rather than altering the SteamOS root filesystem. We can then use Distrobox or another user-local approach.
2. Install elan into your home directory

Downloading the installer first lets you inspect it instead of piping directly into a shell:

Bash

```
curl \
--proto '=https' \
--tlsv1.2 \
-fsSL \
https://elan.lean-lang.org/elan-init.sh \
-o /tmp/elan-init.sh
```

Optional inspection:

Bash

```
less /tmp/elan-init.sh
```

Run it:

Bash

```
sh /tmp/elan-init.sh
```

Choose the default installation option when prompted. It should install under:

text

```
/home/deck/.elan/
```

Then activate it in the current terminal:

Bash

source "$HOME/.elan/env"

Ensure future interactive Bash shells load it:

Bash

```
grep -Fqx \
'source "$HOME/.elan/env"' \
"$HOME/.bashrc" \
|| printf '\nsource "$HOME/.elan/env"\n' \
>> "$HOME/.bashrc"
```

Remove the installer:

Bash

```
rm -f /tmp/elan-init.sh
```

3. Verify Lean, Lake, and Elan

Bash

```
command -v elan
command -v lean
command -v lake
```

```
elan --version
elan show
lean --version
lake --version
```

The executables should resolve under:

text

/home/deck/.elan/bin/

If the commands are not found:

Bash

```
source "$HOME/.elan/env"
export PATH="$HOME/.elan/bin:$PATH"
```

Do not run:

Bash

```
sudo pacman -S lean
sudo pacman -S elan
sudo steamos-readonly disable
```

A user-local elan installation avoids modifying the SteamOS system image.
4. Create a pinned AIEF Lean/Mathlib project

Create it inside the SSRH project, but separate from Python code:

Bash

```
cd /home/deck/AI_Programs/SSRH
```

```
mkdir -p lean
cd lean
```

Create a new project with a Mathlib dependency:

Bash

```
lake +leanprover/lean4:stable \
new AIEFProofs math
```

The Mathlib project documentation recommends the math template, followed by updating dependencies and downloading the compiled Mathlib cache 1.

Enter the project:

Bash

```
cd AIEFProofs
```

Resolve and pin dependencies:

Bash

```
lake update
```

Download Mathlib’s compiled cache rather than building all of Mathlib from source:

Bash

```
lake exe cache get
```

Then build:

Bash

```
lake build
```

If the cache download is interrupted, rerun:

Bash

```
lake exe cache get
```

Do not repeatedly run lake update after the initial setup. It can move Mathlib to a newer revision. The project should subsequently be controlled by:

text

lean-toolchain
lake-manifest.json
lakefile.toml or lakefile.lean

5. Create a smoke theorem

Inspect the generated project:

Bash

find . -maxdepth 2 -type f -print

Create:

Bash

cat > AIEFProofs/Smoke.lean <<'EOF'
import Mathlib

set_option autoImplicit false

namespace AIEFProofs

example (x y : ℝ) : x + y = y + x := by
ring

example (x : ℝ) : x - x = 0 := by
ring

end AIEFProofs
EOF

Compile that file:

Bash

```
lake env lean AIEFProofs/Smoke.lean
```

Build the project:

Bash

```
lake build
```

A clean result with no errors confirms that Lean and Mathlib work.
6. Record the exact Lean/Mathlib environment

Bash

```
printf '%s\n' '--- lean-toolchain ---'
cat lean-toolchain

printf '%s\n' '--- elan show ---'
elan show

printf '%s\n' '--- Lean version ---'
lean --version

printf '%s\n' '--- Lake version ---'
lake --version
```

Record lock-file hashes:

Bash

```
sha256sum \
lean-toolchain \
lake-manifest.json \
lakefile.* \
```

> AIEF_LEAN_PROJECT_LOCK_HASHES.txt

cat AIEF_LEAN_PROJECT_LOCK_HASHES.txt

If the SSRH directory is under Git, commit at least:

text

lean/AIEFProofs/lean-toolchain
lean/AIEFProofs/lake-manifest.json
lean/AIEFProofs/lakefile.*
lean/AIEFProofs/AIEFProofs/
lean/AIEFProofs/AIEF_LEAN_PROJECT_LOCK_HASHES.txt

Check Git status first:

Bash

git status

Do not blindly initialize a new Git repository inside an existing one.
7. Recommended initial source structure

Inside:

text

/home/deck/AI_Programs/SSRH/lean/AIEFProofs/

use:

text

AIEFProofs/
Basic.lean
ControllerFiber.lean

ExceptionalK3.lean
ProposalEqualization.lean
RadialFilter.lean
DiscreteHalanay.lean
Assumptions.lean
ClaimRegistry.lean

Create empty starting files:

Bash

```
touch \
AIEFProofs/Basic.lean \
AIEFProofs/ControllerFiber.lean \
AIEFProofs/ExceptionalK3.lean \
AIEFProofs/ProposalEqualization.lean \
AIEFProofs/RadialFilter.lean \
AIEFProofs/DiscreteHalanay.lean \
AIEFProofs/Assumptions.lean \
AIEFProofs/ClaimRegistry.lean
```

Do not attempt all of them immediately. The first pilot should be:

text

```
ControllerFiber.lean
RadialFilter.lean
```

8. Optional editor setup

Lean works from the command line, but the official Lean documentation recommends VS Code with the official leanprover.lean4 extension because of its interactive Infoview 2 4.

First see whether VS Code is already available:

Bash

```
command -v code || true
```

If it is:

Bash

code --install-extension leanprover.lean4

Open the directory containing lakefile.* and lean-toolchain:

Bash

cd /home/deck/AI_Programs/SSRH/lean/AIEFProofs
code .

If you use a Flatpak version of VS Code, install the Lean 4 extension through the Extensions panel. Flatpak sandboxing can occasionally complicate access to host-installed toolchains. If that happens, keep using:

Bash

lake build
lake env lean <file>

while addressing editor integration separately. Do not change the successful command-line Lean installation merely to satisfy the editor.
9. Proof-quality checks

For every formal source file use:

lean

set_option autoImplicit false

Never permit:

lean

sorry
admit
axiom desired_result : ...

Search before freezing:

Bash

```
grep -RInE \
'\bsorry\b|\badmit\b|sorryAx' \
AIEFProofs \
&& {
echo "Unfinished proof marker found"
exit 1
} || true
```

Build everything:

Bash

```
lake build
```

For important theorems, add temporary audits such as:

lean

```
#print axioms AIEFProofs.someTheorem
```

10. Back up the project

Your persistent project will be in your home directory:

text

```
/home/deck/AI_Programs/SSRH/lean/AIEFProofs
```

Keep an external or Git backup, especially before SteamOS updates or storage changes.
Final installation checklist

These commands must all succeed:

Bash

source "$HOME/.elan/env"

elan --version
lean --version
lake --version

cd /home/deck/AI_Programs/SSRH/lean/AIEFProofs

lake exe cache get
lake env lean AIEFProofs/Smoke.lean
lake build

Once that works, the next task should be to create a minimal ControllerFiber.lean formalization of the 2×32×3 controller matrix and prove its kernel is spanned by (1,−1,−1)(1,−1,−1).

Nothing seemed to happen after step 5:

text

(deck@steamdeck ~)$ uname -m
command -v curl
command -v git
df -h "$HOME"
x86_64
/usr/bin/curl
/usr/bin/git
Filesystem Size Used Avail Use% Mounted on
/dev/nvme0n1p8 227G 195G 32G 86% /home
(deck@steamdeck ~)$ curl \
--proto '=https' \

--tlsv1.2 \
-fsSL \
https://elan.lean-lang.org/elan-init.sh \
-o /tmp/elan-init.sh
(deck@steamdeck ~)$ sh /tmp/elan-init.sh
info: downloading installer

Welcome to Lean!

This will download and install Elan, a tool for managing different Lean versions used in packages you create or download. It will also install a default version of Lean and its package manager, lake, for editing files not belonging to any package.

It will add the lake, lean, and elan commands to Elan's bin directory, located at:

/home/deck/.elan/bin

This path will then be added to your PATH environment variable by modifying the profile files located at:

/home/deck/.profile /home/deck/.bash_profile

You can uninstall at any time with elan self uninstall and these changes will be reverted.

Current installation options:

* default toolchain: stable
* modify PATH variable: yes

1) Proceed with installation (default)
2) Customize installation
3) Cancel installation
y

info: aborting installation
'(deck@steamdeck ~)$ sh /tmp/elan-init.sh
info: downloading installer

Welcome to Lean!

This will download and install Elan, a tool for managing different Lean
versions used in packages you create or download. It will also install a
default version of Lean and its package manager, lake, for editing files not
belonging to any package.

It will add the lake, lean, and elan commands to Elan's bin directory, located
at:

/home/deck/.elan/bin

This path will then be added to your PATH environment variable by modifying the
profile files located at:

/home/deck/.profile /home/deck/.bash_profile

You can uninstall at any time with elan self uninstall and these changes will
be reverted.

Current installation options:

* default toolchain: stable
* modify PATH variable: yes

1) Proceed with installation (default)
2) Customize installation
3) Cancel installation
2

I'm going to ask you the value of each these installation options.
You may simply press the Enter key to leave unchanged.

Default toolchain? (stable/beta/nightly/<specific version>/none)

Modify PATH variable? (y/n)
y

Current installation options:

* default toolchain: stable
* modify PATH variable: yes

1) Proceed with installation (default)
2) Customize installation
3) Cancel installation
1

info: default toolchain set to 'stable'

Elan is installed now. Great!

To get started you need Elan's bin directory ($HOME/.elan/bin) in your PATH
environment variable. Next time you log in this will be done automatically.

To configure your current shell run source $HOME/.elan/env
(deck@steamdeck ~)$ source "$HOME/.elan/env"
(deck@steamdeck ~)$ grep -Fqx \
'source "$HOME/.elan/env"' \
"$HOME/.bashrc" \
|| printf '\nsource "$HOME/.elan/env"\n' \
>> "$HOME/.bashrc"
(deck@steamdeck ~)$ source "$HOME/.elan/env"
(deck@steamdeck ~)$ rm -f /tmp/elan-init.sh
(deck@steamdeck ~)$ command -v elan
command -v lean

```
command -v lake


elan --version
elan show
lean --version
lake --version
/home/deck/.elan/bin/elan
/home/deck/.elan/bin/lean
/home/deck/.elan/bin/lake
elan 4.2.3 (b6cec7e10 2026-06-08)
warning: could not canonicalize path: '/home/deck/.elan/toolchains'
warning: could not canonicalize path: '/home/deck/.elan/toolchains'
warning: could not canonicalize path: '/home/deck/.elan/toolchains'
leanprover/lean4:v4.33.1 (resolved from default 'stable')
(toolchain will be installed on first use)
warning: could not canonicalize path: '/home/deck/.elan/toolchains'
warning: could not canonicalize path: '/home/deck/.elan/toolchains'
warning: could not canonicalize path: '/home/deck/.elan/toolchains'
info: downloading https://releases.lean-lang.org/lean4/v4.33.1/lean-4.33.1-linux.tar.zst
544.0 MiB / 544.0 MiB (100 %) 51.9 MiB/s ETA: 0 s
info: installing /home/deck/.elan/toolchains/leanprover--lean4---v4.33.1
Lean (version 4.33.1, x86_64-unknown-linux-gnu, commit
819816b2e0a3bf405af45ae5c7af2491d8f5bee6, Release)
Lake version 5.0.0-src+819816b (Lean version 4.33.1)
(deck@steamdeck ~)$ cd /home/deck/AI_Programs/SSRH


mkdir -p lean
cd lean
(deck@steamdeck lean)$ lake +leanprover/lean4:stable \
new AIEFProofs math
info: AIEFProofs: no previous manifest, creating one from scratch
info: leanprover-community/mathlib: cloning https://github.com/leanprover-community/mathlib4
info: leanprover-community/mathlib: checking out revision
'0df444a360eaa60ab8c11dca51a86af692955474'
info: plausible: cloning https://github.com/leanprover-community/plausible
info: plausible: checking out revision 'b7eb3304aeae834b12dda98993a37f6a41f6f0bb'
info: LeanSearchClient: cloning https://github.com/leanprover-community/LeanSearchClient
info: LeanSearchClient: checking out revision '5f4d51b81cbd3f6b32b156bfad9056621a040404'
info: importGraph: cloning https://github.com/leanprover-community/import-graph
info: importGraph: checking out revision '16f02aa7642864af59f1ff0e384a015994db9118'
info: proofwidgets: cloning https://github.com/leanprover-community/ProofWidgets4
info: proofwidgets: checking out revision '4be2e3d5087eeb272cf5a8853b8f9dd025ef5957'
```

```
info: aesop: cloning https://github.com/leanprover-community/aesop
info: aesop: checking out revision '3448c0bcc5ce01b2d1546e483ec3620e32df3d0e'
info: Qq: cloning https://github.com/leanprover-community/quote4
info: Qq: checking out revision '92c15be17b7caf78c2ad767ec40f89052d908d81'
info: batteries: cloning https://github.com/leanprover-community/batteries
info: batteries: checking out revision '4488d40d070b9700d4d5a6aa342f0d40c31b2a2d'
info: Cli: cloning https://github.com/leanprover/lean4-cli
info: Cli: checking out revision '6130a47896ce867c6a4a55373441e59e565bad0f'
info: mathlib: running post-update hooks
Current branch: HEAD
Using cache from origin: (some leanprover-community/mathlib4)
Attempting to download 8690 file(s) from leanprover-community/mathlib4 cache at
https://lakecache.blob.core.windows.net/mathlib4-master
Downloaded: 8690 file(s) [attempted 8690/8690 = 100%, 50 KB/s], Decompressed: 2398
Decompressed 8690 file(s)
Already decompressed 8690 file(s)
(deck@steamdeck lean)$ cd AIEFProofs
(deck@steamdeck AIEFProofs)$ lake update
info: toolchain not updated; already up-to-date
info: mathlib: running post-update hooks
Current branch: HEAD
Using cache from origin: (some leanprover-community/mathlib4)
No files to download
Already decompressed 8690 file(s)
(deck@steamdeck AIEFProofs)$ ^C
(130)(deck@steamdeck AIEFProofs)$ find . -maxdepth 2 -type f -print
./lakefile.toml
./.git/description
./.git/HEAD
./.git/config
./lake-manifest.json
./AIEFProofs/Basic.lean
./lean-toolchain
./README.md
./.gitignore
./AIEFProofs.lean
(deck@steamdeck AIEFProofs)$ cat > AIEFProofs/Smoke.lean <<'EOF'
import Mathlib

set_option autoImplicit false

namespace AIEFProofs
```

```
example (x y : ℝ) : x + y = y + x := by
ring

example (x : ℝ) : x - x = 0 := by
ring

end AIEFProofs
EOF
(deck@steamdeck AIEFProofs)$ lake env lean AIEFProofs/Smoke.lean

Stopped
```

This is the current project Directory map

```
eck@steamdeck SSRH)$ ls -R
.:
AI_AGENT_HANDOFF_PROMPT.md AIEF_1_4_OPERATOR_THEORY_MANUSCRIPT.md
AIEF_1_3_analysis.py aief_1_4_parameter_free_triangular_closure
aief_1_3_closure_diagnostics AIEF_1_4_parameter_free_triangular_closure.py
AIEF_1_3_closure_diagnostics.py aief_1_4_phase_balance_source_dynamics
AIEF_1_3_closure_diagnostics_updated.py AIEF_1_4_phase_balance_source_dynamics.py
AIEF_1_3_falsification.py aief_1_4_phase_cancellation_audit
AIEF_1_3_falsification_updated.py aief_1_4_phase_source_decomposition
AIEF_1_3_paired_statistics.py aief_1_4_phase_source_decomposition.py
AIEF_1_3_paired_statistics_updated.py aief_1_4_phase_source_decomposition_v2.py
aief_1_4_active_boundary_analysis aief_1_4_phase_source_locking
AIEF_1_4_active_boundary_analysis.py AIEF_1_4_phase_source_locking_analysis.py
aief_1_4_actual_admissibility_branch aief_1_4_propagation_gap_geometric_control
AIEF_1_4_actual_admissibility_branch_audit.py AIEF_1_4_propagation_gap_geometric_control.py
aief_1_4_augmented_duhamel_feedback aief_1_4_proposal_equalization_bridge
AIEF_1_4_augmented_duhamel_feedback_analysis.py
AIEF_1_4_proposal_equalization_bridge_analysis.py
aief_1_4_augmented_spectrum
AIEF_1_4_PROPOSAL_EQUALIZATION_BRIDGE_INTERPRETATION.md
aief_1_4_augmented_spectrum.py aief_1_4_radial_observable_subsystem
aief_1_4_augmented_tangent_cocycle AIEF_1_4_radial_observable_subsystem_analysis.py
AIEF_1_4_augmented_tangent_cocycle_analysis.py aief_1_4_resonant_tracking_angular
aief_1_4_biorthogonal_triangular_closure AIEF_1_4_resonant_tracking_angular_analysis.py
AIEF_1_4_biorthogonal_triangular_closure.py aief_1_4_results_analysis
```

aief_1_4_channel_invariance AIEF_1_4_results_analysis.py
AIEF_1_4_channel_invariance_analysis.py aief_1_4_scalar_spectral_gap_reduction
aief_1_4_cocycle_controls_coupled AIEF_1_4_scalar_spectral_gap_reduction.py
AIEF_1_4_cocycle_controls_coupled_comparison.py aief_1_4_slow_bundle_geometry
aief_1_4_component_controller_audit aief_1_4_slow_bundle_geometry.py
AIEF_1_4_component_controller_audit.py aief_1_4_slow_eigenvector_analysis
AIEF_1_4_CONDITIONAL_PROPOSAL_EQUALIZATION_THEOREM.md
aief_1_4_slow_eigenvector_analysis.py
aief_1_4_controller_boundary_margin aief_1_4_slow_subspace_projection
aief_1_4_controller_boundary_margin.py aief_1_4_slow_subspace_projection.py
AIEF_1_4_CONTROLLER_FIBER_THEOREM_PACKAGE.md aief_1_4_stationarity_perturbation
aief_1_4_coupling_lock_audit AIEF_1_4_stationarity_perturbation_analysis_updated.py
aief_1_4_coupling_lock_audit.py AIEF_1_4_superseded_claims.csv
aief_1_4_defect_causal_ema_trace aief_1_4_survival_outlier_spectral
AIEF_1_4_defect_causal_ema_trace.py AIEF_1_4_survival_outlier_spectral.py
aief_1_4_defect_recurrence_analytic_mode aief_1_4_tangent_spectrum_switch
AIEF_1_4_defect_recurrence_analytic_mode.py AIEF_1_4_tangent_spectrum_switch_analysis.py
aief_1_4_duhamel_memory_alignment aief_1_4_trajectory_recovery
AIEF_1_4_duhamel_memory_alignment_analysis.py AIEF_1_4_trajectory_recovery_equivalence.py
aief_1_4_entropy_geometry aief_1_4_transverse_phase_jacobian
AIEF_1_4_entropy_geometry_diagnostics.py aief_1_4_transverse_phase_jacobian.py
aief_1_4_exceptional_K3_transition AIEF_1_4_Twindow_pivot_seed_contrasts.py
AIEF_1_4_exceptional_K3_transition_audit.py AIEF_1_4_Twindow_sensitivity_analysis_v3.py
aief_1_4_exceptional_tangent AIEF_1_4_Twindow_sensitivity.py
AIEF_1_4_exceptional_tangent_analysis.py aief_1_4_two_channel_alignment_v4
aief_1_4_existing_clique_invariant_subspaces aief_1_4_two_channel_alignment_v4.py
AIEF_1_4_existing_clique_invariant_subspaces.py aief_1_4_two_channel_alignment_v5
aief_1_4_face_spectrum aief_1_4_two_channel_alignment_v5.py
AIEF_1_4_face_spectrum_analysis.py AIEF_CODING_AGENT_PROMPT.md
aief_1_4_feedback_rank_memory_reduction aief_experiment_engine_v1.py
'aief_1_4_feedback_rank_memory_reduction 01' AIEF_Experiments
AIEF_1_4_feedback_rank_memory_reduction.py AIEF_HISTORIC_UNFOLDING.md
AIEF_1_4_FORWARD_PLAN.md AIEF_MODULAR_DRIVER_CODING_AGENT_PROMPT.md
AIEF_1_4_freeze_controller_fiber_theorem.py check_001.py
aief_1_4_frozen_driver_cocycle check_01.py
AIEF_1_4_frozen_driver_cocycle_analysis.py check_02.py
aief_1_4_intertwining_controller check.py
'aief_1_4_intertwining_controller 01' diagnose_import.py
'aief_1_4_intertwining_controller 02' gap_phase_scan_v9.py
AIEF_1_4_intertwining_controller_analysis.py lean
aief_1_4_k3_structure_scan 'Older files'
aief_1_4_k3_structure_scan.py paired_seed_analysis

aief_1_4_local_update_comparison Papers
AIEF_1_4_local_update_comparison.py probe_01.py
aief_1_4_matched_phase_memory_ablation probe_02.py
AIEF_1_4_matched_phase_memory_ablation.py pycache
AIEF_1_4_memory_ablation.py Results.pdf
AIEF_1_4_memory_ablation_updated_07.py runs
AIEF_1_4_memory_ablation_updated.py runs_aief14_smoke
aief_1_4_multiseed_geometry runs_pivot_study
aief_1_4_multiseed_geometry.py run_twindow_pivot_study.py
aief_1_4_nested_operator_family 'Sci- stuff.txt'
AIEF_1_4_nested_operator_family_audit.py test_01.py
aief_1_4_observable_jacobian test_output
aief_1_4_observable_jacobian.py Test_Terminal_Output
AIEF_1_4_OPERATOR_COCYCLE_CLOSURE_THEOREM_NOTE.md
twindow_frozen_authority_hashes.txt
AIEF_1_4_OPERATOR_COCYCLE_THOUGHT_TREE.md twindow_source_hashes_before.txt
aief_1_4_operator_compression workspace-019ff86e-8322-7d0d-bcaf-ed27a8018c0f
aief_1_4_operator_compression.py workspace-019ff86e-8322-7d0d-bcaf-ed27a8018c0f.zip

./aief_1_3_closure_diagnostics:
baseline_persistence_by_seed.csv closure_report.txt matrix_scalar_consistency.csv
baseline_persistence_unique_values_by_seed.csv closure_status.json
projection_consistency_by_seed.csv
baseline_persistence_unique_values.csv cohort_seed_coverage.csv
projection_diagnostics_summary.csv
baseline_persistence_unique_values_pooled.csv cohort_seed_overlap.csv

./aief_1_4_active_boundary_analysis:
active_boundary_report.txt analysis_manifest.json coupled_epsilon_dt_summary.csv
active_stratified_comparisons.csv boundary_field_convergence_orders.csv hessian_eigenmodes.csv
active_stratified_orders_by_seed.csv boundary_field_finite_difference.csv
hessian_eigenspace_summary_by_seed.csv
active_stratified_order_summary.csv coupled_epsilon_dt_rows.csv hessian_eigenspace_summary.csv

./aief_1_4_actual_admissibility_branch:
actual_admissibility_branch_report.txt actual_branch_key_matrices.npz analysis_manifest.json
actual_branch_edge_trajectory.csv actual_branch_node_trajectory.csv branch_jacobian_validation.csv
actual_branch_events.csv actual_branch_step_summary.csv directional_derivative_validation.csv

./aief_1_4_augmented_duhamel_feedback:

analysis_manifest.json duhamel_trajectory.csv forcing_contribution_to_final.csv
tangent_ablation_summary.csv
augmented_duhamel_feedback_report.txt duhamel_vectors.npz forcing_time_bin_attribution.csv
tangent_ablation_trajectory.csv

./aief_1_4_augmented_spectrum:
analysis_manifest.json augmented_eigenvalues.csv augmented_spectrum_report.txt

./aief_1_4_augmented_tangent_cocycle:
analysis_manifest.json augmented_tangent_report.txt matched_horizon_amplitude_collapse.csv
augmented_tangent_comparison.csv augmented_tangent_summary.csv

./aief_1_4_biorthogonal_triangular_closure:
analysis_manifest.json biorthogonal_covector_trajectory.csv biorthogonal_rollout_summary.csv
biorthogonal_closure_arrays.npz biorthogonal_one_step_summary.csv
biorthogonal_rollout_trajectory.csv
biorthogonal_coordinate_summary.csv biorthogonal_one_step_trajectory.csv
biorthogonal_triangular_report.txt

./aief_1_4_channel_invariance:
analysis_manifest.json channel_multiplicities_by_seed.csv
geometry_spectrum_consistency_tolerant.csv
channel_invariance_by_seed.csv exceptional_channel_stages.csv modal_success_summary.json
channel_invariance_report.txt exceptional_transition_metadata.json
unique_direction_modal_success.csv

./aief_1_4_cocycle_controls_coupled:
analysis_manifest.json cocycle_controls_coupled_report.txt coupled_frozen_centered_summary.csv
stable_cocycle_spectra.csv
cocycle_control_matrices.npz coupled_frozen_centered_comparison.csv
dense_sliding_window_amplification.csv

./aief_1_4_component_controller_audit:
analysis_manifest.json component_controller_report.txt controller_residual_by_node.csv
state_mechanism_summary.csv
component_controller_matrices.npz component_mechanism_audit.csv edge_locking_balance_audit.csv

./aief_1_4_controller_boundary_margin:

analysis_manifest.json controller_margin_intervals.csv controller_margin_report.txt
controller_margin_trajectory.csv

./aief_1_4_coupling_lock_audit:
analysis_manifest.json coupling_lock_intervals.csv coupling_lock_report.txt
coupling_lock_trajectory.csv

./aief_1_4_defect_causal_ema_trace:
analysis_manifest.json defect_causal_identity_checks.csv defect_causal_trace_trajectory.csv
defect_causal_ema_report.txt defect_causal_trace_intervals.csv

./aief_1_4_defect_recurrence_analytic_mode:
analysis_manifest.json analytic_defect_recurrence_arrays.npz defect_coefficient_trajectory.csv
analytic_defect_POD_comparison.csv analytic_defect_recurrence_report.txt
defect_recurrence_summary.csv

./aief_1_4_duhamel_memory_alignment:
analysis_manifest.json duhamel_memory_alignment_report.txt
forcing_alignment_and_final_influence.csv selected_ablation_states.csv
duhamel_mechanism_summary.csv exact_crossover_and_angles.csv
recent_forcing_memory_curve.csv

./aief_1_4_entropy_geometry:
analysis_manifest.json entropy_geometry_report.txt interior_geometry_summary.csv
intrinsic_distances_summary.csv raw_node_margins.csv
boundary_occupancy_by_seed.csv interior_geometry_by_seed.csv intrinsic_distances_by_seed.csv
raw_edge_weights.csv

./aief_1_4_exceptional_K3_transition:
analysis_manifest.json exceptional_K3_node_trajectory.csv
exceptional_K3_transition_window_nodes.csv
boundary_tolerance_switch_sensitivity.csv exceptional_K3_step_summary.csv
exceptional_K3_transition_window_summary.csv
exceptional_K3_edge_trajectory.csv exceptional_K3_transition_report.txt
exceptional_K3_key_matrices.npz exceptional_K3_transition_window_edges.csv

./aief_1_4_exceptional_tangent:
analysis_manifest.json corrected_tangent_trajectories.csv exceptional_seed_switch_matrices.npz
corrected_tangent_by_seed.csv exceptional_seed_summary.json exceptional_seed_timeseries.csv

corrected_tangent_summary.csv exceptional_seed_switches.csv exceptional_tangent_report.txt

./aief_1_4_existing_clique_invariant_subspaces:
analysis_manifest.json clique_invariant_subspace_by_component.csv
clique_invariant_subspace_report.txt
clique_fiber_direction_checks.csv clique_invariant_subspace_matrices.npz

./aief_1_4_face_spectrum:
active_face_fingerprints.csv filtered_anomalous_directions.csv
modal_trajectory_prediction_summary.csv
analysis_manifest.json fingerprint_spectrum_consistency.csv tangent_direction_summary.csv
direction_agreement_summary.json jacobian_direction_diagnostics.csv tangent_jacobian_matrices.npz
exceptional_stage_spectra.csv jacobian_eigenvalues.csv tangent_rate_face_crosstab_rows.csv
exceptional_transition_summary.json jacobian_step_sensitivity.csv
tangent_rate_face_crosstab_summary.csv
face_spectrum_report.txt modal_trajectory_predictions.csv

./aief_1_4_feedback_rank_memory_reduction:
analysis_manifest.json late_channel_scalar_projection.csv pod_first_mode_stability_summary.csv
target_time_memory_curves.csv
feedback_rank_memory_report.txt late_scalar_transverse_summary.csv pod_rank_by_interval.csv
target_time_memory_fits.csv
instantaneous_feedback_rank.csv memory_stationarity_summary.csv
propagated_forcing_rank_by_target.csv

'./aief_1_4_feedback_rank_memory_reduction 01':
analysis_manifest.json instantaneous_feedback_rank.csv pod_rank_by_interval.csv
target_time_memory_curves.csv
feedback_rank_memory_report.txt late_channel_scalar_projection.csv
propagated_forcing_rank_by_target.csv target_time_memory_fits.csv

./aief_1_4_frozen_driver_cocycle:
analysis_manifest.json frozen_cocycle_nonlinear_comparison.csv frozen_cocycle_propagator.csv
frozen_cocycle_step_diagnostics.csv
frozen_cocycle_matrices.npz frozen_cocycle_nonlinear_summary.csv
frozen_cocycle_sliding_windows.csv frozen_driver_cocycle_report.txt

./aief_1_4_intertwining_controller:
analysis_manifest.json exceptional_channel_stages.csv modal_success_summary.json

channel_invariance_by_seed.csv exceptional_transition_metadata.json
node_controller_equilibrium_by_seed.csv
channel_invariance_report.txt geometry_spectrum_consistency_tolerant.csv
unique_direction_modal_success.csv
channel_multiplicities_by_seed.csv intertwining_controller_matrices.npz

'./aief_1_4_intertwining_controller 01':
analysis_manifest.json exceptional_channel_stages.csv modal_success_summary.json
channel_invariance_by_seed.csv exceptional_transition_metadata.json
node_controller_equilibrium_by_seed.csv
channel_invariance_report.txt geometry_spectrum_consistency_tolerant.csv
unique_direction_modal_success.csv
channel_multiplicities_by_seed.csv intertwining_controller_matrices.npz

'./aief_1_4_intertwining_controller 02':
analysis_manifest.json exceptional_channel_stages.csv modal_success_summary.json
channel_invariance_by_seed.csv exceptional_transition_metadata.json
node_controller_equilibrium_by_seed.csv
channel_invariance_report.txt geometry_spectrum_consistency_tolerant.csv
unique_direction_modal_success.csv
channel_multiplicities_by_seed.csv intertwining_controller_matrices.npz

./aief_1_4_k3_structure_scan:
analysis_manifest.json k3_prevalence.csv k3_prevalence_report.txt

./aief_1_4_local_update_comparison:
analysis_manifest.json one_step_comparisons_by_seed.csv one_step_convergence_orders_by_seed.csv
one_step_methods_by_seed.csv
local_update_comparison_report.txt one_step_comparison_summary.csv
one_step_convergence_order_summary.csv one_step_method_summary.csv

./aief_1_4_matched_phase_memory_ablation:
analysis_manifest.json frozen_anchor_validation.csv matched_initial_state_audit.csv

./aief_1_4_multiseed_geometry:
k3_structure_scan.csv

./aief_1_4_nested_operator_family:

analysis_manifest.json nested_family_decay_fits.csv nested_family_key_times.csv
nested_family_trajectory.csv
nested_family_arrays.npz nested_family_defect_pod.csv nested_family_parameters.csv
nested_operator_family_report.txt

./aief_1_4_observable_jacobian:
analysis_manifest.json eigenvector_decomposition.csv observable_jacobian.csv
observable_jacobian_report.txt

./aief_1_4_operator_compression:
analysis_manifest.json operator_compression.csv operator_compression_report.txt

./aief_1_4_parameter_free_triangular_closure:
analysis_manifest.json triangular_closure_arrays.npz triangular_one_step_summary.csv
triangular_rollout_summary.csv
parameter_free_triangular_report.txt triangular_gap_summary.csv triangular_one_step_trajectory.csv
triangular_rollout_trajectory.csv

./aief_1_4_phase_balance_source_dynamics:
analysis_manifest.json phase_balance_interval_summary.csv phase_balance_trajectory.csv
phase_balance_identity_checks.csv phase_balance_source_report.txt

./aief_1_4_phase_cancellation_audit:
analysis_manifest.json phase_cancellation_intervals.csv phase_cancellation_report.txt
phase_cancellation_trajectory.csv

./aief_1_4_phase_source_decomposition:
analysis_manifest.json source_decomposition_intervals.csv source_decomposition_report.txt
source_decomposition_trajectory.csv

./aief_1_4_phase_source_locking:
analysis_manifest.json phase_source_locking_intervals.csv phase_source_locking_trajectory.csv
phase_source_locking_identity_checks.csv phase_source_locking_report.txt

./aief_1_4_propagation_gap_geometric_control:
analysis_manifest.json forcing_state_identifiability.csv propagated_contribution_envelopes.csv
direct_propagation_gap_tests.csv geometric_vs_POD_direction_control.csv
propagation_gap_geometric_report.txt

./aief_1_4_proposal_equalization_bridge:
analysis_manifest.json proposal_equalization_interval_summary.csv
proposal_equalization_trajectory.csv
proposal_equalization_identity_summary.csv proposal_equalization_report.txt

./aief_1_4_radial_observable_subsystem:
analysis_manifest.json radial_observable_identity_checks.csv radial_observable_report.txt
observable_subsystem_fits.csv radial_observable_interval_summary.csv
radial_observable_trajectory.csv

./aief_1_4_resonant_tracking_angular:
alignment_component_asymptotics.csv angular_radius_summary.csv
resonant_tracking_fit_summary.csv resonant_tracking_trajectory.csv
analysis_manifest.json resonant_tracking_angular_report.txt resonant_tracking_identity_checks.csv

./aief_1_4_results_analysis:
AIEF_1_4_analysis_report.txt B_EMA_interactions.csv frobenius_distance_summary.csv
projection_summary.csv
analysis_manifest.json density_trajectories.png paired_branch_effects_by_seed.csv
similarity_trajectories.png
B_EMA_interactions_by_seed.csv frobenius_distances_by_seed.csv paired_branch_effects.csv
trajectory_summary.csv

./aief_1_4_scalar_spectral_gap_reduction:
analysis_manifest.json scalar_projection_trajectory.csv scalar_spectral_gap_report.txt
two_state_model_summary.csv
scalar_projection_summary.csv scalar_reduction_arrays.npz target_spectral_gap_tests.csv
two_state_rollout_validation.csv

./aief_1_4_slow_bundle_geometry:
analysis_manifest.json compressed_operator_entries.csv slow_bundle_geometry_report.txt

./aief_1_4_slow_eigenvector_analysis:
analysis_manifest.json eigenvector_overlaps.csv participation_factors.csv singular_values.csv
slow_eigenvector_report.txt slow_eigenvectors.csv

./aief_1_4_slow_subspace_projection:
analysis_manifest.json eigenvector_decomposition.csv observable_projections.csv
slow_subspace_report.txt variance_fractions.csv

./aief_1_4_stationarity_perturbation:
analysis_manifest.json coupled_perturbation_trajectories.csv mature_stationarity_by_seed.csv
coupled_perturbations_by_seed.csv frozen_fast_perturbations_by_seed.csv
mature_stationarity_timeseries.csv
coupled_perturbation_summary.csv frozen_fast_perturbation_summary.csv
stationarity_perturbation_report.txt

./aief_1_4_survival_outlier_spectral:
analysis_manifest.json KM_similarity90_recovery.png resetB_residual_first_mode_edges.csv
split_state_features.csv
feature_recovery_associations.csv paired_recovery_contrasts.csv
resetB_residual_localization_by_seed.csv survival_outlier_spectral_report.txt
KM_curves.csv recovery_by_seed.csv resetB_residual_svd.csv
KM_density90_recovery.png recovery_summary.csv spectral_metrics_split_late_final.csv

./aief_1_4_tangent_spectrum_switch:
analysis_manifest.json exceptional_switch_events.csv tangent_direction_summary.csv
tangent_jacobian_summary.csv
anomalous_tangent_directions.csv exceptional_switch_node_window.csv
tangent_jacobian_direction_details.csv tangent_spectrum_switch_report.txt
direction_agreement_summary.json exceptional_switch_summary.json tangent_jacobian_matrices.npz
tangent_transient_growth.csv

./aief_1_4_trajectory_recovery:
analysis_manifest.json equivalence_tests.csv trajectory_envelopes.csv trajectory_metrics_summary.csv
density_recovery_envelope.png similarity_recovery_envelope.png trajectory_metrics_by_seed.csv
trajectory_recovery_report.txt

./aief_1_4_transverse_phase_jacobian:
analysis_manifest.json transverse_jacobian_intervals.csv transverse_jacobian_report.txt
transverse_jacobian_trajectory.csv

./aief_1_4_two_channel_alignment_v4:
analysis_manifest.json two_channel_decomposition_report.txt two_channel_identity_checks.csv
two_channel_summary.csv two_channel_trajectory.csv

./aief_1_4_two_channel_alignment_v5:

analysis_manifest.json two_channel_decomposition_report.txt two_channel_identity_checks.csv
two_channel_summary.csv two_channel_trajectory.csv

./AIEF_Experiments:
aief_modular_platform_v1_analysis_v4 aief_modular_platform_v1_analysis_v4.zip

./AIEF_Experiments/aief_modular_platform_v1_analysis_v4:
AIEF_1_4_FROZEN_5a9f093a2fe0.authority.lock.json AIEF_modular_driver_v1.py
PROJECT_INTEGRATION_MAP.md tests_modular
AIEF_1_4_Twindow_sensitivity_analysis_v4.py ANALYSIS_V4_DESIGN.md pytest.ini
aief_modular MODULAR_IMPLEMENTATION_REPORT.md README_modular.md

./AIEF_Experiments/aief_modular_platform_v1_analysis_v4/aief_modular:
artifacts.py branches.py collectors.py init.py registry.py spec.py validation.py
authority.py catalog.py factors.py oracle.py runner.py studies

./AIEF_Experiments/aief_modular_platform_v1_analysis_v4/aief_modular/studies:
aief_1_4_s01_twindow.py init.py

./AIEF_Experiments/aief_modular_platform_v1_analysis_v4/tests_modular:
conftest.py test_authority_lock.py test_factors_hardening.py test_spec_canonicalization.py
test_unsupported_policies.py
test_analysis_v4.py test_builtin_registry.py test_registry.py test_twindow_one_condition.py
test_artifact_paths.py test_catalog.py test_rng_namespace.py test_twindow_oracle_equivalence.py
test_atomic_writes.py test_condition_expansion.py test_runner_regeneration.py
test_twindow_spec_expansion.py

./lean:
AIEFProofs

./lean/AIEFProofs:
AIEFProofs AIEFProofs.lean lakefile.toml lake-manifest.json lean-toolchain README.md

./lean/AIEFProofs/AIEFProofs:
Basic.lean Smoke.lean

'./Older files':

87dfe5798954 AIEF_1_4_matched_phase_memory_ablation_01.py
aief_1_4_multiseed_geometry_1.py
AIEF_1_4_augmented_duhamel_feedback_analysis_01.py
AIEF_1_4_matched_phase_memory_ablation_07.py AIEF_1_4_phase_source_locking_analysis.py
AIEF_1_4_augmented_duhamel_feedback_analysis.py
AIEF_1_4_matched_phase_memory_ablation.py AIEF_1_4_resonant_tracking_angular_analysis.py
aief_1_4_augmented_spectrum_02.py AIEF_1_4_memory_ablation_updated_01.py
aief_1_4_slow_bundle_geometry_01.py
aief_1_4_augmented_spectrum.py AIEF_1_4_memory_ablation_updated.py
aief_1_4_slow_subspace_projection_01.py
AIEF_1_4_augmented_tangent_cocycle_analysis.py aief_1_4_multiseed_geometry_01.py
aief_1_4_slow_subspace_projection.py
aief_1_4_coupling_lock_audit.py aief_1_4_multiseed_geometry_02.py
aief_1_4_two_channel_alignment_v4.py
AIEF_1_4_duhamel_memory_alignment_analysis.py aief_1_4_multiseed_geometry_03.py
aief_1_4_two_channel_alignment_v5_01.py
aief_1_4_face_spectrum aief_1_4_multiseed_geometry_05.py
aief_1_4_two_channel_alignment_v5.py
AIEF_1_4_face_spectrum_analysis.py aief_1_4_multiseed_geometry_08.py b57562a8ee77
AIEF_1_4_feedback_rank_memory_reduction.py aief_1_4_multiseed_geometry_10.py
AIEF_1_4_freeze_controller_fiber_theorem.py aief_1_4_multiseed_geometry_11.py

'./Older files/87dfe5798954':
AIEF_1_3_87dfe5798954_edge_diagnostics.npz baseline_seed_9042.npz no_delay_seed_9042.npz
no_mu_seed_10042.npz shuffle_R_seed_1042.npz
AIEF_1_3_87dfe5798954_scalar_summary.csv manifest.json no_gap_seed_10042.npz
no_mu_seed_1042.npz shuffle_R_seed_11042.npz
baseline_seed_10042.npz no_delay_seed_10042.npz no_gap_seed_1042.npz no_mu_seed_11042.npz
shuffle_R_seed_12042.npz
baseline_seed_1042.npz no_delay_seed_1042.npz no_gap_seed_11042.npz no_mu_seed_12042.npz
shuffle_R_seed_13042.npz
baseline_seed_11042.npz no_delay_seed_11042.npz no_gap_seed_12042.npz no_mu_seed_13042.npz
shuffle_R_seed_14042.npz
baseline_seed_12042.npz no_delay_seed_12042.npz no_gap_seed_13042.npz no_mu_seed_14042.npz
shuffle_R_seed_15042.npz
baseline_seed_13042.npz no_delay_seed_13042.npz no_gap_seed_14042.npz no_mu_seed_15042.npz
shuffle_R_seed_16042.npz
baseline_seed_14042.npz no_delay_seed_14042.npz no_gap_seed_15042.npz no_mu_seed_16042.npz
shuffle_R_seed_17042.npz
baseline_seed_15042.npz no_delay_seed_15042.npz no_gap_seed_16042.npz no_mu_seed_17042.npz
shuffle_R_seed_18042.npz

baseline_seed_16042.npz no_delay_seed_16042.npz no_gap_seed_17042.npz no_mu_seed_18042.npz shuffle_R_seed_19042.npz
baseline_seed_17042.npz no_delay_seed_17042.npz no_gap_seed_18042.npz no_mu_seed_19042.npz shuffle_R_seed_2042.npz
baseline_seed_18042.npz no_delay_seed_18042.npz no_gap_seed_19042.npz no_mu_seed_2042.npz shuffle_R_seed_3042.npz
baseline_seed_19042.npz no_delay_seed_19042.npz no_gap_seed_2042.npz no_mu_seed_3042.npz shuffle_R_seed_4042.npz
baseline_seed_2042.npz no_delay_seed_2042.npz no_gap_seed_3042.npz no_mu_seed_4042.npz shuffle_R_seed_42.npz
baseline_seed_3042.npz no_delay_seed_3042.npz no_gap_seed_4042.npz no_mu_seed_42.npz shuffle_R_seed_5042.npz
baseline_seed_4042.npz no_delay_seed_4042.npz no_gap_seed_42.npz no_mu_seed_5042.npz shuffle_R_seed_6042.npz
baseline_seed_42.npz no_delay_seed_42.npz no_gap_seed_5042.npz no_mu_seed_6042.npz shuffle_R_seed_7042.npz
baseline_seed_5042.npz no_delay_seed_5042.npz no_gap_seed_6042.npz no_mu_seed_7042.npz shuffle_R_seed_8042.npz
baseline_seed_6042.npz no_delay_seed_6042.npz no_gap_seed_7042.npz no_mu_seed_8042.npz shuffle_R_seed_9042.npz
baseline_seed_7042.npz no_delay_seed_7042.npz no_gap_seed_8042.npz no_mu_seed_9042.npz
baseline_seed_8042.npz no_delay_seed_8042.npz no_gap_seed_9042.npz shuffle_R_seed_10042.npz

'./Older files/aief_1_4_face_spectrum':
active_face_fingerprints.csv exceptional_transition_summary.json jacobian_direction_diagnostics.csv tangent_jacobian_matrices.npz
analysis_manifest.json face_spectrum_report.txt jacobian_eigenvalues.csv tangent_rate_face_crosstab_rows.csv
direction_agreement_summary.json filtered_anomalous_directions.csv jacobian_step_sensitivity.csv tangent_rate_face_crosstab_summary.csv
exceptional_stage_spectra.csv fingerprint_spectrum_consistency.csv tangent_direction_summary.csv

'./Older files/b57562a8ee77':
baseline_equivalence_by_seed.json baseline_equivalence_certificate.json manifest.json

./paired_seed_analysis:
analysis_manifest.json paired_binary_effects.csv paired_cohort_heterogeneity.csv paired_effects.csv paired_report.txt paired_seed_deltas.csv

./Papers:
2804.05652v1.pdf 'The Holland Atom_Paper_1_30.May.2026_v1.pdf'

'H-Atom_Paper 3 Geometric Radius Scaling Law 19.Jul.2026_v1.0.pdf' 'The Holland Atom_Paper 2 Radius Variation 23.Jun.2026_v1.0.pdf'

./pycache:
AIEF_1_4_actual_admissibility_branch_audit.cpython-313.pyc
AIEF_1_4_nested_operator_family_audit.cpython-313.pyc
AIEF_1_4_augmented_tangent_cocycle_analysis.cpython-313.pyc
AIEF_1_4_proposal_equalization_bridge_analysis.cpython-313.pyc
AIEF_1_4_component_controller_audit.cpython-313.pyc
AIEF_1_4_stationarity_perturbation_analysis_updated.cpython-313.pyc
AIEF_1_4_exceptional_K3_transition_audit.cpython-313.pyc
AIEF_1_4_tangent_spectrum_switch_analysis.cpython-313.pyc
AIEF_1_4_exceptional_tangent_analysis.cpython-313.pyc
AIEF_1_4_trajectory_recovery_equivalence.cpython-313.pyc
AIEF_1_4_face_spectrum_analysis.cpython-313.pyc
AIEF_1_4_Twindow_sensitivity_analysis_v3.cpython-313.pyc
AIEF_1_4_frozen_driver_cocycle_analysis.cpython-313.pyc
AIEF_1_4_Twindow_sensitivity.cpython-313.pyc
AIEF_1_4_matched_phase_memory_ablation.cpython-313.pyc
aief_experiment_engine_v1.cpython-313.pyc
AIEF_1_4_memory_ablation_updated.cpython-313.pyc

./runs:
0ddf42b3e20a 2e2843f5e26c 5a338105785a 73dca37cf93f 92dd4ddbdba1 a3dcc58f4939 b57562a8ee77 fccebfc14855
123b2cadbb0d 46fb82c784ef 5a9f093a2fe0 87dfe5798954 9fb83552f0ce a937d5e96efb c3da1ac707bf

./runs/0ddf42b3e20a:
AIEF_1_3_0ddf42b3e20a_edge_diagnostics.npz baseline_seed_9042.npz no_delay_seed_9042.npz no_mu_seed_10042.npz shuffle_R_seed_1042.npz
AIEF_1_3_0ddf42b3e20a_scalar_summary.csv manifest.json no_gap_seed_10042.npz no_mu_seed_1042.npz shuffle_R_seed_11042.npz
baseline_seed_10042.npz no_delay_seed_10042.npz no_gap_seed_1042.npz no_mu_seed_11042.npz shuffle_R_seed_12042.npz
baseline_seed_1042.npz no_delay_seed_1042.npz no_gap_seed_11042.npz no_mu_seed_12042.npz shuffle_R_seed_13042.npz
baseline_seed_11042.npz no_delay_seed_11042.npz no_gap_seed_12042.npz no_mu_seed_13042.npz shuffle_R_seed_14042.npz
baseline_seed_12042.npz no_delay_seed_12042.npz no_gap_seed_13042.npz no_mu_seed_14042.npz shuffle_R_seed_15042.npz

baseline_seed_13042.npz no_delay_seed_13042.npz no_gap_seed_14042.npz no_mu_seed_15042.npz shuffle_R_seed_16042.npz
baseline_seed_14042.npz no_delay_seed_14042.npz no_gap_seed_15042.npz no_mu_seed_16042.npz shuffle_R_seed_17042.npz
baseline_seed_15042.npz no_delay_seed_15042.npz no_gap_seed_16042.npz no_mu_seed_17042.npz shuffle_R_seed_18042.npz
baseline_seed_16042.npz no_delay_seed_16042.npz no_gap_seed_17042.npz no_mu_seed_18042.npz shuffle_R_seed_19042.npz
baseline_seed_17042.npz no_delay_seed_17042.npz no_gap_seed_18042.npz no_mu_seed_19042.npz shuffle_R_seed_2042.npz
baseline_seed_18042.npz no_delay_seed_18042.npz no_gap_seed_19042.npz no_mu_seed_2042.npz shuffle_R_seed_3042.npz
baseline_seed_19042.npz no_delay_seed_19042.npz no_gap_seed_2042.npz no_mu_seed_3042.npz shuffle_R_seed_4042.npz
baseline_seed_2042.npz no_delay_seed_2042.npz no_gap_seed_3042.npz no_mu_seed_4042.npz shuffle_R_seed_42.npz
baseline_seed_3042.npz no_delay_seed_3042.npz no_gap_seed_4042.npz no_mu_seed_42.npz shuffle_R_seed_5042.npz
baseline_seed_4042.npz no_delay_seed_4042.npz no_gap_seed_42.npz no_mu_seed_5042.npz shuffle_R_seed_6042.npz
baseline_seed_42.npz no_delay_seed_42.npz no_gap_seed_5042.npz no_mu_seed_6042.npz shuffle_R_seed_7042.npz
baseline_seed_5042.npz no_delay_seed_5042.npz no_gap_seed_6042.npz no_mu_seed_7042.npz shuffle_R_seed_8042.npz
baseline_seed_6042.npz no_delay_seed_6042.npz no_gap_seed_7042.npz no_mu_seed_8042.npz shuffle_R_seed_9042.npz
baseline_seed_7042.npz no_delay_seed_7042.npz no_gap_seed_8042.npz no_mu_seed_9042.npz
baseline_seed_8042.npz no_delay_seed_8042.npz no_gap_seed_9042.npz shuffle_R_seed_10042.npz

./runs/123b2cadbb0d:
AIEF_1_3_123b2cadbb0d_edge_diagnostics.npz baseline_seed_9042.npz no_delay_seed_9042.npz no_mu_seed_10042.npz shuffle_R_seed_1042.npz
AIEF_1_3_123b2cadbb0d_scalar_summary.csv manifest.json no_gap_seed_10042.npz no_mu_seed_1042.npz shuffle_R_seed_11042.npz
baseline_seed_10042.npz no_delay_seed_10042.npz no_gap_seed_1042.npz no_mu_seed_11042.npz shuffle_R_seed_12042.npz
baseline_seed_1042.npz no_delay_seed_1042.npz no_gap_seed_11042.npz no_mu_seed_12042.npz shuffle_R_seed_13042.npz
baseline_seed_11042.npz no_delay_seed_11042.npz no_gap_seed_12042.npz no_mu_seed_13042.npz shuffle_R_seed_14042.npz
baseline_seed_12042.npz no_delay_seed_12042.npz no_gap_seed_13042.npz no_mu_seed_14042.npz shuffle_R_seed_15042.npz

baseline_seed_13042.npz no_delay_seed_13042.npz no_gap_seed_14042.npz no_mu_seed_15042.npz shuffle_R_seed_16042.npz
baseline_seed_14042.npz no_delay_seed_14042.npz no_gap_seed_15042.npz no_mu_seed_16042.npz shuffle_R_seed_17042.npz
baseline_seed_15042.npz no_delay_seed_15042.npz no_gap_seed_16042.npz no_mu_seed_17042.npz shuffle_R_seed_18042.npz
baseline_seed_16042.npz no_delay_seed_16042.npz no_gap_seed_17042.npz no_mu_seed_18042.npz shuffle_R_seed_19042.npz
baseline_seed_17042.npz no_delay_seed_17042.npz no_gap_seed_18042.npz no_mu_seed_19042.npz shuffle_R_seed_2042.npz
baseline_seed_18042.npz no_delay_seed_18042.npz no_gap_seed_19042.npz no_mu_seed_2042.npz shuffle_R_seed_3042.npz
baseline_seed_19042.npz no_delay_seed_19042.npz no_gap_seed_2042.npz no_mu_seed_3042.npz shuffle_R_seed_4042.npz
baseline_seed_2042.npz no_delay_seed_2042.npz no_gap_seed_3042.npz no_mu_seed_4042.npz shuffle_R_seed_42.npz
baseline_seed_3042.npz no_delay_seed_3042.npz no_gap_seed_4042.npz no_mu_seed_42.npz shuffle_R_seed_5042.npz
baseline_seed_4042.npz no_delay_seed_4042.npz no_gap_seed_42.npz no_mu_seed_5042.npz shuffle_R_seed_6042.npz
baseline_seed_42.npz no_delay_seed_42.npz no_gap_seed_5042.npz no_mu_seed_6042.npz shuffle_R_seed_7042.npz
baseline_seed_5042.npz no_delay_seed_5042.npz no_gap_seed_6042.npz no_mu_seed_7042.npz shuffle_R_seed_8042.npz
baseline_seed_6042.npz no_delay_seed_6042.npz no_gap_seed_7042.npz no_mu_seed_8042.npz shuffle_R_seed_9042.npz
baseline_seed_7042.npz no_delay_seed_7042.npz no_gap_seed_8042.npz no_mu_seed_9042.npz
baseline_seed_8042.npz no_delay_seed_8042.npz no_gap_seed_9042.npz shuffle_R_seed_10042.npz

./runs/2e2843f5e26c:
AIEF_1_3_2e2843f5e26c_edge_diagnostics.npz baseline_seed_9042.npz no_delay_seed_9042.npz no_mu_seed_10042.npz shuffle_R_seed_1042.npz
AIEF_1_3_2e2843f5e26c_scalar_summary.csv manifest.json no_gap_seed_10042.npz no_mu_seed_1042.npz shuffle_R_seed_11042.npz
baseline_seed_10042.npz no_delay_seed_10042.npz no_gap_seed_1042.npz no_mu_seed_11042.npz shuffle_R_seed_12042.npz
baseline_seed_1042.npz no_delay_seed_1042.npz no_gap_seed_11042.npz no_mu_seed_12042.npz shuffle_R_seed_13042.npz
baseline_seed_11042.npz no_delay_seed_11042.npz no_gap_seed_12042.npz no_mu_seed_13042.npz shuffle_R_seed_14042.npz
baseline_seed_12042.npz no_delay_seed_12042.npz no_gap_seed_13042.npz no_mu_seed_14042.npz shuffle_R_seed_15042.npz

baseline_seed_13042.npz no_delay_seed_13042.npz no_gap_seed_14042.npz no_mu_seed_15042.npz shuffle_R_seed_16042.npz
baseline_seed_14042.npz no_delay_seed_14042.npz no_gap_seed_15042.npz no_mu_seed_16042.npz shuffle_R_seed_17042.npz
baseline_seed_15042.npz no_delay_seed_15042.npz no_gap_seed_16042.npz no_mu_seed_17042.npz shuffle_R_seed_18042.npz
baseline_seed_16042.npz no_delay_seed_16042.npz no_gap_seed_17042.npz no_mu_seed_18042.npz shuffle_R_seed_19042.npz
baseline_seed_17042.npz no_delay_seed_17042.npz no_gap_seed_18042.npz no_mu_seed_19042.npz shuffle_R_seed_2042.npz
baseline_seed_18042.npz no_delay_seed_18042.npz no_gap_seed_19042.npz no_mu_seed_2042.npz shuffle_R_seed_3042.npz
baseline_seed_19042.npz no_delay_seed_19042.npz no_gap_seed_2042.npz no_mu_seed_3042.npz shuffle_R_seed_4042.npz
baseline_seed_2042.npz no_delay_seed_2042.npz no_gap_seed_3042.npz no_mu_seed_4042.npz shuffle_R_seed_42.npz
baseline_seed_3042.npz no_delay_seed_3042.npz no_gap_seed_4042.npz no_mu_seed_42.npz shuffle_R_seed_5042.npz
baseline_seed_4042.npz no_delay_seed_4042.npz no_gap_seed_42.npz no_mu_seed_5042.npz shuffle_R_seed_6042.npz
baseline_seed_42.npz no_delay_seed_42.npz no_gap_seed_5042.npz no_mu_seed_6042.npz shuffle_R_seed_7042.npz
baseline_seed_5042.npz no_delay_seed_5042.npz no_gap_seed_6042.npz no_mu_seed_7042.npz shuffle_R_seed_8042.npz
baseline_seed_6042.npz no_delay_seed_6042.npz no_gap_seed_7042.npz no_mu_seed_8042.npz shuffle_R_seed_9042.npz
baseline_seed_7042.npz no_delay_seed_7042.npz no_gap_seed_8042.npz no_mu_seed_9042.npz
baseline_seed_8042.npz no_delay_seed_8042.npz no_gap_seed_9042.npz shuffle_R_seed_10042.npz

./runs/46fb82c784ef:
AIEF_1_3_46fb82c784ef_edge_diagnostics.npz baseline_seed_8042.npz no_delay_seed_8042.npz no_gap_seed_9042.npz shuffle_R_seed_10042.npz
AIEF_1_3_46fb82c784ef_scalar_summary.csv baseline_seed_9042.npz no_delay_seed_9042.npz no_mu_seed_10042.npz shuffle_R_seed_1042.npz
AIEF_1_3_analysis_results.csv manifest.json no_gap_seed_10042.npz no_mu_seed_1042.npz shuffle_R_seed_11042.npz
baseline_seed_10042.npz no_delay_seed_10042.npz no_gap_seed_1042.npz no_mu_seed_11042.npz shuffle_R_seed_12042.npz
baseline_seed_1042.npz no_delay_seed_1042.npz no_gap_seed_11042.npz no_mu_seed_12042.npz shuffle_R_seed_13042.npz
baseline_seed_11042.npz no_delay_seed_11042.npz no_gap_seed_12042.npz no_mu_seed_13042.npz shuffle_R_seed_14042.npz

baseline_seed_12042.npz no_delay_seed_12042.npz no_gap_seed_13042.npz no_mu_seed_14042.npz shuffle_R_seed_15042.npz
baseline_seed_13042.npz no_delay_seed_13042.npz no_gap_seed_14042.npz no_mu_seed_15042.npz shuffle_R_seed_16042.npz
baseline_seed_14042.npz no_delay_seed_14042.npz no_gap_seed_15042.npz no_mu_seed_16042.npz shuffle_R_seed_17042.npz
baseline_seed_15042.npz no_delay_seed_15042.npz no_gap_seed_16042.npz no_mu_seed_17042.npz shuffle_R_seed_18042.npz
baseline_seed_16042.npz no_delay_seed_16042.npz no_gap_seed_17042.npz no_mu_seed_18042.npz shuffle_R_seed_19042.npz
baseline_seed_17042.npz no_delay_seed_17042.npz no_gap_seed_18042.npz no_mu_seed_19042.npz shuffle_R_seed_2042.npz
baseline_seed_18042.npz no_delay_seed_18042.npz no_gap_seed_19042.npz no_mu_seed_2042.npz shuffle_R_seed_3042.npz
baseline_seed_19042.npz no_delay_seed_19042.npz no_gap_seed_2042.npz no_mu_seed_3042.npz shuffle_R_seed_4042.npz
baseline_seed_2042.npz no_delay_seed_2042.npz no_gap_seed_3042.npz no_mu_seed_4042.npz shuffle_R_seed_42.npz
baseline_seed_3042.npz no_delay_seed_3042.npz no_gap_seed_4042.npz no_mu_seed_42.npz shuffle_R_seed_5042.npz
baseline_seed_4042.npz no_delay_seed_4042.npz no_gap_seed_42.npz no_mu_seed_5042.npz shuffle_R_seed_6042.npz
baseline_seed_42.npz no_delay_seed_42.npz no_gap_seed_5042.npz no_mu_seed_6042.npz shuffle_R_seed_7042.npz
baseline_seed_5042.npz no_delay_seed_5042.npz no_gap_seed_6042.npz no_mu_seed_7042.npz shuffle_R_seed_8042.npz
baseline_seed_6042.npz no_delay_seed_6042.npz no_gap_seed_7042.npz no_mu_seed_8042.npz shuffle_R_seed_9042.npz
baseline_seed_7042.npz no_delay_seed_7042.npz no_gap_seed_8042.npz no_mu_seed_9042.npz

./runs/5a338105785a:
AIEF_1_3_5a338105785a_edge_diagnostics.npz baseline_seed_9042.npz no_delay_seed_9042.npz no_mu_seed_10042.npz shuffle_R_seed_1042.npz
AIEF_1_3_5a338105785a_scalar_summary.csv manifest.json no_gap_seed_10042.npz no_mu_seed_1042.npz shuffle_R_seed_11042.npz
baseline_seed_10042.npz no_delay_seed_10042.npz no_gap_seed_1042.npz no_mu_seed_11042.npz shuffle_R_seed_12042.npz
baseline_seed_1042.npz no_delay_seed_1042.npz no_gap_seed_11042.npz no_mu_seed_12042.npz shuffle_R_seed_13042.npz
baseline_seed_11042.npz no_delay_seed_11042.npz no_gap_seed_12042.npz no_mu_seed_13042.npz shuffle_R_seed_14042.npz

baseline_seed_12042.npz no_delay_seed_12042.npz no_gap_seed_13042.npz no_mu_seed_14042.npz shuffle_R_seed_15042.npz
baseline_seed_13042.npz no_delay_seed_13042.npz no_gap_seed_14042.npz no_mu_seed_15042.npz shuffle_R_seed_16042.npz
baseline_seed_14042.npz no_delay_seed_14042.npz no_gap_seed_15042.npz no_mu_seed_16042.npz shuffle_R_seed_17042.npz
baseline_seed_15042.npz no_delay_seed_15042.npz no_gap_seed_16042.npz no_mu_seed_17042.npz shuffle_R_seed_18042.npz
baseline_seed_16042.npz no_delay_seed_16042.npz no_gap_seed_17042.npz no_mu_seed_18042.npz shuffle_R_seed_19042.npz
baseline_seed_17042.npz no_delay_seed_17042.npz no_gap_seed_18042.npz no_mu_seed_19042.npz shuffle_R_seed_2042.npz
baseline_seed_18042.npz no_delay_seed_18042.npz no_gap_seed_19042.npz no_mu_seed_2042.npz shuffle_R_seed_3042.npz
baseline_seed_19042.npz no_delay_seed_19042.npz no_gap_seed_2042.npz no_mu_seed_3042.npz shuffle_R_seed_4042.npz
baseline_seed_2042.npz no_delay_seed_2042.npz no_gap_seed_3042.npz no_mu_seed_4042.npz shuffle_R_seed_42.npz
baseline_seed_3042.npz no_delay_seed_3042.npz no_gap_seed_4042.npz no_mu_seed_42.npz shuffle_R_seed_5042.npz
baseline_seed_4042.npz no_delay_seed_4042.npz no_gap_seed_42.npz no_mu_seed_5042.npz shuffle_R_seed_6042.npz
baseline_seed_42.npz no_delay_seed_42.npz no_gap_seed_5042.npz no_mu_seed_6042.npz shuffle_R_seed_7042.npz
baseline_seed_5042.npz no_delay_seed_5042.npz no_gap_seed_6042.npz no_mu_seed_7042.npz shuffle_R_seed_8042.npz
baseline_seed_6042.npz no_delay_seed_6042.npz no_gap_seed_7042.npz no_mu_seed_8042.npz shuffle_R_seed_9042.npz
baseline_seed_7042.npz no_delay_seed_7042.npz no_gap_seed_8042.npz no_mu_seed_9042.npz
baseline_seed_8042.npz no_delay_seed_8042.npz no_gap_seed_9042.npz shuffle_R_seed_10042.npz

./runs/5a9f093a2fe0:
AIEF_1_4_5a9f093a2fe0_branch_diagnostics.npz seed_15403081841477312463.npz
seed_1888415619847841128.npz seed_4619122171158768524.npz
AIEF_1_4_5a9f093a2fe0_scalar_summary.csv seed_15853699719446171583.npz
seed_1918007468868772336.npz seed_4932511216876650745.npz
manifest.json seed_16329425431442249417.npz seed_2461295749352584108.npz
seed_5213047409853278064.npz
seed_10431384713483787270.npz seed_16627818730624180313.npz seed_283404770198602602.npz
seed_5415544643585892346.npz
seed_10856823803646276806.npz seed_16952466944611321358.npz seed_305986450614148690.npz
seed_6023008189364820510.npz

seed_1124242675370235686 5.npz seed_172180395821707403.npz seed_3258463328937640595.npz
seed_6108867655028980287.npz
seed_1212351980795175165 0.npz seed_17279552413280415429.npz
seed_3582616062465030267.npz seed_6942591650975756121.npz
seed_1230287949579537403.npz seed_17521310920850592782.npz seed_3704716633526084363.npz
seed_9036321132460399255.npz
seed_12851640484293199234.npz seed_17715532056777442089.npz
seed_3952397966055989215.npz seed_9082055256280075172.npz
seed_14040336450255043598.npz seed_17883847597205629106.npz
seed_4040285944958847150.npz seed_9189988769602851417.npz
seed_14924267685654216746.npz seed_18127452951968972828.npz
seed_4219039243791140697.npz

./runs/73dca37cf93f:
AIEF_1_3_73dca37cf93f_edge_diagnostics.npz no_delay_seed_5727400265110223831.npz
no_mu_seed_17587848749391215576.npz
AIEF_1_3_73dca37cf93f_scalar_summary.csv no_delay_seed_577339501563627643.npz
no_mu_seed_17852680212380503812.npz
AIEF_1_3_analysis_results.csv no_delay_seed_5776146606078111419.npz
no_mu_seed_18195599253746093149.npz
baseline_seed_11078602408408019102.npz no_delay_seed_5996580515567889036.npz
no_mu_seed_18405604921195991417.npz
baseline_seed_11197595738654224886.npz no_delay_seed_6384490036851063518.npz
no_mu_seed_2799856556347805964.npz
baseline_seed_1246923751594727930.npz no_delay_seed_6597351216039696310.npz
no_mu_seed_2841400781150679534.npz
baseline_seed_13104999914199871252.npz no_delay_seed_6810304725465717906.npz
no_mu_seed_3050482280156901559.npz
baseline_seed_13168160597277742379.npz no_delay_seed_6833618658979281150.npz
no_mu_seed_4042692546438370928.npz
baseline_seed_13168918443122321 82.npz no_delay_seed_7550509291629200394.npz
no_mu_seed_4274086224959828596.npz
baseline_seed_14101137954994639910.npz no_delay_seed_7706845139332910089.npz
no_mu_seed_4365343787365008077.npz
baseline_seed_14247790844233709566.npz no_delay_seed_7938073305027935059.npz
no_mu_seed_4723126346505719299.npz
baseline_seed_14413360560776613994.npz no_delay_seed_8063201614575931673.npz
no_mu_seed_4866117689794208263.npz
baseline_seed_14498610575954930932.npz no_delay_seed_8103184518762191864.npz
no_mu_seed_5014145038466735176.npz
baseline_seed_14668531591501948822.npz no_delay_seed_8264568941032381199.npz
no_mu_seed_5304325715998943684.npz

baseline_seed_14743978782784222765.npz no_delay_seed_8526943454688901591.npz
no_mu_seed_536452804929066902.npz
baseline_seed_15011858390678511204.npz no_delay_seed_9127451059220389752.npz
no_mu_seed_5727400265110223831.npz
baseline_seed_159500863463728126.npz no_delay_seed_9156045106123447864.npz
no_mu_seed_577339501563627643.npz
baseline_seed_16649945166774720395.npz no_delay_seed_9198312127684667017.npz
no_mu_seed_5776146606078111419.npz
baseline_seed_1698991527437907300.npz no_delay_seed_956874784835626302.npz
no_mu_seed_5996580515567889036.npz
baseline_seed_17587848749391215576.npz no_gap_seed_11078602408408019102.npz
no_mu_seed_6384490036851063518.npz
baseline_seed_17852680212380503812.npz no_gap_seed_11197595738654224886.npz
no_mu_seed_6597351216039696310.npz
baseline_seed_18195599253746093149.npz no_gap_seed_1246923751594727930.npz
no_mu_seed_6810304725465717906.npz
baseline_seed_18405604921195991417.npz no_gap_seed_13104999914199871252.npz
no_mu_seed_6833618658979281150.npz
baseline_seed_2799856556347805964.npz no_gap_seed_13168160597277742379.npz
no_mu_seed_7550509291629200394.npz
baseline_seed_2841400781150679534.npz no_gap_seed_1316891844312232182.npz
no_mu_seed_7706845139332910089.npz
baseline_seed_3050482280156901559.npz no_gap_seed_14101137954994639910.npz
no_mu_seed_7938073305027935059.npz
baseline_seed_4042692546438370928.npz no_gap_seed_14247790844233709566.npz
no_mu_seed_8063201614575931673.npz
baseline_seed_4274086224959828596.npz no_gap_seed_14413360560776613994.npz
no_mu_seed_8103184518762191864.npz
baseline_seed_4365343787365008077.npz no_gap_seed_14498610575954930932.npz
no_mu_seed_8264568941032381199.npz
baseline_seed_4723126346505719299.npz no_gap_seed_14668531591501948822.npz
no_mu_seed_8526943454688901591.npz
baseline_seed_4866117689794208263.npz no_gap_seed_14743978782784222765.npz
no_mu_seed_9127451059220389752.npz
baseline_seed_5014145038466735176.npz no_gap_seed_15011858390678511204.npz
no_mu_seed_9156045106123447864.npz
baseline_seed_5304325715998943684.npz no_gap_seed_159500863463728126.npz
no_mu_seed_9198312127684667017.npz
baseline_seed_536452804929066902.npz no_gap_seed_16649945166774720395.npz
no_mu_seed_956874784835626302.npz
baseline_seed_5727400265110223831.npz no_gap_seed_1698991527437907300.npz
shuffle_R_seed_11078602408408019102.npz

baseline_seed_577339501563627643.npz no_gap_seed_17587848749391215576.npz
shuffle_R_seed_11197595738654224886.npz
baseline_seed_5776146606078111419.npz no_gap_seed_17852680212380503812.npz
shuffle_R_seed_1246923751594727930.npz
baseline_seed_5996580515567889036.npz no_gap_seed_18195599253746093149.npz
shuffle_R_seed_13104999914199871252.npz
baseline_seed_6384490036851063518.npz no_gap_seed_18405604921195991417.npz
shuffle_R_seed_13168160597277742379.npz
baseline_seed_6597351216039696310.npz no_gap_seed_2799856556347805964.npz
shuffle_R_seed_1316891844312232182.npz
baseline_seed_6810304725465717906.npz no_gap_seed_2841400781150679534.npz
shuffle_R_seed_14101137954994639910.npz
baseline_seed_6833618658979281150.npz no_gap_seed_3050482280156901559.npz
shuffle_R_seed_14247790844233709566.npz
baseline_seed_7550509291629200394.npz no_gap_seed_4042692546438370928.npz
shuffle_R_seed_14413360560776613994.npz
baseline_seed_7706845139332910089.npz no_gap_seed_4274086224959828596.npz
shuffle_R_seed_14498610575954930932.npz
baseline_seed_7938073305027935059.npz no_gap_seed_4365343787365008077.npz
shuffle_R_seed_14668531591501948822.npz
baseline_seed_8063201614575931673.npz no_gap_seed_4723126346505719299.npz
shuffle_R_seed_14743978782784222765.npz
baseline_seed_8103184518762191864.npz no_gap_seed_4866117689794208263.npz
shuffle_R_seed_15011858390678511204.npz
baseline_seed_8264568941032381199.npz no_gap_seed_5014145038466735176.npz
shuffle_R_seed_159500863463728126.npz
baseline_seed_8526943454688901591.npz no_gap_seed_5304325715998943684.npz
shuffle_R_seed_16649945166774720395.npz
baseline_seed_9127451059220389752.npz no_gap_seed_536452804929066902.npz
shuffle_R_seed_1698991527437907300.npz
baseline_seed_9156045106123447864.npz no_gap_seed_5727400265110223831.npz
shuffle_R_seed_17587848749391215576.npz
baseline_seed_9198312127684667017.npz no_gap_seed_577339501563627643.npz
shuffle_R_seed_17852680212380503812.npz
baseline_seed_956874784835626302.npz no_gap_seed_5776146606078111419.npz
shuffle_R_seed_18195599253746093149.npz
manifest.json no_gap_seed_5996580515567889036.npz shuffle_R_seed_18405604921195991417.npz
no_delay_seed_11078602408408019102.npz no_gap_seed_6384490036851063518.npz
shuffle_R_seed_2799856556347805964.npz
no_delay_seed_11197595738654224886.npz no_gap_seed_6597351216039696310.npz
shuffle_R_seed_2841400781150679534.npz

no_delay_seed_1246923751594727930.npz no_gap_seed_6810304725465717906.npz
shuffle_R_seed_3050482280156901559.npz
no_delay_seed_13104999914199871252.npz no_gap_seed_6833618658979281150.npz
shuffle_R_seed_4042692546438370928.npz
no_delay_seed_13168160597277742379.npz no_gap_seed_7550509291629200394.npz
shuffle_R_seed_427408624959828596.npz
no_delay_seed_13168918443122321 82.npz no_gap_seed_7706845139332910089.npz
shuffle_R_seed_4365343787365008077.npz
no_delay_seed_14101137954994639910.npz no_gap_seed_7938073305027935059.npz
shuffle_R_seed_4723126346505719299.npz
no_delay_seed_14247790844233709566.npz no_gap_seed_8063201614575931673.npz
shuffle_R_seed_4866117689794208263.npz
no_delay_seed_14413360560776613994.npz no_gap_seed_8103184518762191864.npz
shuffle_R_seed_5014145038466735176.npz
no_delay_seed_14498610575954930932.npz no_gap_seed_8264568941032381199.npz
shuffle_R_seed_5304325715998943684.npz
no_delay_seed_14668531591501948822.npz no_gap_seed_8526943454688901591.npz
shuffle_R_seed_536452804929066902.npz
no_delay_seed_14743978782784222765.npz no_gap_seed_9127451059220389752.npz
shuffle_R_seed_5727400265110223831.npz
no_delay_seed_15011858390678511204.npz no_gap_seed_9156045106123447864.npz
shuffle_R_seed_577339501563627643.npz
no_delay_seed_159500863463728126.npz no_gap_seed_9198312127684667017.npz
shuffle_R_seed_5776146606078111419.npz
no_delay_seed_16649945166774720395.npz no_gap_seed_956874784835626302.npz
shuffle_R_seed_5996580515567889036.npz
no_delay_seed_16989915274379073 00.npz no_mu_seed_11078602408408019102.npz
shuffle_R_seed_6384490036851063518.npz
no_delay_seed_17587848749391215576.npz no_mu_seed_11197595738654224886.npz
shuffle_R_seed_659735121603969 6310.npz
no_delay_seed_17852680212380503812.npz no_mu_seed_1246923751594727930.npz
shuffle_R_seed_6810304725465717906.npz
no_delay_seed_18195599253746093149.npz no_mu_seed_13104999914199871252.npz
shuffle_R_seed_6833618658979281150.npz
no_delay_seed_18405604921195991417.npz no_mu_seed_13168160597277742379.npz
shuffle_R_seed_7550509291629200394.npz
no_delay_seed_2799856556347805964.npz no_mu_seed_13168918443122321 82.npz
shuffle_R_seed_7706845139332910089.npz
no_delay_seed_2841400781150679534.npz no_mu_seed_14101137954994639910.npz
shuffle_R_seed_7938073305027935059.npz
no_delay_seed_3050482280156901559.npz no_mu_seed_14247790844233709566.npz
shuffle_R_seed_8063201614575931673.npz

no_delay_seed_4042692546438370928.npz no_mu_seed_14413360560776613994.npz
shuffle_R_seed_8103184518762191864.npz
no_delay_seed_4274086224959828596.npz no_mu_seed_14498610575954930932.npz
shuffle_R_seed_8264568941032381199.npz
no_delay_seed_4365343787365008077.npz no_mu_seed_14668531591501948822.npz
shuffle_R_seed_8526943454688901591.npz
no_delay_seed_4723126346505719299.npz no_mu_seed_14743978782784222765.npz
shuffle_R_seed_9127451059220389752.npz
no_delay_seed_4866117689794208263.npz no_mu_seed_15011858390678511204.npz
shuffle_R_seed_9156045106123447864.npz
no_delay_seed_5014145038466735176.npz no_mu_seed_159500863463728126.npz
shuffle_R_seed_9198312127684667017.npz
no_delay_seed_5304325715998943684.npz no_mu_seed_16649945166774720395.npz
shuffle_R_seed_956874784835626302.npz
no_delay_seed_536452804929066902.npz no_mu_seed_1698991527437907300.npz

./runs/87dfe5798954:
AIEF_1_3_87dfe5798954_edge_diagnostics.npz baseline_seed_8042.npz no_delay_seed_8042.npz
no_gap_seed_9042.npz shuffle_R_seed_10042.npz
AIEF_1_3_87dfe5798954_scalar_summary.csv baseline_seed_9042.npz no_delay_seed_9042.npz
no_mu_seed_10042.npz shuffle_R_seed_1042.npz
AIEF_1_3_analysis_results.csv manifest.json no_gap_seed_10042.npz no_mu_seed_1042.npz
shuffle_R_seed_11042.npz
baseline_seed_10042.npz no_delay_seed_10042.npz no_gap_seed_1042.npz no_mu_seed_11042.npz
shuffle_R_seed_12042.npz
baseline_seed_1042.npz no_delay_seed_1042.npz no_gap_seed_11042.npz no_mu_seed_12042.npz
shuffle_R_seed_13042.npz
baseline_seed_11042.npz no_delay_seed_11042.npz no_gap_seed_12042.npz no_mu_seed_13042.npz
shuffle_R_seed_14042.npz
baseline_seed_12042.npz no_delay_seed_12042.npz no_gap_seed_13042.npz no_mu_seed_14042.npz
shuffle_R_seed_15042.npz
baseline_seed_13042.npz no_delay_seed_13042.npz no_gap_seed_14042.npz no_mu_seed_15042.npz
shuffle_R_seed_16042.npz
baseline_seed_14042.npz no_delay_seed_14042.npz no_gap_seed_15042.npz no_mu_seed_16042.npz
shuffle_R_seed_17042.npz
baseline_seed_15042.npz no_delay_seed_15042.npz no_gap_seed_16042.npz no_mu_seed_17042.npz
shuffle_R_seed_18042.npz
baseline_seed_16042.npz no_delay_seed_16042.npz no_gap_seed_17042.npz no_mu_seed_18042.npz
shuffle_R_seed_19042.npz
baseline_seed_17042.npz no_delay_seed_17042.npz no_gap_seed_18042.npz no_mu_seed_19042.npz
shuffle_R_seed_2042.npz

baseline_seed_18042.npz no_delay_seed_18042.npz no_gap_seed_19042.npz no_mu_seed_2042.npz shuffle_R_seed_3042.npz
baseline_seed_19042.npz no_delay_seed_19042.npz no_gap_seed_2042.npz no_mu_seed_3042.npz shuffle_R_seed_4042.npz
baseline_seed_2042.npz no_delay_seed_2042.npz no_gap_seed_3042.npz no_mu_seed_4042.npz shuffle_R_seed_42.npz
baseline_seed_3042.npz no_delay_seed_3042.npz no_gap_seed_4042.npz no_mu_seed_42.npz shuffle_R_seed_5042.npz
baseline_seed_4042.npz no_delay_seed_4042.npz no_gap_seed_42.npz no_mu_seed_5042.npz shuffle_R_seed_6042.npz
baseline_seed_42.npz no_delay_seed_42.npz no_gap_seed_5042.npz no_mu_seed_6042.npz shuffle_R_seed_7042.npz
baseline_seed_5042.npz no_delay_seed_5042.npz no_gap_seed_6042.npz no_mu_seed_7042.npz shuffle_R_seed_8042.npz
baseline_seed_6042.npz no_delay_seed_6042.npz no_gap_seed_7042.npz no_mu_seed_8042.npz shuffle_R_seed_9042.npz
baseline_seed_7042.npz no_delay_seed_7042.npz no_gap_seed_8042.npz no_mu_seed_9042.npz

./runs/92dd4ddbdba1:
AIEF_1_3_92dd4ddbdba1_edge_diagnostics.npz baseline_seed_9042.npz no_delay_seed_9042.npz no_mu_seed_10042.npz shuffle_R_seed_1042.npz
AIEF_1_3_92dd4ddbdba1_scalar_summary.csv manifest.json no_gap_seed_10042.npz no_mu_seed_1042.npz shuffle_R_seed_11042.npz
baseline_seed_10042.npz no_delay_seed_10042.npz no_gap_seed_1042.npz no_mu_seed_11042.npz shuffle_R_seed_12042.npz
baseline_seed_1042.npz no_delay_seed_1042.npz no_gap_seed_11042.npz no_mu_seed_12042.npz shuffle_R_seed_13042.npz
baseline_seed_11042.npz no_delay_seed_11042.npz no_gap_seed_12042.npz no_mu_seed_13042.npz shuffle_R_seed_14042.npz
baseline_seed_12042.npz no_delay_seed_12042.npz no_gap_seed_13042.npz no_mu_seed_14042.npz shuffle_R_seed_15042.npz
baseline_seed_13042.npz no_delay_seed_13042.npz no_gap_seed_14042.npz no_mu_seed_15042.npz shuffle_R_seed_16042.npz
baseline_seed_14042.npz no_delay_seed_14042.npz no_gap_seed_15042.npz no_mu_seed_16042.npz shuffle_R_seed_17042.npz
baseline_seed_15042.npz no_delay_seed_15042.npz no_gap_seed_16042.npz no_mu_seed_17042.npz shuffle_R_seed_18042.npz
baseline_seed_16042.npz no_delay_seed_16042.npz no_gap_seed_17042.npz no_mu_seed_18042.npz shuffle_R_seed_19042.npz
baseline_seed_17042.npz no_delay_seed_17042.npz no_gap_seed_18042.npz no_mu_seed_19042.npz shuffle_R_seed_2042.npz

baseline_seed_18042.npz no_delay_seed_18042.npz no_gap_seed_19042.npz no_mu_seed_2042.npz shuffle_R_seed_3042.npz
baseline_seed_19042.npz no_delay_seed_19042.npz no_gap_seed_2042.npz no_mu_seed_3042.npz shuffle_R_seed_4042.npz
baseline_seed_2042.npz no_delay_seed_2042.npz no_gap_seed_3042.npz no_mu_seed_4042.npz shuffle_R_seed_42.npz
baseline_seed_3042.npz no_delay_seed_3042.npz no_gap_seed_4042.npz no_mu_seed_42.npz shuffle_R_seed_5042.npz
baseline_seed_4042.npz no_delay_seed_4042.npz no_gap_seed_42.npz no_mu_seed_5042.npz shuffle_R_seed_6042.npz
baseline_seed_42.npz no_delay_seed_42.npz no_gap_seed_5042.npz no_mu_seed_6042.npz shuffle_R_seed_7042.npz
baseline_seed_5042.npz no_delay_seed_5042.npz no_gap_seed_6042.npz no_mu_seed_7042.npz shuffle_R_seed_8042.npz
baseline_seed_6042.npz no_delay_seed_6042.npz no_gap_seed_7042.npz no_mu_seed_8042.npz shuffle_R_seed_9042.npz
baseline_seed_7042.npz no_delay_seed_7042.npz no_gap_seed_8042.npz no_mu_seed_9042.npz
baseline_seed_8042.npz no_delay_seed_8042.npz no_gap_seed_9042.npz shuffle_R_seed_10042.npz

./runs/9fb83552f0ce:
AIEF_1_3_9fb83552f0ce_edge_diagnostics.npz baseline_seed_9042.npz no_delay_seed_9042.npz no_mu_seed_10042.npz shuffle_R_seed_1042.npz
AIEF_1_3_9fb83552f0ce_scalar_summary.csv manifest.json no_gap_seed_10042.npz no_mu_seed_1042.npz shuffle_R_seed_11042.npz
baseline_seed_10042.npz no_delay_seed_10042.npz no_gap_seed_1042.npz no_mu_seed_11042.npz shuffle_R_seed_12042.npz
baseline_seed_1042.npz no_delay_seed_1042.npz no_gap_seed_11042.npz no_mu_seed_12042.npz shuffle_R_seed_13042.npz
baseline_seed_11042.npz no_delay_seed_11042.npz no_gap_seed_12042.npz no_mu_seed_13042.npz shuffle_R_seed_14042.npz
baseline_seed_12042.npz no_delay_seed_12042.npz no_gap_seed_13042.npz no_mu_seed_14042.npz shuffle_R_seed_15042.npz
baseline_seed_13042.npz no_delay_seed_13042.npz no_gap_seed_14042.npz no_mu_seed_15042.npz shuffle_R_seed_16042.npz
baseline_seed_14042.npz no_delay_seed_14042.npz no_gap_seed_15042.npz no_mu_seed_16042.npz shuffle_R_seed_17042.npz
baseline_seed_15042.npz no_delay_seed_15042.npz no_gap_seed_16042.npz no_mu_seed_17042.npz shuffle_R_seed_18042.npz
baseline_seed_16042.npz no_delay_seed_16042.npz no_gap_seed_17042.npz no_mu_seed_18042.npz shuffle_R_seed_19042.npz
baseline_seed_17042.npz no_delay_seed_17042.npz no_gap_seed_18042.npz no_mu_seed_19042.npz shuffle_R_seed_2042.npz

baseline_seed_18042.npz no_delay_seed_18042.npz no_gap_seed_19042.npz no_mu_seed_2042.npz shuffle_R_seed_3042.npz
baseline_seed_19042.npz no_delay_seed_19042.npz no_gap_seed_2042.npz no_mu_seed_3042.npz shuffle_R_seed_4042.npz
baseline_seed_2042.npz no_delay_seed_2042.npz no_gap_seed_3042.npz no_mu_seed_4042.npz shuffle_R_seed_42.npz
baseline_seed_3042.npz no_delay_seed_3042.npz no_gap_seed_4042.npz no_mu_seed_42.npz shuffle_R_seed_5042.npz
baseline_seed_4042.npz no_delay_seed_4042.npz no_gap_seed_42.npz no_mu_seed_5042.npz shuffle_R_seed_6042.npz
baseline_seed_42.npz no_delay_seed_42.npz no_gap_seed_5042.npz no_mu_seed_6042.npz shuffle_R_seed_7042.npz
baseline_seed_5042.npz no_delay_seed_5042.npz no_gap_seed_6042.npz no_mu_seed_7042.npz shuffle_R_seed_8042.npz
baseline_seed_6042.npz no_delay_seed_6042.npz no_gap_seed_7042.npz no_mu_seed_8042.npz shuffle_R_seed_9042.npz
baseline_seed_7042.npz no_delay_seed_7042.npz no_gap_seed_8042.npz no_mu_seed_9042.npz
baseline_seed_8042.npz no_delay_seed_8042.npz no_gap_seed_9042.npz shuffle_R_seed_10042.npz

./runs/a3dcc58f4939:
AIEF_1_3_a3dcc58f4939_edge_diagnostics.npz baseline_seed_9042.npz no_delay_seed_9042.npz no_mu_seed_10042.npz shuffle_R_seed_1042.npz
AIEF_1_3_a3dcc58f4939_scalar_summary.csv manifest.json no_gap_seed_10042.npz no_mu_seed_1042.npz shuffle_R_seed_11042.npz
baseline_seed_10042.npz no_delay_seed_10042.npz no_gap_seed_1042.npz no_mu_seed_11042.npz shuffle_R_seed_12042.npz
baseline_seed_1042.npz no_delay_seed_1042.npz no_gap_seed_11042.npz no_mu_seed_12042.npz shuffle_R_seed_13042.npz
baseline_seed_11042.npz no_delay_seed_11042.npz no_gap_seed_12042.npz no_mu_seed_13042.npz shuffle_R_seed_14042.npz
baseline_seed_12042.npz no_delay_seed_12042.npz no_gap_seed_13042.npz no_mu_seed_14042.npz shuffle_R_seed_15042.npz
baseline_seed_13042.npz no_delay_seed_13042.npz no_gap_seed_14042.npz no_mu_seed_15042.npz shuffle_R_seed_16042.npz
baseline_seed_14042.npz no_delay_seed_14042.npz no_gap_seed_15042.npz no_mu_seed_16042.npz shuffle_R_seed_17042.npz
baseline_seed_15042.npz no_delay_seed_15042.npz no_gap_seed_16042.npz no_mu_seed_17042.npz shuffle_R_seed_18042.npz
baseline_seed_16042.npz no_delay_seed_16042.npz no_gap_seed_17042.npz no_mu_seed_18042.npz shuffle_R_seed_19042.npz
baseline_seed_17042.npz no_delay_seed_17042.npz no_gap_seed_18042.npz no_mu_seed_19042.npz shuffle_R_seed_2042.npz

baseline_seed_18042.npz no_delay_seed_18042.npz no_gap_seed_19042.npz no_mu_seed_2042.npz shuffle_R_seed_3042.npz
baseline_seed_19042.npz no_delay_seed_19042.npz no_gap_seed_2042.npz no_mu_seed_3042.npz shuffle_R_seed_4042.npz
baseline_seed_2042.npz no_delay_seed_2042.npz no_gap_seed_3042.npz no_mu_seed_4042.npz shuffle_R_seed_42.npz
baseline_seed_3042.npz no_delay_seed_3042.npz no_gap_seed_4042.npz no_mu_seed_42.npz shuffle_R_seed_5042.npz
baseline_seed_4042.npz no_delay_seed_4042.npz no_gap_seed_42.npz no_mu_seed_5042.npz shuffle_R_seed_6042.npz
baseline_seed_42.npz no_delay_seed_42.npz no_gap_seed_5042.npz no_mu_seed_6042.npz shuffle_R_seed_7042.npz
baseline_seed_5042.npz no_delay_seed_5042.npz no_gap_seed_6042.npz no_mu_seed_7042.npz shuffle_R_seed_8042.npz
baseline_seed_6042.npz no_delay_seed_6042.npz no_gap_seed_7042.npz no_mu_seed_8042.npz shuffle_R_seed_9042.npz
baseline_seed_7042.npz no_delay_seed_7042.npz no_gap_seed_8042.npz no_mu_seed_9042.npz
baseline_seed_8042.npz no_delay_seed_8042.npz no_gap_seed_9042.npz shuffle_R_seed_10042.npz

./runs/a937d5e96efb:
baseline_seed_12232032881337366345.npz baseline_seed_13443745119513809312.npz
baseline_seed_9723627428708784598.npz
baseline_seed_12681060683294206227.npz baseline_seed_15655786368575661155.npz manifest.json
baseline_seed_13048312801923903802.npz baseline_seed_7157263871600225889.npz

./runs/b57562a8ee77:
AIEF_1_4_Twindow_sensitivity_b57562a8ee77_branch_diagnostics.npz
condition_seed_1888415619847841128_rho_2.00.npz
AIEF_1_4_Twindow_sensitivity_b57562a8ee77_scalar_summary.csv
condition_seed_1918007468868772336_rho_0.50.npz
analysis condition_seed_1918007468868772336_rho_0.75.npz
baseline_equivalence_by_seed.json condition_seed_1918007468868772336_rho_1.00.npz
baseline_equivalence_certificate.json condition_seed_1918007468868772336_rho_1.50.npz
condition_seed_10431384713483787270_rho_0.50.npz
condition_seed_1918007468868772336_rho_2.00.npz
condition_seed_10431384713483787270_rho_0.75.npz
condition_seed_2461295749352584108_rho_0.50.npz
condition_seed_10431384713483787270_rho_1.00.npz
condition_seed_2461295749352584108_rho_0.75.npz
condition_seed_10431384713483787270_rho_1.50.npz
condition_seed_2461295749352584108_rho_1.00.npz

condition_seed_10431384713483787270_rho_2.00.npz
condition_seed_2461295749352584108_rho_1.50.npz
condition_seed_10856823803646276806_rho_0.50.npz
condition_seed_2461295749352584108_rho_2.00.npz
condition_seed_10856823803646276806_rho_0.75.npz
condition_seed_283404770198602602_rho_0.50.npz
condition_seed_10856823803646276806_rho_1.00.npz
condition_seed_283404770198602602_rho_0.75.npz
condition_seed_10856823803646276806_rho_1.50.npz
condition_seed_283404770198602602_rho_1.00.npz
condition_seed_10856823803646276806_rho_2.00.npz
condition_seed_283404770198602602_rho_1.50.npz
condition_seed_11242426753702356865_rho_0.50.npz
condition_seed_283404770198602602_rho_2.00.npz
condition_seed_11242426753702356865_rho_0.75.npz
condition_seed_305986450614148690_rho_0.50.npz
condition_seed_11242426753702356865_rho_1.00.npz
condition_seed_305986450614148690_rho_0.75.npz
condition_seed_11242426753702356865_rho_1.50.npz
condition_seed_305986450614148690_rho_1.00.npz
condition_seed_11242426753702356865_rho_2.00.npz
condition_seed_305986450614148690_rho_1.50.npz
condition_seed_12123519807951751650_rho_0.50.npz
condition_seed_305986450614148690_rho_2.00.npz
condition_seed_12123519807951751650_rho_0.75.npz
condition_seed_3258463328937640595_rho_0.50.npz
condition_seed_12123519807951751650_rho_1.00.npz
condition_seed_3258463328937640595_rho_0.75.npz
condition_seed_12123519807951751650_rho_1.50.npz
condition_seed_3258463328937640595_rho_1.00.npz
condition_seed_12123519807951751650_rho_2.00.npz
condition_seed_3258463328937640595_rho_1.50.npz
condition_seed_1230287949579537403_rho_0.50.npz
condition_seed_3258463328937640595_rho_2.00.npz
condition_seed_1230287949579537403_rho_0.75.npz
condition_seed_3582616062465030267_rho_0.50.npz
condition_seed_1230287949579537403_rho_1.00.npz
condition_seed_3582616062465030267_rho_0.75.npz
condition_seed_1230287949579537403_rho_1.50.npz
condition_seed_3582616062465030267_rho_1.00.npz
condition_seed_1230287949579537403_rho_2.00.npz
condition_seed_3582616062465030267_rho_1.50.npz

condition_seed_12851640484293199234_rho_0.50.npz
condition_seed_3582616062465030267_rho_2.00.npz
condition_seed_12851640484293199234_rho_0.75.npz
condition_seed_3704716633526084363_rho_0.50.npz
condition_seed_12851640484293199234_rho_1.00.npz
condition_seed_3704716633526084363_rho_0.75.npz
condition_seed_12851640484293199234_rho_1.50.npz
condition_seed_3704716633526084363_rho_1.00.npz
condition_seed_12851640484293199234_rho_2.00.npz
condition_seed_3704716633526084363_rho_1.50.npz
condition_seed_14040336450255043598_rho_0.50.npz
condition_seed_3704716633526084363_rho_2.00.npz
condition_seed_14040336450255043598_rho_0.75.npz
condition_seed_3952397966055989215_rho_0.50.npz
condition_seed_14040336450255043598_rho_1.00.npz
condition_seed_3952397966055989215_rho_0.75.npz
condition_seed_14040336450255043598_rho_1.50.npz
condition_seed_3952397966055989215_rho_1.00.npz
condition_seed_14040336450255043598_rho_2.00.npz
condition_seed_3952397966055989215_rho_1.50.npz
condition_seed_14924267685654216746_rho_0.50.npz
condition_seed_3952397966055989215_rho_2.00.npz
condition_seed_14924267685654216746_rho_0.75.npz
condition_seed_4040285944958847150_rho_0.50.npz
condition_seed_14924267685654216746_rho_1.00.npz
condition_seed_4040285944958847150_rho_0.75.npz
condition_seed_14924267685654216746_rho_1.50.npz
condition_seed_4040285944958847150_rho_1.00.npz
condition_seed_14924267685654216746_rho_2.00.npz
condition_seed_4040285944958847150_rho_1.50.npz
condition_seed_15403081841477312463_rho_0.50.npz
condition_seed_4040285944958847150_rho_2.00.npz
condition_seed_15403081841477312463_rho_0.75.npz
condition_seed_4219039243791140697_rho_0.50.npz
condition_seed_15403081841477312463_rho_1.00.npz
condition_seed_4219039243791140697_rho_0.75.npz
condition_seed_15403081841477312463_rho_1.50.npz
condition_seed_4219039243791140697_rho_1.00.npz
condition_seed_15403081841477312463_rho_2.00.npz
condition_seed_4219039243791140697_rho_1.50.npz
condition_seed_15853699719446171583_rho_0.50.npz
condition_seed_4219039243791140697_rho_2.00.npz

condition_seed_15853699719446171583_rho_0.75.npz
condition_seed_46191221711587685 24_rho_0.50.npz
condition_seed_15853699719446171583_rho_1.00.npz
condition_seed_46191221711587685 24_rho_0.75.npz
condition_seed_15853699719446171583_rho_1.50.npz
condition_seed_46191221711587685 24_rho_1.00.npz
condition_seed_15853699719446171583_rho_2.00.npz
condition_seed_46191221711587685 24_rho_1.50.npz
condition_seed_16329425431442249417_rho_0.50.npz
condition_seed_46191221711587685 24_rho_2.00.npz
condition_seed_16329425431442249417_rho_0.75.npz
condition_seed_49325112168766507 45_rho_0.50.npz
condition_seed_16329425431442249417_rho_1.00.npz
condition_seed_49325112168766507 45_rho_0.75.npz
condition_seed_16329425431442249417_rho_1.50.npz
condition_seed_49325112168766507 45_rho_1.00.npz
condition_seed_16329425431442249417_rho_2.00.npz
condition_seed_49325112168766507 45_rho_1.50.npz
condition_seed_16627818730624180313_rho_0.50.npz
condition_seed_49325112168766507 45_rho_2.00.npz
condition_seed_16627818730624180313_rho_0.75.npz
condition_seed_52130474098532780 64_rho_0.50.npz
condition_seed_16627818730624180313_rho_1.00.npz
condition_seed_52130474098532780 64_rho_0.75.npz
condition_seed_16627818730624180313_rho_1.50.npz
condition_seed_52130474098532780 64_rho_1.00.npz
condition_seed_16627818730624180313_rho_2.00.npz
condition_seed_52130474098532780 64_rho_1.50.npz
condition_seed_16952466944611321358_rho_0.50.npz
condition_seed_52130474098532780 64_rho_2.00.npz
condition_seed_16952466944611321358_rho_0.75.npz
condition_seed_54155446435858923 46_rho_0.50.npz
condition_seed_16952466944611321358_rho_1.00.npz
condition_seed_54155446435858923 46_rho_0.75.npz
condition_seed_16952466944611321358_rho_1.50.npz
condition_seed_54155446435858923 46_rho_1.00.npz
condition_seed_16952466944611321358_rho_2.00.npz
condition_seed_54155446435858923 46_rho_1.50.npz
condition_seed_172180395821707403_rho_0.50.npz
condition_seed_54155446435858923 46_rho_2.00.npz
condition_seed_172180395821707403_rho_0.75.npz
condition_seed_60230081893648205 10_rho_0.50.npz

condition_seed_172180395821707403_rho_1.00.npz
condition_seed_6023008189364820510_rho_0.75.npz
condition_seed_172180395821707403_rho_1.50.npz
condition_seed_6023008189364820510_rho_1.00.npz
condition_seed_172180395821707403_rho_2.00.npz
condition_seed_6023008189364820510_rho_1.50.npz
condition_seed_17279552413280415429_rho_0.50.npz
condition_seed_6023008189364820510_rho_2.00.npz
condition_seed_17279552413280415429_rho_0.75.npz
condition_seed_6108867655028980287_rho_0.50.npz
condition_seed_17279552413280415429_rho_1.00.npz
condition_seed_6108867655028980287_rho_0.75.npz
condition_seed_17279552413280415429_rho_1.50.npz
condition_seed_6108867655028980287_rho_1.00.npz
condition_seed_17279552413280415429_rho_2.00.npz
condition_seed_6108867655028980287_rho_1.50.npz
condition_seed_17521310920850592782_rho_0.50.npz
condition_seed_6108867655028980287_rho_2.00.npz
condition_seed_17521310920850592782_rho_0.75.npz
condition_seed_6942591650975756121_rho_0.50.npz
condition_seed_17521310920850592782_rho_1.00.npz
condition_seed_6942591650975756121_rho_0.75.npz
condition_seed_17521310920850592782_rho_1.50.npz
condition_seed_6942591650975756121_rho_1.00.npz
condition_seed_17521310920850592782_rho_2.00.npz
condition_seed_6942591650975756121_rho_1.50.npz
condition_seed_17715532056777442089_rho_0.50.npz
condition_seed_6942591650975756121_rho_2.00.npz
condition_seed_17715532056777442089_rho_0.75.npz
condition_seed_9036321132460399255_rho_0.50.npz
condition_seed_17715532056777442089_rho_1.00.npz
condition_seed_9036321132460399255_rho_0.75.npz
condition_seed_17715532056777442089_rho_1.50.npz
condition_seed_9036321132460399255_rho_1.00.npz
condition_seed_17715532056777442089_rho_2.00.npz
condition_seed_9036321132460399255_rho_1.50.npz
condition_seed_17883847597205629106_rho_0.50.npz
condition_seed_9036321132460399255_rho_2.00.npz
condition_seed_17883847597205629106_rho_0.75.npz
condition_seed_9082055256280075172_rho_0.50.npz
condition_seed_17883847597205629106_rho_1.00.npz
condition_seed_9082055256280075172_rho_0.75.npz

condition_seed_17883847597205629106_rho_1.50.npz
condition_seed_9082055256280075172_rho_1.00.npz
condition_seed_17883847597205629106_rho_2.00.npz
condition_seed_9082055256280075172_rho_1.50.npz
condition_seed_18127452951968972828_rho_0.50.npz
condition_seed_9082055256280075172_rho_2.00.npz
condition_seed_18127452951968972828_rho_0.75.npz
condition_seed_9189988769602851417_rho_0.50.npz
condition_seed_18127452951968972828_rho_1.00.npz
condition_seed_9189988769602851417_rho_0.75.npz
condition_seed_18127452951968972828_rho_1.50.npz
condition_seed_9189988769602851417_rho_1.00.npz
condition_seed_18127452951968972828_rho_2.00.npz
condition_seed_9189988769602851417_rho_1.50.npz
condition_seed_1888415619847841128_rho_0.50.npz
condition_seed_9189988769602851417_rho_2.00.npz
condition_seed_1888415619847841128_rho_0.75.npz manifest.json
condition_seed_1888415619847841128_rho_1.00.npz twindow_condition_status.csv
condition_seed_1888415619847841128_rho_1.50.npz

./runs/b57562a8ee77/analysis:
analysis_manifest.json twindow_cross_condition_topology.csv
twindow_recovery_interactions_summary.csv
twindow_branch_DID_secondary_summary.csv twindow_paired_regime_contrasts.csv
twindow_recovery_summary.csv
twindow_branch_DID_summary.csv twindow_paired_regime_contrasts_fixed_physical.csv
twindow_regime_secondary_summary.csv
twindow_branch_interactions.csv twindow_paired_regime_contrasts_normalized_EMA.csv
twindow_regime_summary.csv
twindow_branch_interactions_fixed_physical.csv twindow_recovery_by_seed.csv
twindow_sensitivity_report.txt
twindow_branch_interactions_normalized_EMA.csv twindow_recovery_interactions.csv
twindow_topology_summary.csv

./runs/c3da1ac707bf:
AIEF_1_3_analysis_results.csv no_delay_seed_17942362857188767334.npz
no_mu_seed_12990827071890121233.npz
AIEF_1_3_c3da1ac707bf_edge_diagnostics.npz no_delay_seed_18120325990998516096.npz
no_mu_seed_13011308780887378248.npz
AIEF_1_3_c3da1ac707bf_scalar_summary.csv no_delay_seed_3464540282706013453.npz
no_mu_seed_13144254950108472985.npz

baseline_seed_10138059242446602359.npz no_delay_seed_3867900559517386202.npz
no_mu_seed_13475711373734757509.npz
baseline_seed_10162605287015011567.npz no_delay_seed_4004306315412470776.npz
no_mu_seed_13877067520498498525.npz
baseline_seed_10280671141729910962.npz no_delay_seed_4026263611424279724.npz
no_mu_seed_14789669151170384611.npz
baseline_seed_10414151041756256180.npz no_delay_seed_4213031461048503485.npz
no_mu_seed_15538769139318490183.npz
baseline_seed_10460494971182033875.npz no_delay_seed_4225958022653391162.npz
no_mu_seed_15696189914383701419.npz
baseline_seed_10500527434474296483.npz no_delay_seed_4297606350873079899.npz
no_mu_seed_16004921505170220512.npz
baseline_seed_10619621485132495208.npz no_delay_seed_4773450812113592330.npz
no_mu_seed_16165962154861038927.npz
baseline_seed_1071435888660092917.npz no_delay_seed_4884909319511143462.npz
no_mu_seed_16301226009792754359.npz
baseline_seed_11552247507883116273.npz no_delay_seed_4989123116796778357.npz
no_mu_seed_17260448256843220567.npz
baseline_seed_11699415664738731472.npz no_delay_seed_6799375687132975133.npz
no_mu_seed_17619579101733829571.npz
baseline_seed_11760291472930266701.npz no_delay_seed_6799580967308193508.npz
no_mu_seed_17629994763630970995.npz
baseline_seed_12101127255601305672.npz no_delay_seed_6847020926293287670.npz
no_mu_seed_17813068995467247720.npz
baseline_seed_12423041489110211363.npz no_delay_seed_7443702551313363331.npz
no_mu_seed_17942362857188767334.npz
baseline_seed_12672897826997023262.npz no_delay_seed_7518080831024782954.npz
no_mu_seed_18120325990998516096.npz
baseline_seed_12828118411696793387.npz no_delay_seed_7568763936099751332.npz
no_mu_seed_3464540282706013453.npz
baseline_seed_12932465548281524219.npz no_delay_seed_9166811211305776210.npz
no_mu_seed_3867900559517386202.npz
baseline_seed_12990827071890121233.npz no_gap_seed_10138059242446602359.npz
no_mu_seed_4004306315412470776.npz
baseline_seed_13011308780887378248.npz no_gap_seed_10162605287015011567.npz
no_mu_seed_4026263611424279724.npz
baseline_seed_13144254950108472985.npz no_gap_seed_10280671141729910962.npz
no_mu_seed_4213031461048503485.npz
baseline_seed_13475711373734757509.npz no_gap_seed_10414151041756256180.npz
no_mu_seed_4225958022653391162.npz
baseline_seed_13877067520498498525.npz no_gap_seed_10460494971182033875.npz
no_mu_seed_4297606350873079899.npz

baseline_seed_14789669151170384611.npz no_gap_seed_10500527434474296483.npz
no_mu_seed_4773450812113592330.npz
baseline_seed_15538769139318490183.npz no_gap_seed_10619621485132495208.npz
no_mu_seed_4884909319511143462.npz
baseline_seed_15696189914383701419.npz no_gap_seed_1071435888660092917.npz
no_mu_seed_4989123116796778357.npz
baseline_seed_16004921505170220512.npz no_gap_seed_11552247507883116273.npz
no_mu_seed_6799375687132975133.npz
baseline_seed_16165962154861038927.npz no_gap_seed_11699415664738731472.npz
no_mu_seed_6799580967308193508.npz
baseline_seed_16301226009792754359.npz no_gap_seed_11760291472930266701.npz
no_mu_seed_6847020926293287670.npz
baseline_seed_17260448256843220567.npz no_gap_seed_12101127255601305672.npz
no_mu_seed_7443702551313363331.npz
baseline_seed_17619579101733829571.npz no_gap_seed_12423041489110211363.npz
no_mu_seed_7518080831024782954.npz
baseline_seed_17629994763630970995.npz no_gap_seed_12672897826997023262.npz
no_mu_seed_7568763936099751332.npz
baseline_seed_17813068995467247720.npz no_gap_seed_12828118411696793387.npz
no_mu_seed_9166811211305776210.npz
baseline_seed_17942362857188767334.npz no_gap_seed_12932465548281524219.npz
shuffle_R_seed_10138059242446602359.npz
baseline_seed_18120325990998516096.npz no_gap_seed_12990827071890121233.npz
shuffle_R_seed_10162605287015011567.npz
baseline_seed_3464540282706013453.npz no_gap_seed_13011308780887378248.npz
shuffle_R_seed_10280671141729910962.npz
baseline_seed_3867900559517386202.npz no_gap_seed_13144254950108472985.npz
shuffle_R_seed_10414151041756256180.npz
baseline_seed_4004306315412470776.npz no_gap_seed_13475711373734757509.npz
shuffle_R_seed_10460494971182033875.npz
baseline_seed_4026263611424279724.npz no_gap_seed_13877067520498498525.npz
shuffle_R_seed_1050052743447429648 3.npz
baseline_seed_4213031461048503485.npz no_gap_seed_14789669151170384611.npz
shuffle_R_seed_10619621485132495208.npz
baseline_seed_4225958022653391162.npz no_gap_seed_15538769139318490183.npz
shuffle_R_seed_1071435888660092917.npz
baseline_seed_4297606350873079899.npz no_gap_seed_15696189914383701419.npz
shuffle_R_seed_11552247507883116273.npz
baseline_seed_4773450812113592330.npz no_gap_seed_16004921505170220512.npz
shuffle_R_seed_11699415664738731472.npz
baseline_seed_4884909319511143462.npz no_gap_seed_16165962154861038927.npz
shuffle_R_seed_11760291472930266701.npz

baseline_seed_4989123116796778357.npz no_gap_seed_16301226009792754359.npz
shuffle_R_seed_12101127255601305672.npz
baseline_seed_6799375687132975133.npz no_gap_seed_17260448256843220567.npz
shuffle_R_seed_12423041489110211363.npz
baseline_seed_6799580967308193508.npz no_gap_seed_17619579101733829571.npz
shuffle_R_seed_12672897826997023262.npz
baseline_seed_6847020926293287670.npz no_gap_seed_17629994763630970995.npz
shuffle_R_seed_12828118411696793387.npz
baseline_seed_7443702551313363331.npz no_gap_seed_17813068995467247720.npz
shuffle_R_seed_12932465548281524219.npz
baseline_seed_7518080831024782954.npz no_gap_seed_17942362857188767334.npz
shuffle_R_seed_12990827071890121233.npz
baseline_seed_7568763936099751332.npz no_gap_seed_18120325990998516096.npz
shuffle_R_seed_13011308780887378248.npz
baseline_seed_9166811211305776210.npz no_gap_seed_3464540282706013453.npz
shuffle_R_seed_13144254950108472985.npz
manifest.json no_gap_seed_3867900559517386202.npz shuffle_R_seed_13475711373734757509.npz
no_delay_seed_10138059242446602359.npz no_gap_seed_4004306315412470776.npz
shuffle_R_seed_13877067520498498525.npz
no_delay_seed_10162605287015011567.npz no_gap_seed_4026263611424279724.npz
shuffle_R_seed_14789669151170384611.npz
no_delay_seed_10280671141729910962.npz no_gap_seed_4213031461048503485.npz
shuffle_R_seed_15538769139318490183.npz
no_delay_seed_10414151041756256180.npz no_gap_seed_4225958022653391162.npz
shuffle_R_seed_15696189914383701419.npz
no_delay_seed_10460494971182033875.npz no_gap_seed_4297606350873079899.npz
shuffle_R_seed_16004921505170220512.npz
no_delay_seed_10500527434474296483.npz no_gap_seed_4773450812113592330.npz
shuffle_R_seed_16165962154861038927.npz
no_delay_seed_10619621485132495208.npz no_gap_seed_4884909319511143462.npz
shuffle_R_seed_16301226009792754359.npz
no_delay_seed_1071435888660092917.npz no_gap_seed_4989123116796778357.npz
shuffle_R_seed_17260448256843220567.npz
no_delay_seed_11552247507883116273.npz no_gap_seed_6799375687132975133.npz
shuffle_R_seed_17619579101733829571.npz
no_delay_seed_11699415664738731472.npz no_gap_seed_6799580967308193508.npz
shuffle_R_seed_17629994763630970995.npz
no_delay_seed_11760291472930266701.npz no_gap_seed_6847020926293287670.npz
shuffle_R_seed_17813068995467247720.npz
no_delay_seed_12101127255601305672.npz no_gap_seed_7443702551313363331.npz
shuffle_R_seed_17942362857188767334.npz

no_delay_seed_12423041489110211363.npz no_gap_seed_7518080831024782954.npz
shuffle_R_seed_18120325990998516096.npz
no_delay_seed_12672897826997023262.npz no_gap_seed_7568763936099751332.npz
shuffle_R_seed_3464540282706013453.npz
no_delay_seed_12828118411696793387.npz no_gap_seed_9166811211305776210.npz
shuffle_R_seed_3867900559517386202.npz
no_delay_seed_12932465548281524219.npz no_mu_seed_10138059242446602359.npz
shuffle_R_seed_4004306315412470776.npz
no_delay_seed_12990827071890121233.npz no_mu_seed_10162605287015011567.npz
shuffle_R_seed_4026263611424279724.npz
no_delay_seed_13011308780887378248.npz no_mu_seed_10280671141729910962.npz
shuffle_R_seed_4213031461048503485.npz
no_delay_seed_13144254950108472985.npz no_mu_seed_10414151041756256180.npz
shuffle_R_seed_4225958022653391162.npz
no_delay_seed_13475711373734757509.npz no_mu_seed_10460494971182033875.npz
shuffle_R_seed_4297606350873079899.npz
no_delay_seed_13877067520498498525.npz no_mu_seed_10500527434474296483.npz
shuffle_R_seed_4773450812113592330.npz
no_delay_seed_14789669151170384611.npz no_mu_seed_10619621485132495208.npz
shuffle_R_seed_4884909319511143462.npz
no_delay_seed_15538769139318490183.npz no_mu_seed_1071435888660092917.npz
shuffle_R_seed_4989123116796778357.npz
no_delay_seed_15696189914383701419.npz no_mu_seed_11552247507883116273.npz
shuffle_R_seed_6799375687132975133.npz
no_delay_seed_16004921505170220512.npz no_mu_seed_11699415664738731472.npz
shuffle_R_seed_6799580967308193508.npz
no_delay_seed_16165962154861038927.npz no_mu_seed_11760291472930266701.npz
shuffle_R_seed_6847020926293287670.npz
no_delay_seed_16301226009792754359.npz no_mu_seed_12101127255601305672.npz
shuffle_R_seed_7443702551313363331.npz
no_delay_seed_17260448256843220567.npz no_mu_seed_12423041489110211363.npz
shuffle_R_seed_7518080831024782954.npz
no_delay_seed_17619579101733829571.npz no_mu_seed_12672897826997023262.npz
shuffle_R_seed_7568763936099751332.npz
no_delay_seed_17629994763630970995.npz no_mu_seed_12828118411696793387.npz
shuffle_R_seed_9166811211305776210.npz
no_delay_seed_17813068995467247720.npz no_mu_seed_12932465548281524219.npz

./runs/fccebfc14855:
AIEF_1_3_fccebfc14855_edge_diagnostics.npz baseline_seed_9042.npz no_delay_seed_9042.npz
no_mu_seed_10042.npz shuffle_R_seed_1042.npz

AIEF_1_3_fccebfc14855_scalar_summary.csv manifest.json no_gap_seed_10042.npz
no_mu_seed_1042.npz shuffle_R_seed_11042.npz
baseline_seed_10042.npz no_delay_seed_10042.npz no_gap_seed_1042.npz no_mu_seed_11042.npz
shuffle_R_seed_12042.npz
baseline_seed_1042.npz no_delay_seed_1042.npz no_gap_seed_11042.npz no_mu_seed_12042.npz
shuffle_R_seed_13042.npz
baseline_seed_11042.npz no_delay_seed_11042.npz no_gap_seed_12042.npz no_mu_seed_13042.npz
shuffle_R_seed_14042.npz
baseline_seed_12042.npz no_delay_seed_12042.npz no_gap_seed_13042.npz no_mu_seed_14042.npz
shuffle_R_seed_15042.npz
baseline_seed_13042.npz no_delay_seed_13042.npz no_gap_seed_14042.npz no_mu_seed_15042.npz
shuffle_R_seed_16042.npz
baseline_seed_14042.npz no_delay_seed_14042.npz no_gap_seed_15042.npz no_mu_seed_16042.npz
shuffle_R_seed_17042.npz
baseline_seed_15042.npz no_delay_seed_15042.npz no_gap_seed_16042.npz no_mu_seed_17042.npz
shuffle_R_seed_18042.npz
baseline_seed_16042.npz no_delay_seed_16042.npz no_gap_seed_17042.npz no_mu_seed_18042.npz
shuffle_R_seed_19042.npz
baseline_seed_17042.npz no_delay_seed_17042.npz no_gap_seed_18042.npz no_mu_seed_19042.npz
shuffle_R_seed_2042.npz
baseline_seed_18042.npz no_delay_seed_18042.npz no_gap_seed_19042.npz no_mu_seed_2042.npz
shuffle_R_seed_3042.npz
baseline_seed_19042.npz no_delay_seed_19042.npz no_gap_seed_2042.npz no_mu_seed_3042.npz
shuffle_R_seed_4042.npz
baseline_seed_2042.npz no_delay_seed_2042.npz no_gap_seed_3042.npz no_mu_seed_4042.npz
shuffle_R_seed_42.npz
baseline_seed_3042.npz no_delay_seed_3042.npz no_gap_seed_4042.npz no_mu_seed_42.npz
shuffle_R_seed_5042.npz
baseline_seed_4042.npz no_delay_seed_4042.npz no_gap_seed_42.npz no_mu_seed_5042.npz
shuffle_R_seed_6042.npz
baseline_seed_42.npz no_delay_seed_42.npz no_gap_seed_5042.npz no_mu_seed_6042.npz
shuffle_R_seed_7042.npz
baseline_seed_5042.npz no_delay_seed_5042.npz no_gap_seed_6042.npz no_mu_seed_7042.npz
shuffle_R_seed_8042.npz
baseline_seed_6042.npz no_delay_seed_6042.npz no_gap_seed_7042.npz no_mu_seed_8042.npz
shuffle_R_seed_9042.npz
baseline_seed_7042.npz no_delay_seed_7042.npz no_gap_seed_8042.npz no_mu_seed_9042.npz
baseline_seed_8042.npz no_delay_seed_8042.npz no_gap_seed_9042.npz shuffle_R_seed_10042.npz

./runs_aief14_smoke:
ae136b3a165b

./runs_aief14_smoke/ae136b3a165b:
AIEF_1_4_ae136b3a165b_branch_diagnostics.npz manifest.json seed_8824921114796074700.npz
AIEF_1_4_ae136b3a165b_scalar_summary.csv seed_14673009024601869667.npz

./runs_pivot_study:
ebfec9a5c419 PIVOT_STUDY_LABEL.json

./runs_pivot_study/ebfec9a5c419:
AIEF_1_4_Twindow_sensitivity_ebfec9a5c419_branch_diagnostics.npz
condition_seed_1918007468868772336_rho_0.75.npz
AIEF_1_4_Twindow_sensitivity_ebfec9a5c419_scalar_summary.csv
condition_seed_1918007468868772336_rho_1.00.npz
baseline_equivalence_by_seed.json condition_seed_1918007468868772336_rho_1.50.npz
baseline_equivalence_certificate.json condition_seed_2461295749352584108_rho_0.75.npz
condition_seed_10431384713483787270_rho_0.75.npz
condition_seed_2461295749352584108_rho_1.00.npz
condition_seed_10431384713483787270_rho_1.00.npz
condition_seed_2461295749352584108_rho_1.50.npz
condition_seed_10431384713483787270_rho_1.50.npz
condition_seed_283404770198602602_rho_0.75.npz
condition_seed_10856823803646276806_rho_0.75.npz
condition_seed_283404770198602602_rho_1.00.npz
condition_seed_10856823803646276806_rho_1.00.npz
condition_seed_283404770198602602_rho_1.50.npz
condition_seed_10856823803646276806_rho_1.50.npz
condition_seed_305986450614148690_rho_0.75.npz
condition_seed_11242426753702356865_rho_0.75.npz
condition_seed_305986450614148690_rho_1.00.npz
condition_seed_11242426753702356865_rho_1.00.npz
condition_seed_305986450614148690_rho_1.50.npz
condition_seed_11242426753702356865_rho_1.50.npz
condition_seed_3258463328937640595_rho_0.75.npz
condition_seed_12123519807951751650_rho_0.75.npz
condition_seed_3258463328937640595_rho_1.00.npz
condition_seed_12123519807951751650_rho_1.00.npz
condition_seed_3258463328937640595_rho_1.50.npz
condition_seed_12123519807951751650_rho_1.50.npz
condition_seed_3582616062465030267_rho_0.75.npz
condition_seed_1230287949579537403_rho_0.75.npz
condition_seed_3582616062465030267_rho_1.00.npz
condition_seed_1230287949579537403_rho_1.00.npz
condition_seed_3582616062465030267_rho_1.50.npz

condition_seed_1230287949579537403_rho_1.50.npz
condition_seed_3704716633526084363_rho_0.75.npz
condition_seed_12851640484293199234_rho_0.75.npz
condition_seed_3704716633526084363_rho_1.00.npz
condition_seed_12851640484293199234_rho_1.00.npz
condition_seed_3704716633526084363_rho_1.50.npz
condition_seed_12851640484293199234_rho_1.50.npz
condition_seed_3952397966055989215_rho_0.75.npz
condition_seed_14040336450255043598_rho_0.75.npz
condition_seed_3952397966055989215_rho_1.00.npz
condition_seed_14040336450255043598_rho_1.00.npz
condition_seed_3952397966055989215_rho_1.50.npz
condition_seed_14040336450255043598_rho_1.50.npz
condition_seed_4040285944958847150_rho_0.75.npz
condition_seed_14924267685654216746_rho_0.75.npz
condition_seed_4040285944958847150_rho_1.00.npz
condition_seed_14924267685654216746_rho_1.00.npz
condition_seed_4040285944958847150_rho_1.50.npz
condition_seed_14924267685654216746_rho_1.50.npz
condition_seed_4219039243791140697_rho_0.75.npz
condition_seed_15403081841477312463_rho_0.75.npz
condition_seed_4219039243791140697_rho_1.00.npz
condition_seed_15403081841477312463_rho_1.00.npz
condition_seed_4219039243791140697_rho_1.50.npz
condition_seed_15403081841477312463_rho_1.50.npz
condition_seed_4619122171158768524_rho_0.75.npz
condition_seed_15853699719446171583_rho_0.75.npz
condition_seed_4619122171158768524_rho_1.00.npz
condition_seed_15853699719446171583_rho_1.00.npz
condition_seed_4619122171158768524_rho_1.50.npz
condition_seed_15853699719446171583_rho_1.50.npz
condition_seed_4932511216876650745_rho_0.75.npz
condition_seed_16329425431442249417_rho_0.75.npz
condition_seed_4932511216876650745_rho_1.00.npz
condition_seed_16329425431442249417_rho_1.00.npz
condition_seed_4932511216876650745_rho_1.50.npz
condition_seed_16329425431442249417_rho_1.50.npz
condition_seed_5213047409853278064_rho_0.75.npz
condition_seed_16627818730624180313_rho_0.75.npz
condition_seed_5213047409853278064_rho_1.00.npz
condition_seed_16627818730624180313_rho_1.00.npz
condition_seed_5213047409853278064_rho_1.50.npz

condition_seed_16627818730624180313_rho_1.50.npz
condition_seed_5415544643585892346_rho_0.75.npz
condition_seed_16952466944611321358_rho_0.75.npz
condition_seed_5415544643585892346_rho_1.00.npz
condition_seed_16952466944611321358_rho_1.00.npz
condition_seed_5415544643585892346_rho_1.50.npz
condition_seed_16952466944611321358_rho_1.50.npz
condition_seed_6023008189364820510_rho_0.75.npz
condition_seed_172180395821707403_rho_0.75.npz
condition_seed_6023008189364820510_rho_1.00.npz
condition_seed_172180395821707403_rho_1.00.npz
condition_seed_6023008189364820510_rho_1.50.npz
condition_seed_172180395821707403_rho_1.50.npz
condition_seed_6108867655028980287_rho_0.75.npz
condition_seed_17279552413280415429_rho_0.75.npz
condition_seed_6108867655028980287_rho_1.00.npz
condition_seed_17279552413280415429_rho_1.00.npz
condition_seed_6108867655028980287_rho_1.50.npz
condition_seed_17279552413280415429_rho_1.50.npz
condition_seed_6942591650975756121_rho_0.75.npz
condition_seed_17521310920850592782_rho_0.75.npz
condition_seed_6942591650975756121_rho_1.00.npz
condition_seed_17521310920850592782_rho_1.00.npz
condition_seed_6942591650975756121_rho_1.50.npz
condition_seed_17521310920850592782_rho_1.50.npz
condition_seed_9036321132460399255_rho_0.75.npz
condition_seed_17715532056777442089_rho_0.75.npz
condition_seed_9036321132460399255_rho_1.00.npz
condition_seed_17715532056777442089_rho_1.00.npz
condition_seed_9036321132460399255_rho_1.50.npz
condition_seed_17715532056777442089_rho_1.50.npz
condition_seed_9082055256280075172_rho_0.75.npz
condition_seed_17883847597205629106_rho_0.75.npz
condition_seed_9082055256280075172_rho_1.00.npz
condition_seed_17883847597205629106_rho_1.00.npz
condition_seed_9082055256280075172_rho_1.50.npz
condition_seed_17883847597205629106_rho_1.50.npz
condition_seed_9189988769602851417_rho_0.75.npz
condition_seed_18127452951968972828_rho_0.75.npz
condition_seed_9189988769602851417_rho_1.00.npz
condition_seed_18127452951968972828_rho_1.00.npz
condition_seed_9189988769602851417_rho_1.50.npz

condition_seed_18127452951968972828_rho_1.50.npz manifest.json
condition_seed_1888415619847841128_rho_0.75.npz pivot_seed_contrasts
condition_seed_1888415619847841128_rho_1.00.npz PIVOT_STUDY_LABEL.json
condition_seed_1888415619847841128_rho_1.50.npz twindow_condition_status.csv

./runs_pivot_study/ebfec9a5c419/pivot_seed_contrasts:
pivot_analysis_manifest.json pivot_branch_did_summary.csv pivot_branch_gap_summary.csv
pivot_regime_summary.csv
pivot_branch_did_by_seed.csv pivot_branch_gaps_by_seed.csv pivot_regime_contrasts_by_seed.csv
pivot_seed_contrasts_report.txt

./test_output:
analysis_manifest.json matched_phase_branch_diagnostics.npz matched_phase_report.txt
matched_phase_seed_status.csv
frozen_anchor_validation.csv matched_phase_paired_contrasts.csv
matched_phase_scalar_summary.csv
matched_initial_state_audit.csv matched_phase_recovery_summary.csv
matched_phase_seed_level_contrasts.csv

./Test_Terminal_Output:
terminal_output_01.txt

./workspace-019ff86e-8322-7d0d-bcaf-ed27a8018c0f:
aief_modular AIEF_modular_driver_v1.py MODULAR_IMPLEMENTATION_REPORT.md pytest.ini
README_modular.md tests_modular

./workspace-019ff86e-8322-7d0d-bcaf-ed27a8018c0f/aief_modular:
artifacts.py branches.py catalog.py collectors.py factors.py init.py oracle.py registry.py runner.py
spec.py studies validation.py

./workspace-019ff86e-8322-7d0d-bcaf-ed27a8018c0f/aief_modular/studies:
aief_1_4_s01_twindow.py init.py

./workspace-019ff86e-8322-7d0d-bcaf-ed27a8018c0f/tests_modular:
conftest.py test_builtin_registry.py test_registry.py test_twindow_one_condition.py
test_artifact_paths.py test_catalog.py test_rng_namespace.py test_twindow_oracle_equivalence.py
test_atomic_writes.py test_condition_expansion.py test_spec_canonicalization.py
test_unsupported_policies.py

What you observed is probably normal. This command:

Bash

lake env lean AIEFProofs/Smoke.lean

prints nothing when the file compiles successfully. Your Smoke.lean contains proofs but no #eval or #check commands that produce visible output.

Your directory listing confirms that the project was successfully created:

text

lean/AIEFProofs/
AIEFProofs/
Basic.lean
Smoke.lean
AIEFProofs.lean
lakefile.toml
lake-manifest.json
lean-toolchain

The Lean installation also succeeded:

text

Lean 4.33.1
Lake 5.0.0
Mathlib cache: 8690 files downloaded/decompressed

The early could not canonicalize path warnings occurred before the toolchain directory existed and are no longer concerning.
1. Finish/verify the interrupted update

You interrupted lake update after it reported that the cache was already decompressed. Run it once more and let it return to the prompt:

Bash

```
cd /home/deck/AI_Programs/SSRH/lean/AIEFProofs

source "$HOME/.elan/env"

time lake update
```

It should be quick and report something similar to:

text

```
toolchain not updated
No files to download
Already decompressed ...
```

Then explicitly check the cache:

Bash

```
time lake exe cache get
```

Do not run lake new again. The project already exists.
2. Run the smoke test with an explicit success message

Bash

```
time lake env lean AIEFProofs/Smoke.lean \
&& echo "LEAN SMOKE TEST PASSED"
```

Then inspect the exit status:

Bash

```
echo "Exit status: $?"
```

A successful result is:

text

LEAN SMOKE TEST PASSED
Exit status: 0

Lean normally remains silent when theorem checking succeeds.
3. Run a visibly evaluated test

Create a temporary test that prints a result:

Bash

```
cat > /tmp/aief_lean_visible_test.lean <<'EOF'
import Mathlib

#eval (2 + 2 : Nat)

#check Matrix
#check Real.exp
EOF
```

Run it in the project environment:

Bash

```
lake env lean /tmp/aief_lean_visible_test.lean
```

Expected output should include:

text

4
Matrix ...
Real.exp ...

Then remove it:

Bash

```
rm -f /tmp/aief_lean_visible_test.lean
```

4. Build the complete project

Bash

```
time lake build
```

A successful build may produce only a small amount of output if everything is already cached.

Check:

Bash

```
echo "Lake build exit status: $?"
```

Expected:

text

```
Lake build exit status: 0
```

5. Verify which toolchain the project pins

From inside lean/AIEFProofs:

Bash

```
printf '%s\n' "--- lean-toolchain ---"
cat lean-toolchain
```

```
printf '%s\n' "--- elan show ---"
elan show

printf '%s\n' "--- Lean ---"
lean --version

printf '%s\n' "--- Lake ---"
lake --version
```

Inside the project, elan show should report that the toolchain comes from the project’s lean-toolchain file.

6. Check current disk usage

You began with 32 GB free. Mathlib and its cache consume nontrivial space, so check again:

Bash

```
df -h "$HOME"

du -sh \
"$HOME/.elan" \
/home/deck/AI_Programs/SSRH/lean/AIEFProofs \
2>/dev/null
```

Do not delete the project’s .lake directory or Mathlib cache unless necessary; those files make builds much faster.

7. Record the project lock state

Bash

```
cd /home/deck/AI_Programs/SSRH/lean/AIEFProofs

sha256sum \
lean-toolchain \
lake-manifest.json \
lakefile.toml \
```

> AIEF_LEAN_PROJECT_LOCK_HASHES.txt

cat AIEF_LEAN_PROJECT_LOCK_HASHES.txt

These files pin:

text

Lean toolchain
Mathlib revision
Lake dependency graph

Do not run lake update again unless intentionally upgrading the formalization environment.
8. Check Git status

lake new created a separate Git repository inside AIEFProofs, as shown by:

text

lean/AIEFProofs/.git/

Inspect it:

Bash

git status --short

You should see Smoke.lean and possibly the lock-hash file as new files.

Commit when ready:

Bash

git add \
lean-toolchain \
lake-manifest.json \

lakefile.toml \
AIEFProofs.lean \
AIEFProofs \
AIEF_LEAN_PROJECT_LOCK_HASHES.txt \
README.md \
.gitignore

git commit -m \
"Initialize pinned AIEF Lean proof project"

If Git asks for a name/email, configure them locally rather than globally if preferred:

Bash

git config user.name "Your Name"
git config user.email "your-email@example.invalid"

9. Your project organization now looks appropriate

The important new top-level locations are:

text

AIEF_Experiments/
aief_modular_platform_v1_analysis_v4/
aief_modular/
tests_modular/
AIEF_1_4_Twindow_sensitivity_analysis_v4.py
AIEF_modular_driver_v1.py

lean/
AIEFProofs/
lean-toolchain
lake-manifest.json
lakefile.toml
AIEFProofs/
Basic.lean
Smoke.lean

One caution: you now have two modular-platform copies:

text

AIEF_Experiments/aief_modular_platform_v1_analysis_v4/
workspace-019ff86e-8322-7d0d-bcaf-ed27a8018c0f/

Treat the first as the candidate canonical version and the workspace-... directory as scratch/older material. When running modular tests, explicitly cd into the canonical directory so Python does not import the wrong copy.
10. If the Lean command truly remains stuck

If lake env lean AIEFProofs/Smoke.lean does not return to a prompt after several minutes, open a second Konsole and inspect:

Bash

ps -eo pid,etime,pcpu,pmem,cmd \
| grep -E 'lean|lake' \
| grep -v grep

If Lean is using CPU, allow the first build to finish. If it has zero CPU for a long time, interrupt it once and rerun:

Bash

lake exe cache get
lake env lean AIEFProofs/Smoke.lean

Bottom line

Your Lean installation and project creation appear successful. The “nothing happened” behavior is most likely Lean’s normal silent success. Confirm with:

Bash

lake env lean AIEFProofs/Smoke.lean \
&& echo "LEAN SMOKE TEST PASSED"

After that, the environment is ready for the first real formalization file, ControllerFiber.lean.

text

(deck@steamdeck ~)$ uname -m
command -v curl
command -v git
df -h "$HOME"
x86_64
/usr/bin/curl
/usr/bin/git
Filesystem Size Used Avail Use% Mounted on
/dev/nvme0n1p8 227G 195G 32G 86% /home
(deck@steamdeck ~)$ curl \
--proto '=https' \
--tlsv1.2 \
-fsSL \
https://elan.lean-lang.org/elan-init.sh \
-o /tmp/elan-init.sh
(deck@steamdeck ~)$ sh /tmp/elan-init.sh
info: downloading installer

Welcome to Lean!

This will download and install Elan, a tool for managing different Lean versions used in packages you create or download. It will also install a default version of Lean and its package manager, lake, for editing files not belonging to any package.

It will add the lake, lean, and elan commands to Elan's bin directory, located at:

/home/deck/.elan/bin

This path will then be added to your PATH environment variable by modifying the profile files located at:

/home/deck/.profile /home/deck/.bash_profile

You can uninstall at any time with elan self uninstall and these changes will be reverted.

Current installation options:

* default toolchain: stable
* modify PATH variable: yes

1) Proceed with installation (default)
2) Customize installation
3) Cancel installation
y

info: aborting installation
'(deck@steamdeck ~)$ sh /tmp/elan-init.sh
info: downloading installer

Welcome to Lean!

This will download and install Elan, a tool for managing different Lean versions used in packages you create or download. It will also install a default version of Lean and its package manager, lake, for editing files not belonging to any package.

It will add the lake, lean, and elan commands to Elan's bin directory, located at:

/home/deck/.elan/bin

This path will then be added to your PATH environment variable by modifying the profile files located at:

/home/deck/.profile /home/deck/.bash_profile

You can uninstall at any time with elan self uninstall and these changes will
be reverted.

Current installation options:

* default toolchain: stable
* modify PATH variable: yes

1) Proceed with installation (default)
2) Customize installation
3) Cancel installation
2

I'm going to ask you the value of each these installation options.
You may simply press the Enter key to leave unchanged.

Default toolchain? (stable/beta/nightly/<specific version>/none)

Modify PATH variable? (y/n)
y

Current installation options:

* default toolchain: stable
* modify PATH variable: yes

1) Proceed with installation (default)
2) Customize installation
3) Cancel installation
1

info: default toolchain set to 'stable'

Elan is installed now. Great!

To get started you need Elan's bin directory ($HOME/.elan/bin) in your PATH
environment variable. Next time you log in this will be done automatically.

To configure your current shell run source $HOME/.elan/env
(deck@steamdeck ~)$ source "$HOME/.elan/env"
(deck@steamdeck ~)$ grep -Fqx \
'source "$HOME/.elan/env"' \
"$HOME/.bashrc" \
|| printf '\nsource "$HOME/.elan/env"\n' \
>> "$HOME/.bashrc"
(deck@steamdeck ~)$ source "$HOME/.elan/env"
(deck@steamdeck ~)$ rm -f /tmp/elan-init.sh
(deck@steamdeck ~)$ command -v elan
command -v lean
command -v lake

elan --version
elan show
lean --version
lake --version
/home/deck/.elan/bin/elan
/home/deck/.elan/bin/lean
/home/deck/.elan/bin/lake
elan 4.2.3 (b6cec7e10 2026-06-08)
warning: could not canonicalize path: '/home/deck/.elan/toolchains'
warning: could not canonicalize path: '/home/deck/.elan/toolchains'
warning: could not canonicalize path: '/home/deck/.elan/toolchains'
leanprover/lean4:v4.33.1 (resolved from default 'stable')
(toolchain will be installed on first use)
warning: could not canonicalize path: '/home/deck/.elan/toolchains'
warning: could not canonicalize path: '/home/deck/.elan/toolchains'
warning: could not canonicalize path: '/home/deck/.elan/toolchains'
info: downloading https://releases.lean-lang.org/lean4/v4.33.1/lean-4.33.1-linux.tar.zst
544.0 MiB / 544.0 MiB (100 %) 51.9 MiB/s ETA: 0 s
info: installing /home/deck/.elan/toolchains/leanprover--lean4---v4.33.1
Lean (version 4.33.1, x86_64-unknown-linux-gnu, commit
819816b2e0a3bf405af45ae5c7af2491d8f5bee6, Release)

Lake version 5.0.0-src+819816b (Lean version 4.33.1)
(deck@steamdeck ~)$ cd /home/deck/AI_Programs/SSRH

mkdir -p lean
cd lean
(deck@steamdeck lean)$ lake +leanprover/lean4:stable \
new AIEFProofs math
info: AIEFProofs: no previous manifest, creating one from scratch
info: leanprover-community/mathlib: cloning https://github.com/leanprover-community/mathlib4
info: leanprover-community/mathlib: checking out revision
'0df444a360eaa60ab8c11dca51a86af692955474'
info: plausible: cloning https://github.com/leanprover-community/plausible
info: plausible: checking out revision 'b7eb3304aeae834b12dda98993a37f6a41f6f0bb'
info: LeanSearchClient: cloning https://github.com/leanprover-community/LeanSearchClient
info: LeanSearchClient: checking out revision '5f4d51b81cbd3f6b32b156bfad9056621a040404'
info: importGraph: cloning https://github.com/leanprover-community/import-graph
info: importGraph: checking out revision '16f02aa7642864af59f1ff0e384a015994db9118'
info: proofwidgets: cloning https://github.com/leanprover-community/ProofWidgets4
info: proofwidgets: checking out revision '4be2e3d5087eeb272cf5a8853b8f9dd025ef5957'
info: aesop: cloning https://github.com/leanprover-community/aesop
info: aesop: checking out revision '3448c0bcc5ce01b2d1546e483ec3620e32df3d0e'
info: Qq: cloning https://github.com/leanprover-community/quote4
info: Qq: checking out revision '92c15be17b7caf78c2ad767ec40f89052d908d81'
info: batteries: cloning https://github.com/leanprover-community/batteries
info: batteries: checking out revision '4488d40d070b9700d4d5a6aa342f0d40c31b2a2d'
info: Cli: cloning https://github.com/leanprover/lean4-cli
info: Cli: checking out revision '6130a47896ce867c6a4a55373441e59e565bad0f'
info: mathlib: running post-update hooks
Current branch: HEAD
Using cache from origin: (some leanprover-community/mathlib4)
Attempting to download 8690 file(s) from leanprover-community/mathlib4 cache at
https://lakecache.blob.core.windows.net/mathlib4-master
Downloaded: 8690 file(s) [attempted 8690/8690 = 100%, 50 KB/s], Decompressed: 2398
Decompressed 8690 file(s)
Already decompressed 8690 file(s)
(deck@steamdeck lean)$ cd AIEFProofs
(deck@steamdeck AIEFProofs)$ lake update
info: toolchain not updated; already up-to-date
info: mathlib: running post-update hooks
Current branch: HEAD
Using cache from origin: (some leanprover-community/mathlib4)
No files to download

```
Already decompressed 8690 file(s)
(deck@steamdeck AIEFProofs)$ ^C
(130)(deck@steamdeck AIEFProofs)$ find . -maxdepth 2 -type f -print
./lakefile.toml
./.git/description
./.git/HEAD
./.git/config
./lake-manifest.json
./AIEFProofs/Basic.lean
./lean-toolchain
./README.md
./.gitignore
./AIEFProofs.lean
(deck@steamdeck AIEFProofs)$ cat > AIEFProofs/Smoke.lean <<'EOF'
import Mathlib

set_option autoImplicit false

namespace AIEFProofs

example (x y : ℝ) : x + y = y + x := by
ring

example (x : ℝ) : x - x = 0 := by
ring

end AIEFProofs
EOF
(deck@steamdeck AIEFProofs)$ lake env lean AIEFProofs/Smoke.lean
lake build
(deck@steamdeck AIEFProofs)$ lake build
Build completed successfully (4 jobs).
(deck@steamdeck AIEFProofs)$ echo "Lake build exit status: $?"
Lake build exit status: 0
(deck@steamdeck AIEFProofs)$ printf '%s\n' "--- lean-toolchain ---"
cat lean-toolchain

printf '%s\n' "--- elan show ---"
elan show
```

```
printf '%s\n' "--- Lean ---"
lean --version


printf '%s\n' "--- Lake ---"
lake --version
--- lean-toolchain ---
leanprover/lean4:v4.33.1
--- elan show ---
leanprover/lean4:v4.33.1 (overridden by '/home/deck/AI_Programs/SSRH/lean/AIEFProofs/lean-toolchain')
Lean (version 4.33.1, x86_64-unknown-linux-gnu, commit 819816b2e0a3bf405af45ae5c7af2491d8f5bee6, Release)
--- Lean ---
Lean (version 4.33.1, x86_64-unknown-linux-gnu, commit 819816b2e0a3bf405af45ae5c7af2491d8f5bee6, Release)
--- Lake ---
Lake version 5.0.0-src+819816b (Lean version 4.33.1)
(deck@steamdeck AIEFProofs)$ lean/AIEFProofs/.git/
bash: lean/AIEFProofs/.git/: No such file or directory
(127)(deck@steamdeck AIEFProofs)$ git status --short
?? .github/
?? .gitignore
?? AIEFProofs.lean
?? AIEFProofs/
?? README.md
?? lake-manifest.json
?? lakefile.toml
?? lean-toolchain
(deck@steamdeck AIEFProofs)$ git add \
lean-toolchain \
lake-manifest.json \
lakefile.toml \
AIEFProofs.lean \
AIEFProofs \
AIEF_LEAN_PROJECT_LOCK_HASHES.txt \
README.md \
.gitignore


git commit -m \
"Initialize pinned AIEF Lean proof project"
fatal: pathspec 'AIEF_LEAN_PROJECT_LOCK_HASHES.txt' did not match any files
Author identity unknown
```

*** Please tell me who you are.

Run

git config --global user.email "you@example.com"
git config --global user.name "Your Name"

to set your account's default identity.
Omit --global to set the identity only in this repository.

fatal: unable to auto-detect email address (got 'deck@steamdeck.(none)')
(128)(deck@steamdeck AIEFProofs)$ git config user.name "jacobbrainard84-knightninja"
(deck@steamdeck AIEFProofs)$
git config user.email "jacobbrainard84@gmail.com"
(deck@steamdeck AIEFProofs)$ git add \
lean-toolchain \
lake-manifest.json \
lakefile.toml \
AIEFProofs.lean \
AIEFProofs \
AIEF_LEAN_PROJECT_LOCK_HASHES.txt \
README.md \
.gitignore

git commit -m \
fatal: pathspec ' ' did not match any files
bash: lean-toolchain: command not found
error: switch `m' requires a value
(129)(deck@steamdeck AIEFProofs)$ lean-toolchain lake-manifest.json lakefile.toml AIEFProofs.lean
AIEFProofs AIEF_LEAN_PROJECT_LOCK_HASHES.txt README.md .gitignore
bash: lean-toolchain: command not found
(127)(deck@steamdeck AIEFProofs)$

Lean and Mathlib are installed correctly. The important lines are:

text

Build completed successfully (4 jobs).
Lake build exit status: 0
leanprover/lean4:v4.33.1

Your remaining errors are only Git command-entry issues.
Why the Git commands failed
Wrong relative path

You were already inside:

text

/home/deck/AI_Programs/SSRH/lean/AIEFProofs

so this:

Bash

lean/AIEFProofs/.git/

looked for:

text

.../lean/AIEFProofs/lean/AIEFProofs/.git/

The repository is simply:

text

.git/

Missing lock-hash file

You attempted to add:

text

AIEF_LEAN_PROJECT_LOCK_HASHES.txt

before creating it.
Backslash followed by spaces

This is invalid shell continuation:

Bash

```
git add \
```

The backslash must be the final character on the line, with no trailing spaces. The easiest solution is to avoid continuations and use:

Bash

```
git add -A
```

Incomplete commit command

This:

Bash

```
git commit -m \
```

was executed without a message argument. Use one line.
Run these commands exactly

Bash

```
cd /home/deck/AI_Programs/SSRH/lean/AIEFProofs
```

source "$HOME/.elan/env"

pwd
ls -ld .git

Expected working directory:

text

/home/deck/AI_Programs/SSRH/lean/AIEFProofs

1. Confirm .lake is ignored

Bash

cat .gitignore

Check whether .lake is ignored:

Bash

git check-ignore -v .lake || true

If no ignore rule is printed, add one:

Bash

grep -Fqx '.lake/' .gitignore \
|| printf '\n.lake/\n' >> .gitignore

Do not commit .lake; it contains the large Mathlib checkout and compiled cache.
2. Create the missing lock-hash file

Bash

sha256sum \
lean-toolchain \
lake-manifest.json \
lakefile.toml \
> AIEF_LEAN_PROJECT_LOCK_HASHES.txt

Inspect it:

Bash

cat AIEF_LEAN_PROJECT_LOCK_HASHES.txt

3. Confirm local Git identity

You already set it, but verify:

Bash

git config --local --get user.name
git config --local --get user.email

Both commands should print a value.
4. Stage the project

Use one simple command:

Bash

git add -A

Inspect what will be committed:

Bash

git status --short

You should see files such as:

text

```
A .github/...
A .gitignore
A AIEFProofs.lean
A AIEFProofs/Basic.lean
A AIEFProofs/Smoke.lean
A README.md
A lake-manifest.json
A lakefile.toml
A lean-toolchain
A AIEF_LEAN_PROJECT_LOCK_HASHES.txt
```

You should not see thousands of files under .lake/.

Check the staged summary:

Bash

```
git diff --cached --stat
```

5. Commit on one line

Bash

```
git commit -m "Initialize pinned AIEF Lean proof project"
```

6. Confirm success

Bash

```
git status --short
git log -1 --oneline
```

Expected:

git status --short prints nothing;
git log -1 --oneline shows your new commit.

Final Lean verification

After committing:

Bash

```
lake env lean AIEFProofs/Smoke.lean \
&& echo "LEAN SMOKE TEST PASSED"

lake build \
&& echo "LEAN PROJECT BUILD PASSED"
```

Current status

You have successfully established:

text

```
Elan 4.2.3
Lean 4.33.1
Lake 5.0.0
Mathlib revision pinned in lake-manifest.json
Mathlib compiled cache downloaded
Smoke theorem compiled
Complete project build passed
```

Once the Git commit succeeds, the Lean environment setup is complete.

text

```
(deck@steamdeck ~)$ uname -m
command -v curl
command -v git
```

df -h "$HOME"
x86_64
/usr/bin/curl
/usr/bin/git
Filesystem Size Used Avail Use% Mounted on
/dev/nvme0n1p8 227G 195G 32G 86% /home
(deck@steamdeck ~)$ curl \
--proto '=https' \
--tlsv1.2 \
-fsSL \
https://elan.lean-lang.org/elan-init.sh \
-o /tmp/elan-init.sh
(deck@steamdeck ~)$ sh /tmp/elan-init.sh
info: downloading installer

Welcome to Lean!

This will download and install Elan, a tool for managing different Lean versions used in packages you create or download. It will also install a default version of Lean and its package manager, lake, for editing files not belonging to any package.

It will add the lake, lean, and elan commands to Elan's bin directory, located at:

/home/deck/.elan/bin

This path will then be added to your PATH environment variable by modifying the profile files located at:

/home/deck/.profile /home/deck/.bash_profile

You can uninstall at any time with elan self uninstall and these changes will be reverted.

Current installation options:

* default toolchain: stable

* modify PATH variable: yes

1) Proceed with installation (default)
2) Customize installation
3) Cancel installation
y

info: aborting installation
'(deck@steamdeck ~)$ sh /tmp/elan-init.sh
info: downloading installer

Welcome to Lean!

This will download and install Elan, a tool for managing different Lean versions used in packages you create or download. It will also install a default version of Lean and its package manager, lake, for editing files not belonging to any package.

It will add the lake, lean, and elan commands to Elan's bin directory, located at:

/home/deck/.elan/bin

This path will then be added to your PATH environment variable by modifying the profile files located at:

/home/deck/.profile /home/deck/.bash_profile

You can uninstall at any time with elan self uninstall and these changes will be reverted.

Current installation options:

* default toolchain: stable
* modify PATH variable: yes

1) Proceed with installation (default)
2) Customize installation
3) Cancel installation
2

I'm going to ask you the value of each these installation options.
You may simply press the Enter key to leave unchanged.

Default toolchain? (stable/beta/nightly/<specific version>/none)

Modify PATH variable? (y/n)
y

Current installation options:

* default toolchain: stable
* modify PATH variable: yes

1) Proceed with installation (default)
2) Customize installation
3) Cancel installation
1

info: default toolchain set to 'stable'

Elan is installed now. Great!

To get started you need Elan's bin directory ($HOME/.elan/bin) in your PATH
environment variable. Next time you log in this will be done automatically.

To configure your current shell run source $HOME/.elan/env
(deck@steamdeck ~)$ source "$HOME/.elan/env"

```
(deck@steamdeck ~)$ grep -Fqx \
'source "$HOME/.elan/env"' \
"$HOME/.bashrc" \
|| printf '\nsource "$HOME/.elan/env"\n' \
>> "$HOME/.bashrc"
(deck@steamdeck ~)$ source "$HOME/.elan/env"
(deck@steamdeck ~)$ rm -f /tmp/elan-init.sh
(deck@steamdeck ~)$ command -v elan
command -v lean
command -v lake


elan --version
elan show
lean --version
lake --version
/home/deck/.elan/bin/elan
/home/deck/.elan/bin/lean
/home/deck/.elan/bin/lake
elan 4.2.3 (b6cec7e10 2026-06-08)
warning: could not canonicalize path: '/home/deck/.elan/toolchains'
warning: could not canonicalize path: '/home/deck/.elan/toolchains'
warning: could not canonicalize path: '/home/deck/.elan/toolchains'
leanprover/lean4:v4.33.1 (resolved from default 'stable')
(toolchain will be installed on first use)
warning: could not canonicalize path: '/home/deck/.elan/toolchains'
warning: could not canonicalize path: '/home/deck/.elan/toolchains'
warning: could not canonicalize path: '/home/deck/.elan/toolchains'
info: downloading https://releases.lean-lang.org/lean4/v4.33.1/lean-4.33.1-linux.tar.zst
544.0 MiB / 544.0 MiB (100 %) 51.9 MiB/s ETA: 0 s
info: installing /home/deck/.elan/toolchains/leanprover--lean4---v4.33.1
Lean (version 4.33.1, x86_64-unknown-linux-gnu, commit
819816b2e0a3bf405af45ae5c7af2491d8f5bee6, Release)
Lake version 5.0.0-src+819816b (Lean version 4.33.1)
(deck@steamdeck ~)$ cd /home/deck/AI_Programs/SSRH


mkdir -p lean
cd lean
(deck@steamdeck lean)$ lake +leanprover/lean4:stable \
new AIEFProofs math
info: AIEFProofs: no previous manifest, creating one from scratch
info: leanprover-community/mathlib: cloning https://github.com/leanprover-community/mathlib4
```

```
info: leanprover-community/mathlib: checking out revision
'0df444a360eaa60ab8c11dca51a86af692955474'
info: plausible: cloning https://github.com/leanprover-community/plausible
info: plausible: checking out revision 'b7eb3304aeae834b12dda98993a37f6a41f6f0bb'
info: LeanSearchClient: cloning https://github.com/leanprover-community/LeanSearchClient
info: LeanSearchClient: checking out revision '5f4d51b81cbd3f6b32b156bfad9056621a040404'
info: importGraph: cloning https://github.com/leanprover-community/import-graph
info: importGraph: checking out revision '16f02aa7642864af59f1ff0e384a015994db9118'
info: proofwidgets: cloning https://github.com/leanprover-community/ProofWidgets4
info: proofwidgets: checking out revision '4be2e3d5087eeb272cf5a8853b8f9dd025ef5957'
info: aesop: cloning https://github.com/leanprover-community/aesop
info: aesop: checking out revision '3448c0bcc5ce01b2d1546e483ec3620e32df3d0e'
info: Qq: cloning https://github.com/leanprover-community/quote4
info: Qq: checking out revision '92c15be17b7caf78c2ad767ec40f89052d908d81'
info: batteries: cloning https://github.com/leanprover-community/batteries
info: batteries: checking out revision '4488d40d070b9700d4d5a6aa342f0d40c31b2a2d'
info: Cli: cloning https://github.com/leanprover/lean4-cli
info: Cli: checking out revision '6130a47896ce867c6a4a55373441e59e565bad0f'
info: mathlib: running post-update hooks
Current branch: HEAD
Using cache from origin: (some leanprover-community/mathlib4)
Attempting to download 8690 file(s) from leanprover-community/mathlib4 cache at
https://lakecache.blob.core.windows.net/mathlib4-master
Downloaded: 8690 file(s) [attempted 8690/8690 = 100%, 50 KB/s], Decompressed: 2398
Decompressed 8690 file(s)
Already decompressed 8690 file(s)
(deck@steamdeck lean)$ cd AIEFProofs
(deck@steamdeck AIEFProofs)$ lake update
info: toolchain not updated; already up-to-date
info: mathlib: running post-update hooks
Current branch: HEAD
Using cache from origin: (some leanprover-community/mathlib4)
No files to download
Already decompressed 8690 file(s)
(deck@steamdeck AIEFProofs)$ ^C
(130)(deck@steamdeck AIEFProofs)$ find . -maxdepth 2 -type f -print
./lakefile.toml
./.git/description
./.git/HEAD
./.git/config
./lake-manifest.json
./AIEFProofs/Basic.lean
```

```
./lean-toolchain
./README.md
./.gitignore
./AIEFProofs.lean
(deck@steamdeck AIEFProofs)$ cat > AIEFProofs/Smoke.lean <<'EOF'
import Mathlib

set_option autoImplicit false

namespace AIEFProofs

example (x y : ℝ) : x + y = y + x := by
ring

example (x : ℝ) : x - x = 0 := by
ring

end AIEFProofs
EOF
(deck@steamdeck AIEFProofs)$ lake env lean AIEFProofs/Smoke.lean
lake build
(deck@steamdeck AIEFProofs)$ lake build
Build completed successfully (4 jobs).
(deck@steamdeck AIEFProofs)$ echo "Lake build exit status: $?"
Lake build exit status: 0
(deck@steamdeck AIEFProofs)$ printf '%s\n' "--- lean-toolchain ---"
cat lean-toolchain

printf '%s\n' "--- elan show ---"
elan show

printf '%s\n' "--- Lean ---"
lean --version

printf '%s\n' "--- Lake ---"
lake --version
--- lean-toolchain ---
leanprover/lean4:v4.33.1
--- elan show ---
```

```
leanprover/lean4:v4.33.1 (overridden by '/home/deck/AI_Programs/SSRH/lean/AIEFProofs/lean-toolchain')
Lean (version 4.33.1, x86_64-unknown-linux-gnu, commit 819816b2e0a3bf405af45ae5c7af2491d8f5bee6, Release)
--- Lean ---
Lean (version 4.33.1, x86_64-unknown-linux-gnu, commit 819816b2e0a3bf405af45ae5c7af2491d8f5bee6, Release)
--- Lake ---
Lake version 5.0.0-src+819816b (Lean version 4.33.1)
(deck@steamdeck AIEFProofs)$ lean/AIEFProofs/.git/
bash: lean/AIEFProofs/.git/: No such file or directory
(127)(deck@steamdeck AIEFProofs)$ git status --short
?? .github/
?? .gitignore
?? AIEFProofs.lean
?? AIEFProofs/
?? README.md
?? lake-manifest.json
?? lakefile.toml
?? lean-toolchain
(deck@steamdeck AIEFProofs)$ git add \
lean-toolchain \
lake-manifest.json \
lakefile.toml \
AIEFProofs.lean \
AIEFProofs \
AIEF_LEAN_PROJECT_LOCK_HASHES.txt \
README.md \
.gitignore


git commit -m \
"Initialize pinned AIEF Lean proof project"
fatal: pathspec 'AIEF_LEAN_PROJECT_LOCK_HASHES.txt' did not match any files
Author identity unknown


*** Please tell me who you are.


Run


git config --global user.email "you@example.com"
git config --global user.name "Your Name"
```

to set your account's default identity.
Omit --global to set the identity only in this repository.

fatal: unable to auto-detect email address (got 'deck@steamdeck.(none)')
(128)(deck@steamdeck AIEFProofs)$ git config user.name "jacobbrainard84-knightninja"
(deck@steamdeck AIEFProofs)$
git config user.email "jacobbrainard84@gmail.com"
(deck@steamdeck AIEFProofs)$ git add \
lean-toolchain \
lake-manifest.json \
lakefile.toml \
AIEFProofs.lean \
AIEFProofs \
AIEF_LEAN_PROJECT_LOCK_HASHES.txt \
README.md \
.gitignore

git commit -m \
fatal: pathspec ' ' did not match any files
bash: lean-toolchain: command not found
error: switch `m' requires a value
(129)(deck@steamdeck AIEFProofs)$ lean-toolchain lake-manifest.json lakefile.toml AIEFProofs.lean
AIEFProofs AIEF_LEAN_PROJECT_LOCK_HASHES.txt README.md .gitignore
bash: lean-toolchain: command not found
(127)(deck@steamdeck AIEFProofs)$ cd /home/deck/AI_Programs/SSRH/lean/AIEFProofs

source "$HOME/.elan/env"

pwd
ls -ld .git
/home/deck/AI_Programs/SSRH/lean/AIEFProofs
drwxr-xr-x 6 deck deck 4096 Aug 23 19:57 .git
(deck@steamdeck AIEFProofs)$ cat .gitignore
/.lake
(deck@steamdeck AIEFProofs)$ git check-ignore -v .lake || true
.gitignore:1:/.lake .lake
(deck@steamdeck AIEFProofs)$ sha256sum \
lean-toolchain \
lake-manifest.json \

```
lakefile.toml \
> AIEF_LEAN_PROJECT_LOCK_HASHES.txt
(deck@steamdeck AIEFProofs)$ cat AIEF_LEAN_PROJECT_LOCK_HASHES.txt
3aac669c7a910ec2389f4e4f921b605adf6ebf2d1e0c9b9cd0be4d33f3f5db71 lean-toolchain
baa1bc12c21222b19b8436aaf1b7800b54b484b1cfe8065b6911ff3463271e08 lake-manifest.json
59df7f74d25059e734ef241a9e891c81077fb440726b1334864a9dd5a072e84b lakefile.toml
(deck@steamdeck AIEFProofs)$ git config --local --get user.name
git config --local --get user.email
jacobbrainard84-knightninja
jacobbrainard84@gmail.com
(deck@steamdeck AIEFProofs)$ git add -A
(deck@steamdeck AIEFProofs)$ git status --short
A .github/workflows/create-release.yml
A .github/workflows/lean_action_ci.yml
A .github/workflows/update.yml
A .gitignore
A AIEFProofs.lean
A AIEFProofs/Basic.lean
A AIEFProofs/Smoke.lean
A AIEF_LEAN_PROJECT_LOCK_HASHES.txt
A README.md
A lake-manifest.json
A lakefile.toml
A lean-toolchain
(deck@steamdeck AIEFProofs)$ git diff --cached --stat
.github/workflows/create-release.yml | 22 ++++++++++++++++++++
.github/workflows/lean_action_ci.yml | 21 ++++++++++++++++++++
.github/workflows/update.yml | 41 ++++++++++++++++++++++++++++++++++++++++
.gitignore | 1 +
AIEFProofs.lean | 1 +
AIEFProofs/Basic.lean | 1 +
AIEFProofs/Smoke.lean | 13 +++++++++++++
AIEF_LEAN_PROJECT_LOCK_HASHES.txt | 3 +++
README.md | 13 +++++++++++++
lake-manifest.json | 96 ++++++++++++++++++++++++++++++++++++++++++++++++++++++
++++++++++++++++++++++++++++++++++++
lakefile.toml | 18 ++++++++++++++++++
lean-toolchain | 1 +
12 files changed, 231 insertions(+)
(deck@steamdeck AIEFProofs)$ git commit -m "Initialize pinned AIEF Lean proof project"
[master (root-commit) 27cbb2e] Initialize pinned AIEF Lean proof project
12 files changed, 231 insertions(+)
```

create mode 100644 .github/workflows/create-release.yml
create mode 100644 .github/workflows/lean_action_ci.yml
create mode 100644 .github/workflows/update.yml
create mode 100644 .gitignore
create mode 100644 AIEFProofs.lean
create mode 100644 AIEFProofs/Basic.lean
create mode 100644 AIEFProofs/Smoke.lean
create mode 100644 AIEF_LEAN_PROJECT_LOCK_HASHES.txt
create mode 100644 README.md
create mode 100644 lake-manifest.json
create mode 100644 lakefile.toml
create mode 100644 lean-toolchain
(deck@steamdeck AIEFProofs)$ git status --short
git log -1 --oneline
27cbb2e (HEAD -> master) Initialize pinned AIEF Lean proof project
(deck@steamdeck AIEFProofs)$ lake env lean AIEFProofs/Smoke.lean \
&& echo "LEAN SMOKE TEST PASSED"

lake build \
&& echo "LEAN PROJECT BUILD PASSED"

The installation and project initialization are now complete.
Confirmed successful

text

Elan: 4.2.3
Lean: 4.33.1
Lake: 5.0.0
Mathlib: pinned in lake-manifest.json
Smoke compile: previously succeeded silently
Project build: succeeded
Git commit: 27cbb2e
Git state: clean

The Git repository is correctly located at:

text

/home/deck/AI_Programs/SSRH/lean/AIEFProofs/.git

Your lock hashes are:

text

lean-toolchain:
3aac669c7a910ec2389f4e4f921b605adf6ebf2d1e0c9b9cd0be4d33f3f5db71

lake-manifest.json:
baa1bc12c21222b19b8436aaf1b7800b54b484b1cfe8065b6911ff3463271e08

lakefile.toml:
59df7f74d25059e734ef241a9e891c81077fb440726b1334864a9dd5a072e84b

1. If the last command still appears silent

Run each command individually with an explicit status:

Bash

```
cd /home/deck/AI_Programs/SSRH/lean/AIEFProofs
source "$HOME/.elan/env"

time lake env lean AIEFProofs/Smoke.lean
smoke_status=$?
echo "Smoke status: $smoke_status"
```

Then:

Bash

```
time lake build
build_status=$?
echo "Build status: $build_status"
```

A successful Lean proof check is normally silent. The required result is:

text

Smoke status: 0
Build completed successfully ...
Build status: 0

If a command has not returned to the prompt, open a second Konsole and inspect it:

Bash

```
ps -eo pid,etime,pcpu,pmem,cmd \
| grep -E 'lean|lake' \
| grep -v grep
```

Because lake build already completed successfully earlier, the installation is known to work.
2. Verify the lock hashes

Bash

```
cd /home/deck/AI_Programs/SSRH/lean/AIEFProofs
```

```
sha256sum -c AIEF_LEAN_PROJECT_LOCK_HASHES.txt
```

Expected:

text

lean-toolchain: OK
lake-manifest.json: OK
lakefile.toml: OK

Do not run lake update again unless you deliberately intend to change Lean or Mathlib versions.
3. Check for unfinished proof markers

Bash

```
if grep -RInE \
'\bsorry\b|\badmit\b|sorryAx' \
AIEFProofs
then
echo "ERROR: unfinished proof marker found"
else
echo "No unfinished proof markers found"
fi
```

4. Review the generated automatic-update workflow

The Mathlib template added:

text

.github/workflows/update.yml

Inspect it:

Bash

```
sed -n '1,240p' \
.github/workflows/update.yml
```

Because this is intended to be a pinned formal-proof environment, do not allow an automatic workflow to update lean-toolchain or lake-manifest.json without explicit review.

If it performs automatic dependency updates, remove it:

Bash

```
git rm .github/workflows/update.yml

git commit -m \
"Disable automatic Lean dependency updates"
```

The CI build workflow is useful and can remain:

text

.github/workflows/lean_action_ci.yml

The release workflow can remain if it only operates on explicitly created tags. Inspect it before pushing:

Bash

```
sed -n '1,240p' \
.github/workflows/create-release.yml
```

5. Check disk use after Mathlib installation

Bash

```
df -h "$HOME"
```

```
du -sh \
"$HOME/.elan" \
/home/deck/AI_Programs/SSRH/lean/AIEFProofs \
2>/dev/null
```

The .lake directory is correctly ignored and should not be committed.
6. Remember that local Git is not yet an external backup

Check whether a remote exists:

Bash

```
git remote -v
```

If nothing is printed, the repository currently exists only on the Steam Deck. Before substantial formalization work, add a private remote or make an external backup.

7. Final environment record

Bash

```
{
date -u
lean --version
lake --version
elan --version
cat lean-toolchain
sha256sum \
lean-toolchain \
lake-manifest.json \
lakefile.toml
git rev-parse HEAD
git status --short
} | tee AIEF_LEAN_ENVIRONMENT_RECORD.txt
```

Commit it:

Bash

```
git add AIEF_LEAN_ENVIRONMENT_RECORD.txt

git commit -m \
"Record certified Lean proof environment"
```

8. Next formalization step

The environment is ready for:

text

```
AIEFProofs/ControllerFiber.lean
```

The first theorem package should remain small:

define the 2×32×3 controller matrix;
define v=(1,−1,−1)v=(1,−1,−1);
prove CJv=0CJv=0;
characterize every kernel vector as a scalar multiple of vv;
audit theorem axioms;
run lake build.

Do not begin with the full delayed system. The controller-kernel theorem is the safest first Lean milestone.

Think about this:

text

/-
Copyright (c) 2026 AIEF / SSRH formalization.

Controller-fiber kernel (first Lean milestone)

Formalizes only the exact 2×3 controller-incidence identity used for the
exceptional K3 fiber. It does not formalize delay, EMA, admissibility,
or synchronization.

Scientific mapping (comments only; not used as axioms)

Ordered K3 edges: e_0=(2,7), e_1=(2,11), e_2=(7,11)
Audited winners: e_0↦7, e_1↦2, e_2↦7
Controller rows: I={2,7}

C = ![![1, 1, 0], ![1, 0, 1]]
v = (1, -1, -1)

Proved: C v = 0, and C x = 0 ↔ ∃ a, x = a • v, over ℚ.

Install on the Deck project

Copy to:
lean/AIEFProofs/AIEFProofs/ControllerFiber.lean

Add one import to the library root file (`AIEFProofs.lean` or `AIEFProofs/Basic.lean`):
import AIEFProofs.ControllerFiber

Then:
cd /home/deck/AI_Programs/SSRH/lean/AIEFProofs
source "$HOME/.elan/env"
lake build

Do not run `lake update`.
-/

```
import Mathlib.Data.Matrix.Basic
import Mathlib.Data.Matrix.Notation
import Mathlib.Tactic

namespace AIEFProofs
namespace ControllerFiber

open Matrix

/-- Unsigned incidence of controller nodes `{2,7}` on the ordered K3 edges
`( (2,7), (2,11), (7,11) )`. -/
def CCtrl : Matrix (Fin 2) (Fin 3) ℚ :=
!![1, 1, 0;
1, 0, 1]

/-- Canonical generator of `ker CCtrl` (unnormalized). -/
def vCtrl : Fin 3 → ℚ :=
![1, -1, -1]

lemma mulVec_CCtrl (x : Fin 3 → ℚ) (i : Fin 2) :
(CCtrl *ᵥ x) i =
CCtrl i 0 * x 0 + CCtrl i 1 * x 1 + CCtrl i 2 * x 2 := by
simp [mulVec, dotProduct, Fin.sum_univ_three]
```

```
/-- The controller matrix annihilates the fiber generator. -/
theorem CCtrl_mulVec_vCtrl : CCtrl *ᵥ vCtrl = 0 := by
ext i
fin_cases i
· -- row 0: 1*1 + 1*(-1) + 0*(-1) = 0
rw [mulVec_CCtrl]
native_decide
· -- row 1: 1*1 + 0*(-1) + 1*(-1) = 0
rw [mulVec_CCtrl]
native_decide


/-- Row form of the kernel equation. -/
lemma mulVec_CCtrl_eq_zero_iff (x : Fin 3 → ℚ) :
CCtrl *ᵥ x = 0 ↔ x 0 + x 1 = 0 ∧ x 0 + x 2 = 0 := by
constructor
· intro h
constructor
· have h0 := congrFun h 0
rw [mulVec_CCtrl] at h0
-- C row0 = (1,1,0)
have : (1 : ℚ) * x 0 + 1 * x 1 + 0 * x 2 = 0 := by
convert h0
native_decide
linarith
· have h1 := congrFun h 1
rw [mulVec_CCtrl] at h1
-- C row1 = (1,0,1)
have : (1 : ℚ) * x 0 + 0 * x 1 + 1 * x 2 = 0 := by
convert h1
native_decide
linarith
· intro ⟨h01, h02⟩
ext i
fin_cases i
· rw [mulVec_CCtrl]
have : CCtrl 0 0 * x 0 + CCtrl 0 1 * x 1 + CCtrl 0 2 * x 2 =
x 0 + x 1 := by native_decide
simpa [this] using h01
· rw [mulVec_CCtrl]
have : CCtrl 1 0 * x 0 + CCtrl 1 1 * x 1 + CCtrl 1 2 * x 2 =
x 0 + x 2 := by native_decide
simpa [this] using h02
```

```
/-- Every kernel vector is a scalar multiple of `vCtrl`. -/
theorem mem_ker_CCtrl_iff (x : Fin 3 → ℚ) :
CCtrl *ᵥ x = 0 ↔ ∃ a : ℚ, x = a • vCtrl := by
constructor
· intro hx
rcases (mulVec_CCtrl_eq_zero_iff x).mp hx with ⟨h01, h02⟩
refine ⟨x 0, ?_⟩
funext i
fin_cases i
· -- a • v = x 0 * 1
change x 0 = x 0 * vCtrl 0
have : vCtrl 0 = 1 := by native_decide
simp [this]
· -- x 1 = -x 0 = x 0 * (-1)
change x 1 = x 0 * vCtrl 1
have : vCtrl 1 = (-1 : ℚ) := by native_decide
rw [this]
linarith
· change x 2 = x 0 * vCtrl 2
have : vCtrl 2 = (-1 : ℚ) := by native_decide
rw [this]
linarith
· rintro ⟨a, rfl⟩
have hsmul : CCtrl *ᵥ (a • vCtrl) = a • (CCtrl *ᵥ vCtrl) := by
simp [mulVec_smul]
rw [hsmul, CCtrl_mulVec_vCtrl, smul_zero]


theorem vCtrl_ne_zero : vCtrl ≠ 0 := by
intro h
have := congrFun h 0
have hv : vCtrl 0 = 1 := by native_decide
simp [hv] at this


/-- Documentation: this file contains no unfinished proof markers.
Dependencies are Lean + Mathlib matrix/`ℚ` arithmetic only. -/
theorem controllerFiber_marker : True := trivial


end ControllerFiber
end AIEFProofs
```

This is an excellent first formalization target: small, exact, and clearly separated from delay, EMA, admissibility, and synchronization claims.

However, I would not copy the proposed file unchanged. Several uses of native_decide involve arbitrary symbolic values x, not closed expressions. Those proofs are unlikely to compile and, even where native_decide works, it is unnecessary for elementary rational linear algebra.
Scientific scope

The proposed theorem correctly formalizes:
C=(110101),v=(1,−1,−1),
C=(111001),v=(1,−1,−1),

and:
Cx=0 ⟺ ∃a∈Q, x=av.
Cx=0⟺∃a∈Q, x=av.

This is an exact algebraic result over QQ.

It does not yet prove:

that this matrix is the controller matrix produced by Engine v1;
persistence of the winner map;
branch invariance;
any delayed-system result;
any asymptotic claim;
anything about the normalized real vector v/3v/3
.

The edge/controller mapping remains an externally audited correspondence recorded in comments and evidence lineage.
Main Lean issue: symbolic native_decide

This kind of proof is problematic:

lean

have :
(1 : ℚ) * x 0 + 1 * x 1 + 0 * x 2

= x 0 + x 1 := by
native_decide

The proposition contains the arbitrary variable x. It cannot generally be established by evaluating a closed Boolean decision procedure.

Use:

lean

simp

or:

lean

ring

instead.

Similarly, matrix coefficients should be reduced with:

lean

simp [CCtrl, Matrix.mulVec, dotProduct, Fin.sum_univ_three]

rather than repeated convert and native_decide.
Recommended corrected first file

Copy this version to:

text

lean/AIEFProofs/AIEFProofs/ControllerFiber.lean

lean

/-
Copyright (c) 2026 AIEF / SSRH formalization.

Controller-fiber kernel

This file formalizes only the exact 2×3 controller-incidence
identity used for the exceptional K3 fiber.

It does not formalize delay, EMA, admissibility, branch
invariance, synchronization, or asymptotic behavior.

Scientific correspondence, not a formal assumption:

* Ordered edges:
e_0 = (2,7)
e_1 = (2,11)
e_2 = (7,11)

* Controller rows:
node 2
node 7

* Controller-incidence matrix:

C = ![![1, 1, 0],
![1, 0, 1]]

* Unnormalized kernel generator:

v = (1, -1, -1)
-/

import Mathlib.Data.Matrix.Basic
import Mathlib.Data.Matrix.Notation
import Mathlib.Tactic

```
set_option autoImplicit false

namespace AIEFProofs
namespace ControllerFiber

open Matrix

/-- Unsigned incidence matrix of controller nodes `{2,7}`
on the ordered K3 edges `((2,7), (2,11), (7,11))`. -/
def CCtrl : Matrix (Fin 2) (Fin 3) ℚ :=
!![1, 1, 0;
1, 0, 1]

/-- Canonical unnormalized generator of the controller
kernel. -/
def vCtrl : Fin 3 → ℚ :=
![1, -1, -1]

/-- Evaluation of the first controller row. -/
@[simp]
lemma CCtrl_mulVec_apply_zero
(x : Fin 3 → ℚ) :
(CCtrl *ᵥ x) 0 = x 0 + x 1 := by
simp [
CCtrl,
Matrix.mulVec,
dotProduct,
Fin.sum_univ_three
]

/-- Evaluation of the second controller row. -/
@[simp]
lemma CCtrl_mulVec_apply_one
(x : Fin 3 → ℚ) :
(CCtrl *ᵥ x) 1 = x 0 + x 2 := by
simp [
CCtrl,
Matrix.mulVec,
```

```
dotProduct,
Fin.sum_univ_three
]

/-- The controller matrix annihilates the canonical
fiber generator. -/
theorem CCtrl_mulVec_vCtrl :
CCtrl *ᵥ vCtrl = 0 := by
funext i
fin_cases i <;> simp [vCtrl]

/-- Coordinate form of the controller-kernel equation. -/
theorem CCtrl_mulVec_eq_zero_iff
(x : Fin 3 → ℚ) :
CCtrl *ᵥ x = 0 ↔
x 0 + x 1 = 0 ∧
x 0 + x 2 = 0 := by
constructor
· intro h
constructor
· have h0 := congrFun h (0 : Fin 2)
simpa using h0
· have h1 := congrFun h (1 : Fin 2)
simpa using h1
· rintro ⟨h01, h02⟩
funext i
fin_cases i
· simpa using h01
· simpa using h02

/-- Every vector annihilated by `CCtrl` is a scalar
multiple of `vCtrl`, and every scalar multiple of
`vCtrl` is annihilated by `CCtrl`. -/
theorem CCtrl_mulVec_eq_zero_iff_exists_smul
(x : Fin 3 → ℚ) :
CCtrl *ᵥ x = 0 ↔
∃ a : ℚ, x = a • vCtrl := by
constructor
· intro hx
rcases
(CCtrl_mulVec_eq_zero_iff x).mp hx
```

```
with ⟨h01, h02⟩
refine ⟨x 0, ?_⟩
funext i
fin_cases i
· simp [vCtrl]
· simp [vCtrl]
linarith
· simp [vCtrl]
linarith
· rintro ⟨a, rfl⟩
apply
(CCtrl_mulVec_eq_zero_iff
(a • vCtrl)).2
constructor <;> simp [vCtrl]


/-- The canonical controller-fiber generator is
nonzero. -/
theorem vCtrl_ne_zero :
vCtrl ≠ 0 := by
intro h
have h0 := congrFun h (0 : Fin 3)
norm_num [vCtrl] at h0


end ControllerFiber
end AIEFProofs
```

Why this version is preferable

It removes:

symbolic native_decide;
unnecessary coefficient-by-coefficient have statements;
reliance on Matrix.mulVec_smul;
the vacuous controllerFiber_marker : True.

It replaces those with:

two explicit row-evaluation lemmas;
coordinate kernel characterization;

direct finite-function extensionality;
elementary simp and linarith.

Every proof remains kernel-checked by Lean.
About the marker theorem

This theorem:

lean

theorem controllerFiber_marker : True := trivial

does not establish that the file contains no sorry. It proves only True.

Use external checks instead:

Bash

grep -RInE \
'\bsorry\b|\badmit\b|sorryAx' \
AIEFProofs

and:

Bash

lake build

Import it through the library root

Edit:

text

AIEFProofs.lean

It probably currently contains:

lean

import AIEFProofs.Basic

Change it to:

lean

import AIEFProofs.Basic
import AIEFProofs.ControllerFiber

Do not import ControllerFiber from Basic.lean. Basic.lean should remain lower-level so later theorem files can import it without dependency cycles.
Compile the file before changing the root

From:

text

/home/deck/AI_Programs/SSRH/lean/AIEFProofs

run:

Bash

source "$HOME/.elan/env"

lake env lean \
AIEFProofs/ControllerFiber.lean

If successful:

Bash

printf '\nimport AIEFProofs.ControllerFiber\n' \
>> AIEFProofs.lean

lake build

Check for unfinished markers:

Bash

if grep -RInE \
'\bsorry\b|\badmit\b|sorryAx' \
AIEFProofs
then
echo "ERROR: unfinished proof marker found"
exit 1
else
echo "No unfinished proof markers found"
fi

Axiom audit

Create a temporary audit file:

Bash

cat > /tmp/aief_controller_axioms.lean <<'EOF'
import AIEFProofs.ControllerFiber

#print axioms \
AIEFProofs.ControllerFiber.CCtrl_mulVec_vCtrl

#print axioms \
AIEFProofs.ControllerFiber.CCtrl_mulVec_eq_zero_iff

#print axioms \
AIEFProofs.ControllerFiber.CCtrl_mulVec_eq_zero_iff_exists_smul

#print axioms \
AIEFProofs.ControllerFiber.vCtrl_ne_zero
EOF

Run:

Bash

lake env lean /tmp/aief_controller_axioms.lean

rm -f /tmp/aief_controller_axioms.lean

Record the output. Do not automatically assume that any printed foundational Mathlib axioms indicate a defect; inspect them individually. The critical prohibition is introducing a project-specific axiom equivalent to the theorem.
Git commit

After successful compilation:

Bash

git status --short

git add \
AIEFProofs.lean \
AIEFProofs/ControllerFiber.lean

git commit -m \
"Formalize exceptional K3 controller kernel"

Avoid trailing spaces after shell continuation backslashes. A one-line alternative is safer:

Bash

git add AIEFProofs.lean AIEFProofs/ControllerFiber.lean
git commit -m "Formalize exceptional K3 controller kernel"

Next mathematical refinement

This first file proves an elementwise characterization:
CCtrl x=0 ⟺ ∃a, x=a vCtrl.
CCtrlx=0⟺∃a, x=avCtrl.

A later theorem can represent CCtrl $*_v$ · as a LinearMap and prove the literal submodule equality:
ker(C)=spanQ{vCtrl}.
ker(C)=spanQ{vCtrl}.

After that, add a real-valued corollary over RR, which connects more directly to:
13(1,−1,−1).
3
1(1,−1,−1).
Final assessment

The theorem target and scientific scope are excellent. The original proof script is unnecessarily brittle because it asks native_decide to solve symbolic arithmetic. Use the simplified simp/linarith version, compile it directly, audit its axioms, and only then add it to the project root.

now this:

text

/-- Canonical unnormalized generator of the controller kernel. -/
def vCtrl : Fin 3 → ℚ :=
![1, -1, -1]

/-- Evaluation of the first controller row. -/
@[simp]
lemma CCtrl_mulVec_apply_zero (x : Fin 3 → ℚ) :
(CCtrl $*_v$ x) 0 = x 0 + x 1 := by
simp [CCtrl, Matrix.mulVec, dotProduct, Fin.sum_univ_three]

/-- Evaluation of the second controller row. -/
@[simp]
lemma CCtrl_mulVec_apply_one (x : Fin 3 → ℚ) :
(CCtrl $*_v$ x) 1 = x 0 + x 2 := by

```
simp [CCtrl, Matrix.mulVec, dotProduct, Fin.sum_univ_three]


/-- The controller matrix annihilates the canonical fiber generator. -/
theorem CCtrl_mulVec_vCtrl : CCtrl *ᵥ vCtrl = 0 := by
funext i
fin_cases i <;> simp [vCtrl]


/-- Coordinate form of the controller-kernel equation. -/
theorem CCtrl_mulVec_eq_zero_iff (x : Fin 3 → ℚ) :
CCtrl *ᵥ x = 0 ↔ x 0 + x 1 = 0 ∧ x 0 + x 2 = 0 := by
constructor
· intro h
constructor
· have h0 := congrFun h (0 : Fin 2)
simpa using h0
· have h1 := congrFun h (1 : Fin 2)
simpa using h1
· rintro ⟨h01, h02⟩
funext i
fin_cases i
· simpa using h01
· simpa using h02


/-- Locked-ratio form of the two controller equations. -/
theorem CCtrl_coords_iff (x : Fin 3 → ℚ) :
x 0 + x 1 = 0 ∧ x 0 + x 2 = 0 ↔
x 1 = -x 0 ∧ x 2 = -x 0 := by
constructor
· intro ⟨h01, h02⟩
constructor <;> linarith
· intro ⟨h1, h2⟩
constructor <;> linarith


/-- Explicit fiber coordinate form:
the two independent constraints leave one free scalar. -/
theorem CCtrl_eq_zero_iff_locked_ratio (x : Fin 3 → ℚ) :
CCtrl *ᵥ x = 0 ↔ x 1 = -x 0 ∧ x 2 = -x 0 := by
rw [CCtrl_mulVec_eq_zero_iff, CCtrl_coords_iff]


/-- Every vector annihilated by `CCtrl` is a scalar multiple of `vCtrl`,
```

```
and every scalar multiple of `vCtrl` is annihilated by `CCtrl`. -/
theorem CCtrl_mulVec_eq_zero_iff_exists_smul (x : Fin 3 → ℚ) :
CCtrl *ᵥ x = 0 ↔ ∃ a : ℚ, x = a • vCtrl := by
constructor
· intro hx
rcases (CCtrl_mulVec_eq_zero_iff x).mp hx with ⟨h01, h02⟩
refine ⟨x 0, ?_⟩
funext i
fin_cases i
· simp [vCtrl]
· simp [vCtrl]
linarith
· simp [vCtrl]
linarith
· rintro ⟨a, rfl⟩
apply (CCtrl_mulVec_eq_zero_iff (a • vCtrl)).2
constructor <;> simp [vCtrl]


/-- The canonical controller-fiber generator is nonzero. -/
theorem vCtrl_ne_zero : vCtrl ≠ 0 := by
intro h
have h0 := congrFun h (0 : Fin 3)
norm_num [vCtrl] at h0


end ControllerFiber
end AIEFProofs
```

This is a good improvement. The added coordinate theorem makes the one-dimensional fiber structure explicit:
CCtrl x=0 ⟺ x1=−x0∧x2=−x0 ⟺ x=x0(1,−1,−1).
CCtrlx=0⟺x1=−x0∧x2=−x0⟺x=x0(1,−1,−1).
Verdict

Likely compile-ready, assuming the file still contains:

the imports;
namespace openings;
definition of CCtrl;
set_option autoImplicit false.

The proofs use ordinary kernel-checked tactics rather than symbolic native_decide.
One recommended code change

This proof:

lean

```
theorem CCtrl_eq_zero_iff_locked_ratio
(x : Fin 3 → ℚ) :
CCtrl *ᵥ x = 0 ↔
x 1 = -x 0 ∧ x 2 = -x 0 := by
rw [CCtrl_mulVec_eq_zero_iff, CCtrl_coords_iff]
```

will probably work, but explicitly composing the two equivalences is clearer and less dependent on rewrite matching:

lean

```
theorem CCtrl_eq_zero_iff_signed_coords
(x : Fin 3 → ℚ) :
CCtrl *ᵥ x = 0 ↔
x 1 = -x 0 ∧ x 2 = -x 0 := by
exact
(CCtrl_mulVec_eq_zero_iff x).trans
(CCtrl_coords_iff x)
```

Avoid “locked-ratio” terminology

The theorem does not use a ratio. At the zero vector,
x0=x1=x2=0,
x0=x1=x2=0,

the ratios x1/x0x1/x0 and x2/x0x2/x0 are undefined.

Also, “locked” is already used in the project for rotating-frequency lock. Using it here could conflate two unrelated concepts.

Prefer names such as:

lean

CCtrl_coords_iff
CCtrl_eq_zero_iff_signed_coords
CCtrl_eq_zero_iff_neg_coordinates

and documentation:

lean

/-- Signed-coordinate form of the two controller equations. -/

Optional simplification of the span proof

The existing proof is valid in structure. It can use the new signed-coordinate theorem directly:

lean

```
theorem CCtrl_mulVec_eq_zero_iff_exists_smul
(x : Fin 3 → ℚ) :
CCtrl *ᵥ x = 0 ↔
∃ a : ℚ, x = a • vCtrl := by
constructor
· intro hx
rcases
(CCtrl_eq_zero_iff_signed_coords x).mp hx
with ⟨h1, h2⟩
refine ⟨x 0, ?_⟩
funext i
fin_cases i
· simp [vCtrl]
· simpa [vCtrl] using h1
· simpa [vCtrl] using h2
· rintro ⟨a, rfl⟩
apply
(CCtrl_mulVec_eq_zero_iff
(a • vCtrl)).2
```

constructor <;> simp [vCtrl]

If the simpa orientation on h1 or h2 causes a minor elaboration problem, the existing linarith version is perfectly acceptable and more robust:

lean

· simp [vCtrl]
linarith

There is no need to optimize a proof that already compiles.
Scientific interpretation

This file proves an exact algebraic statement over QQ:
ker(CCtrl)=
ker(CCtrl)=

{a(1,-1,-1):a\in\mathbb Q}.
]

More precisely, it currently proves the elementwise characterization rather than literal equality of LinearMap.ker and Submodule.span.

That is appropriate for the first milestone.

It does not prove that:

the engine remains on the exceptional controller branch;
controller winners persist;
the K3 stays isolated;
the fiber is dynamically invariant;
the normalized real vector is selected;
any delay or EMA theorem holds.

Those remain separate claims.
Compile sequence

Save the complete file as:

text

AIEFProofs/ControllerFiber.lean

Then:

Bash

```
cd /home/deck/AI_Programs/SSRH/lean/AIEFProofs
source "$HOME/.elan/env"

lake env lean AIEFProofs/ControllerFiber.lean \
&& echo "CONTROLLER FIBER FILE PASSED"
```

If it passes, ensure the root file contains:

lean

```
import AIEFProofs.Basic
import AIEFProofs.ControllerFiber
```

Then:

Bash

```
lake build \
&& echo "FULL LEAN PROJECT PASSED"
```

Audit unfinished markers:

Bash

```
if grep -RInE \
'\bsorry\b|\badmit\b|sorryAx' \
```

AIEFProofs
then
echo "ERROR: unfinished proof marker found"
exit 1
else
echo "No unfinished proof markers found"
fi

Commit:

Bash

git add AIEFProofs.lean AIEFProofs/ControllerFiber.lean
git commit -m "Formalize exceptional K3 controller kernel"

Final assessment

The theorem chain is now appropriately structured:

text

row evaluation
→ coordinate equations
→ signed-coordinate form
→ scalar-multiple characterization
→ nonzero generator

That is a sound first formal milestone.

warning: AIEFProofs/ControllerFiber.lean:29:0: The module doc-string for a file should be the first command after the imports.
Please, add a module doc-string before ``.

Note: This linter can be disabled with set_option linter.style.header false
info: AIEFProofs/ControllerFiber.lean:138:0: linter.style.header:68:12: error: expected token

linter.style.header:74:12: error: expected token

linter.style.header:78:36: error: expected token

linter.style.header:84:11: error: expected token

linter.style.header:110:11: error: expected token

linter.style.header:115:11: error: expected token
Build completed successfully (3009 jobs).
Root build status: 0
(deck@steamdeck AIEFProofs)$ lake build
✖ [1228/1230] Building AIEFProofs.ControllerFiber
trace: .> LEAN_PATH=/home/deck/AI_Programs/SSRH/lean/AIEFProofs/.lake/packages/Cli/.lake/build/lib/lean:/home/deck/AI_Programs/SSRH/lean/AIEFProofs/.lak
e/packages/batteries/.lake/build/lib/lean:/home/deck/AI_Programs/SSRH/lean/AIEFProofs/.lake/packages/Qq/.lake/build/lib/lean:/home/deck/AI_Programs/SSRH
/lean/AIEFProofs/.lake/packages/aesop/.lake/build/lib/lean:/home/deck/AI_Programs/SSRH/lean/AIEFProofs/.lake/packages/proofwidgets/.lake/build/lib/lean:
/home/deck/AI_Programs/SSRH/lean/AIEFProofs/.lake/packages/importGraph/.lake/build/lib/lean:/home/deck/AI_Programs/SSRH/lean/AIEFProofs/.lake/packages/L
eanSearchClient/.lake/build/lib/lean:/home/deck/AI_Programs/SSRH/lean/AIEFProofs/.lake/packages/plausible/.lake/build/lib/lean:/home/deck/AI_Programs/SS
RH/lean/AIEFProofs/.lake/packages/mathlib/.lake/build/lib/lean:/home/deck/AI_Programs/SSRH/lean/AIEFProofs/.lake/build/lib/lean /home/deck/.elan/toolcha
ins/leanprover--lean4---v4.33.1/bin/lean
/home/deck/AI_Programs/SSRH/lean/AIEFProofs/AIEFProofs/ControllerFiber.lean -o
/home/deck/AI_Programs/SSRH/lean
/AIEFProofs/.lake/build/lib/lean/AIEFProofs/ControllerFiber.olean -i
/home/deck/AI_Programs/SSRH/lean/AIEFProofs/.lake/build/lib/lean/AIEFProofs/Control
lerFiber.ilean -c /home/deck/AI_Programs/SSRH/lean/AIEFProofs/.lake/build/ir/AIEFProofs/
ControllerFiber.c --setup /home/deck/AI_Programs/SSRH/lean/AIEFP
roofs/.lake/build/ir/AIEFProofs/ControllerFiber.setup.json --json
error: AIEFProofs/ControllerFiber.lean:64:35: Unknown constant Fin.sum_univ_three
error: AIEFProofs/ControllerFiber.lean:63:34: unsolved goals
x : Fin 3 → ℚ
⊢ ∑ x_1, CCtrl 0 x_1 *x x_1 = x 0 + x 1 *
error: AIEFProofs/ControllerFiber.lean:70:35: Unknown constant Fin.sum_univ_three **
*error: AIEFProofs/ControllerFiber.lean:69:34: unsolved goals *
*x : Fin 3 → ℚ *
⊢ ∑ x_1, CCtrl 1 x_1 x x_1 = x 0 + x 2
error: Lean exited with code 1
Some required targets logged failures:

AIEFProofs.ControllerFiber
error: build failed
(1)(deck@steamdeck AIEFProofs)$

The installation is fine. The final lake build correctly exposed two source issues:

Fin.sum_univ_three does not exist in your pinned Mathlib/Lean version.
The module documentation comment is not positioned according to the Mathlib header linter.

The actual build is currently failed, despite the earlier root build status. The earlier build likely did not yet include ControllerFiber.lean as a root dependency.
1. Replace Fin.sum_univ_three

Your Lean/Mathlib version provides the recursive theorem:

lean

Fin.sum_univ_succ

Confirm it:

Bash

cd /home/deck/AI_Programs/SSRH/lean/AIEFProofs
source "$HOME/.elan/env"

cat > /tmp/check_fin_sum.lean <<'EOF'
import Mathlib

#check Fin.sum_univ_succ
EOF

lake env lean /tmp/check_fin_sum.lean
rm -f /tmp/check_fin_sum.lean

Then replace both row lemmas with:

lean

/-- Evaluation of the first controller row. -/
@[simp]
lemma CCtrl_mulVec_apply_zero
(x : Fin 3 → ℚ) :
(CCtrl *ᵥ x) 0 = x 0 + x 1 := by
simp [
CCtrl,
Matrix.mulVec,
dotProduct,
Fin.sum_univ_succ
]

/-- Evaluation of the second controller row. -/
@[simp]
lemma CCtrl_mulVec_apply_one
(x : Fin 3 → ℚ) :
(CCtrl *ᵥ x) 1 = x 0 + x 2 := by
simp [
CCtrl,
Matrix.mulVec,
dotProduct,
Fin.sum_univ_succ
]

The essential change is:

lean

Fin.sum_univ_three

to:

lean

Fin.sum_univ_succ

You can make the textual replacement directly:

Bash

```
sed -i \
's/Fin\.sum_univ_three/Fin.sum_univ_succ/g' \
AIEFProofs/ControllerFiber.lean
```

2. Correct the module-header layout

Lean distinguishes:

lean

```
/- ordinary block comment -/
```

from:

lean

```
/-! module documentation comment -/
```

Use this order at the very beginning of the file:

lean

```
/-
Copyright (c) 2026 AIEF / SSRH formalization.
-/

import Mathlib.Data.Matrix.Basic
import Mathlib.Data.Matrix.Notation
import Mathlib.Tactic
```

/-!
Controller-fiber kernel

This file formalizes only the exact 2×3 controller-incidence
identity used for the exceptional K3 fiber.

It does not formalize delay, EMA, admissibility, branch
invariance, synchronization, or asymptotic behavior.

Scientific correspondence

The correspondence below documents the audited Python/model
interpretation. It is not used as a Lean axiom.

* Ordered edges:
* `$e_0$ = (2,7)`
* `$e_1$ = (2,11)`
* `$e_2$ = (7,11)`

* Controller rows:
* node 2
* node 7

* Controller-incidence matrix:

C = ![![1, 1, 0],
![1, 0, 1]]

* Unnormalized kernel generator:

v = (1, -1, -1)
-/

set_option autoImplicit false

namespace AIEFProofs
namespace ControllerFiber

open Matrix

The /-! ... -/ module documentation must be the first command after all imports. Put:

lean

set_option autoImplicit false

after the module documentation.

The regular copyright comment may remain before imports.
3. Compile the single file first

Bash

set -o pipefail

lake env lean \
AIEFProofs/ControllerFiber.lean \
2>&1 | tee /tmp/controller_fiber_compile.log

echo "Lean file status: ${PIPESTATUS[0]}"

Required result:

text

Lean file status: 0

Warnings from style linters should be inspected, but there must be no error: lines.

Check explicitly:

Bash

```
grep -n "error:" \
/tmp/controller_fiber_compile.log \
|| echo "No Lean errors found"
```

4. If simp still leaves a goal

If the two row lemmas remain unsolved after using Fin.sum_univ_succ, try:

```
lean

norm_num [
CCtrl,
Matrix.mulVec,
dotProduct,
Fin.sum_univ_succ
]
```

Thus the complete fallback first-row proof is:

```
lean

@[simp]
lemma CCtrl_mulVec_apply_zero
(x : Fin 3 → ℚ) :
(CCtrl *ᵥ x) 0 = x 0 + x 1 := by
norm_num [
CCtrl,
Matrix.mulVec,
dotProduct,
Fin.sum_univ_succ
]
```

Use the same replacement for the second row.

Start with simp; use norm_num only if needed.
5. Verify the library root import

Check:

Bash

```
cat AIEFProofs.lean
```

It should contain:

lean

```
import AIEFProofs.Basic
import AIEFProofs.ControllerFiber
```

Avoid duplicate imports:

Bash

```
grep -n \
'import AIEFProofs.ControllerFiber' \
AIEFProofs.lean
```

6. Build the complete project

Only after the direct file compilation succeeds:

Bash

```
set -o pipefail

lake build \
2>&1 | tee /tmp/aief_lean_build.log

build_status=${PIPESTATUS[0]}
echo "Full project status: $build_status"

test "$build_status" -eq 0
```

7. Inspect any remaining header errors

If the header linter still reports line-specific errors, print the file with line numbers:

Bash

```
nl -ba AIEFProofs/ControllerFiber.lean \
| sed -n '1,170p'
```

The required ordering is:

text

```
ordinary copyright comment
imports
module doc comment beginning /-!
set_option
namespaces
declarations
```

Do not disable the header linter merely to hide malformed documentation.
8. Audit unfinished markers and axioms

Bash

```
if grep -RInE \
'\bsorry\b|\badmit\b|sorryAx' \
AIEFProofs
then
echo "ERROR: unfinished proof marker found"
exit 1
else
echo "No unfinished proof markers found"
fi
```

Axiom audit:

Bash

```
cat > /tmp/controller_fiber_axioms.lean <<'EOF'
import AIEFProofs.ControllerFiber

#print axioms \
AIEFProofs.ControllerFiber.CCtrl_mulVec_vCtrl

#print axioms \
AIEFProofs.ControllerFiber.CCtrl_mulVec_eq_zero_iff

#print axioms \
AIEFProofs.ControllerFiber.CCtrl_mulVec_eq_zero_iff_exists_smul

#print axioms \
AIEFProofs.ControllerFiber.vCtrl_ne_zero
EOF

lake env lean /tmp/controller_fiber_axioms.lean \
2>&1 | tee /tmp/controller_fiber_axioms.log

rm -f /tmp/controller_fiber_axioms.lean
```

9. Commit only after the clean build

Bash

```
git status --short

git add \
AIEFProofs.lean \
AIEFProofs/ControllerFiber.lean

git commit -m \
"Formalize exceptional K3 controller kernel"
```

Bottom line

The mathematical proof strategy remains sound. The build failure comes from a Mathlib API mismatch:

text

Fin.sum_univ_three

is unavailable in Lean 4.33.1/your pinned Mathlib revision. Replace it with recursive finite-sum simplification via:

lean

Fin.sum_univ_succ

and move the /-! ... -/ module documentation immediately after the imports.

text

```
/-
Copyright (c) 2026 AIEF Proofs. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: AIEF / SSRH
-/
import Mathlib.Algebra.BigOperators.Fin
import Mathlib.Data.Matrix.Basic
import Mathlib.Tactic.FinCases
import Mathlib.Tactic.Linarith

/-!
# Controller-fiber kernel

This file formalizes only the exact 2×3 controller-incidence identity used
for the exceptional K3 fiber.

It does not formalize delay, EMA, admissibility, winner-map persistence,
synchronization, Engine v1 emission of this matrix, or any dynamical
```

dimension claim.

Scientific mapping of edges
`((2,7), (2,11), (7,11))` to columns, and of controller nodes `{2,7}` to
rows, remains an external audit correspondence.
-/

set_option autoImplicit false
set_option linter.style.header false

namespace AIEFProofs
namespace ControllerFiber

open Matrix Finset

/-- Unsigned incidence of controller nodes `{2,7}` on ordered K3 edges
`((2,7), (2,11), (7,11))`. -/
def CCtrl : Matrix (Fin 2) (Fin 3) ℚ
| 0, 0 => 1
| 0, 1 => 1
| 0, 2 => 0
| 1, 0 => 1
| 1, 1 => 0
| 1, 2 => 1

/-- Canonical unnormalized generator of `ker CCtrl`. -/
def vCtrl : Fin 3 → ℚ
| 0 => 1
| 1 => -1
| 2 => -1

@[simp] lemma CCtrl_00 : CCtrl 0 0 = 1 := rfl
@[simp] lemma CCtrl_01 : CCtrl 0 1 = 1 := rfl
@[simp] lemma CCtrl_02 : CCtrl 0 2 = 0 := rfl
@[simp] lemma CCtrl_10 : CCtrl 1 0 = 1 := rfl
@[simp] lemma CCtrl_11 : CCtrl 1 1 = 0 := rfl
@[simp] lemma CCtrl_12 : CCtrl 1 2 = 1 := rfl

@[simp] lemma vCtrl_zero : vCtrl 0 = 1 := rfl

```
@[simp] lemma vCtrl_one : vCtrl 1 = -1 := rfl
@[simp] lemma vCtrl_two : vCtrl 2 = -1 := rfl


/-- Explicit expansion of a sum over `Fin 3`. -/
lemma sum_fin3 (f : Fin 3 → ℚ) : ∑ i, f i = f 0 + f 1 + f 2 := by
rw [Fin.sum_univ_succ, Fin.sum_univ_succ, Fin.sum_univ_zero]
ring


/-- Evaluation of the first controller row. -/
@[simp]
lemma CCtrl_mulVec_apply_zero (x : Fin 3 → ℚ) :
(CCtrl *ᵥ x) 0 = x 0 + x 1 := by
simp [Matrix.mulVec, dotProduct, sum_fin3]


/-- Evaluation of the second controller row. -/
@[simp]
lemma CCtrl_mulVec_apply_one (x : Fin 3 → ℚ) :
(CCtrl *ᵥ x) 1 = x 0 + x 2 := by
simp [Matrix.mulVec, dotProduct, sum_fin3]


/-- The controller matrix annihilates the canonical fiber generator. -/
theorem CCtrl_mulVec_vCtrl : CCtrl *ᵥ vCtrl = 0 := by
funext i
fin_cases i <;> simp


/-- Coordinate form of the controller-kernel equation. -/
theorem CCtrl_mulVec_eq_zero_iff (x : Fin 3 → ℚ) :
CCtrl *ᵥ x = 0 ↔ x 0 + x 1 = 0 ∧ x 0 + x 2 = 0 := by
constructor
· intro h
constructor
· have h0 := congrFun h (0 : Fin 2)
simpa using h0
· have h1 := congrFun h (1 : Fin 2)
simpa using h1
· rintro ⟨h01, h02⟩
funext i
fin_cases i
· simpa using h01
· simpa using h02
```

```
/-- Locked-ratio form of the two controller equations. -/
theorem CCtrl_coords_iff (x : Fin 3 → ℚ) :
x 0 + x 1 = 0 ∧ x 0 + x 2 = 0 ↔
x 1 = -x 0 ∧ x 2 = -x 0 := by
constructor
· intro ⟨h01, h02⟩
constructor <;> linarith
· intro ⟨h1, h2⟩
constructor <;> linarith


/-- Explicit fiber coordinate form. -/
theorem CCtrl_eq_zero_iff_locked_ratio (x : Fin 3 → ℚ) :
CCtrl *ᵥ x = 0 ↔ x 1 = -x 0 ∧ x 2 = -x 0 := by
rw [CCtrl_mulVec_eq_zero_iff, CCtrl_coords_iff]


/-- Every kernel vector is a scalar multiple of `vCtrl`. -/
theorem CCtrl_mulVec_eq_zero_iff_exists_smul (x : Fin 3 → ℚ) :
CCtrl *ᵥ x = 0 ↔ ∃ a : ℚ, x = a • vCtrl := by
constructor
· intro hx
rcases (CCtrl_mulVec_eq_zero_iff x).mp hx with ⟨h01, h02⟩
refine ⟨x 0, ?_⟩
funext i
fin_cases i
· simp
· simp
linarith
· simp
linarith
· rintro ⟨a, rfl⟩
apply (CCtrl_mulVec_eq_zero_iff (a • vCtrl)).2
constructor <;> simp


theorem vCtrl_ne_zero : vCtrl ≠ 0 := by
intro h
have h0 := congrFun h (0 : Fin 3)
simp at h0


end ControllerFiber
```

end AIEFProofs

text

(1)(deck@steamdeck AIEFProofs)$ source "$HOME/.elan/env"
lake build
echo "Build status: $?"
✖ [1233/1235] Building AIEFProofs.ControllerFiber
trace: .> LEAN_PATH=/home/deck/AI_Programs/SSRH/lean/AIEFProofs/.lake/packages/Cli/.lake/build/lib/lean:/home/deck/AI_Programs/SSRH/lean/AIEFProofs/.lake/packages/batteries/.lake/build/lib/lean:/home/deck/AI_Programs/SSRH/lean/AIEFProofs/.lake/packages/Qq/.lake/build/lib/lean:/home/deck/AI_Programs/SSRH/lean/AIEFProofs/.lake/packages/aesop/.lake/build/lib/lean:/home/deck/AI_Programs/SSRH/lean/AIEFProofs/.lake/packages/proofwidgets/.lake/build/lib/lean:/home/deck/AI_Programs/SSRH/lean/AIEFProofs/.lake/packages/importGraph/.lake/build/lib/lean:/home/deck/AI_Programs/SSRH/lean/AIEFProofs/.lake/packages/LeanSearchClient/.lake/build/lib/lean:/home/deck/AI_Programs/SSRH/lean/AIEFProofs/.lake/packages/plausible/.lake/build/lib/lean:/home/deck/AI_Programs/SSRH/lean/AIEFProofs/.lake/packages/mathlib/.lake/build/lib/lean:/home/deck/AI_Programs/SSRH/lean/AIEFProofs/.lake/build/lib/lean /home/deck/.elan/toolchains/leanprover--lean4---v4.33.1/bin/lean
/home/deck/AI_Programs/SSRH/lean/AIEFProofs/AIEFProofs/ControllerFiber.lean -o
/home/deck/AI_Programs/SSRH/lean/AIEFProofs/.lake/build/lib/lean/AIEFProofs/ControllerFiber.olean -i
/home/deck/AI_Programs/SSRH/lean/AIEFProofs/.lake/build/lib/lean/AIEFProofs/ControllerFiber.ilean -c /home/deck/AI_Programs/SSRH/lean/AIEFProofs/.lake/build/ir/AIEFProofs/ControllerFiber.c --setup
/home/deck/AI_Programs/SSRH/lean/AIEFProofs/.lake/build/ir/AIEFProofs/ControllerFiber.setup.json --json
error: AIEFProofs/ControllerFiber.lean:63:44: Tactic `rewrite` failed: Did not find an occurrence of the pattern
∑ i, ?f i
in the target expression
f 0 + (f (Fin.succ 0) + ∑ i, f i.succ.succ) = f 0 + f 1 + f 2

f : Fin 3 → ℚ
⊢ f 0 + (f (Fin.succ 0) + ∑ i, f i.succ.succ) = f 0 + f 1 + f 2
error: Lean exited with code 1
Some required targets logged failures:
- AIEFProofs.ControllerFiber
error: build failed
Build status: 1

The error is straightforward: a sum over Fin 3 requires three applications of Fin.sum_univ_succ before reaching Fin 0.

After two rewrites, Lean correctly reached:

lean

```
∑ i : Fin 1, f i.succ.succ
```

so Fin.sum_univ_zero could not apply yet.
Fix sum_fin3

Replace:

lean

```
lemma sum_fin3 (f : Fin 3 → ℚ) :
∑ i, f i = f 0 + f 1 + f 2 := by
rw [
Fin.sum_univ_succ,
Fin.sum_univ_succ,
Fin.sum_univ_zero
]
ring
```

with:

lean

```
lemma sum_fin3 (f : Fin 3 → ℚ) :
∑ i, f i = f 0 + f 1 + f 2 := by
rw [
Fin.sum_univ_succ,
Fin.sum_univ_succ,
Fin.sum_univ_succ,
Fin.sum_univ_zero
]
ring
```

The progression is:

text

Fin 3
→ first element + sum over Fin 2
→ second element + sum over Fin 1
→ third element + sum over Fin 0
→ empty sum

Make the edit safely

Bash

```
cd /home/deck/AI_Programs/SSRH/lean/AIEFProofs

python3 - <<'PY'
from pathlib import Path

path = Path("AIEFProofs/ControllerFiber.lean")
text = path.read_text(encoding="utf-8")

old = """ rw [Fin.sum_univ_succ, Fin.sum_univ_succ, Fin.sum_univ_zero]"""

new = """ rw [
Fin.sum_univ_succ,
Fin.sum_univ_succ,
Fin.sum_univ_succ,
Fin.sum_univ_zero
]"""

if old not in text:
raise SystemExit(
"Expected old sum_fin3 line not found; edit manually."
)
```

```
path.write_text(
text.replace(old, new, 1),
encoding="utf-8",
)

print("Updated sum_fin3.")
PY
```

Inspect the relevant lines:

Bash

```
nl -ba AIEFProofs/ControllerFiber.lean \
| sed -n '55,80p'
```

Compile the single file

Bash

```
source "$HOME/.elan/env"

set -o pipefail

lake env lean \
AIEFProofs/ControllerFiber.lean \
2>&1 | tee /tmp/controller_fiber_compile.log

status=${PIPESTATUS[0]}
echo "ControllerFiber status: $status"
```

If successful:

text

```
ControllerFiber status: 0
```

Then build the root:

Bash

```
set -o pipefail

lake build \
2>&1 | tee /tmp/aief_proofs_build.log

status=${PIPESTATUS[0]}
echo "Root build status: $status"
```

If the four-rewrite version leaves an associativity goal

Replace the whole proof with:

lean

```
lemma sum_fin3 (f : Fin 3 → ℚ) :
∑ i, f i = f 0 + f 1 + f 2 := by
simp [Fin.sum_univ_succ, add_assoc]
```

Then compile again. Use this only if the explicit four-rewrite proof still reports a residual goal.
Header option

Your module documentation is now in the correct location: immediately after the imports. Therefore this line should no longer be necessary:

lean

```
set_option linter.style.header false
```

It does not affect the theorem, so first get a successful build. Afterward remove it and rebuild to confirm the header linter is clean.
After a successful build

Bash

```
git add AIEFProofs.lean AIEFProofs/ControllerFiber.lean

git commit -m \
"Formalize exceptional K3 controller kernel"
```

Do not run lake update; the problem is only the finite-sum proof.

text

```
(deck@steamdeck AIEFProofs)$ lake build
echo "Build status: $?"
✖ [1233/1235] Building AIEFProofs.ControllerFiber
trace: .> LEAN_PATH=/home/deck/AI_Programs/SSRH/lean/AIEFProofs/.lake/packages/Cli/.lake/
build/lib/lean:/home/deck/AI_Programs/SSRH/lean/AIEFProofs/.lake/packages/batteries/.lake/build/
lib/lean:/home/deck/AI_Programs/SSRH/lean/AIEFProofs/.lake/packages/Qq/.lake/build/lib/lean:/
home/deck/AI_Programs/SSRH/lean/AIEFProofs/.lake/packages/aesop/.lake/build/lib/lean:/home/
deck/AI_Programs/SSRH/lean/AIEFProofs/.lake/packages/proofwidgets/.lake/build/lib/lean:/home/
deck/AI_Programs/SSRH/lean/AIEFProofs/.lake/packages/importGraph/.lake/build/lib/lean:/home/
deck/AI_Programs/SSRH/lean/AIEFProofs/.lake/packages/LeanSearchClient/.lake/build/lib/lean:/
home/deck/AI_Programs/SSRH/lean/AIEFProofs/.lake/packages/plausible/.lake/build/lib/lean:/home/
deck/AI_Programs/SSRH/lean/AIEFProofs/.lake/packages/mathlib/.lake/build/lib/lean:/home/deck/
AI_Programs/SSRH/lean/AIEFProofs/.lake/build/lib/lean /home/deck/.elan/toolchains/leanprover--
lean4---v4.33.1/bin/lean
/home/deck/AI_Programs/SSRH/lean/AIEFProofs/AIEFProofs/ControllerFiber.lean -o
/home/deck/AI_Programs/SSRH/lean/AIEFProofs/.lake/build/lib/lean/AIEFProofs/
ControllerFiber.olean -i
/home/deck/AI_Programs/SSRH/lean/AIEFProofs/.lake/build/lib/lean/AIEFProofs/
ControllerFiber.ilean -c /home/deck/AI_Programs/SSRH/lean/AIEFProofs/.lake/build/ir/AIEFProofs/
ControllerFiber.c --setup
/home/deck/AI_Programs/SSRH/lean/AIEFProofs/.lake/build/ir/AIEFProofs/ControllerFiber.setup.json
--json
info: AIEFProofs/ControllerFiber.lean:64:2: Try this:
[apply] ring_nf
The `ring` tactic failed to close the goal. Use `ring_nf` to obtain a normal form.
Note that `ring` works primarily in *commutative* rings. If you have a noncommutative ring, abelian
group or module, consider using `noncomm_ring`, `abel` or `module` instead.
error: AIEFProofs/ControllerFiber.lean:62:63: unsolved goals
f : Fin 3 → ℚ
```

⊢ f 0 + f (Fin.succ 0) + f (Fin.succ 0).succ = f 0 + f 1 + f 2
error: Lean exited with code 1
Some required targets logged failures:
- AIEFProofs.ControllerFiber
error: build failed
Build status: 1
(deck@steamdeck AIEFProofs)$

The remaining problem is no longer the sum expansion. The three successor rewrites worked. The unsolved goal is that Lean has not normalized these equivalent Fin 3 indices:

lean

Fin.succ (0 : Fin 2)

and:

lean

(1 : Fin 3)

Likewise for index 2.

ring treats the function applications as different atoms, so it cannot close the goal. ring_nf may leave the same index-identification problem.
Replace sum_fin3 with this proof

lean

/-- Explicit expansion of a sum over `Fin 3`. -/
lemma sum_fin3 (f : Fin 3 → ℚ) :
∑ i, f i = f 0 + f 1 + f 2 := by
rw [
Fin.sum_univ_succ,
Fin.sum_univ_succ,
Fin.sum_univ_succ,
Fin.sum_univ_zero
]

```
have h1 :
Fin.succ (0 : Fin 2) = (1 : Fin 3) := by
decide

have h2 :
Fin.succ (Fin.succ (0 : Fin 1)) =
(2 : Fin 3) := by
decide

simpa [h1, h2]
```

Here decide is appropriate because h1 and h2 are closed finite-index equalities. This is different from asking native_decide to prove an equality containing an arbitrary symbolic function x.
Make the replacement

Edit:

text

AIEFProofs/ControllerFiber.lean

and replace only the sum_fin3 lemma.

Then compile the single file:

Bash

```
cd /home/deck/AI_Programs/SSRH/lean/AIEFProofs
source "$HOME/.elan/env"

set -o pipefail

lake env lean \
AIEFProofs/ControllerFiber.lean \
2>&1 | tee /tmp/controller_fiber_compile.log
```

status=${PIPESTATUS[0]}
echo "ControllerFiber status: $status"

If that reports status zero, build the project:

Bash

set -o pipefail

lake build \
2>&1 | tee /tmp/aief_proofs_build.log

status=${PIPESTATUS[0]}
echo "Root build status: $status"

Do not run lake update.
Alternative shorter proof to try later

Once the file is working, you can test whether your Mathlib simplifier handles this shorter proof:

lean

lemma sum_fin3_short (f : Fin 3 → ℚ) :
∑ i, f i = f 0 + f 1 + f 2 := by
simp [Fin.sum_univ_succ, add_assoc]

But use the explicit h1/h2 proof first because it addresses the exact goal shown by Lean.
Header linter

Your header is now correctly organized:

text

copyright comment
imports
module docstring
set_option

namespaces

After the proof builds, remove:

lean

set_option linter.style.header false

and rebuild. It should no longer be necessary. Keep it temporarily while resolving the theorem error if you prefer.
Current status

The build has progressed beyond all matrix definitions and later theorem declarations. The only reported failure is sum_fin3. Fixing the finite-index normalization should allow Lean to expose whether any later proof needs adjustment.

should I do this?

text

I cannot write onto the Steam Deck from here. The artifact is updated. Apply the same edit locally with this:

```bash
cd /home/deck/AI_Programs/SSRH/lean/AIEFProofs
python3 - <<'PY'
from pathlib import Path
path = Path("AIEFProofs/ControllerFiber.lean")
text = path.read_text(encoding="utf-8")
old = """/-- Explicit expansion of a sum over `Fin 3`. -/
lemma sum_fin3 (f : Fin 3 → ℚ) : ∑ i, f i = f 0 + f 1 + f 2 := by
rw [Fin.sum_univ_succ, Fin.sum_univ_succ, Fin.sum_univ_succ, Fin.sum_univ_zero]
ring"""
old2 = old.replace(" ring", " simp")
new = """/-- Explicit expansion of a sum over `Fin 3`. -/
lemma univ_fin3 : (univ : Finset (Fin 3)) = {0, 1, 2} := by
ext i
constructor
```

```
· intro
fin_cases i <;> simp
· intro
exact mem_univ i


lemma sum_fin3 (f : Fin 3 → ℚ) : ∑ i, f i = f 0 + f 1 + f 2 := by
rw [univ_fin3, sum_insert, sum_insert, sum_singleton]
· ring
· decide
· decide"""
if old in text:
text = text.replace(old, new, 1)
elif old2 in text:
text = text.replace(old2, new, 1)
else:
raise SystemExit("Could not find sum_fin3 block; print the lemma and edit manually.")
path.write_text(text, encoding="utf-8")
print("Updated", path)
PY


source "$HOME/.elan/env"
lake build
echo "Build status: $?"
```

This version no longer uses Fin.succ or Fin.sum_univ_zero. It names the three indices and lets decide discharge the two ∉ side goals of sum_insert.

ring is only used after the sum is already f 0 + f 1 + f 2, where it is legitimate.

If decide fails on a ∉ goal, paste that goal. Do not run lake update.

Yes. This finite-set enumeration proof is a reasonable and likely robust solution. It avoids the problematic definitional identification between:

lean

```
Fin.succ (0 : Fin 2)
```

and:

lean

(1 : Fin 3)

The two decide calls are appropriate because the non-membership goals are closed finite propositions.

I would make univ_fin3 private and use a marker-based replacement rather than relying on exact whitespace matching.
1. Back up the current file

Bash

cd /home/deck/AI_Programs/SSRH/lean/AIEFProofs

cp \
AIEFProofs/ControllerFiber.lean \
AIEFProofs/ControllerFiber.lean.before-sum-fix

2. Apply the safer replacement

Bash

python3 - <<'PY'
from pathlib import Path

path = Path("AIEFProofs/ControllerFiber.lean")
text = path.read_text(encoding="utf-8")

start_marker = (
"/-- Explicit expansion of a sum over `Fin 3`. -/"
)
end_marker = (
"/-- Evaluation of the first controller row. -/"
)

```
start = text.find(start_marker)
end = text.find(end_marker, start)


if start < 0:
raise SystemExit(
"Could not find the sum_fin3 start marker."
)


if end < 0:
raise SystemExit(
"Could not find the first-row end marker."
)


replacement = r"""/-- The universal finite set on `Fin 3` consists of
the three explicitly named indices. -/
private lemma univ_fin3 :
(univ : Finset (Fin 3)) = {0, 1, 2} := by
ext i
constructor
· intro _hi
fin_cases i <;> simp
· intro _hi
exact mem_univ i


/-- Explicit expansion of a sum over `Fin 3`. -/
lemma sum_fin3 (f : Fin 3 → ℚ) :
∑ i, f i = f 0 + f 1 + f 2 := by
rw [
univ_fin3,
sum_insert,
sum_insert,
sum_singleton
]
· ring
· decide
· decide


"""


updated = text[:start] + replacement + text[end:]
```

path.write_text(updated, encoding="utf-8")

print(f"Updated {path}")
PY

This script replaces only the block between the two documentation markers, regardless of the current formatting of the old proof.
3. Inspect the change

Bash

git diff -- AIEFProofs/ControllerFiber.lean

Also inspect the relevant lines:

Bash

nl -ba AIEFProofs/ControllerFiber.lean \
| sed -n '50,100p'

4. Compile only ControllerFiber.lean

Bash

source "$HOME/.elan/env"
set -o pipefail

lake env lean \
AIEFProofs/ControllerFiber.lean \
2>&1 | tee /tmp/controller_fiber_compile.log

status=${PIPESTATUS[0]}
echo "ControllerFiber status: $status"

Required:

text

ControllerFiber status: 0

If it fails, do not run lake update; paste the new exact error.
5. Build the complete project

If the direct file compilation succeeds:

Bash

set -o pipefail

lake build \
2>&1 | tee /tmp/aief_proofs_build.log

status=${PIPESTATUS[0]}
echo "Root build status: $status"

Required:

text

Build completed successfully
Root build status: 0

6. Remove the backup only after success

Bash

rm \
AIEFProofs/ControllerFiber.lean.before-sum-fix

Then commit:

Bash

git add AIEFProofs/ControllerFiber.lean AIEFProofs.lean

git commit -m \
"Prove explicit Fin 3 controller sum expansion"

Why this proof is sound

It establishes:
univFin 3=
univFin3=

{0,1,2},
]

then rewrites:
∑i:Fin 3f(i)
i:Fin3∑f(i)

as three finite-set insertions. The side conditions are exactly:

text

0 ∉ {1,2}
1 ∉ {2}

which decide proves as closed finite propositions. Once expanded, ring handles only associativity/normalization in QQ, where it is appropriate.

So yes: apply this version, compile the single file, and only then run the full project build.

think about the relevance of these things:

So, usually when I uh go talk to somebody about my research, um they'll

say, "Oh, you work in quantum computing. That's cool." You know, I have something I'm trying to compute and it's very slow. Can your quantum computer do it? Um and I I wish the answer were yes, but usually the answer is no. They're interested in some tasks that maybe I don't know neural networks need like matrix multiplication. Um but quantum computers are very very specialized. So, uh many people say they can do many things. Um, often those are slightly overstated. Uh, for many tasks that we actually care about, quantum computers are largely useless. Um, virtually their only like very secure application that we know that they're going to be good at is factoring. Uh, but even like the utility for improving the world by factoring is also a little unclear. And so then they'll sometimes tell me a problem that like maybe quantum computers are useful at. and they'll say, "Oh, I'm simulating some some molecule uh and okay, I don't know any chemistry, but they do." And they say, "Uh, here's a problem uh that I have. Can you help?" And what I'd really like to tell them is that yes, I could totally help. You know, uh, nature is quantum mechanical. Damn it. And we need a quantum computer to simulate it. Um but sadly what they'll say is uh they'll have some particular system and um I don't know if they can actually uh you know get a quantum speed up for that

particular system. And so what I have to say is that well I can promise that there's a very contrived uh quantum system that I have. It also does like arbitrary quantum computations including factoring. Um but you could write as a molecule and quantum computers could do that faster. uh and then they you know give me a molecule and I'm like well rip I don't know what to say about this and so yeah this talk is trying to overcome that and say try to build a theory for quantum advantage where we say oh here's your physical system yes a quantum computer can do better than a classical computer that's kind of the focus of my research uh at the moment and uh this requires understanding how physics informs computation and this is like a very long history I think many of us know more physics than computation I'll tell you a bit about computational complexity so we're on the same page. Um so classical computes P and NP. Um roughly speaking very informally P is something that your laptop can do very quickly. And NP is a set of problems that your laptop can check the answer very quickly. Like I'll give you the answer and your laptop says yes that is the answer. That's that's what NP does. Um but crucially the little vin diagram I drew where P is inside NP is slightly contrived in the

definitions. So what the problems in in the like uh NP but not P section of this ven diagram are are problems that are uh hard in the worst case. So you might have heard for instance that the traveling salesman problem is hard. a salesman wants to go to a bunch of houses and he wants to end up where he started and he wants to do it in less than I don't know 10 miles since he's going by foot. Um and some of these problems are easy like this one on the left. I I mean I if I give you these houses you could probably draw the circle and be like well that's the shortest route houses np hard. Um but the point is that some of them are also much harder and so this like it's a statement that there exists some problem that is hard. um it's not necessarily the problem that you care about. Um okay so the same story holds in the quantum setting where we have uh like BQP instead of P and QMA instead of NP and we want to find problems that might have quantum advantage for some problem instance. Um and that's in complexity theory traditionally a worst case statement but we want to kind of go further and make for your particular molecule we have quantum advantage sort of claim. Um this is where phase transitions come in. So let's recall a bit of physics. Um you might have seen magnets. They have like spin up and spin down things. Uh so they would all try to

line to be spin up. For example, this is my what my little picture is showing. We can write that as a Hamiltonian. Again, apologies about the type setting. Um where the uh the spins are represented by these configurations plus or minus one to the n. These are the x's and we have some coefficients. The partition function is this equation. You sum over all of the spin configurations and exponentiate it with some inverse temperature beta. So beta is like one over temperature. Um knowing this quantity z or more precisely it's log up to additive error is enough to determine thermal expectations. So if you want to say uh you know what is this like local expectation in my system of thermal equilibrium, it's enough for me to compute log z and then I could tell you that number. Um so now we can talk about phase transitions in a somewhat more unusual language perhaps than you're used to. You know that you know water has three phases maybe more I don't know. Um but uh you can also write it kind of in mathematical physics in a more formal way. So you write it in terms of the partition function. Um temperature we all know is a real number since our thermometer doesn't have imaginary numbers on it. But uh the partition function you can evaluate it for fun at

an imaginary temperature. And you'll find that sometimes it comes out to zero. This is a bit weird since if you remember the definition it was like summing e to the minus beta h of x and h of x is a real number. So you really have to go to complex numbers in order to get zeros. Um a phase transition is when this partition function um the log of it becomes non-analytic. So it's defined in a thermodynamic limit of a large system size. as you take the system size to be infinity, you find that uh there's like a pinching of zeros on the real beta axis at a particular point and that's where the phase transition occurs. Now, this is very cool because this allows you to write down a a classical algorithm just from knowing this theory. Um you can try to compute the partition function by tailor expanding and if you do that to run to zero, your radius of convergence, you know, dies pretty quickly. So you can't go all the way to some temperature close to the phase transition but you can kind of tailor expand with an analytic continuation to get there and that gives you a classic algorithm. Unfortunately um because quantum computation is dominated by computer science people um this algorithm was not particularly appreciated in the community and so people thought for example that there was a particular problem that had quantum advantage uh which was for

looking at random fionic systems but it turns out that it's classically easy. So

uh knowing this sort of theory is very helpful.

But then you might ask where can I find quantum advantage if not here? Well you

can look at the other side of the phase transition where I can't play this trick

of an analytic continuation. Um so on this side of the phase

transition uh you have a nice story of of the phase transition actually

coinciding with computational hardness uh for some problems. So spin glasses

are a an example that people are interested in. So the Gibbs measure over

here this this E to the minus beta H of X is a probability distribution over the

spin configurations X and the support of this probability distribution is

supported on clusters. So you can think of this as like one cluster being all

spins close to being up or all spins being close to down. Um and for some

models at the phase transition you get uh like a yeah discontinuous change in

support of this which obstracts algorithms. In particular, if your

algorithm is one that changes the distribution it outputs very smoothly,

it breaks at these phase transitions. And so you can use that in your proof to

show that it's hard. Um, so this idea, for instance, you can take to look at

problems that people thought were in BQP and QMA uh outside of P and NP and show that they're actually inside P and NP. I won't get into that too much, but I think these sorts of things are interesting. So now we have things that are classically easy, things that are quantumly hard. We still don't have the magical thing that's quantumly easy and classically hard that we can say we have quantum advantage for that. So let's look a bit harder. Um you can look at a problem uh uh that looks at first nothing like thermodynamics but rewrite it in the language of thermodynamics. So if you cared about uh sampling from a probability distribution that let's start by writing it as e minus beta h of x um you might say oh you know I have my diffusion model or something and I want to get close in vistine gan or something I want to get close in kl divergence um and so you write down like the kale divergence kc lily divergence from information theory and you just do a bit of algebra and you realize that you're just looking at the gibbs free energy and that this is actually just a thermodynamics problem and once Again computing sort of partition function quantities is all you need. Um so the problem that we can apply this to is community detection which I'll very briefly tell you the definition of. We start with some vertices and we put uh random edges between them with uh

independent identical probability. This is called an Erdishi graph. Um and then we add community labels um randomly. In this case when I did it the green ones fell on the left and the bright red ones on the right. Um but they're random. You don't know what they are. And then between vertices that are in the same community, you add a few more edges with a probability beta. Beta here named very cleverly like it's inverse temperature since that's what it'll be. Um, and once you have this graph, you array the community labels and ask uh, you know, a computer, here's my graph, find the communities in the graph. This is what community detection is. Now it turns out that if you redo all this story but make it a hyperraph so each edge instead of having two vertices has maybe three or more uh a very interesting thing happens which is that you have a sort of phase in this diagram. So if I look at where the phase transition that I defined earlier was it's going to fall at some place where recovering the community labels is actually transitioning from being information theoretically impossible to being possible. Unfortunately, it's very computationally hard uh until some larger beta at which point becomes easy. But we're able to show quantum advantage over there. Um and I guess looking for these sort of

quantum advantage examples is quite rare outside of things like factoring. So this was um selected as a Google XP prize finalist project for applications for quantum computers. We won't know the winners until next year. Um but it's something I'm very excited about and happy to talk more about. Um yeah, so I'll end there. There are many pro problems where looking through this lens of free energy and phase transitions can teach you a lot. I talked about a few of them, but there are many more. So yeah, I really like this thing. Thanks.

Well, um, hello, I'm Nikil. I'm an incoming fellow um, 2026 fellow and I'm currently at Los Alas National Laboratory. So my question was when so going back to the slide you had on where you so I'm guessing you try to analytically continue to complex beta. Yeah. So the way the formalism I've usually seen for characterizing phase transitions is that for for given like a free energy profile for two phases the phase transition temperature uh is the boundary where like the free energy for two different phases uh intersects and then one becomes lower than the other depending on above or below. Your formalism has that the zeros get pinched at the boundary of this region and this region being where there are no zeros for wagsy. So how do you reconcile those two um characterizations of a phase transition?

So this is like the it's perhaps people use first of all I should say people use phase transitions sometimes um a bit more sloppily. So like people will describe these computational going from easy to hard transitions sometimes as phase transitions. I just want to clarify that's not what's going on here.

Um this is known as the Leyang theory of zeros or fisher zeros for the case of beta being the parameter that the phase transition is happening in. Um and yeah it really needs a sense of log z having a nonanalytic uh point at some temperature in the thermodynamic limit. Um this often coincides with other phenomena. So you might have uh yeah like some some intersection going on as well. Um but I guess formally this is what a phase transition is defined as I I don't know if I have a better answer than that.

I see. And as for where you're saying that log Z has a pole in beta does the order of the pole usually matter in terms of characterizing what what the phase mission or at least what that region is?

Uh yeah. So that's a that's a great question. Um you can look at like the order of the phase transition and things through this lens as well. Um I guess for the purposes here, all I care about is the fact that there is phase

transition. Uh but you can look at like the density of the zeros as they approach the real axis. Um and the phase transition occurs as long as you have at least an extensive density, but then how rapidly they approach can change things a little bit as well. I see. So it's more of like a heristic thing than like a hard qualitative thing. uh this quantity is is very quantitative. It's it's saying like in the limit of n going to infinity log z is nonalytic at some beta star where beta star is where they're approaching.

Yeah, this is a very formal sort of thing.

I see. Okay, I think that kind of answers my question.

Cool. Thank you.

Hi. Uh I'm Ivan. I'm from Stanford. Thank you for this really beautiful talk. I'm curious if you could maybe provide some more intuition as to why if we continuously vary beta to higher and higher values, we see these like sudden transitions between different complexity classes and then also what are those boundary values of beta and is there an interpretation for what they mean?

Uh yeah, it's a a great question uh and one with a lot of history behind it. So um these sorts of although I draw pictures and you know talk in somewhat vague terms uh this was first empirically like measured in a lab. It's

how we got started in spin glasses and uh so the the idea is that you should think of these clusters if you want like a physical interpretation as metastable states. So if you were to cool your system starting from infinite temperature, you might get stuck in some sort of metastable state. And so you'd be stuck in like one of the clusters on the far right. Um and to actually sample from the Gibbs measure, you could try like reheating up your system and then cooling it back down and hopefully landing another cluster. Um the issue is that these clusters are of different sizes and like the free energy wells in the free energy landscape is picture of being different sizes and different heights. Um, and so, uh, yeah, there's no guarantee that you'll actually be able to sample from the Gibbs measure. And that is something that people realized non-rigorously with the replica trick from physics back in the 80s. Um, and then was recently made rigorous for these classical spin glasses uh, in yeah the last few decades. Um, it is a very hard thing. I think they got an Abel prize just for showing that the limit exists of the free energy like never mind computing it. competing it was a fields metal. Um so yeah making it rigorous in the quantum case where I'm interested in has still been is is still

open and so that's a big part of what I'm interested in is making that tie to computational complexity like a formal result.

Hi um I'm 2023 fellow. Uh thanks for this great talk. Um you mentioned that replica symmetry breaking comes into play and I guess that separates some uh complexity classes in this case. Colloquially we often think about uh like uh the origin of replica symmetry breaking as being a requirement when you have some increased amount of disorder like for instance like there's this classical result from um I think late 80s early 90s that for instance like solving the like finding the optimal free energy for a lattice protein uh model for instance becomes NP hard but that's because there's a certain amount of disorder in the folding energy lens. Cape and of course classical optimization on some smooth surface then is otherwise very easy and it doesn't fall into that class. So what in the I guess subset of problems that are quantumly easy or or not like separating BQP and QMA um comes from disorder that's like introduced into the actual problem.

Yes, that's a great question. um the it turns out that in order to study sort of asmtoic complexity of these sorts of questions uh you can't say here's a particular molecule since I can't take a large n limit so like I can't say that

well in the limit of a very large molecule of this nature um it's hard or

easy because it would just approach like a single number maybe and that's like a

constant sized question now so um we look at random ensembles of Hamiltonians

and it turns out that in the quantum case although classically having a large

random ensemble usually you have replica symmetry breaking uh in the quantum case

you might not and so this example that about fionic systems like a random

fionic system actually the randomness makes it replica symmetric for all

constant beta um and even though there might be adversarial coefficients that

make it hard um so yeah it's uh these are all all the things that I was

talking about are all large random systems

But the physics can change quite a bit from system to system. Thanks.

Hello, Eric Tal. I study neuroscience at NYU. Uh so u I'm wondering if if you

could spell out in more detail what the connection is between um log z being

analytic and and falling within this sort of classical complexity theory

class. Sure. So uh

let's go back to the tailor expansion. Uh we try to tell expand but this t

expansions work in a disk. And so as soon as you hit a zero you can no longer

say that your tail expansion converges. Um so what you try to do is you uh say okay I'm going to look at a point in the right half of my disc let's say and try expanding around that but I'm going to estimate the value there by my old tailor expansion. And so you repeat this process of patching discs together iteratively until you can get an effective to expansion for something that's far away. And this is what analytic contation is. And it turns out that if you have a new phase transition because you can guarantee that there are no zeros next to this real axis strip, you can always find a region and control the degree of this uh that you get from this tailor expansion and eleation and that's how you end up getting classical algorithms out for it. This is a bit glib. Technically this strategy will get you a quasi polinomial time algorithm. Um, but there are often tricks to make it polinomial time by using cluster expansions and ideas from mathematical physics.

Hello. Uh, I'm Eric, new fellow, recently graduated from UCSB. I wanted to ask, I think you touched on this a little bit, but what are you calling a phase transition when it become goes from like impossible to solve to hard to solve? Um, and can you just speak a little bit more about what exactly that is? Yeah. So that phase transition from impossible to hard is like a a phase

transition in this sense. It's a very formal one. So there the inverse temperature is the like signal to noise ratio of the the communities in the graph. Um there's another transition that people sometimes call a phase transition that happens between hard and easy. Uh but that's not formally a phase transition in this sense. And so like the the physics of it is quite different. What's really happening is that you have many metastable states. um at approximations of a free energy turning into one at that same level of approximation. So it's it's somewhat more subtle thing. Um yeah, maybe that answers your question. Hi um I'm WJ. I'm a 22 and2 fellow. So tying back to what you talked about at the beginning of the presentation, I was wondering if um you were trying to hint at a more general relationship between um hard problems um that can be formulated in this phase transition sense and whether or not there exists um a quantum algorithm capable of providing advantage to solve those problems. Yeah. So um my argument is basically that in the high temperature phase it's classically and quantumly easy and then out of phase transition we actually know that in the worst case of phase transition will obstruct quantum algorithms as well like you can show

things become n hard and unless you believe that say um quantum computers

can do nmplete things then yeah you're going to get stuck there in the worst

case um so the question is really are there natural families of Hamiltonians

that people would care about where quantum computers somehow can get past

this transition but classical computers can't and that's why I was looking at

community detection as sort of example that's a very special example and it's

the only one we have really and very related problems to it um but I think

understanding like the structure of these metastable states in a deeper way

will sort of elucidate what sort of structure you need uh to get this

quantum advantage so that's yeah something I think a lot about

I think that is it for questions if we can get one last round of applause for

this block of fellow showcases

text

Yes. After reading the podcast transcript with the entire AIEF thread in mind, I think there are several **genuinely useful conceptual nuggets**—but the strongest ones are methodological and mathematical, not “quantum computing is somehow analogous to AIEF.”

1. The most resonant idea: phase transitions are about structure changing, not merely a number crossing a threshold

The speaker emphasizes that a formally defined phase transition is associated with a qualitative change in the analytic structure—zeros pinching the real axis, nonanalyticity in the thermodynamic limit—not merely with somebody drawing a line on a parameter plot.

That resonates strongly with one of the lessons from our AIEF work:

We should be very careful about saying:

$$
\rho,;T_{\rm window},;\text{or some margin}
$$

“causes a phase transition” merely because an observable changes sharply.

Our controller boundary is much more modestly established:

$$
m_{25},m_{29},m_{59}>0
$$

through the finite replay, while approaching zero.

That is a **boundary approach**, not yet a phase transition.

Likewise, the T-window study can identify an intervention response, but a change in slope or branch behavior does not automatically constitute a dynamical phase transition.

That distinction from the podcast is worth importing directly into our terminology.

---

2. The podcast's discussion of "easy/hard" versus a true phase transition is almost exactly a warning for our spectral work

The speaker explicitly distinguishes a computational transition from a formally defined thermodynamic phase transition.

That maps beautifully onto the trouble we had with:

> "there is a slow bundle / spectral gap / transition."

A numerical eigenvalue cluster is a **spectral observation**.

A branch switch is a **dynamical regime change**.

A controller boundary is a **geometric boundary**.

A true bifurcation or phase transition would require a much stronger mathematical object.

So one lesson I'd steal is:

[
\boxed{
\text{Don't let a useful metaphor become the formal definition.}
}
]

That is exactly the "map-territory" issue we've been returning to.

---

3. Metastability may be a useful lens for the K3—but only conditionally

The speaker describes clusters of configurations as metastable states, with dynamics potentially getting stuck in one basin before transitioning to another.

This is interesting for AIEF because you have:

* frozen winner maps,
* controller-boundary margins,
* branch persistence,
* delayed feedback,
* and potentially multiple admissible structural configurations.

The natural question becomes:

> Is the exceptional K3 one **metastable branch** inside a larger collection of winner-map/controller configurations?

That is a much more precise question than calling it a "phase."

You could potentially define:

$$
\mathcal B_{\mathcal W}
=======================

{\text{states retaining winner map }\mathcal W}
$$

and ask how long the trajectory remains in that region.

Then the controller margins

$$
m_{25},m_{29},m_{59}
$$

become **distance-to-branch-boundary coordinates**.

That would give the boundary diagnostics a broader dynamical interpretation.

Not proof of metastability—but a useful hypothesis.

---

4. The most interesting mathematical connection is the role of analyticity

The podcast's central computational insight is:

> if the relevant partition function remains analytic in a region, one can move through that region using controlled analytic continuation; zeros obstruct this process.

This actually resonates with your **T-window sensitivity methodology** in an abstract way.

Suppose your observables depend smoothly on a parameter:

[
Y(\rho).
]

If the system remains on the same winner/support regime and all relevant maps are smooth, then one might expect locally smooth parameter dependence:

[
Y(\rho)
=======

Y(1)
+
Y'(1)(\rho-1)
+
O((\rho-1)^2).
]

But a branch boundary can destroy that smooth description.

So the important object isn't merely:

[
Y(\rho).
]

It's the **regime structure underlying (Y(\rho))**.

That dovetails perfectly with your runner philosophy:

> first gate on support topology, winner branch, component class, etc.

The podcast gives a conceptual justification for why that is important: **smooth continuation is meaningful only while you remain in the same analytic/regime region.**

---

5. The "zeros obstruct continuation" idea has a possible AIEF analogue

I would not push this too far, but there is an interesting mathematical analogy.

In the podcast:

[
Z(\beta)=0
]

creates an obstruction to analytic continuation.

In AIEF, a controller boundary corresponds to something like:

[
m_j=0.
]

Before the boundary:

[
m_j>0
]

and the winner map is stable.

At the boundary:

[
m_j=0
]

and the branch becomes nonunique / switching becomes possible.

Beyond it:

[
m_j<0
]

the branch ordering changes.

So, at a purely structural level:

[
\boxed{
\text{analytic continuation region}
\leftrightarrow
\text{fixed-regime branch region}
}
]

and

[
\boxed{
\text{singular/obstructing set}
\leftrightarrow
\text{controller boundary}.
}
]

That's not a theorem yet. But it's an excellent way to think about **regime-conditioned continuation in parameter space**.

It may even suggest a formal object for the modular experiments:

[
\mathcal R
==========

{\rho:\text{support + winner map + clipping regime remain fixed}}.
]

Then study smoothness only within each connected component of (\mathcal R).

That would be much more rigorous than fitting one global sensitivity curve.

---

6. The podcast's "cluster" language resonates with your controller fibers

The speaker talks about probability distributions supported on clusters of configurations.

Our Lean controller result gives a different kind of cluster/fiber structure:

[
Cx=0
\iff
x=a(1,-1,-1).
]

That's a **linear fiber**, not a metastable probability cluster.

But there is a shared abstraction:

[
\text{high-dimensional configuration space}
\rightarrow
\text{restricted structured subset}.
]

This reinforces why I think formalizing the controller incidence geometry is a better mathematical foundation than chasing individual numerical eigenvectors.

The Lean theorem is telling us exactly what the allowed direction is.

The spectral diagnostics were trying to discover what the allowed dynamics are.

Those are different tasks.

---

7. There is a very useful connection to your multi-seed strategy

The podcast makes a strong point that complexity statements are often **ensemble statements**, rather than statements about one special instance. The speaker explicitly distinguishes one molecule from a family of random Hamiltonians because a single finite instance doesn't support an asymptotic complexity claim.

That resonates very strongly with your move from:

> one exceptional K3 trajectory

to:

> 40-seed intervention studies.

The methodological lesson is:

$$
\boxed{
\text{one trajectory} \neq \text{general mechanism}.
}
$$

Our S01 seed is valuable for discovery and exact replay analysis.

The 40-seed T-window experiments are what let you ask whether the discovered structure survives ensemble variation.

That is exactly the right evolution.

---

8. The podcast also gives a reason to preserve "component classes"

One of the questions raised is how disorder and structure change the computational regime.

That resonates with the existing plan to examine K4/K5/K6 component classes before generating enormous new seed cohorts.

Why?

Because topology may determine the dimension and geometry of the invariant fiber.

You already have:

$$
\dim\ker C_{K3}=1.
$$

A K4 may have a different incidence rank and therefore a different lineality dimension.

So rather than asking:

> "Does AIEF always have a rank-one fiber?"

the better question is:

$$
\boxed{
\text{How does topology determine the dimension of the admissible/controller subspace?}
}
$$

That's a real mathematical question.

And it is exactly the type of structural classification the podcast's discussion of ensembles and phases is nudging toward.

---

9. The most valuable phrase to steal: "the structure of the metastable states"

Near the end the speaker says that understanding the structure of metastable states may illuminate what is required for the computational phenomenon.

That could be repurposed almost verbatim for AIEF:

> **Understand the structure of the admissible controller/winner states before trying to explain the asymptotic dynamics.**

We've spent a lot of time asking:

$$
\text{Why does the observable decay at } \alpha?
$$

But perhaps the prior question is:

$$
\boxed{
\text{What is the geometry of the branch region that supports the slow trajectory?}
}
$$

Your controller-fiber Lean theorem is a first exact piece of that geometry.

The boundary-margin diagnostic is an empirical piece.

The multi-seed study tests persistence of that structure.

That is a much more coherent program.

One connection I would *not* make

I would not take the podcast's use of words like:

* phase transition,
* quantum advantage,
* replica symmetry breaking,
* spin glass,

and import them into AIEF just because the vocabulary sounds resonant.

The speaker himself is careful about distinguishing formal phase transitions from loose computational language.

We should be equally strict.

There is currently **no evidence that the AIEF controller boundary is a thermodynamic phase transition** in the formal sense.

The idea I find most promising

The podcast gives me a new way of organizing the AIEF program:

Regime space

Define regions of state/parameter space where:

$$
[
\text{winner map},\quad
\text{support topology},\quad
\text{controller active set}
]
$$

are constant.

Within a regime

Study:

$$
\text{fiber geometry}
\rightarrow
\text{fast contraction}
\rightarrow
\text{memory-driven slow evolution}.
$$

At regime boundaries

Study:

$$
m_j=0
$$

as the analogue of an obstruction to continuation.

Across regimes

Do **not** assume smooth continuation.

Instead classify transitions.

That would connect beautifully to the modular driver architecture:

$$
\boxed{
\text{Engine}
\rightarrow
\text{Regime specification}
}
$$

\rightarrow
\text{Intervention}
\rightarrow
\text{Seed ensemble}
\rightarrow
\text{Analysis}
}
]

and it gives your Lean formalization a natural role: prove the structural geometry of each regime where possible, while the computational experiments establish persistence and transition behavior.

So yes: **there are real nuggets here**. The biggest one isn't "AIEF resembles a spin glass." It is the much more useful idea that **structure and regime boundaries should be treated as first-class mathematical objects**, and that continuity/sensitivity claims should be made only inside validated structural regions.

That fits remarkably well with where your project has independently been heading.

There is a genuinely useful idea here, but several claims need substantial narrowing. The strongest connection is not to quantum advantage or thermodynamic phase transitions. It is to piecewise-smooth/analytic continuation within a fixed structural stratum.
Central assessment

The useful synthesis is:
AIEF has structural strata on which its finite-step map has a fixed formula.
AIEF has structural strata on which its finite-step map has a fixed formula.

Within such a stratum, parameter continuation may be smooth or analytic. At a boundary, the selected formula, derivative, support, or scientific observable may change.

That is a valuable mathematical program. But it is not Lee–Yang theory, a thermodynamic phase transition, metastability, or an asymptotic complexity transition.
1. Phase-transition terminology

The caution is correct:

A sharp change in an AIEF plot is not sufficient evidence of a phase transition.

AIEF currently has:

N=12N=12, not a thermodynamic limit;
finite simulation horizons;
nonsmooth update rules;
hard topology thresholds;
only five TwindowTwindow levels in S01;
no established singular limiting free energy or analogous potential.

Therefore, current observations should be described as:

text

parameter sensitivity
regime change
support change
controller switch
active-set transition
boundary approach
nonsmooth branch crossing

not a formal phase transition.

A finite-dimensional bifurcation might eventually be studied, but that would require:

invariant or recurrent asymptotic objects;
stability changes;
continuation in a parameter;
control of the delay-augmented state;
behavior beyond a finite 320-unit future.

2. Controller boundaries versus partition-function zeros

The analogy:
$Z(\beta)=0 \leftrightarrow mj=0$
$Z(\beta)=0 \leftrightarrow mj=0$

is conceptually suggestive but mathematically weak.

A Fisher/Lee–Yang zero is a zero of a complexified partition function whose accumulation may generate a thermodynamic-limit nonanalyticity.

A controller margin zero is a finite-dimensional switching surface where:

two controller scales tie;
a selected branch formula may cease to be unique;
a classical derivative may change or fail to exist;
the full map may nevertheless remain continuous.

Thus mj=0mj=0 is not necessarily a singularity of the state trajectory. It is more accurately:

an obstruction to continuing one fixed branch representation with one fixed classical Jacobian.

That is still important, but it is a piecewise-smooth/nonsmooth-analysis statement, not a partition-function-zero statement.
3. A more rigorous “regime space”

The proposed regime set should contain more information than winner map and support.

For the augmented AIEF map, a structural signature should include:

Positive proposal support
xij>0orxij=0.
xij>0orxij=0.

Global gate state

text

active / inactive

Controller active set for every edge
unique winning endpoint;
tied endpoints;

inactive endpoints.

Capacity-row active set

Threshold topology
Bij>Bc,
Bij>Bc,

noting that this affects observables but not the continuous weight update itself.

Connected support-component partition

Coherence magnitude zero set
Cij=0,
Cij=0,

because |Cij||Cij| is not differentiable at zero as a real map.

Phase-coordinate chart

The implemented mod 2π coordinate has chart discontinuities even though the corresponding circle dynamics may remain smooth.

A regime cell could therefore be defined as:
Rσ=
Rσ=

{X:\operatorname{signature}(X)=\sigma},
]

where XX is the complete delay-augmented state.

This is better called a stratum, active-set cell, or branch cell than a phase.
4. Conditional analyticity is a real theorem direction

The analyticity idea can be made much more concrete than the thermodynamic analogy.

For ρ>0ρ>0,
α(ρ)=
α(ρ)=

\exp\left(
-\frac{dt}{\rho T_*}
\right)
]

is analytic in ρρ.

On a strict branch cell:

maximum(proposal, 0) is a fixed linear selection;
minimum(1, scale) has a fixed active set;
minimum.outer has a fixed controller selection;
denominators contain the positive safeguard;
symmetrization is linear;
sine and complex exponential are analytic;
|C||C| is real-analytic away from C=0C=0;
a fixed future-shuffle seed produces the same sequence of edge permutations.

Therefore, conditionally on a fixed finite branch itinerary and nonzero relevant coherence entries, the finite-horizon state map may be real-analytic in ρρ:
ρ⟼XH(ρ).
ρ⟼XH(ρ).

A defensible prospective theorem is:

For a fixed seed, finite horizon HH, fixed RNG realization, and an open parameter interval on which the positivity support, global gate, controller active sets, phase chart, and coherence nonzero conditions remain fixed, the HH-step augmented state is real-analytic in ρρ.

That is much closer to the implemented system than the partition-function analogy.
Outcome analyticity is separate

Even if XH(ρ)XH(ρ) is analytic, every reported outcome need not be.

Examples:
Potentially smooth within a strict cell

BfinalBfinal;
continuous-weight cosine similarity, away from zero norm;
total weight;
some mean-weight observables.

Hard-threshold dependent
Pij=1{Bij>Bc}
Pij=1{Bij>Bc}

can jump when a weight crosses BcBc.
Discretization dependent

Normalized horizon boundaries use:
nW(ρ)=
nW(ρ)=

\operatorname{round}
\left(
\frac{\rho T_*}{dt}
\right).
]

Thus normalized-EMA summaries can change definition when window_steps changes, even if the underlying trajectory varies smoothly.

The fixed-physical primary outcomes avoid this particular horizon-rounding change, but their topology-incidence components still use hard thresholds.
5. Five rho values do not establish smooth continuation

S01 samples:
ρ∈{0.5,0.75,1,1.5,2}.
ρ∈{0.5,0.75,1,1.5,2}.

That is enough for prespecified paired sensitivity contrasts. It is not enough to establish:

differentiability at ρ=1ρ=1;
analyticity;
a critical rho;
a branch boundary;
a bifurcation;
a phase transition.

A future continuation study would need a distinct estimand and design:

select a seed and structural object;
fully regenerate at every rho;
classify its complete branch signature;
adaptively refine rho near signature changes;
distinguish trajectory changes from collector-boundary changes;
use common seed/RNG coupling across rho;
report one-sided behavior at switching values.

Such a study should use the existing seeds rather than immediately introducing another broad cohort.
6. “Metastable K3 branch” is premature

The exceptional K3 remained on one observed winner map throughout the saved finite replay while margins approached a tie.

That establishes finite branch persistence. It does not yet establish metastability.

Metastability normally requires something such as:

a long residence timescale relative to local relaxation;
eventual exits;
stochastic perturbations;
a family of initial conditions;
quasi-stationary distributions;
basin and transition-rate structure.

No effective positive-edge winner transition was observed in the saved exceptional trajectory.

Therefore, call it:

text

persistent strict branch
long-lived branch cell
finite-replay branch residence
one-sided approach to a tie boundary

A metastability hypothesis could later be tested under controlled perturbations or noise, but it is not currently a result.
7. Controller margins are slacks, not automatically distances

The thought says:

controller margins become distance-to-boundary coordinates.

More precisely, they are signed boundary slacks.

If a branch boundary is locally:
$g(X)=0$,
$g(X)=0$,

then $g(X)g(X)$ is not generally the Euclidean distance to the boundary. For a linear hyperplane in a chosen norm, a normalized distance may be:
$|g(X)|\|\nabla g\|_*$.
$\|\nabla g\|_*|g(X)|$.

For nonlinear proposal/controller expressions, even that is only a local construction.

Use:

text

signed controller margin
boundary slack
one-sided branch coordinate

unless a norm and normalization theorem are supplied.

8. Topology alone does not determine fiber dimension

This is another important correction.

The exceptional K3 has:
dimkerCJ=1
dimkerCJ=1

because it uses the partial controller-row set:
J={2,7}.
J={2,7}.

At the exact tied/full-controller K3:
CK3=
CK3=

\begin{pmatrix}
1&1&0\
1&0&1\
0&1&1
\end{pmatrix},
\qquad
\ker C_{K_3}={0}.
]

The support topology is K3 in both cases, but the fiber dimension differs.

Thus the relevant object is not topology alone. It is:
support graph+active controller rows+tie structure
support graph+active controller rows+tie structure

More generally:
dimkerCJ=
dimkerCJ=

|E|-\operatorname{rank}(C_J).
]

K4/K5/K6 classification should therefore be indexed by the controller-incidence pattern, not merely clique size.

Lean is well suited to formalizing these finite combinatorial rank/kernel statements.
9. The ensemble analogy is partly misapplied

The complexity-theory point is sound:

A single finite instance cannot establish an asymptotic family-level complexity result.

But the thought then overstates what S01 does.

The 40-seed TwindowTwindow study tests seed-level variation in aggregate network outcomes. It does not establish cross-seed variation of the exceptional partial-controller K3 mechanism.

The corrected component audit indicates approximately:

text

26 actual isolated K3 components
25 ordinary tied/full-controller K3s
1 exceptional strict/partial-controller K3

Therefore:

The observed partial-controller isolated-K3 class has one instance. Cross-seed variation for that class cannot be estimated.

S01 is a 40-seed parameter sensitivity experiment, not a 40-replicate test of the exceptional K3 mechanism.

This distinction is essential.
10. The Lean theorem and dynamical invariance

The Lean result:
$Cx=0 \iff x=a(1,-1,-1)$
$Cx=0 \iff x=a(1,-1,-1)$

proves the exact kernel of one controller-incidence matrix.

It does not prove that this direction is:

dynamically invariant;
attracting;
slow;
realized by perturbed trajectories;
preserved through a controller tie;
present in every K3.

Its role is to establish the exact geometry of a specified branch matrix. Dynamical claims require the intertwining/closure conditions and branch persistence assumptions.

The thought recognizes this distinction in places, but phrases such as “the allowed direction” should be replaced with:

the algebraic lineality direction of the specified controller fiber.

11. The modular system’s appropriate role

The modular platform could make regime structure first-class through a dedicated collector and validator.

A future regime-audit record might contain:

text

seed
parameter level
time step
positive proposal support
B_c threshold support
component partition
global gate state
active row constraints
controller winner/tie sets

controller margins
minimum positivity slack
minimum capacity slack
coherence-zero flags
branch signature hash

The resulting object is a branch itinerary:
σ0,σ1,…,σH.
σ0,σ1,…,σH.

Then parameter continuation can ask:
Iσ=
Iσ=

{\rho:
\text{the finite-horizon branch itinerary equals }\sigma
}.
]

But current S01 artifacts do not necessarily contain every proposal/controller quantity at every step. A full branch-itinerary analysis may require regenerated trajectories with a new collector.

That would be a repeated workflow using the same seeds, not an independent experiment.
12. Better mathematical program

The most rigorous version of the proposed direction is:
A. Define branch strata

Formalize the strict inequalities and active sets that select one explicit update formula.
B. Prove local formula regularity

Within a strict stratum, establish continuity, differentiability, or real analyticity of the selected finite-step map.
C. Prove finite-itinerary parameter regularity

If an entire finite trajectory remains a positive distance from all switching surfaces, prove smooth/analytic dependence on rho over a neighborhood.
D. Classify boundary events

Separate:

text

positivity support event
global gate event
controller tie
B_c topology event
coherence-zero event
phase-chart event

E. Study one-sided continuation

At a switching surface, derive one-sided branch maps and directional derivatives where available.
F. Only then consider asymptotic bifurcations

Do not infer an attractor transition from finite branch changes.
Relevance ranking
High relevance

formal versus metaphorical transition;
structural strata;
branch-conditioned continuation;
hard boundaries as first-class objects;
component/controller classification;
finite instance versus family-level claim.

Medium relevance

metastability as a future perturbative hypothesis;
analytic continuation as a broad conceptual analogy;
clusters as structured subsets.

Low/directly irrelevant

quantum advantage;
BQP/QMA separation;

replica symmetry breaking;
spin-glass free-energy structure;
partition-function zeros as a literal AIEF mechanism.

Bottom line

The strongest useful conclusion is:
AIEF should be treated as a stratified, piecewise-analytic delayed map,
AIEF should be treated as a stratified, piecewise-analytic delayed map,

with sensitivity claims conditioned on a fixed finite structural itinerary.

But three corrections are essential:

A controller margin is a signed slack, not automatically a distance.
Controller-fiber dimension depends on active controller/tie structure, not topology alone.
The 40-seed S01 study does not provide 40 replicates of the exceptional partial-controller K3 mechanism.

The modular runner can support this program by recording structural signatures and boundary events. Lean can formalize the exact geometry and within-stratum identities. Neither finite experiments nor the kernel theorem alone establish metastability, bifurcation, or phase-transition behavior.

Can our modular system be used to probe metastability, bifurcation, or phase-transition behavior?

Try again.

Can our modular system be used to probe metastability, bifurcation, or phase-transition behavior?

hello

Can our modular system be used to probe metastability, bifurcation, or phase-transition behavior?

Can our modular system be used to probe metastability, bifurcation, or phase-transition behavior?

hello

hello

text

```
(deck@steamdeck AIEFProofs)$ cd /home/deck/AI_Programs/SSRH/lean/AIEFProofs
python3 - <<'PY'
from pathlib import Path
path = Path("AIEFProofs/ControllerFiber.lean")
text = path.read_text(encoding="utf-8")
old = """/-- Explicit expansion of a sum over `Fin 3`. -/
lemma sum_fin3 (f : Fin 3 → ℚ) : ∑ i, f i = f 0 + f 1 + f 2 := by
rw [Fin.sum_univ_succ, Fin.sum_univ_succ, Fin.sum_univ_succ, Fin.sum_univ_zero]
ring"""
old2 = old.replace(" ring", " simp")
new = """/-- Explicit expansion of a sum over `Fin 3`. -/
lemma univ_fin3 : (univ : Finset (Fin 3)) = {0, 1, 2} := by
ext i
constructor
· intro
fin_cases i <;> simp
· intro
exact mem_univ i

lemma sum_fin3 (f : Fin 3 → ℚ) : ∑ i, f i = f 0 + f 1 + f 2 := by
rw [univ_fin3, sum_insert, sum_insert, sum_singleton]
· ring
· decide
· decide"""
if old in text:
text = text.replace(old, new, 1)
elif old2 in text:
text = text.replace(old2, new, 1)
else:
raise SystemExit("Could not find sum_fin3 block; print the lemma and edit manually.")
path.write_text(text, encoding="utf-8")
print("Updated", path)
PY
Updated AIEFProofs/ControllerFiber.lean
(deck@steamdeck AIEFProofs)$ source "$HOME/.elan/env"
lake build
echo "Build status: $?"
```

```
Build completed successfully (1235 jobs).
Build status: 0
(deck@steamdeck AIEFProofs)$ source "$HOME/.elan/env"

grep -RInE '\bsorry\b|\badmit\b|sorryAx' AIEFProofs
grep -n 'ControllerFiber' AIEFProofs.lean

cat > /tmp/aief_controller_axioms.lean <<'EOF'
import AIEFProofs.ControllerFiber
open AIEFProofs.ControllerFiber
#print axioms CCtrl_mulVec_vCtrl
#print axioms CCtrl_mulVec_eq_zero_iff
#print axioms CCtrl_eq_zero_iff_locked_ratio
#print axioms CCtrl_mulVec_eq_zero_iff_exists_smul
#print axioms vCtrl_ne_zero
EOF
lake env lean /tmp/aief_controller_axioms.lean

git add AIEFProofs.lean AIEFProofs/ControllerFiber.lean
git status
3:import AIEFProofs.ControllerFiber
'AIEFProofs.ControllerFiber.CCtrl_mulVec_vCtrl' depends on axioms: [propext, Classical.choice,
Quot.sound]
'AIEFProofs.ControllerFiber.CCtrl_mulVec_eq_zero_iff' depends on axioms: [propext,
Classical.choice, Quot.sound]
'AIEFProofs.ControllerFiber.CCtrl_eq_zero_iff_locked_ratio' depends on axioms: [propext,
Classical.choice, Quot.sound]
'AIEFProofs.ControllerFiber.CCtrl_mulVec_eq_zero_iff_exists_smul' depends on axioms: [propext,
Classical.choice,
Quot.sound]
'AIEFProofs.ControllerFiber.vCtrl_ne_zero' depends on axioms: [propext, Classical.choice,
Quot.sound]
On branch master
Changes to be committed:
(use "git restore --staged <file>..." to unstage)
modified: AIEFProofs.lean
new file: AIEFProofs/ControllerFiber.lean

(deck@steamdeck AIEFProofs)$
```

```
text


(deck@steamdeck AIEFProofs)$ cd /home/deck/AI_Programs/SSRH/lean/AIEFProofs
python3 - <<'PY'
from pathlib import Path
path = Path("AIEFProofs/ControllerFiber.lean")
text = path.read_text(encoding="utf-8")
old = """/-- Explicit expansion of a sum over `Fin 3`. -/
lemma sum_fin3 (f : Fin 3 → ℚ) : ∑ i, f i = f 0 + f 1 + f 2 := by
rw [Fin.sum_univ_succ, Fin.sum_univ_succ, Fin.sum_univ_succ, Fin.sum_univ_zero]
ring"""
old2 = old.replace(" ring", " simp")
new = """/-- Explicit expansion of a sum over `Fin 3`. -/
lemma univ_fin3 : (univ : Finset (Fin 3)) = {0, 1, 2} := by
ext i
constructor
· intro
fin_cases i <;> simp
· intro
exact mem_univ i


lemma sum_fin3 (f : Fin 3 → ℚ) : ∑ i, f i = f 0 + f 1 + f 2 := by
rw [univ_fin3, sum_insert, sum_insert, sum_singleton]
· ring
· decide
· decide"""
if old in text:
text = text.replace(old, new, 1)
elif old2 in text:
text = text.replace(old2, new, 1)
else:
raise SystemExit("Could not find sum_fin3 block; print the lemma and edit manually.")
path.write_text(text, encoding="utf-8")
print("Updated", path)
PY
Updated AIEFProofs/ControllerFiber.lean
(deck@steamdeck AIEFProofs)$ source "$HOME/.elan/env"
lake build
echo "Build status: $?"
Build completed successfully (1235 jobs).
Build status: 0
(deck@steamdeck AIEFProofs)$ source "$HOME/.elan/env"
```

```
grep -RInE '\bsorry\b|\badmit\b|sorryAx' AIEFProofs
grep -n 'ControllerFiber' AIEFProofs.lean

cat > /tmp/aief_controller_axioms.lean <<'EOF'
import AIEFProofs.ControllerFiber
open AIEFProofs.ControllerFiber
#print axioms CCtrl_mulVec_vCtrl
#print axioms CCtrl_mulVec_eq_zero_iff
#print axioms CCtrl_eq_zero_iff_locked_ratio
#print axioms CCtrl_mulVec_eq_zero_iff_exists_smul
#print axioms vCtrl_ne_zero
EOF
lake env lean /tmp/aief_controller_axioms.lean

git add AIEFProofs.lean AIEFProofs/ControllerFiber.lean
git status
3:import AIEFProofs.ControllerFiber
'AIEFProofs.ControllerFiber.CCtrl_mulVec_vCtrl' depends on axioms: [propext, Classical.choice,
Quot.sound]
'AIEFProofs.ControllerFiber.CCtrl_mulVec_eq_zero_iff' depends on axioms: [propext,
Classical.choice, Quot.sound]
'AIEFProofs.ControllerFiber.CCtrl_eq_zero_iff_locked_ratio' depends on axioms: [propext,
Classical.choice, Quot.sound]
'AIEFProofs.ControllerFiber.CCtrl_mulVec_eq_zero_iff_exists_smul' depends on axioms: [propext,
Classical.choice,
Quot.sound]
'AIEFProofs.ControllerFiber.vCtrl_ne_zero' depends on axioms: [propext, Classical.choice,
Quot.sound]
On branch master
Changes to be committed:
(use "git restore --staged <file>..." to unstage)
modified: AIEFProofs.lean
new file: AIEFProofs/ControllerFiber.lean

(deck@steamdeck AIEFProofs)$
```